



Introduction to Clinical Data Analysis in R

A Practical Guide to Data Management, Statistical Analysis, and Interpretation

Bernard Isekah Osang'ir
Vương Mỹ Lượng

Introduction to Clinical Data Analysis in R

A Practical Guide to Data Management, Statistical Analysis, and Interpretation

Bernard Isekah Osang'ir
Vương Mỹ Lượng



Introduction to Clinical Data Analysis in R

A Practical Guide to Data Management, Statistical Analysis, and Interpretation

First edition, 2026

The data used throughout this book are simulated for teaching: they resemble a real multicentre study of hypertension care, but the results illustrate methods and are not clinical findings.

Every result, table and figure in this book was produced by the R code printed alongside it, using R, knitr and the tidyverse, and the book was typeset with LuaLaTeX.

Copyright © Neudata, 2026

All property rights and copyright are reserved. This book contains proprietary information and materials developed by Neudata and may not be copied, reproduced, distributed, published or cited, in whole or in part, without the prior written consent of Neudata. Where explicit written permission has been granted, appropriate attribution must be included, followed by “by courtesy of Neudata”.

Neudata Consulting Ltd · www.neu-data.com · contact@neu-data.com

Contents

Preface	vi
1 Getting Started with R and RStudio	1
1.1 What R and RStudio are	2
1.2 R as a calculator	4
1.3 Objects and assignment	7
1.4 Data types	10
1.5 Vectors	15
1.6 Functions, arguments and help	21
1.7 Packages	23
1.8 Projects, working directories and file paths	25
1.9 The case study and importing data	26
1.10 A first inspection of the data	29
1.11 The pipe and a first taste of dplyr	37
1.12 Writing a reproducible script	40
1.13 Summary	42
1.14 Further reading	42
1.15 Exercises	43
2 Understanding and Cleaning Clinical Data	45
2.1 Why cleaning matters	46
2.2 Tidy data and variable types	47
2.3 Importing with explicit missing-value codes	49
2.4 Missing data: concepts and an audit	52
2.5 Duplicate records	56
2.6 Inconsistent categories and whitespace	58
2.7 Validating impossible values	62
2.8 Recoding and deriving variables	66
2.9 Dates	71

2.10	Factors, level order and reference levels	73
2.11	Final validation, saving and the cleaning log	77
2.12	Summary	81
2.13	Further reading	82
2.14	Exercises	82
3	Descriptive Statistics, Tables and Figures	85
3.1	Why describe before testing?	86
3.2	Types of variables and the summaries that match them	87
3.3	Measures of central tendency	88
3.4	Measures of spread	91
3.5	The shape of a distribution	94
3.6	Missing values and the <code>na.rm</code> trap	99
3.7	Grouped summaries	100
3.8	From description to inference: standard error and confidence interval.	102
3.9	Describing categorical variables	106
3.10	Building Table 1 with <code>gtsummary</code>	110
3.11	Figures with <code>ggplot2</code>	114
3.12	Exporting tables and figures	120
3.13	Summary	121
3.14	Further reading	122
3.15	Exercises	122
4	Common Statistical Tests in Medical Research	124
4.1	From description to inference	125
4.2	Defining the analytic population.	132
4.3	Checking the normality assumption	133
4.4	Comparing two groups: a continuous outcome.	136
4.5	Comparing more than two groups	142
4.6	Two categorical variables	147
4.7	Correlation between two continuous variables	154
4.8	Matching the question to the test.	156
4.9	A bridge to regression: why one variable at a time is not enough	157
4.10	Reporting results correctly	158
4.11	Summary	159

4.12 Further reading	160
4.13 Exercises	160
5 Regression Modelling and Reporting Results	163
5.1 Why regression?	164
5.2 Setting up the analysis	165
5.3 Linear regression for a continuous outcome	166
5.4 Binary outcomes: probability, odds and log-odds	171
5.5 The logistic regression model	173
5.6 Fitting the logistic model in R	176
5.7 Confounding: crude versus adjusted estimates	183
5.8 Building the model	187
5.9 Checking the model	190
5.10 Presenting the results	196
5.11 Writing the results	199
5.12 Reproducible reporting	201
5.13 Summary	204
5.14 Further reading	205
5.15 Exercises	205
A Solutions to the Exercises	208
A.1 Chapter 1: Getting Started with R and RStudio	208
A.2 Chapter 2: Understanding and Cleaning Clinical Data	214
A.3 Chapter 3: Descriptive Statistics, Tables and Figures	225
A.4 Chapter 4: Common Statistical Tests in Medical Research	233
A.5 Chapter 5: Regression Modelling and Reporting Results	244
References	255

Preface

Every clinical research project ends in numbers: a proportion of patients who were treated, a difference in mean blood pressure, an odds ratio with its confidence interval. Those numbers shape guidelines, funding decisions and, ultimately, the care that patients receive. Yet the path from a spreadsheet of raw measurements to a sentence in a manuscript is often hidden from view. It may live in a sequence of mouse clicks that nobody recorded, in a spreadsheet that was edited by hand, or in the memory of a single analyst who has since moved on. This book is about making that path visible, careful and repeatable.

Introduction to Clinical Data Analysis in R: A Practical Guide to Data Management, Statistical Analysis, and Interpretation teaches you to analyse clinical data with R, a free programming language for statistical computing (R Core Team 2026). It is written for health professionals who want to understand and carry out their own analyses, not for programmers who happen to work in health. We start from the very beginning, assuming no previous experience of programming, and finish with a complete, reproducible analysis of a realistic clinical study, from importing a messy raw data file to writing the Results section of a manuscript.

Why R for clinical research?

There are many statistical packages on the market, and most of them can produce a t-test or a logistic regression. The case for R does not rest on any single statistical procedure. It rests on a way of working.

The analysis as code

When you analyse data in R, you write instructions in a script: import this file, recode that variable, exclude these impossible values, fit this model. The script *is* the analysis. Anyone who has the script and the raw data can run it and obtain exactly the same tables and figures. If a reviewer asks how a variable was derived, the answer is written down. If you discover an error in the raw data six months later, you correct it and re-run the script, and every downstream number updates consistently.

This is the essence of **reproducible research**: the idea that a published result should be accompanied by the data and code needed to regenerate it (Peng 2011). Peng argued that in computational science, where results depend on long chains of data processing, reproducibility is the minimum standard against which findings can be checked, even before anyone attempts to replicate a study independently. Clinical research is now thoroughly computational. Even a modest cross-sectional survey involves dozens of decisions about missing values, outliers, categories and model specification, and each decision can change the answer.

Point-and-click software encourages us to make those decisions silently. A script makes them explicit. That is why we will insist throughout this book that you write your work in scripts rather

than typing commands into the console and forgetting them.

Transparency and trust

Clinical research is increasingly expected to be open. Journals ask for data-sharing statements, funders ask for data-management plans, and reporting guidelines ask authors to describe their statistical methods in enough detail for others to judge them. An R script is the most complete methods section you can write. It records not only *which* test was used but *how* the data were prepared for it, which is where most real errors occur.

Transparency also protects you. Writing analysis code forces you to think about each step, and reading it back later lets you, a colleague or a supervisor find mistakes before they reach print. Wilson et al. (2017) describe a set of “good enough” practices for scientific computing: keep raw data untouched, write code in scripts, give files and variables meaningful names, organise each project in its own folder, and record the steps that turn raw data into results. None of these practices requires advanced skills, and all of them are easy to follow in R. We adopt them from the first chapter.

Free, open and widely used

R is free and open source. You can install it on your own laptop, on a hospital computer or on a university server without a licence fee, and you can keep using it when you change institutions or countries. For researchers in low- and middle-income settings, and for anyone working on a limited budget, this matters a great deal. It also means that your collaborators can run your code without buying anything.

Because R is open, thousands of statisticians and researchers have contributed *packages*: bundles of functions that extend what R can do. Many of the methods used in medical statistics were first implemented in R, and new methods often appear there before anywhere else. In this book we rely heavily on the **tidyverse**, a coherent collection of packages for importing, tidying, transforming and visualising data that share a common design and grammar (Wickham et al. 2019). The tidyverse makes R code read almost like a description of what you want to do, which is a great help to beginners and to anyone who must read your code later.

An honest word about the learning curve

R asks more of you at the start than a menu-driven package. You will make typing mistakes, meet error messages and occasionally wonder why a comma matters so much. This is normal, and it passes faster than most people expect. The investment pays back quickly: once you have written a script that cleans and describes one dataset, adapting it to the next study takes minutes rather than days, and you never have to remember which menu options you chose.

Good practice

Treat your R script as part of the study record, in the same way as the protocol and the data-management plan. Save it with the project, give it a clear name such as `02_clean_data.R`, and add short comments that explain *why* each step is done, not only *what* it does.

Who this book is for

This book is written for people who work in or around health care and who want to analyse data themselves:

- **clinicians and medical doctors** who conduct audits, quality-improvement projects or research alongside clinical work;
- **pharmacists** interested in medicines use, adherence and pharmacoepidemiology;
- **nurses and midwives** who collect and use patient data in practice and research;
- **public-health practitioners, epidemiologists and programme staff** who monitor services and evaluate interventions;
- **postgraduate and undergraduate students** in medicine, nursing, pharmacy, public health and allied disciplines who need to analyse data for a dissertation or thesis;
- **research coordinators and data managers** who prepare datasets and want to understand what happens to them next.

You do **not** need any programming background. We explain every piece of code the first time it appears, and we build up slowly from typing a calculation to fitting a multivariable model. Nor do you need advanced mathematics. Where a formula helps understanding, we show it and explain each term in words, but the emphasis is always on knowing *when* a method is appropriate, *how* to run it correctly and *what* the result means for patients.

What you do need is curiosity, your clinical reasoning and a willingness to learn from mistakes. Clinical knowledge is a genuine advantage here: you already know that a systolic blood pressure of 700 mmHg is impossible and that a 200-year-old patient is a data-entry error. Much of good data analysis is applying exactly that kind of judgement systematically.

Readers who already use another statistical package, such as SPSS or Stata, will find that the statistical content is familiar and can concentrate on the R implementation and on reproducible workflow. Readers new to statistics will find the key concepts explained from first principles, with references to standard medical statistics texts for those who want to go deeper (Altman 1991; Kirkwood and Sterne 2003; Bland 2015).

What you will be able to do

By the end of the book you will be able to:

1. navigate R and RStudio confidently, working in scripts within an RStudio Project;
2. create and use objects, vectors and data frames, and install and load packages;
3. import clinical data from CSV and Excel files and inspect them systematically;
4. recognise common data-quality problems and identify variable types (numeric, character, factor, date);
5. clean data reproducibly: handle missing-value codes, remove duplicates, harmonise inconsistent categories and validate impossible values;
6. derive new clinical variables, such as body mass index and blood-pressure categories, and set sensible reference levels for factors;

7. compute descriptive statistics correctly, including the proper handling of missing data;
8. build a manuscript-ready "Table 1" and publication-quality figures, and export them;
9. choose an appropriate hypothesis test for a given clinical question, run it, check its assumptions and interpret p-values and confidence intervals honestly;
10. fit univariable and multivariable logistic regression models, interpret odds ratios with 95% confidence intervals and check the model;
11. report your results in the style expected by clinical journals, with a complete script that regenerates every number.

These skills are cumulative. Each chapter takes the output of the previous one as its starting point, so that by the final chapter you have performed a complete analysis of a single study.

How this book is organised

The book has five chapters, an appendix of solutions and a bibliography. All chapters use the same case study, described later in this preface, so that the skills you learn build directly on one another.

Chapter 1: Getting started with R and RStudio. We install R and RStudio, tour the RStudio interface and learn why scripts and Projects are the foundation of reproducible work. We use R as a calculator, create objects and vectors, meet the main data types, and learn to install and load packages. The chapter ends by importing the raw case-study data from both a CSV file and an Excel workbook and taking a first, structured look at what they contain. By the end of Chapter 1 you will have the raw data in R and a sense that something is not quite right with it.

Chapter 2: Understanding and cleaning clinical data. Real clinical data are rarely clean, and analysis of dirty data produces confident but wrong answers. This chapter works through the problems deliberately planted in the raw file: missing-value codes such as 999 and -99, duplicate patient records, inconsistent spellings of categories, stray spaces, physiologically impossible measurements and dates recorded in several formats. We derive new variables, convert categorical variables to factors with clinically sensible reference levels, audit the remaining missing data and save a clean, analysis-ready file. Every later chapter starts from this file.

Chapter 3: Descriptive statistics, tables and figures. Before testing anything, we must describe the study population well. This chapter covers measures of central tendency and spread, frequency tables and cross-tabulations, and the common trap of missing values in summary functions. We build a manuscript-quality "Table 1" with the **gtsummary** package (Sjoberg et al. 2021) and produce histograms, bar charts, box plots and scatter plots with **ggplot2** (Wickham 2016), paying attention to the principles of clear graphical presentation.

Chapter 4: Common statistical tests in medical research. Here we turn from description to inference. We explain what a p-value is and is not (Wasserstein and Lazar 2016; Greenland et al. 2016), how to check distributional assumptions, and how to choose between parametric and non-parametric tests. We compare groups with t-tests, Wilcoxon tests and analysis of variance, examine associations between categorical variables with chi-square and Fisher's exact tests, and measure correlation. Each test is applied to a genuine question from the case study, with careful attention to effect sizes and confidence intervals rather than p-values alone.

Chapter 5: Regression modelling and reporting results. The final chapter brings everything together. We introduce logistic regression for a binary outcome, interpret odds ratios, and move

from crude to adjusted models while discussing confounding and the principles of model building (Hosmer et al. 2013; Heinze et al. 2018). We check the model for multicollinearity and assess its discrimination with the area under the ROC curve, then present the results in a publication-ready regression table and write a short Results section in clinical-manuscript style.

Appendix: Solutions to the exercises. Each chapter ends with a set of exercises, graded from straightforward to challenging. The appendix gives a worked solution for every exercise, with the R code, its output and a short interpretation. Use it to check your work after you have made a genuine attempt.

Bibliography. All the sources cited in the text are listed at the end of the book. They include standard textbooks of medical statistics, the original papers that introduced the methods we use and the references for the software packages. Each chapter also closes with a short "Further reading" section that points to the most useful of these sources and explains why.

How to read and use this book

The chapters are designed to be read in order, because each one builds on the data and skills produced by the last. If you already know some R, you may skim Chapter 1, but please do read Chapter 2: the clean data file produced there is used everywhere else, and the decisions taken in cleaning it affect every later result.

R code and output

R code appears in shaded blocks. The output that R prints when the code is run appears immediately below, with each line beginning with `#>`. For example:

```
# Mean of four systolic blood pressure readings (mmHg)
sbp <- c(120, 130, 140, 150)
mean(sbp)
```

R

```
#> [1] 135
```

The first line of the block, beginning with `#`, is a *comment*: R ignores everything after a `#` on a line, so comments are notes for human readers. The second line creates an object called `sbp` holding four values, and the third line calculates their mean. The line `#> [1] 135` is R's answer. The `#>` prefix is there so that you can tell code and output apart at a glance; when you run the code yourself, the output appears in the RStudio console without it. (The `[1]` simply tells you that the answer is the first element of the result; it will be explained in Chapter 1.)

All the output in this book was produced by actually running the code on the case-study data while the book was being prepared. If you run the same code on the same data, you should see the same results, apart from minor differences in formatting between versions of R and its packages.

Within the text, names of functions, variables, packages and files are written in a fixed-width font, for example `mean()`, `treatment_uptake` and `analysis_data.rds`. Function names are followed by brackets to distinguish them from other objects.

Callout boxes

Four kinds of highlighted box appear throughout the book. Each has a particular purpose, and recognising them will help you navigate the text.

A **Clinical interpretation** box explains what a statistical result means for patients or for clinical practice:

Clinical interpretation

An odds ratio of 2.0 for diabetes would mean that, among patients diagnosed with hypertension, the odds of being on treatment are twice as high in those who also have diabetes as in those who do not, after accounting for the other variables in the model. It does not mean that diabetes doubles the *probability* of treatment, and in a cross-sectional study it does not, on its own, show cause and effect.

A **Good practice** box offers a habit or technique that will make your work more reliable, efficient or readable:

Good practice

Never edit the raw data file by hand. Keep it exactly as you received it, and make every correction in your R script, so that each change is documented and can be reviewed or reversed.

A **Common mistake** box warns about an error that beginners, and sometimes experienced analysts, often make:

Common mistake

Running `mean(x)` on a variable that contains missing values returns `NA` rather than a number. Adding `na.rm = TRUE` removes the missing values before the calculation, but you should always report how many values were missing, rather than silently dropping them.

Finally, every chapter closes with a **Key points** box that summarises the essential messages:

Key points

- R scripts make an analysis transparent and reproducible.
- Every chapter uses the same case study, so skills build on one another.
- Type the code yourself, run it, and read the output carefully.

Exercises and solutions

Each chapter ends with five to eight exercises, all of which can be answered using the case-study data. They begin with short tasks that consolidate the main techniques and progress to more open questions that ask you to choose an approach and interpret the result clinically. Worked solutions are collected in the appendix. You will learn far more by attempting each exercise before looking at the solution, and by comparing your code with ours afterwards: there is usually more than one correct way to write it.

Type, do not copy

It is tempting to copy and paste code from the book into R. We encourage you instead to **type it yourself**. Typing slows you down just enough to notice each function name, each argument and each bracket. You will make small mistakes, see the resulting error messages and learn to fix them, and that is exactly the skill you will need when analysing your own data. After a few chapters, much of the syntax will have become second nature.

When you do meet an error, read the message carefully. It usually tells you which function failed and often why. Check for unmatched brackets or quotation marks, misspelt object names (R is case-sensitive, so `Age` and `age` are different), and missing commas between arguments. Then experiment: change one thing at a time and run the code again.

Software and set-up

Installing R and RStudio

You need two pieces of free software. **R** is the language and the engine that performs the calculations. **RStudio** is an integrated development environment: a comfortable workspace that sits on top of R, with a script editor, a console, a view of the objects in memory, and panes for files, plots, packages and help. You install R first, then RStudio.

1. Download and install R from the Comprehensive R Archive Network (CRAN) at <https://cran.r-project.org>. Choose the version for your operating system (Windows, macOS or Linux) and accept the default installation options.
2. Download and install RStudio Desktop (the free edition) from <https://posit.co/download/rstudio-desktop/>.
3. Open RStudio. It finds R automatically, and you will rarely need to open R on its own.

Both programs are updated regularly. Any recent version of R will work with this book; if you already have an older version installed, it is worth updating before you start, because some packages require a reasonably current version of R.

Packages used in this book

R's core installation provides a great deal of statistical functionality, but we also use a number of contributed packages:

- **tidyverse**: a collection of packages for data import, manipulation and visualisation, including **dplyr**, **tidyr**, **readr**, **forcats** and **stringr** (Wickham et al. 2019);
- **readxl**: imports data from Excel workbooks;
- **janitor**: tidies column names and helps to find duplicates and tabulate data;
- **lubridate**: parses and manipulates dates;
- **gtsummary**: produces publication-ready summary and regression tables (Sjoberg et al. 2021);
- **broom**: converts model output into tidy data frames;

- **ggplot2**: creates graphics based on the grammar of graphics (it is installed with the tidyverse) (Wickham 2016);
- **car**: provides regression diagnostics, including variance inflation factors;
- **pROC**: computes and plots ROC curves and the area under the curve.

A package needs to be *installed* only once on each computer, which downloads it from CRAN. The following code installs everything used in the book. It requires an internet connection and may take several minutes the first time:

```
# Run ONCE per computer: download and install the packages used in this book
install.packages(c(
  "tidyverse", "readxl", "janitor", "lubridate",
  "gtsummary", "broom", "ggplot2", "car", "pROC"
))
```

R

Once installed, a package must be *loaded* with `library()` in every new R session before its functions can be used, for example `library(tidyverse)`. Confusing these two steps is one of the most common sources of early frustration: you install once, but you load every time.

Common mistake

Do not leave `install.packages()` at the top of your analysis scripts. Reinstalling packages every time a script runs is slow, needs an internet connection and may silently change package versions in the middle of a project. Install once from the console; load with `library()` in the script.

Working in RStudio Projects

R always works relative to a folder called the *working directory*. If your code reads "Data/analysis_data.rds", R looks for a Data folder inside the working directory. The most reliable way to manage this is an **RStudio Project**: a folder containing a small `.Rproj` file that tells RStudio "this is the root of the analysis".

To create one, choose *File > New Project*, then either *New Directory* or *Existing Directory*, and select the folder that holds (or will hold) your data and scripts. Whenever you open the project, RStudio sets the working directory to that folder automatically, so every path in your code can be *relative* to it. Your analysis will then run unchanged on a colleague's computer, on a server or on your own laptop a year later.

A simple, effective folder structure for a clinical analysis project is:

- Data/ for raw and cleaned data files (with the raw files never edited);
- Scripts/ for R scripts, numbered in the order they should be run;
- Outputs/ for tables and figures produced by the scripts;
- a short README describing the project and how to run it.

Good practice

Avoid `setwd("C:/Users/yourname/Documents/...")` in scripts. A hard-coded absolute path works only on one computer. Inside an RStudio Project, use relative paths such as `"Data/hypertension_phc_raw.csv"`.

The software versions used in this book

R and its packages evolve over time, and occasionally a new version changes how a function behaves or how its output is printed. The code in this book was run with the versions shown below. If your output looks slightly different from ours, compare your versions with these.

```
# Version of R used to produce this book
R.version.string
```

R

```
#> [1] "R version 4.6.1 (2026-06-24 ucrt)"
```

```
# Versions of the main packages used in this book
pkgs <- c("tidyverse", "readxl", "janitor", "lubridate",
          "gtsummary", "broom", "ggplot2", "car", "pROC")
data.frame(
  package = pkgs,
  version = vapply(pkgs, function(p) as.character(packageVersion(p)),
                  character(1)),
  row.names = NULL
)
```

R

```
#>   package version
#> 1 tidyverse   2.0.0
#> 2  readxl    1.5.0
#> 3  janitor    2.2.1
#> 4 lubridate   1.9.5
#> 5 gtsummary   2.5.1
#> 6  broom     1.0.13
#> 7 ggplot2     4.0.3
#> 8    car      3.1.5
#> 9    pROC     1.19.0.1
```

You can run the same code on your own computer. Small differences in the final digit of a version number rarely matter; a different major version (the first number) occasionally does.

The case study

Throughout the book we analyse one study. Working with a single, coherent dataset from start to finish has a great advantage: you see how decisions made while cleaning the data affect the descriptive tables, how the descriptive tables guide the choice of tests, and how all of these feed into the final regression model and the written report. This mirrors how real clinical research is done.

Clinical background: hypertension and its treatment gap

Raised blood pressure is the leading modifiable risk factor for cardiovascular disease and premature death worldwide. It is a major cause of stroke, ischaemic heart disease, heart failure and chronic kidney disease. Its global burden has grown steadily, and the increase has been especially steep in low- and middle-income countries, where health systems are often organised around acute rather than chronic care (Mills et al. 2020).

A pooled analysis of population-based studies from around the world found that the number of adults aged 30 to 79 years living with hypertension roughly doubled between 1990 and 2019, to well over a billion people (NCD Risk Factor Collaboration (NCD-RisC) 2021). The same analysis documented a large and persistent *treatment gap*: a substantial proportion of people with hypertension were unaware of their condition, fewer than half were receiving treatment, and only a minority had their blood pressure controlled. The gaps were widest in parts of sub-Saharan Africa, South Asia and Oceania.

Effective, inexpensive medicines exist. The World Health Organization guideline on the pharmacological treatment of hypertension recommends starting drug treatment in adults with a confirmed diagnosis and a systolic blood pressure of 140 mmHg or more or a diastolic blood pressure of 90 mmHg or more, with treatment of people at high cardiovascular risk starting at lower thresholds (World Health Organization 2021). The problem in many settings is therefore not a lack of effective therapy but the failure of diagnosed patients to start and continue it. The reasons are many: cost and lack of health insurance, distance to the health facility, limited knowledge about hypertension, competing priorities and the absence of symptoms, as well as characteristics of the health services themselves.

Primary healthcare facilities are where most adults with hypertension are diagnosed and managed. Understanding which patients attending these facilities are, and are not, on treatment can help health services target education, outreach and financial protection to those who need it most. That is the question our case study addresses.

The study

Title. *Determinants of hypertension treatment uptake among adults attending primary healthcare facilities.*

Design. A multicentre cross-sectional study of 1,500 adults attending six primary healthcare facilities. For each participant, the study recorded socio-demographic characteristics, health behaviours, anthropometric measurements, blood pressure, laboratory biomarkers, knowledge of hypertension, access to care, hypertension diagnosis and treatment.

Research question. Among adults who have been diagnosed with hypertension, which factors are associated with being on antihypertensive treatment?

Primary outcome. `treatment_uptake`: whether the participant is currently taking antihypertensive medication (Yes/No, with "No" as the reference category). It is analysed only among participants who have been diagnosed with hypertension, that is, those with `htn_diagnosed == "Yes"`. This restriction is a matter of clinical logic as much as statistics: a person cannot take up treatment for a condition that has not been diagnosed.

Secondary outcomes. Two further outcomes are defined only among participants who are on treatment:

- *adherence*: treatment adherence, recorded as Good or Poor;
- *bp_controlled*: whether blood pressure is controlled, defined as a systolic pressure below 140 mmHg *and* a diastolic pressure below 90 mmHg.

Candidate determinants. The study recorded a range of factors that might plausibly influence whether a diagnosed patient is on treatment:

- *socio-demographic*: age, sex, place of residence (urban or rural), education, occupation, marital status and health insurance;
- *behavioural*: smoking, alcohol intake and physical activity;
- *clinical*: body mass index, family history of hypertension, diabetes, number of comorbidities, systolic and diastolic blood pressure and time since diagnosis;
- *knowledge and access*: a hypertension knowledge score (0 to 20) and distance to the health facility.

In addition, eight laboratory biomarkers (lipids, fasting glucose, creatinine, sodium and potassium) are available for descriptive analysis and exercises.

Common mistake

Because the design is cross-sectional, exposure and outcome are measured at the same time. An association between, say, health insurance and treatment uptake does not establish that insurance *causes* uptake. Throughout the book we speak of *associations* and *determinants* in this cautious sense, and we discuss confounding explicitly in Chapter 5.

The data are simulated

The dataset used in this book was **simulated for teaching**. It was generated by computer to resemble the kind of data collected in a real primary-care survey of hypertension, with realistic distributions, plausible relationships between variables and the sort of data-quality problems found in practice. It does not describe real patients, and the facility names are used purely for illustration.

This has two consequences. First, there are no confidentiality concerns, and you are free to explore, modify and share the data as you learn. Second, and more importantly, **the results in this book illustrate methods, not real clinical findings**. When we report that a particular factor is associated with treatment uptake, that statement is about the simulated data and should not be quoted as evidence about hypertension care anywhere. We shall remind you of this whenever results are interpreted.

The variables

The data dictionary describes each of the 36 variables recorded in the study. It is provided as a CSV file so that R can read it like any other dataset. The code below reads it and prints the main columns:

```
# Read the data dictionary that accompanies the case-study data
dictionary <- read.csv("Data/data_dictionary.csv")
```

R

```
# Keep the columns that describe each variable and show them as a table
knitr::kable(
  dictionary[, c("variable", "label", "type", "units_coding", "role")],
  col.names = c("Variable", "Label", "Type", "Units / coding", "Role"),
  caption = "Variables recorded in the hypertension treatment-uptake study."
)
```

Table 1: Variables recorded in the hypertension treatment-uptake study.

Variable	Label	Type	Units / coding	Role
patient_id	Patient identifier	Character	PHC-0001 ... PHC-1500	ID
facility	Healthcare facility	Categorical	6 PHC facilities	Cluster/covariate
enroll_date	Date of enrolment	Date	Mixed: YYYY-MM-DD, DD/MM/YYYY, DD-Mon-YYYY	Metadata
age	Age	Numeric	Years	Predictor
sex	Sex	Categorical	Female / Male	Predictor
residence	Place of residence	Categorical	Urban / Rural	Predictor
education	Highest education	Ordinal	None < Primary < Secondary < Tertiary	Predictor
occupation	Occupation	Categorical	Unemployed/Farmer/Trader/Professional/Other	Predictor
marital_status	Marital status	Categorical	Single/Married/Divorced/Widowed	Predictor
health_insurance	Has health insurance	Binary	Yes / No	Predictor
height_cm	Height	Numeric	centimetres	Derived input
weight_kg	Weight	Numeric	kilograms	Derived input
bmi	Body mass index	Numeric	kg/m ²	Predictor
smoking	Smoking status	Categorical	Never / Former / Current	Predictor
alcohol	Alcohol intake	Categorical	None / Moderate / Heavy	Predictor
physical_activity	Physical activity level	Ordinal	Low / Moderate / High	Predictor
family_history_htn	Family history of hypertension	Binary	Yes / No	Predictor
diabetes	Diabetes mellitus	Binary	Yes / No	Predictor
sbp_mmhg	Systolic blood pressure	Numeric	mmHg	Clinical
dbp_mmhg	Diastolic blood pressure	Numeric	mmHg	Clinical
total_chol_mmol_l	Total cholesterol	Numeric	mmol/L	Lab biomarker
hdl_mmol_l	HDL cholesterol	Numeric	mmol/L	Lab biomarker
ldl_mmol_l	LDL cholesterol	Numeric	mmol/L	Lab biomarker
triglycerides_mmol_l	Triglycerides	Numeric	mmol/L	Lab biomarker
fasting_glucose_mmol_l	Fasting glucose	Numeric	mmol/L	Lab biomarker
creatinine_umol_l	Serum creatinine	Numeric	umol/L	Lab biomarker

Variable	Label	Type	Units / coding	Role
sodium_mmol_l	Serum sodium	Numeric	mmol/L	Lab biomarker
potassium_mmol_l	Serum potassium	Numeric	mmol/L	Lab biomarker
knowledge_score	Hypertension knowledge score	Numeric	0-20	Predictor
distance_to_facility_km	Distance to facility	Numeric	kilometres	Predictor
comorbidity_count	Number of comorbidities	Count	0+	Predictor
htn_diagnosed	Diagnosed hypertensive	Binary	Yes / No	Filter
months_since_diagnosis	Months since HTN diagnosis	Numeric	Months	Predictor
treatment_uptake	On antihypertensive treatment	Binary	Yes / No	PRIMARY OUTCOME
adherence	Treatment adherence	Categorical	Good / Poor (blank if untreated)	Secondary outcome
bp_controlled	Blood pressure controlled	Binary	Yes / No (blank if untreated)	Secondary outcome

The **Role** column shows how each variable is used. `patient_id` identifies participants and `facility` records the cluster (the health facility) each participant attended. Most variables are candidate predictors. `htn_diagnosed` acts as a *filter* that defines the analysis population, `treatment_uptake` is the primary outcome, and `adherence` and `bp_controlled` are the secondary outcomes. The data dictionary file also contains a column of notes describing the known problems in the raw data, which we use in Chapter 2. Keep the dictionary close at hand: knowing exactly what each variable means and how it was coded is half of good analysis.

In the clean analysis file, two derived variables are added to these 36: `bmi_cat`, the World Health Organization body mass index category, and `bp_category`, a categorisation of measured blood pressure. Both are created in Chapter 2.

The study population at a glance

The clean analysis file, created in Chapter 2, contains one row per participant. The following code loads it and counts the participants, the facilities and the participants diagnosed with hypertension, who form the population for the primary analysis:

```
# Load the clean, analysis-ready data created in Chapter 2
analysis_data <- readRDS("Data/analysis_data.rds")
```

R

```
# Number of rows (participants) and columns (variables)
dim(analysis_data)
```

```
#> [1] 1500 38
```

```
# Key numbers describing the study population
diagnosed <- analysis_data[analysis_data$htn_diagnosed == "Yes", ]
study_size <- data.frame(
  quantity = c("Participants", "Facilities",
```

R

```

      "Diagnosed with hypertension",
      "Diagnosed and on treatment"),
  n = c(nrow(analysis_data),
        length(unique(analysis_data$facility)),
        nrow(diagnosed),
        sum(diagnosed$treatment_uptake == "Yes", na.rm = TRUE))
)
knitr::kable(study_size, col.names = c("Quantity", "n"),
              caption = "The case-study population.")

```

Table 2: The case-study population.

Quantity	n
Participants	1500
Facilities	6
Diagnosed with hypertension	1089
Diagnosed and on treatment	508

The data contain 1,500 participants and 38 variables (the 36 recorded variables plus the two derived categories) from six facilities. Of the participants, 1089 had been diagnosed with hypertension, and these form the denominator for the primary outcome. Among them, 508 (47%) were on antihypertensive treatment. In the chapters that follow we ask who these patients are, how they differ from those who are not on treatment, and which factors remain associated with treatment once others are taken into account.

Clinical interpretation

In these simulated data, fewer than half of the patients known to have hypertension were on treatment. In a real study, a finding like this would point to a substantial gap between diagnosis and care at the primary-care level, and would motivate the search for determinants that the rest of the book carries out.

The data files and the analysis workflow

The data files

The case-study data come in several files, each with a specific purpose. All are stored in a folder called `Data/`, and all the code in this book reads them with relative paths such as `"Data/analysis_data.rds"`.

- `hypertension_phc_raw.csv` and `hypertension_phc_raw.xlsx`: the **raw data**, as they might arrive from a data-collection team, in two common formats. The two files contain the same records. They have deliberately been given the problems that real clinical data so often have: 1,503 rows for 1,500 patients because of three duplicate records; categories spelled in several ways (for example F, f, female and Female); yes/no variables coded variously as Yes/No, Y/N and 1/0; missing values recorded as blanks, NA, 999 or -99; impossible values such as an age of 200 years or a systolic blood pressure of 700 mmHg; stray spaces before and after text; and dates entered in three different formats. These files are used in Chapters 1 and 2.
- `analysis_data.rds`: the **clean, analysis-ready data**, saved in R's own file format. Problems have been corrected, variables have their proper types, categorical variables are factors with

clinically sensible reference levels, and the derived variables have been added. The `.rds` format preserves all of this exactly, which a CSV file cannot. Chapters 3 to 5 start from this file.

- `data_dictionary.csv` and `data_dictionary.md`: the **data dictionary**, describing every variable, its coding, its role in the analysis and its known problems.

Chapter 2 shows you how to produce `analysis_data.rds` from the raw file yourself. A ready-made copy is provided so that you can work on any chapter independently, and so that you can compare your own cleaned data with ours.

Good practice

Keep raw data and analysis-ready data in separate files, and generate the second from the first with a script. If a problem is later found in the raw data, you fix the script and regenerate the clean file, rather than patching the clean file by hand.

The analysis workflow

The chapters follow the stages of a typical clinical data analysis. It is worth keeping this sequence in mind, because it applies to almost any study you will analyse:

1. **Import** the raw data into R and inspect them: how many rows and columns, what types of variable, what looks wrong (Chapter 1).
2. **Clean** the data: handle missing-value codes, remove duplicates, harmonise categories, validate impossible values, parse dates, derive new variables and set factor levels, then save a clean analysis file (Chapter 2).
3. **Describe** the study population with summary statistics, a "Table 1" and well-designed figures, overall and by outcome group (Chapter 3).
4. **Test** specific hypotheses about differences and associations, choosing each test to suit the type of data and checking its assumptions (Chapter 4).
5. **Model** the outcome with regression, moving from crude to adjusted estimates and checking the model's adequacy (Chapter 5).
6. **Report** the findings clearly and honestly, in tables, figures and text that meet the expectations of clinical journals and reporting guidelines such as STROBE for observational studies (Elm et al. 2007) (Chapters 3 and 5).

Each stage is written as code, so the whole pathway from raw file to final report can be re-run at any time. This, in a sentence, is what the book aims to teach.

About the authors

Bernard Isekah Osang'ir is a Senior Biostatistician at Neudata Consulting Ltd.

Vương Mỹ Lượng is a Senior Biostatistician at Neudata Consulting Ltd.

Both authors work on the design and analysis of clinical and public-health studies and on training health professionals to analyse their own data reproducibly in R.

Acknowledgements

This book grew out of teaching material developed for health professionals learning to analyse clinical data in R. We are grateful to the participants who worked through earlier versions of the material: their questions, their errors and their insights revealed where explanations needed to be clearer, where examples needed to be more clinical and where the pace needed to be slower. We also thank the colleagues and reviewers who read drafts of the chapters and whose careful comments improved both the statistics and the prose.

Finally, we thank the R community: the R Core Team, who develop and maintain R itself, and the many authors of the packages used in this book, who share their work freely so that others may build upon it. Any errors that remain are our own.

CHAPTER 1

Getting Started with R and RStudio

Every clinical study eventually becomes a table of numbers and words: one row per patient, one column per measurement. Blood pressures, ages, laboratory results, diagnoses and treatment decisions all end up in a spreadsheet or database, and someone has to turn that table into evidence. How that is done matters. An analysis performed by pointing and clicking leaves no record of what was done, cannot easily be checked by a colleague, and is hard to repeat when the data are corrected or a reviewer asks for a sensitivity analysis. An analysis written as code in R, by contrast, is a complete, readable and re-runnable record of every step from raw data to final table. This chapter takes you from opening R for the first time to importing a real (simulated) clinical dataset of 1,500 adults attending primary healthcare facilities and taking a first careful look at it. Along the way you will learn the small set of ideas on which everything else in this book is built: objects, data types, vectors, functions, packages, projects and data frames.

Learning objectives

- Describe what R and RStudio are, name the four RStudio panes and explain why analysis code belongs in a script rather than the console.
- Use R as a calculator, including operator precedence and built-in mathematical functions, and compute simple clinical quantities such as body mass index and mean arterial pressure.
- Create, name, overwrite and remove objects with the assignment arrow `<=`.
- Distinguish the main data types (numeric, integer, character, logical, factor and Date), check them with `class()` and convert between them, recognising common coercion pitfalls.
- Create and index vectors, perform vectorised arithmetic and logical comparisons, and summarise vectors that contain missing values.
- Call functions with positional and named arguments, find and read help pages, and interpret common error messages.
- Install and load packages, organise work in an RStudio Project and use relative file paths.
- Import a CSV and an Excel file into a data frame, inspect it with `dim()`, `names()`, `glimpse()`, `summary()` and `table()`, and identify data-quality problems.
- Use the pipe `|>` with the `dplyr` verbs `select()`, `filter()`, `arrange()`, `mutate()` and `count()`.
- Write a well-structured, commented and reproducible analysis script.

1.1 What R and RStudio are

1.1.1 R: a language for data analysis

R is a free, open-source programming language and software environment designed for statistical computing and graphics (R Core Team 2026). It was created in the 1990s as an open implementation of the S language developed at Bell Laboratories, and it is now maintained by the R Core Team and a very large community of contributors. Three features explain why R has become one of the standard tools of clinical and epidemiological research.

First, R is **free and open**. Anyone can download it, anyone can inspect the code that performs a calculation, and there are no licence fees standing between a district hospital, a university department or a ministry of health and a state-of-the-art statistical analysis. Second, R is **extensible**. Thousands of add-on *packages* contributed by statisticians and data scientists provide methods ranging from simple frequency tables to survival analysis, multiple imputation, meta-analysis and machine learning. When a new method is published, an R implementation often appears at the same time. Third, and most importantly for research, R is a **language**. You do not tell R what to do by clicking menus; you write instructions. Those instructions can be saved, read, shared, checked and run again, which is the foundation of reproducible research.

1.1.2 RStudio: a comfortable place to work with R

R itself is the engine that performs the computations. On its own it offers only a rather bare window in which you type commands. **RStudio** (developed by the company Posit) is an *integrated development environment*, or IDE: a program that sits on top of R and gives you a script editor, a console, a viewer for your data and plots, a file browser, a help browser and many conveniences such as auto-completion and keyboard shortcuts. A useful analogy is a car: R is the engine and RStudio is the dashboard, steering wheel and seats. You need both, you install R first, and once both are installed you only ever open RStudio, which starts R for you behind the scenes.

Installing the software is straightforward. R is downloaded from the Comprehensive R Archive Network (CRAN) at <https://cran.r-project.org>, choosing the installer for your operating system. RStudio Desktop (free) is downloaded from <https://posit.co/download/rstudio-desktop>. Accept the default options in both installers.

1.1.3 The four RStudio panes

When you open RStudio you see a window divided into (up to) four panes. Knowing what each pane is for removes most of the confusion of the first hour.

If you see only three panes, no script is open yet: choose *File > New File > R Script* (or press `Ctrl+Shift+N` on Windows, `Cmd+Shift+N` on macOS) and the source pane appears. Panes can be resized and rearranged under *Tools > Global Options > Pane Layout*, but the defaults are sensible and this book assumes them.

1.1.4 Console versus script

The **console** is a conversation with R. You type a command after the prompt `>`, press Enter, and R replies immediately. This is perfect for quick questions ("what is the mean of this column?", "what does this function do?"), but the conversation is ephemeral. When you close RStudio, or after a few hundred lines of output, there is no tidy record of what you asked and in what order.

Table 1.1: The four panes of RStudio in their default positions.

Pane	Default position	What it is for
Source (script editor)	Top left	Writing, editing and saving code. This is the permanent record of your analysis.
Console	Bottom left	Where R actually runs code and prints results. Typing here is possible but nothing is saved.
Environment / History	Top right	Lists the objects (datasets, values, results) currently held in memory, and the commands run so far.
Files / Plots / Packages / Help / Viewer	Bottom right	Browsing the project folder, viewing figures, managing packages and reading help pages.

A **script** is a plain text file, with the extension `.R`, that contains R commands written one after the other, together with comments explaining them. You write code in the script, place the cursor on a line and press `Ctrl+Enter` (Windows) or `Cmd+Enter` (macOS); RStudio sends that line to the console, R runs it, and the result appears in the console exactly as if you had typed it there. Highlighting several lines and pressing `Ctrl+Enter` runs all of them; `Ctrl+Shift+Enter` runs the entire script from top to bottom.

In this book R code is shown in grey boxes. Lines that R prints in response begin with `#>`, so that you can tell input from output. For example:

```
2 + 2
#> [1] 4
```

The `[1]` at the start of the output is an index: it says that the first value shown on this line is element number 1 of the result. It becomes useful when R prints long results spread over several lines.

1.1.5 Why scripts make analysis reproducible

An analysis is **reproducible** when another person (or you, six months later) can take the same data and the same code and obtain exactly the same results. Peng (2011) argues that reproducibility is the minimum standard by which computational science should be judged: even when a study cannot be independently *replicated* with new patients, readers should at least be able to verify that the reported numbers follow from the data. In clinical research this matters for very practical reasons. Datasets are corrected after queries are resolved; reviewers ask for an additional adjustment; a data safety monitoring board requests the same tables at the next interim look; a student inherits a supervisor's project. With a script, each of these requests means re-running the code. Without one, it means trying to remember what was clicked.

Wilson et al. (2017) describe a set of "good enough" computing practices that any researcher can adopt without becoming a software engineer. Several of them are introduced in this chapter: keep raw data unchanged and do all modifications in code, write code in scripts with comments, give files and objects meaningful names, organise each piece of work in its own project folder, and use relative rather than absolute file paths. None of these is difficult, and together they protect you from a large share of the errors that creep into manual analyses.

Good practice

Write every command that matters in your script, even if you test it first in the console. A useful rule: if a line produced a number that will appear in a report, it must exist in a saved script. The console is for exploring; the script is the analysis.

1.2 R as a calculator

The simplest way to start with R is to treat it as a very powerful calculator.

1.2.1 Arithmetic operators

R understands the usual arithmetic operators.

```
140 + 90      # addition
```

R

```
#> [1] 230
```

```
140 - 90      # subtraction
```

R

```
#> [1] 50
```

```
140 * 2       # multiplication
```

R

```
#> [1] 280
```

```
140 / 90      # division
```

R

```
#> [1] 1.556
```

```
2^10          # exponentiation (2 to the power 10)
```

R

```
#> [1] 1024
```

```
17 %/% 5      # integer division: how many whole 5s fit into 17
```

R

```
#> [1] 3
```

```
17 %% 5       # remainder (modulo): what is left over
```

R

```
#> [1] 2
```

Everything after a # on a line is a **comment**. R ignores it, but human readers do not, and comments are one of the most important parts of any script. Notice also that R prints 1.556 for `140 / 90`: by

default R shows about seven significant digits (this book uses four to keep output compact), but internally it stores the number to about fifteen or sixteen digits of precision.

1.2.2 Operator precedence

When an expression contains several operators, R follows the conventional mathematical order of operations: parentheses first, then exponentiation, then multiplication and division (from left to right), and finally addition and subtraction (from left to right).

Table 1.2: Order in which R evaluates arithmetic operators (highest first).

Priority	Operator	Meaning
1	()	Parentheses
2	[^]	Exponentiation
3	-x	Unary minus (a negative sign)
4	%%, %/%	Modulo, integer division
5	*, /	Multiplication, division
6	+, -	Addition, subtraction

The consequences are easy to see:

```
2 + 3 * 4      # multiplication first: 2 + 12
```

R

```
#> [1] 14
```

```
(2 + 3) * 4    # parentheses first: 5 * 4
```

R

```
#> [1] 20
```

```
-2^2          # exponent before the minus sign: -(2^2)
```

R

```
#> [1] -4
```

```
(-2)^2        # the square of minus two
```

R

```
#> [1] 4
```

When in doubt, add parentheses. They cost nothing, they make your intention explicit to the reader, and they prevent a whole class of silent errors. A formula that is wrong because of precedence does not produce an error message; it produces a plausible-looking but incorrect number, which is far more dangerous.

1.2.3 Mathematical functions

R contains a large library of built-in mathematical functions. A **function** is a named piece of code that takes one or more inputs (called *arguments*) inside parentheses and returns a result. We will

look at functions in detail in Section 1.6; for now, a few common ones are enough.

```
sqrt(16)          # square root
```

R

```
#> [1] 4
```

```
abs(-7)           # absolute value
```

R

```
#> [1] 7
```

```
exp(1)            # e, the base of natural logarithms
```

R

```
#> [1] 2.718
```

```
log(100)          # natural logarithm (base e)
```

R

```
#> [1] 4.605
```

```
log10(100)        # logarithm base 10
```

R

```
#> [1] 2
```

```
log(8, base = 2)  # logarithm with a chosen base
```

R

```
#> [1] 3
```

```
round(3.14159, 2) # round to 2 decimal places
```

R

```
#> [1] 3.14
```

```
signif(123456, 2) # round to 2 significant figures
```

R

```
#> [1] 120000
```

Note that `log()` in R is the **natural** logarithm, written $\ln$ in many textbooks. This surprises people who are used to calculators where "log" means base 10. Natural logarithms appear throughout biostatistics (for example in logistic regression in Chapter 5, where odds ratios are obtained by exponentiating coefficients with `exp()`), so it is worth remembering now.

1.2.4 A clinical example: body mass index and mean arterial pressure

Let us use R for two calculations that clinicians perform every day. Body mass index (BMI) is weight in kilograms divided by the square of height in metres:

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height (m)}^2}$$

For a patient who weighs 78.5 kg and is 166.2 cm tall, we must first convert height to metres by dividing by 100:

```
78.5 / (166.2 / 100)^2
```

R

```
#> [1] 28.42
```

The parentheses around `166.2 / 100` matter. Without them R would compute `100^2` first (exponentiation has the highest priority) and divide 166.2 by 10,000, giving a nonsensical BMI of over 470,000.

Mean arterial pressure (MAP) estimates the average pressure in the arteries over one cardiac cycle. Because the heart spends roughly two-thirds of each cycle in diastole, a common approximation weights the diastolic blood pressure (DBP) more heavily than the systolic (SBP):

$$\text{MAP} \approx \text{DBP} + \frac{\text{SBP} - \text{DBP}}{3}$$

For a reading of 152/95 mmHg:

```
95 + (152 - 95) / 3
```

R

```
#> [1] 114
```

```
round(95 + (152 - 95) / 3, 1)
```

R

```
#> [1] 114
```

The MAP is about 114 mmHg. Again, the parentheses around `152 - 95` are essential; `95 + 152 - 95 / 3` would subtract only a third of the diastolic pressure and give 215.3 mmHg, a value that is wrong but not obviously absurd to a tired eye.

Clinical interpretation

A MAP of about 114 mmHg is well above the range usually seen in normotensive adults (roughly 70 to 100 mmHg), consistent with the elevated reading of 152/95 mmHg. The point here is not the clinical decision but the computation: once a formula is written correctly in R, it can be applied identically to one patient or to all 1,500 patients in a study, with no transcription errors.

1.3 Objects and assignment

Calculating a value and letting it scroll off the screen is of limited use. To work with a value again we store it in an **object** (sometimes called a variable): a name that refers to a value held in the computer's memory.

1.3.1 The assignment arrow

Objects are created with the **assignment operator** `<-`, typed as a "less than" sign followed by a hyphen. It is read aloud as "gets": `sbp <- 152` reads "sbp gets 152". In RStudio the keyboard shortcut `Alt + -` (Windows) or `Option + -` (macOS) types the arrow with spaces around it.

```
sbp <- 152    # systolic blood pressure of one patient
dbp <- 95     # diastolic blood pressure of the same patient
```

Assignment is silent: R stores the values but prints nothing. To see what an object contains, type its name (which implicitly calls `print()`):

```
sbp
```

```
#> [1] 152
```

```
print(dbp)
```

```
#> [1] 95
```

Objects behave exactly like the values they contain, so they can be used in calculations, and the results can themselves be stored:

```
sbp + 10
```

```
#> [1] 162
```

```
map <- dbp + (sbp - dbp) / 3    # mean arterial pressure
map
```

```
#> [1] 114
```

```
pulse_pressure <- sbp - dbp
pulse_pressure
```

```
#> [1] 57
```

This is the first glimpse of why programming is powerful. The formula for MAP is now written once, in terms of named quantities, and anyone reading the code can see what it means.

Common mistake

R also accepts `=` for assignment at the top level (`sbp = 152` works), and `->` assigns to the right (`152 -> sbp`). The R community convention, followed throughout this book, is to use `<-` for assignment and to reserve `=` for naming arguments inside function calls, for example `round(x, digits = 1)`. Mixing the two makes code harder to read. A related slip is to type `<` and `-` with a space between them: `sbp < - 152` is not an assignment at all but the question "is sbp less than minus 152?", which returns `FALSE` and leaves `sbp` unchanged.

1.3.2 Naming objects

Object names must follow a few rules:

- They may contain letters, digits, full stops (.) and underscores (_).
- They must start with a letter (or a full stop not followed by a digit).
- They cannot contain spaces or other symbols such as -, / or %.
- They cannot be reserved words such as `if`, `else`, `TRUE`, `FALSE`, `NA`, `function`, `for` or `NULL`.

Within these rules you are free, and good names are worth thinking about. Names should say what the object holds, be short enough to type, and follow a consistent style. This book uses **snake case**: lower-case words joined by underscores, which is also the style of the case-study dataset (`sbp_mmhg`, `treatment_uptake`). Including the unit in the name of a measurement, as in `weight_kg` or `creatinine_umol_l`, prevents a common clinical error: mixing up units such as mmol/L and mg/dL.

Table 1.3: Examples of object names.

Name	Valid?	Comment
<code>sbp_mmhg</code>	Yes	Clear, includes the unit, snake case
<code>mean_age_diagnosed</code>	Yes	Descriptive; long but readable
<code>x2</code>	Yes	Valid but says nothing about its content
<code>SBP</code>	Yes	Valid, but a different object from <code>sbp</code>
<code>2nd_visit</code>	No	Starts with a digit; use <code>visit_2</code>
<code>systolic bp</code>	No	Contains a space
<code>bp-control</code>	No	R reads this as <code>bp</code> minus <code>control</code>

R is **case-sensitive**. The objects `sbp`, `SBP` and `Sbp` are three different objects, and asking for one that does not exist produces an error:

```
SBP
```

R

```
#> Error:
#> ! object 'SBP' not found
```

The message "object 'SBP' not found" is probably the most common error message in R. It almost always means a typing slip (a capital letter, a missing underscore) or that the line creating the object was never run.

1.3.3 Overwriting objects and the environment

Assigning a new value to an existing name **replaces** the old value without warning:

```
age <- 60
age
```

R

```
#> [1] 60
```

```
age <- 61      # the old value 60 is gone
age
```

R

```
#> [1] 61
```

```
age <- age + 1 # use the old value to compute the new one
age
```

R

```
#> [1] 62
```

The line `age <- age + 1` looks odd to anyone with a mathematics background, but it is perfectly natural in R: the right-hand side is computed first using the current value of `age`, and the result is then stored back under the same name.

All objects you create live in the **global environment**, which is what the Environment pane in RStudio displays. You can also list them in code, and remove ones you no longer need:

```
ls()      # list the objects in the environment
```

R

```
#> [1] "age"      "dbp"      "map"      "pulse_pressure"
#> [5] "sbp"
```

```
rm(pulse_pressure) # remove one object
ls()
```

R

```
#> [1] "age" "dbp" "map" "sbp"
```

Good practice

Do not rely on the contents of your environment. Objects created by lines you have since deleted from your script may still be lurking in memory, and your script may appear to work only because of them. Restart R regularly (*Session > Restart R*, or `Ctrl+Shift+F10`) and run your script from the top. In *Tools > Global Options > General*, untick "Restore .RData into workspace at startup" and set "Save workspace to .RData on exit" to "Never", so that every session starts clean (Wickham et al. 2023).

1.4 Data types

Every value in R has a **type**, and the type determines what R can do with it. You can add two numbers but not two names; you can sort dates chronologically but sorting them as text gives the wrong order. Most of the data-cleaning work in Chapter 2 is, at heart, the work of making sure every variable has the right type.

1.4.1 The basic types

Numeric values are real numbers stored in "double precision". They are the default for anything that looks like a number. **Integers** are whole numbers; you rarely need to create them explicitly

Table 1.4: The main data types used in clinical data analysis.

Type (class)	Example	Typical clinical use
numeric (double)	152, 23.4, -0.5	Blood pressure, BMI, laboratory values
integer	3L, 0L	Counts such as number of comorbidities
character	"Female", "PHC-0142"	Identifiers, free text, raw category labels
logical	TRUE, FALSE	Yes/no conditions such as "SBP at least 140"
factor	Female, Male (with levels)	Categorical variables: sex, facility, education
Date	2024-03-15	Enrolment date, date of diagnosis

(the suffix L, as in 3L, does so), and for analysis purposes integers and doubles behave almost identically. **Character** values, also called *strings*, are text, and must always be written inside quotation marks, single or double. **Logical** values are TRUE and FALSE (always in capitals) and are produced by comparisons. **Factors** are R's way of storing categorical data: a set of values restricted to a fixed list of categories, called *levels*. **Dates** store calendar dates in a form that allows arithmetic (how many days between enrolment and follow-up?) and correct chronological ordering.

1.4.2 Checking the type: `class()` and the `is.*()` functions

The function `class()` reports the type of any object.

```
class(152)
```

R

```
#> [1] "numeric"
```

```
class(3L)
```

R

```
#> [1] "integer"
```

```
class("Female")
```

R

```
#> [1] "character"
```

```
class(TRUE)
```

R

```
#> [1] "logical"
```

```
class(sbp > 140)
```

R

```
#> [1] "logical"
```

A family of functions beginning with `is.` asks a yes/no question about type:

```
is.numeric(152)
```

R

```
#> [1] TRUE
```

```
is.character(152)
```

R

```
#> [1] FALSE
```

```
is.character("152")
```

R

```
#> [1] TRUE
```

```
is.logical(FALSE)
```

R

```
#> [1] TRUE
```

The last pair of examples deserves attention. 152 and "152" look alike on the screen, but the first is a number and the second is a piece of text that happens to consist of digits. You cannot do arithmetic with the second:

```
"152" + 10
```

R

```
#> Error in `"152" + 10`:  
#> ! non-numeric argument to binary operator
```

"Non-numeric argument to binary operator" means that one side of the + is not a number. When you meet this message on real data, the usual cause is a column that *should* be numeric but was imported as character because a few cells contained text, such as "missing", "<0.5" or "n/a".

1.4.3 Converting types: the `as.*()` functions

Each `is.*` function has an `as.*` partner that attempts to convert a value to the corresponding type. This is called **coercion**.

```
as.numeric("152")      # text to number
```

R

```
#> [1] 152
```

```
as.character(152)      # number to text
```

R

```
#> [1] "152"
```

```
as.numeric(TRUE)      # TRUE becomes 1
```

R

```
#> [1] 1
```

```
as.numeric(FALSE)      # FALSE becomes 0
```

R

```
#> [1] 0
```

```
as.logical("TRUE")     # text to logical
```

R

```
#> [1] TRUE
```

```
as.integer(3.9)        # truncates towards zero; it does NOT round
```

R

```
#> [1] 3
```

1.4.4 Coercion pitfalls

Conversion is not always possible. When R cannot convert a value it does not stop with an error; it produces NA (R's code for a missing value) and issues a warning:

```
as.numeric(c("120", "135", "missing", "<90"))
```

R

```
#> [1] 120 135 NA NA
```

(The warning "NAs introduced by coercion" is suppressed in this book's output but will appear on your screen.) This behaviour is convenient but dangerous: a careless conversion can quietly turn genuine information, such as "below the detection limit", into missing data. Always count missing values before and after a conversion, and investigate any that appear.

A second pitfall arises when values of different types are combined. A vector (see Section 1.5) can hold only one type, so R silently converts everything to the most flexible type present, following the hierarchy logical < integer < numeric < character:

```
c(152, 138, "145")     # one text value turns all values into text
```

R

```
#> [1] "152" "138" "145"
```

```
c(TRUE, FALSE, 3)     # logicals become 0/1 numbers
```

R

```
#> [1] 1 0 3
```

A single stray character entry in a column of blood pressures is therefore enough to turn the whole column into text. This is exactly what has happened to several columns of the raw case-study data, as we will see in Section 1.10.

The third pitfall is the conversion of factors to numbers. A factor is stored internally as integer codes (1, 2, 3, ...) with labels attached, and `as.numeric()` returns the codes, not the labels:

```
dose <- factor(c("10", "5", "20", "5"))
dose
```

```
#> [1] 10 5 20 5
#> Levels: 10 20 5
```

```
as.numeric(dose) # the internal codes: wrong!
```

```
#> [1] 1 3 2 3
```

```
as.numeric(as.character(dose)) # convert to text first: right
```

```
#> [1] 10 5 20 5
```

1.4.5 Factors

Categorical clinical variables, such as sex, facility, education level or smoking status, are best stored as factors. A factor knows its permitted categories (its levels) and their order, which matters for tables, plots and, crucially, for regression models, where the first level becomes the *reference category* (Chapter 5).

```
smoking <- factor(c("Never", "Current", "Former", "Never", "Never"),
                  levels = c("Never", "Former", "Current"))
smoking
```

```
#> [1] Never Current Former Never Never
#> Levels: Never Former Current
```

```
levels(smoking)
```

```
#> [1] "Never" "Former" "Current"
```

```
table(smoking)
```

```
#> smoking
#>   Never Former Current
#>      3      1      1
```

Without the `levels` argument, R would have ordered the levels alphabetically (Current, Former, Never), which is rarely the order a clinician wants to read. A value that is not among the levels becomes NA, which is a useful safeguard against misspellings: `factor("Curent", levels = c("Never", "Former", "Current"))` gives NA rather than creating a fourth, spurious category.

1.4.6 Dates

Dates are stored as the number of days since 1 January 1970, which allows arithmetic, but they print in the familiar year-month-day format. The function `as.Date()` converts text in the international standard format (ISO 8601, YYYY-MM-DD) into a date:

```
enrol <- as.Date("2024-03-15")
followup <- as.Date("2024-09-11")
class(enrol)
```

```
#> [1] "Date"
```

```
followup - enrol # a difference in days
```

```
#> Time difference of 180 days
```

```
as.numeric(followup - enrol) / 7 # in weeks
```

```
#> [1] 25.71
```

```
format(enrol, "%d %B %Y") # print in a friendlier format
```

```
#> [1] "15 March 2024"
```

Dates entered in other formats, such as 15/03/2024 or 15-Mar-2024, need to be told their format, and datasets in which several formats are mixed are common. The **lubridate** package (Grolemund and Wickham 2011) makes parsing such dates much easier, and we use it in Chapter 2.

Clinical interpretation

Types are not a technicality. If treatment uptake is stored as the text "1" and "0" in some rows and "Yes" and "No" in others, a frequency table will show four categories instead of two and every proportion will be wrong. If enrolment dates are stored as text, "01/02/2024" sorts before "2024-01-12" regardless of which came first. Getting types right is the first step of a valid analysis.

1.5 Vectors

So far each object has held a single value. Clinical data, of course, consist of many values: the systolic pressures of all patients in a clinic, the sexes of all participants. R's basic structure for a collection of values is the **vector**: an ordered sequence of values, all of the same type. In fact a single number such as 152 is simply a vector of length one, which is why R prints `[1]` in front of it.

1.5.1 Creating vectors

The function `c()`, for "combine", creates a vector from its arguments.

```
sbp_readings <- c(152, 138, 145, 160, 129, 142) # numeric
sex <- c("Female", "Male", "Female", "Female", "Male", "Female") # character
on_treatment <- c(TRUE, FALSE, TRUE, TRUE, FALSE, FALSE) # logical
sbp_readings
```

```
#> [1] 152 138 145 160 129 142
```

```
length(sbp_readings)
```

```
#> [1] 6
```

These six values could represent the systolic blood pressures of six patients seen during one morning clinic, with their sexes and treatment status in matching positions.

1.5.2 Regular sequences: `seq()`, `:` and `rep()`

Sequences of numbers are needed often, for example to define age bands or follow-up times. The colon operator creates integer sequences, `seq()` creates sequences with any step, and `rep()` repeats values.

```
1:10 # integers from 1 to 10
```

```
#> [1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(from = 18, to = 88, by = 10) # age band boundaries
```

```
#> [1] 18 28 38 48 58 68 78 88
```

```
seq(0, 1, length.out = 5) # five equally spaced values
```

```
#> [1] 0.00 0.25 0.50 0.75 1.00
```

```
rep("Control", times = 3) # repeat a value
```

```
#> [1] "Control" "Control" "Control"
```

```
rep(c("A", "B"), times = 3) # repeat a pattern
```

```
#> [1] "A" "B" "A" "B" "A" "B"
```

```
rep(c("A", "B"), each = 3) # repeat each element
```

```
#> [1] "A" "A" "A" "B" "B" "B"
```

The last two lines show the difference between the `times` and each arguments, which is useful when constructing, for example, a treatment allocation list in a block design.

1.5.3 Indexing: extracting elements

Individual elements of a vector are extracted with square brackets `[ ]`. Positions in R start at 1 (not 0, as in some other languages).

```
sbp_readings[1]           # the first reading
```

R

```
#> [1] 152
```

```
sbp_readings[c(1, 3)]     # the first and third
```

R

```
#> [1] 152 145
```

```
sbp_readings[2:4]         # the second to the fourth
```

R

```
#> [1] 138 145 160
```

```
sbp_readings[-1]          # everything except the first
```

R

```
#> [1] 138 145 160 129 142
```

```
sbp_readings[length(sbp_readings)] # the last
```

R

```
#> [1] 142
```

A negative index removes elements rather than selecting them. Indexing beyond the end of a vector returns NA rather than an error, which is another situation where R quietly gives you a missing value.

The most useful form of indexing uses a **logical vector**: R keeps the elements in the positions where the index is TRUE.

```
sbp_readings[on_treatment] # SBP of patients on treatment
```

R

```
#> [1] 152 145 160
```

```
sbp_readings[sex == "Male"] # SBP of male patients
```

R

```
#> [1] 138 129
```

This is the idea that underlies all filtering of data, from selecting the hypertensive patients in a study to excluding implausible values.

1.5.4 Vectorised arithmetic

Arithmetic in R is **vectorised**: an operation applied to a vector is applied to every element at once, without writing a loop.

```
sbp_readings - 10           # subtract 10 from every reading
```

R

```
#> [1] 142 128 135 150 119 132
```

```
sbp_readings / 7.5         # convert mmHg to kPa (1 kPa = 7.5 mmHg)
```

R

```
#> [1] 20.27 18.40 19.33 21.33 17.20 18.93
```

```
dbp_readings <- c(95, 88, 90, 101, 79, 85)
map_readings <- dbp_readings + (sbp_readings - dbp_readings) / 3
round(map_readings, 1)
```

R

```
#> [1] 114.0 104.7 108.3 120.7 95.7 104.0
```

When two vectors of the same length are combined, R pairs them element by element: the first SBP with the first DBP, the second with the second, and so on. This is precisely what we want when each position represents one patient. (If the lengths differ, R *recycles* the shorter vector, with a warning if the lengths are not multiples of each other. Recycling is occasionally useful but is more often a sign of a mistake.)

1.5.5 Logical comparisons

Comparison operators produce logical vectors.

Table 1.5: Comparison and logical operators in R.

Operator	Meaning	Example
==	equal to	sex == "Female"
!=	not equal to	sex != "Female"
<, <=	less than (or equal)	sbp < 140
>, >=	greater than (or equal)	dbp >= 90
&	and (both true)	sbp >= 140 & dbp >= 90
!	not	!on_treatment
%in%	is in a set	sex %in% c("F", "f")

The table omits one operator because its symbol clashes with the table layout: the vertical bar | means *or*, so `sbp >= 140 | dbp >= 90` is TRUE when at least one of the two conditions is true. Note also the double equals sign == for comparison; a single = is an assignment or an argument name, never a test of equality.

```
high_sbp <- sbp_readings >= 140
high_sbp
```

R

```
#> [1] TRUE FALSE TRUE TRUE FALSE TRUE
```

```
high_bp <- sbp_readings >= 140 | dbp_readings >= 90 # either is raised
high_bp
```

```
#> [1] TRUE FALSE TRUE TRUE FALSE TRUE
```

Because TRUE is treated as 1 and FALSE as 0 in arithmetic, `sum()` of a logical vector counts the TRUE values and `mean()` gives their proportion. This little trick is used constantly in data analysis.

```
sum(high_bp) # how many patients have a raised reading
```

```
#> [1] 4
```

```
mean(high_bp) # the proportion
```

```
#> [1] 0.6667
```

Four of the six patients (67%) have a systolic pressure of at least 140 mmHg or a diastolic pressure of at least 90 mmHg, the conventional thresholds for raised office blood pressure (World Health Organization 2021). In this small example the patients with a raised diastolic pressure happen also to have a raised systolic pressure, so `high_bp` and `high_sbp` coincide; in a larger sample they would not.

1.5.6 Summary functions

R has many functions that reduce a vector to a single summary value. We meet them properly, with their statistical meaning, in Chapter 3; here is a preview.

```
mean(sbp_readings)
```

```
#> [1] 144.3
```

```
median(sbp_readings)
```

```
#> [1] 143.5
```

```
sd(sbp_readings) # standard deviation
```

```
#> [1] 10.82
```

```
min(sbp_readings)
```

```
#> [1] 129
```

```
max(sbp_readings)
```

R

```
#> [1] 160
```

```
range(sbp_readings)
```

R

```
#> [1] 129 160
```

```
summary(sbp_readings) # six-number summary
```

R

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>      129    139    144    144    150    160
```

The output of `summary()` gives, from left to right, the minimum, the first quartile (25% of values lie below it), the median, the mean, the third quartile and the maximum. For these six readings the mean (144.3) and median (143.5) are close, suggesting that the values are fairly symmetric around their centre.

For a character vector, the natural summary is a frequency table:

```
table(sex)
```

R

```
#> sex
#> Female  Male
#>      4      2
```

1.5.7 Missing values: NA

Real data have gaps. A blood sample haemolysed, a patient declined to be weighed, a form was left blank. R represents a missing value by the special symbol `NA` ("not available"), which can appear in a vector of any type. Missing values are contagious: any calculation involving an unknown value is itself unknown.

```
weights <- c(61.3, 68.3, NA, 78.5, 71.6)
weights + 1
```

R

```
#> [1] 62.3 69.3  NA 79.5 72.6
```

```
mean(weights)
```

R

```
#> [1] NA
```

The mean of five numbers of which one is unknown cannot be known, so R answers `NA`. This is the right default, because it forces you to notice the missing value. To compute the mean of the *observed* values, set the argument `na.rm = TRUE` ("NA remove"):

```
mean(weights, na.rm = TRUE)
```

R

```
#> [1] 69.92
```

```
sum(is.na(weights)) # how many values are missing?
```

R

```
#> [1] 1
```

The function `is.na()` returns `TRUE` for each missing element, so `sum(is.na(x))` counts missing values. You cannot test for missingness with `x == NA`, because comparing anything to an unknown value gives an unknown result: `NA == NA` is `NA`.

Common mistake

Do not reach for `na.rm = TRUE` automatically. It is easy to compute a mean "of all patients" that is actually the mean of the 60% who had the test. Always report how many values were missing alongside any summary, and think about why they are missing: values missing for a reason related to the outcome can bias results (Little and Rubin 2019). Missing data are discussed further in Chapters 2 and 3.

1.6 Functions, arguments and help

1.6.1 The anatomy of a function call

You have already used many functions: `sqrt()`, `round()`, `mean()`, `c()`, `seq()`. Every function call has the same shape: the function's name, followed by parentheses containing its **arguments**, separated by commas. Arguments are the inputs that tell the function what to work on and how.

```
round(144.33333, digits = 1)
```

R

```
#> [1] 144.3
```

Here `round` is the function, `144.33333` is the first argument (the number to round) and `digits = 1` is a second argument that controls the behaviour. The function returns a value, which R prints, or which you can store in an object.

1.6.2 Positional and named arguments

Arguments can be supplied in two ways. **Positional** arguments are matched by their order in the function's definition; **named** arguments are matched by name, in any order. The definition of `seq()` begins `seq(from, to, by, ...)`, so these calls are equivalent:

```
seq(18, 88, 10) # positional
```

R

```
#> [1] 18 28 38 48 58 68 78 88
```

```
seq(from = 18, to = 88, by = 10) # named
```

R

```
#> [1] 18 28 38 48 58 68 78 88
```

```
seq(by = 10, to = 88, from = 18) # named, any order
```

R

```
#> [1] 18 28 38 48 58 68 78 88
```

Many arguments have **default values**, used when you do not supply them. The default for `digits` in `round()` is 0, and the default for `na.rm` in `mean()` is `FALSE`, which is why `mean()` returned `NA` above.

Good practice

A good habit is to give the first one or two arguments by position (they are usually the data, and their meaning is obvious) and all others by name, as in `mean(weights, na.rm = TRUE)` or `round(map, digits = 1)`. Named arguments make the code self-documenting and protect you if a function's argument order ever changes.

Functions can be **nested**, with the output of one becoming the input of another. R evaluates them from the inside out:

```
round(mean(weights, na.rm = TRUE), 1)
```

R

```
#> [1] 69.9
```

Here `mean()` is computed first and its result (69.925) is passed to `round()`. Deep nesting quickly becomes hard to read, which is the problem the pipe operator solves (Section 1.11).

1.6.3 Getting help

No one remembers every function and argument. Knowing how to look things up is a core skill. Every function in R has a help page, which you open with `?` or `help()`:

```
?mean           # help page for mean()
help("read_csv") # the same, as a function call
??"standard deviation" # search all help pages for a phrase
example(mean)    # run the examples at the bottom of the help page
vignette("dplyr") # a longer tutorial distributed with a package
```

R

Help pages appear in the Help pane and always have the same structure: *Description* (what the function does), *Usage* (the arguments and their defaults), *Arguments* (what each argument means), *Value* (what is returned), *Details*, *See Also* and, most useful of all, *Examples* at the bottom, which you can copy into the console and run. Help pages are written concisely and can feel terse at first; reading the *Usage* and *Examples* sections first is a good strategy. Beyond R's own help, the free online book *R for Data Science* (Wickham et al. 2023) and package websites are excellent references. When searching the internet, include "R" and the package name along with the error message or task.

1.6.4 Reading error messages

Errors are a normal part of programming, not a sign of failure. R stops and prints a message starting with `Error`; the message describes what went wrong, and learning to read it saves hours. Warnings, starting with `warning`, mean that R did complete the command but something may be amiss; they must never be ignored. A few common errors and their usual causes:

Table 1.6: Common R error messages and their usual causes.

Message	Usual cause
object 'x' not found	Typing error, wrong case, or the line creating x was not run
could not find function "x"	Misspelled function, or its package has not been loaded with <code>library()</code>
non-numeric argument to binary operator	Arithmetic on text, often a number stored as character
unexpected symbol / unexpected ') '	A syntax error: missing comma, bracket or quotation mark
'file.csv' does not exist in current working directory	Wrong file path or wrong working directory
there is no package called 'x'	The package has not been installed

Here is one of them in action. We try to use a function from a package that has not been loaded in the current session:

```
clean_names(data.frame(Patient.ID = 1))
```

R

```
#> Error in `clean_names()`:  
#> ! could not find function "clean_names"
```

The fix is to load the package that provides `clean_names()` (here, **janitor**) or to call the function with its package prefix, as in `janitor::clean_names()`.

A frequent source of confusion is an *incomplete* command. If you forget a closing parenthesis or quotation mark, the console prompt changes from `>` to `+`, meaning that R is waiting for you to finish the command. Either complete it or press `Esc` to cancel and start again.

Common mistake

Forgetting the quotation marks around text is a classic slip. `sex == Female` asks R to compare `sex` with an *object* called `Female`, which does not exist ("object 'Female' not found"). Text values always need quotes: `sex == "Female"`. Conversely, object and column names do not take quotes in most functions: `mean(sbp)`, not `mean("sbp")`.

1.7 Packages

1.7.1 Installing once, loading every session

The functions that come with R ("base R") cover a great deal, but much of R's power comes from **packages**: collections of functions, data and documentation written by the community and distributed through CRAN. In 2026 CRAN hosts more than twenty thousand packages. Using a package involves two distinct steps:

1. **Install** it, which downloads it from the internet and stores it on your computer. You do this *once* per computer (and again only when you upgrade R or want a newer version of the package).
2. **Load** it with `library()`, which makes its functions available in the current R session. You do this *every time* you start R, at the top of each script.

A useful analogy: installing a package is like buying a book and putting it on your shelf; loading it is taking it off the shelf to read. You buy the book once, but you take it down whenever you need it.

```
install.packages("tidyverse") # once per computer: downloads from CRAN
install.packages("readxl")
```

```
library(tidyverse) # every session: data import, wrangling, ggplot2
library(readxl)    # every session: read Excel files
```

When the **tidyverse** is loaded it prints a short message listing the packages it has attached and a few "conflicts", functions in different packages that share a name. For instance, both base R's **stats** package and **dplyr** have a function called `filter()`; after `library(tidyverse)`, the name `filter` refers to the **dplyr** version. If you ever need to be explicit, the double-colon operator `package::function()` calls a function from a specific package, with or without loading it, as in `dplyr::filter()` or `stats::filter()`.

Common mistake

Do not put `install.packages()` in an analysis script that runs every time. It is slow, requires an internet connection, may silently update a package to a version that behaves differently, and fails on computers where you do not have permission to install software. Install packages once from the console (or with a commented-out line, as above); load them with `library()` at the top of every script.

1.7.2 The tidyverse

The **tidyverse** is a collection of packages that share a common design philosophy, grammar and data structures (Wickham et al. 2019). Loading it with `library(tidyverse)` attaches the core members at once:

- **readr** for reading rectangular data files such as CSV;
- **dplyr** for manipulating data frames (selecting, filtering, summarising);
- **tidyr** for reshaping data;
- **ggplot2** for graphics (Wickham 2016);
- **stringr** for working with text, **forcats** for factors and **lubridate** for dates;
- **tibble**, a modern version of the data frame, and **purrr** for repeated operations.

The tidyverse is built around the idea of **tidy data**, in which each variable is a column, each observation is a row and each value is a cell (Wickham 2014). A patient-level clinical dataset with one row per patient is naturally tidy, which is one reason the tidyverse suits clinical research so well. This book uses tidyverse functions for most data handling and plotting, and base R where it

is simpler. Other packages, such as **readxl** for Excel files, **janitor** for cleaning, **gtsummary** for publication-ready tables (Sjoberg et al. 2021) and **broom** for tidy model output, are introduced as they are needed.

1.8 Projects, working directories and file paths

1.8.1 The working directory

When you ask R to read a file, it must know where to look. The **working directory** is the folder that R treats as its current location; a file name without a full path is looked for there.

```
getwd() # print the current working directory
```

R

On the computer used to produce this book, `getwd()` returns a path ending in `.../Course`, the folder that contains the case-study `Data/` directory. On your computer it will be different.

1.8.2 Absolute and relative paths

An **absolute path** gives a file's full location from the root of the disk, for example `C:/Users/amina/Documents/htn_study/Data/hypertension_phc_raw.csv`. A **relative path** gives the location relative to the working directory, for example `Data/hypertension_phc_raw.csv`. Absolute paths work only on the computer where they were written, and only until a folder is renamed; relative paths keep working when the whole project folder is copied to a colleague's laptop, a server or a USB stick.

Note that R uses forward slashes `/` in paths, even on Windows. If you copy a path from Windows Explorer, which uses backslashes, either replace them with `/` or double them (`\\`), because a single backslash has a special meaning inside R strings.

1.8.3 RStudio Projects

The reliable way to manage working directories is an **RStudio Project**. A project is simply a folder containing a small file with the extension `.Rproj`. When you open the project (by double-clicking the `.Rproj` file, or through *File > Open Project*), RStudio starts a fresh R session whose working directory is the project folder. Every relative path in your scripts then resolves correctly, whatever computer the project is on.

To create one, choose *File > New Project*, then either *New Directory* (to start from scratch) or *Existing Directory* (to turn a folder you already have into a project). A simple layout that works well for most clinical analyses, in the spirit of Wilson et al. (2017), is:

```
htn_study/  
  htn_study.Rproj  
  Data/      # raw data (never edited by hand) and cleaned data  
  Scripts/   # R scripts, numbered in the order they run  
  Outputs/   # tables and figures produced by the scripts  
  Reports/   # manuscripts and reports
```

R

Common mistake

Avoid starting scripts with `setwd("C:/Users/yourname/Desktop/analysis")`. The line works on exactly one computer, and the script fails with "cannot change working directory" for anyone else, including you on a new laptop. Use an RStudio Project and relative paths instead. Similarly, avoid saving the data in a different folder from the scripts that analyse them; keep everything belonging to one study in one project.

1.9 The case study and importing data

1.9.1 The case study

All examples in this book use a single dataset, so that you can follow one analysis from raw data to final model. It comes from a multicentre cross-sectional study, *Determinants of hypertension treatment uptake among adults attending primary healthcare facilities*, in which 1,500 adults attending six primary healthcare (PHC) facilities were enrolled. The research question is: among adults already diagnosed with hypertension, which factors determine whether they are currently taking antihypertensive treatment? Hypertension affects well over a billion adults worldwide, and a large share of those diagnosed are not treated (NCD Risk Factor Collaboration (NCD-RisC) 2021; Mills et al. 2020), so understanding the barriers to treatment uptake is a genuine public health priority.

The dataset records demographic characteristics (age, sex, residence, education, occupation, marital status, health insurance), behaviours (smoking, alcohol, physical activity), clinical measurements (height, weight, BMI, blood pressure, comorbidities, family history, diabetes), laboratory biomarkers (lipids, glucose, creatinine, electrolytes), a hypertension knowledge score, distance to the facility, and the outcomes: whether the patient has diagnosed hypertension (`htn_diagnosed`), whether they are on treatment (`treatment_uptake`, the primary outcome), their adherence and whether their blood pressure is controlled. The full definitions are in the data dictionary, `Data/data_dictionary.md`.

The data are **simulated for teaching**. They were generated to resemble real data from primary care, including realistic relationships between variables and, deliberately, the kinds of errors found in real data-collection files. Results obtained from them illustrate methods; they are not clinical findings.

1.9.2 Data frames and tibbles

A vector holds one variable. A study needs many variables measured on the same patients, and R stores these in a **data frame**: a rectangular table in which each column is a vector (all of one type) and all columns have the same length. Each row is one observation, here one patient. Different columns may have different types: a character identifier, a numeric age, a factor for sex. A small data frame can be built by hand:

```
clinic <- data.frame(
  patient_id = c("P01", "P02", "P03", "P04"),
  age        = c(54, 61, 47, 70),
  sex        = c("Female", "Male", "Female", "Male"),
  sbp_mmhg   = c(152, 138, 145, 160)
)
```

R

```
clinic
```

```
#>   patient_id age    sex sbp_mmhg
#> 1      P01   54 Female      152
#> 2      P02   61  Male      138
#> 3      P03   47 Female      145
#> 4      P04   70  Male      160
```

A **tibble** is the tidyverse's modern version of the data frame. It behaves in the same way in almost every respect, but it prints more helpfully (showing the type of each column and only as many rows and columns as fit on the screen) and is stricter in a few situations where data frames silently do something surprising. The tidyverse import functions return tibbles; in this book "data frame" refers to both.

```
as_tibble(clinic)
```

R

```
#> # A tibble: 4 × 4
#>   patient_id age sex    sbp_mmhg
#>   <chr>      <dbl> <chr>    <dbl>
#> 1 P01         54 Female      152
#> 2 P02         61  Male      138
#> 3 P03         47 Female      145
#> 4 P04         70  Male      160
```

The row `<chr> <dbl> <chr> <dbl>` beneath the column names gives each column's type: `chr` for character and `dbl` for double (numeric).

A single column is extracted from a data frame as a vector with the dollar sign `$`, and a column can be added in the same way:

```
clinic$sbp_mmhg
```

R

```
#> [1] 152 138 145 160
```

```
mean(clinic$age)
```

R

```
#> [1] 58
```

```
clinic$high_sbp <- clinic$sbp_mmhg >= 140
clinic
```

R

```
#>   patient_id age    sex sbp_mmhg high_sbp
#> 1      P01   54 Female      152     TRUE
#> 2      P02   61  Male      138     FALSE
#> 3      P03   47 Female      145     TRUE
#> 4      P04   70  Male      160     TRUE
```

1.9.3 Importing a CSV file with `read_csv()`

Clinical data most often arrive as a **CSV** (comma-separated values) file: a plain text file in which each line is a row and values are separated by commas. CSV files can be opened by any software and are the most portable format for sharing data. The **readr** package (part of the tidyverse) provides `read_csv()`:

```
htn <- read_csv("Data/hypertension_phc_raw.csv")
```

Nothing is printed here because this book's settings switch off readr's column report. On your screen, `read_csv()` prints a message such as Rows: 1503 Columns: 36 followed by a summary of the column types it has guessed. readr looks at the first 1,000 rows of each column and chooses the most specific type that fits all of them: logical, then numeric, then date, falling back to character if nothing else fits. By default it treats empty cells and the text NA as missing values, and it removes leading and trailing spaces from values (`trim_ws = TRUE`).

The first thing to note is that the raw file has **1,503 rows**, not the 1,500 patients who were enrolled. Something is already wrong: as the data dictionary warns, three patients have been entered twice. We will remove these duplicates in Chapter 2.

1.9.4 Importing an Excel file with `read_excel()`

Many clinical datasets are collected or shared as Excel workbooks. The **readxl** package (Wickham et al. 2023) reads `.xls` and `.xlsx` files without needing Excel to be installed. A workbook can contain several sheets, so it is good practice to check the sheet names and to name the sheet you want:

```
excel_sheets("Data/hypertension_phc_raw.xlsx")
```

```
#> [1] "data"
```

```
htn_xl <- read_excel("Data/hypertension_phc_raw.xlsx", sheet = "data")
dim(htn)
```

```
#> [1] 1503 36
```

```
dim(htn_xl)
```

```
#> [1] 1503 36
```

```
identical(names(htn), names(htn_xl))
```

```
#> [1] TRUE
```

Both files contain the same 1,503 rows and 36 columns with identical column names. The `identical()` function returns a single `TRUE` only if its two arguments are exactly the same, a convenient way of checking that two versions of a dataset agree. Other formats are just as easy: the **haven** package reads SPSS (`read_sav()`), Stata (`read_dta()`) and SAS (`read_sas()`) files,

which are common in clinical research.

Good practice

Treat the raw data file as read-only. Never edit it by hand in Excel to "fix" an error, because the correction then leaves no trace. Instead, make every change in your R script, so the path from raw data to analysis dataset is fully documented and can be reviewed. If data arrive in a spreadsheet, the recommendations of Broman and Woo (2018) (one variable per column, one value per cell, no colour-coding as data, consistent dates and codes) make them much easier to analyse.

1.10 A first inspection of the data

Before computing any statistic, look at the data. A few minutes of inspection reveals the structure of the dataset, the types R has assigned, and very often the first data problems. The functions in this section form a routine you should run on every new dataset.

1.10.1 Size and names

```
dim(htn)      # number of rows and columns
```

R

```
#> [1] 1503  36
```

```
nrow(htn)
```

R

```
#> [1] 1503
```

```
ncol(htn)
```

R

```
#> [1] 36
```

```
names(htn)
```

R

```
#> [1] "patient_id"      "facility"
#> [3] "enroll_date"     "age"
#> [5] "sex"             "residence"
#> [7] "education"       "occupation"
#> [9] "marital_status"  "health_insurance"
#> [11] "height_cm"       "weight_kg"
#> [13] "bmi"             "smoking"
#> [15] "alcohol"         "physical_activity"
#> [17] "family_history_htn" "diabetes"
#> [19] "sbp_mmhg"        "dbp_mmhg"
#> [21] "total_chol_mmol_l" "hdl_mmol_l"
#> [23] "ldl_mmol_l"      "triglycerides_mmol_l"
#> [25] "fasting_glucose_mmol_l" "creatinine_umol_l"
#> [27] "sodium_mmol_l"    "potassium_mmol_l"
#> [29] "knowledge_score"  "distance_to_facility_km"
```

```
#> [31] "comorbidity_count"      "htn_diagnosed"
#> [33] "months_since_diagnosis" "treatment_uptake"
#> [35] "adherence"              "bp_controlled"
```

`dim()` returns the number of rows and columns, in that order. `names()` lists the 36 variable names. The names are already in a consistent snake-case style and contain units where relevant, which makes them easy to work with.

1.10.2 Printing and looking at the first rows

Typing the name of a tibble prints the first ten rows and as many columns as fit:

htn

R

```
#> # A tibble: 1,503 × 36
#>   patient_id facility enroll_date   age sex  residence education occupation
#>   <chr>      <chr>    <chr>      <dbl> <chr> <chr>      <chr>      <chr>
#> 1 PHC-1224  Igoma HC 01/02/2024    74 Fema... Urban      Primary  Trader
#> 2 PHC-1169  Kisesa ... 2024-09-03    56 Fema... Urban      Primary
Professio...
#> 3 PHC-1391  Bugando... 2024-11-26    54 Fema... Urban      None      Farmer
#> 4 PHC-0142  Ilemela... 2024-10-23    33 Fema... Urban      Primary  Farmer
#> 5 PHC-0112  Kisesa ... 2024-06-03    86 Male  Urban      Primary  Unemployed
#> 6 PHC-0102  Igoma HC 2024-04-22    70 Male  Urban      Primary  Trader
#> 7 PHC-1019  Nyamaga... 08/08/2024    45 Fema... Urban      None      Unemployed
#> 8 PHC-1431  Bugando... 18/02/2024    59 f      Urban      Primary  Professio...
#> 9 PHC-0119  Bugando... 2024-01-12    46 Male  Urban      Primary  Trader
#> 10 PHC-0397 Bugando... 2024-10-19    61 Fema... Rural      None      Trader
#> # i 1,493 more rows
#> # i 28 more variables: marital_status <chr>, health_insurance <chr>,
#> #   height_cm <dbl>, weight_kg <dbl>, bmi <dbl>, smoking <chr>,
#> #   alcohol <chr>, physical_activity <chr>, family_history_htn <chr>,
#> #   diabetes <chr>, sbp_mmhg <dbl>, dbp_mmhg <dbl>, total_chol_mmol_l <dbl>,
#> #   hdl_mmol_l <dbl>, ldl_mmol_l <dbl>, triglycerides_mmol_l <dbl>,
#> #   fasting_glucose_mmol_l <dbl>, creatinine_umol_l <dbl>, ...
```

The header says the tibble has 1,503 rows and 36 columns. Beneath each column name is its type, and the footer lists the columns that did not fit on the screen. `head()` shows a chosen number of rows; to see a few specific columns we can combine it with `select()`, which we meet properly in Section 1.11:

```
head(select(htn, patient_id, facility, age, sex, sbp_mmhg, dbp_mmhg), 5)
```

R

```
#> # A tibble: 5 × 6
#>   patient_id facility   age sex  sbp_mmhg dbp_mmhg
#>   <chr>      <chr>      <dbl> <chr>      <dbl>      <dbl>
#> 1 PHC-1224  Igoma HC      74 Female    140      92
#> 2 PHC-1169  Kisesa HC      56 Female    185      91
#> 3 PHC-1391  Bugando PHC    54 Female    147      85
#> 4 PHC-0142  Ilemela HC     33 Female    124     102
#> 5 PHC-0112  Kisesa HC      86 Male     131      95
```

In RStudio, `view(htn)` opens the data in a spreadsheet-like viewer where you can scroll, sort and

filter (without changing the data). It is useful for browsing but, being interactive, leaves no record, so do not rely on it for anything you would need to repeat.

1.10.3 The structure: `glimpse()` and `str()`

`glimpse()` (from `dplyr`) is the most informative single overview. It lists every column on its own line, with its type and the first few values:

```
glimpse(htn)
```

R

```
#> Rows: 1,503
#> Columns: 36
#> $ patient_id      <chr> "PHC-1224", "PHC-1169", "PHC-1391", "PHC-01...
#> $ facility        <chr> "Igoma HC", "Kisesa HC", "Bugando PHC", "Il...
#> $ enroll_date     <chr> "01/02/2024", "2024-09-03", "2024-11-26", "...
#> $ age             <dbl> 74, 56, 54, 33, 86, 70, 45, 59, 46, 61, 70,...
#> $ sex             <chr> "Female", "Female", "Female", "Female", "Ma...
#> $ residence       <chr> "Urban", "Urban", "Urban", "Urban", "Urban"...
#> $ education       <chr> "Primary", "Primary", "None", "Primary", "P...
#> $ occupation      <chr> "Trader", "Professional", "Farmer", "Farmer...
#> $ marital_status  <chr> "Married", "Married", "Single", "Single", "...
#> $ health_insurance <chr> "Yes", "No", "No", "Yes", "0", "No", "No", ...
#> $ height_cm       <dbl> 164.6, 160.0, 152.8, 166.2, 182.0, 175.6, 1...
#> $ weight_kg       <dbl> 61.3, 68.3, 49.9, 78.5, 71.6, 97.4, 106.7, ...
#> $ bmi             <dbl> 22.6, 26.7, 21.4, 28.4, 21.6, 31.6, NA, 25....
#> $ smoking         <chr> "Former", "Current", "Never", "Never", "For...
#> $ alcohol         <chr> "None", "Moderate", "Moderate", "None", "He...
#> $ physical_activity <chr> "Low", "Low", "Moderate", "Low", "High", "M...
#> $ family_history_htn <chr> "Yes", "1", "1", "1", "0", "No", "No", "No"...
#> $ diabetes        <chr> "No", "No", "No", "No", "0", "No", "0", "No...
#> $ sbp_mmhg        <dbl> 140, 185, 147, 124, 131, 153, 162, 108, 152...
#> $ dbp_mmhg        <dbl> 92, 91, 85, 102, 95, 95, 100, 100, 87, 69, ...
#> $ total_chol_mmol_l <dbl> -99.0, 5.4, 5.4, 5.9, 4.8, 3.6, 5.3, 2.9, 6...
#> $ hdl_mmol_l      <dbl> 1.93, 0.70, 1.67, 0.65, 1.69, 1.20, 1.83, 1...
#> $ ldl_mmol_l      <dbl> 3.5, NA, 2.3, 4.1, 2.4, 1.4, 2.2, NA, 3.4, ...
#> $ triglycerides_mmol_l <dbl> 1.0, 2.1, 2.7, 1.7, 2.0, 1.3, 1.0, 1.5, 2.5...
#> $ fasting_glucose_mmol_l <dbl> 6.6, 5.5, 6.0, 5.2, 5.1, 5.8, 5.5, 4.1, 6.2...
#> $ creatinine_umol_l <dbl> 61, 66, 75, 86, 87, 72, 103, 57, 98, 76, 99...
#> $ sodium_mmol_l   <dbl> 134, 134, 139, 140, 142, 138, 141, 141, 140...
#> $ potassium_mmol_l <dbl> 4.1, 3.6, 4.4, 4.4, 4.2, 4.0, 4.5, 5.0, 4.7...
#> $ knowledge_score <dbl> 14, 7, 9, 12, NA, 12, 17, 8, 13, NA, 11, 6,...
#> $ distance_to_facility_km <dbl> 2.9, 10.5, 2.4, 2.2, 5.4, 6.7, 0.2, 0.4, 5....
#> $ comorbidity_count <dbl> 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0...
#> $ htn_diagnosed    <chr> "1", "Y", "Yes", "Yes", "Yes", "Yes", "Yes"...
#> $ months_since_diagnosis <dbl> 95, 87, 116, 31, 80, 28, 6, 96, 114, 53, 39...
#> $ treatment_uptake <chr> "Yes", "No", "N", "Yes", "No", "No", "1", "...
#> $ adherence       <chr> "Poor", NA, NA, "Good", NA, NA, "Good", NA,...
#> $ bp_controlled    <chr> "No", NA, NA, "No", NA, NA, "No", NA, NA, "...

```

Read this output carefully, column by column, with the data dictionary beside you. It tells a story:

- `patient_id`, `facility`, `sex`, `residence` and the other demographic variables are character (`<chr>`), as expected for text.
- `enroll_date` is character, **not** a date. The first values, "01/02/2024" and "2024-09-03", use different formats, so `readr` could not recognise a single date format and kept the text.

- The measurements (age, height_cm, weight_kg, bmi, sbp_mmhg, the laboratory values) are numeric (<dbl>), which is right.
- total_chol_mmol_l begins with the value -99.0. A negative cholesterol is impossible: this is a **missing-value code** (a "sentinel") used during data entry, which R has taken as a real number.
- health_insurance, family_history_htn, diabetes, htn_diagnosed and treatment_uptake should be Yes/No variables, but we see values such as "1", "0" and "Y" mixed with "Yes" and "No". Because of the mixture, they are stored as character.
- adherence and bp_controlled contain NA for patients not on treatment, for whom these outcomes are not defined.

The base R function `str()` ("structure") gives similar information in a slightly different layout. Applied to a whole tibble it is verbose, so here we show it for a few columns:

```
str(select(htn, age, sex, sbp_mmhg, treatment_uptake))
```

```
#> tibble [1,503 × 4] (S3: tbl_df/tbl/data.frame)
#> $ age          : num [1:1503] 74 56 54 33 86 70 45 59 46 61 ...
#> $ sex          : chr [1:1503] "Female" "Female" "Female" "Female" ...
#> $ sbp_mmhg     : num [1:1503] 140 185 147 124 131 153 162 108 152 127 ...
#> $ treatment_uptake: chr [1:1503] "Yes" "No" "N" "Yes" ...
```

1.10.4 Summaries of numeric variables

`summary()` applied to a numeric vector gives the six-number summary, plus the number of missing values if there are any. It is the quickest way to spot impossible values.

```
summary(htn$age)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>      0.0   43.0   52.0   52.3   62.0   200.0
```

The median age is 52 years and the middle half of patients are between 43 and 62, which is plausible for adults attending primary care. But the minimum is 0 and the maximum is 200. Neither is possible for an adult participant: these are data-entry errors. Applying `summary()` to several columns at once shows more problems:

```
summary(select(htn, sbp_mmhg, dbp_mmhg, weight_kg, height_cm))
```

```
#>      sbp_mmhg      dbp_mmhg      weight_kg      height_cm
#> Min.   : 0      Min.   : 5.0      Min.   : 7.0      Min.   : 17
#> 1st Qu.:126    1st Qu.: 79.0    1st Qu.: 60.9    1st Qu.:158
#> Median :139    Median : 86.0    Median : 70.1    Median :164
#> Mean   :140    Mean   : 85.9    Mean   : 70.9    Mean   :164
#> 3rd Qu.:153    3rd Qu.: 93.0    3rd Qu.: 80.3    3rd Qu.:169
#> Max.   :700    Max.   :121.0    Max.   :132.8    Max.   :193
#>                                     NAs      :45
```

Systolic pressure ranges from 0 to 700 mmHg, diastolic pressure goes down to 5 mmHg, weight has a minimum of 7 kg and height a minimum of 17 cm (probably 170 cm with a misplaced decimal

point). `weight_kg` also has 45 missing values (NA's). None of these values would survive a clinician's glance at a case-report form, yet they would silently distort means, standard deviations and regression coefficients if analysed as they are. The sentinel codes are visible in the laboratory variables too:

```
summary(htn$total_chol_mmol_l)
```

R

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#> -99.00   4.30   5.10   2.59   5.80   8.30    55
```

```
summary(htn$fasting_glucose_mmol_l)
```

R

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#>      3.0    4.8    5.4   32.5    6.1   999.0    35
```

Total cholesterol has a minimum of -99 and fasting glucose a maximum of 999 mmol/L: codes for "missing", not measurements.

1.10.5 Frequency tables of categorical variables

For character and factor variables, `table()` counts each distinct value.

```
table(htn$sex)
```

R

```
#>
#>      f      F female Female      m      M      male      Male
#>    74    73     68    672    40    39     42    495
```

There should be two categories; there are eight. The same two sexes have been typed as Female, female, F and f, and as Male, male, M and m. Because R is case-sensitive and compares text exactly, it treats each spelling as a separate category. A table of sex by treatment uptake, or a regression adjusted for sex, would be meaningless until these are standardised.

```
table(htn$treatment_uptake, useNA = "ifany")
```

R

```
#>
#>    0    1    N    No    Y    Yes
#> 134   79   95  766   52  377
```

The primary outcome is coded in six different ways: Yes, Y and 1 for treated, and No, N and 0 for untreated. The argument `useNA = "ifany"` asks `table()` to show a count of missing values if there are any; by default `table()` silently omits them, which is a trap we return to in Chapter 3. Here there are none, so no NA column appears.

```
table(htn$facility)
```

R

```
#>
#> Bugando PHC  Buzuruga PHC      Igoma HC      Ilemela HC      Kisesa HC
#>      337      175      175      283      228
#> Nyamagana PHC
```

```
#> 305
```

The six facilities appear cleanly, and Bugando PHC contributes the most patients. The data dictionary, however, warns that some facility names contain stray leading or trailing spaces. We do not see them because `read_csv()` trims whitespace by default. Reading the file again with `trim_ws = FALSE` reveals the problem:

```
htn_untrimmed <- read_csv("Data/hypertension_phc_raw.csv", trim_ws = FALSE)
table(htn_untrimmed$facility)
```

```
#>
#> Bugando PHC Buzuruga PHC Igoma HC Ilemela HC
#> 25 16 11 16
#> Kisesa HC Nyamagana PHC Bugando PHC Buzuruga PHC
#> 16 24 312 159
#> Igoma HC Ilemela HC Kisesa HC Nyamagana PHC
#> 164 267 212 281
```

```
# how many values differ from their trimmed version?
sum(htn_untrimmed$facility != str_trim(htn_untrimmed$facility))
```

```
#> [1] 108
```

Now every facility appears twice. The first six counts belong to versions of the names with a space at the start and end (such as " Igoma HC "), which R sorts before the letters; the last six are the clean versions. In total 108 values carry invisible spaces. Without trimming, any table by facility would show twelve facilities instead of six. Other software, and some R import functions, do not trim automatically, so it is reassuring to know what readr is doing on your behalf, and important to check rather than assume.

1.10.6 A first figure

Numbers in a summary are easier to grasp when plotted. A histogram of age shows the whole distribution at once. We use **ggplot2**, which is introduced fully in Chapter 3; for now, read the code as "take the data `htn`, put age on the x-axis, and draw a histogram".

```
ggplot(htn, aes(x = age)) +
  geom_histogram(binwidth = 5, fill = "#1F6F8B", colour = "white") +
  labs(title = "Age of participants, raw data",
       x = "Age (years)", y = "Number of patients")
```

The bulk of the distribution is a single hump between 18 and 95 years, centred in the early fifties and roughly symmetric. The tiny isolated bars at 0 and at 200 are the impossible values identified by `summary()`; each is a single patient, which is why they barely rise above the axis. A histogram makes such outliers impossible to miss, which is why plotting the data is an essential step of any data-quality check.

A bar chart does the same job for a categorical variable. Here we display the number of patients enrolled at each facility, ordered from largest to smallest:

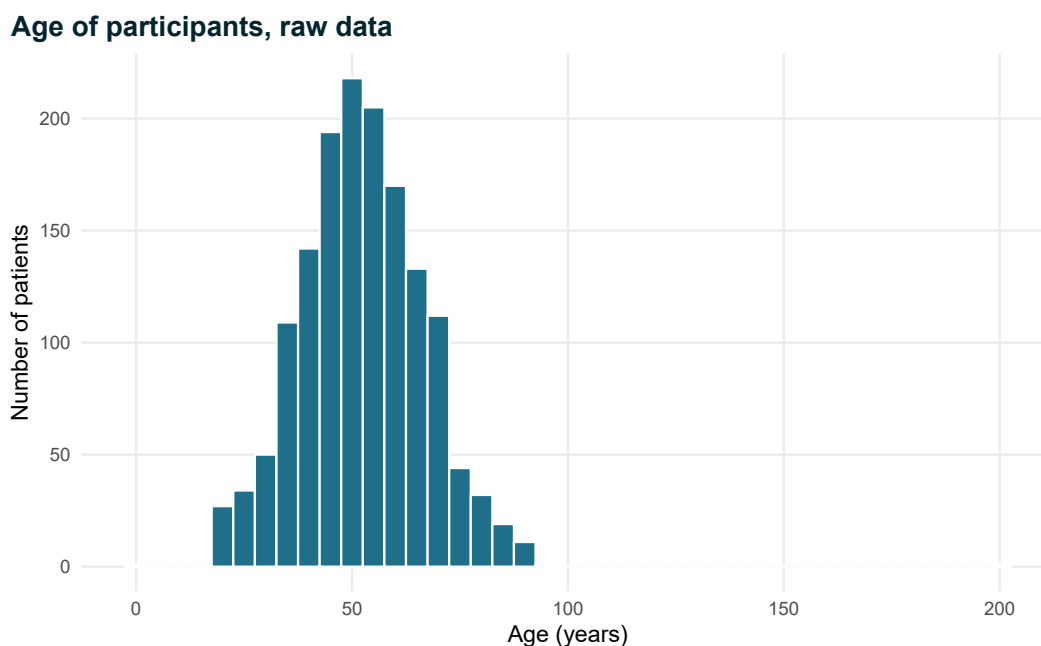


Figure 1.1: Histogram of recorded age in the raw case-study data (n = 1,503). Almost all patients are between 18 and 95 years old, but two impossible values, at 0 and at 200 years, stand out on their own.

```
htn |>
  count(facility) |>
  ggplot(aes(x = n, y = fct_reorder(facility, n))) +
  geom_col(fill = "#1F6F8B") +
  geom_text(aes(label = n, hjust = -0.2, size = 3.5) +
    labs(title = "Records per facility",
      x = "Number of records", y = NULL) +
    expand_limits(x = 380)
```

The facilities differ in size by a factor of almost two, from 175 records at Buzuruga PHC and Igoma HC to 337 at Bugando PHC. In a multicentre study this is worth noting: larger facilities will dominate pooled estimates, and patients attending the same facility may resemble one another more than patients from different facilities.

1.10.7 Taking stock: problems already visible

Without computing a single statistic, the first inspection has revealed most of the data-quality issues documented in the data dictionary. The table below collects them. Deciding how to fix each one is the subject of Chapter 2.

```
problems <- tibble(
  Problem = c("Duplicate records", "Implausible values",
    "Missing-value codes", "Inconsistent spellings",
    "Mixed binary codes", "Mixed date formats",
    "Stray whitespace"),
  Evidence = c("1,503 rows for 1,500 patients",
    "age 0 and 200; SBP 0 and 700; weight 7 kg",
    "cholesterol -99; glucose 999",
    "sex: Female/female/F/f, Male/male/M/m",
    "treatment_uptake: Yes/Y/1, No/N/0",
```

Records per facility

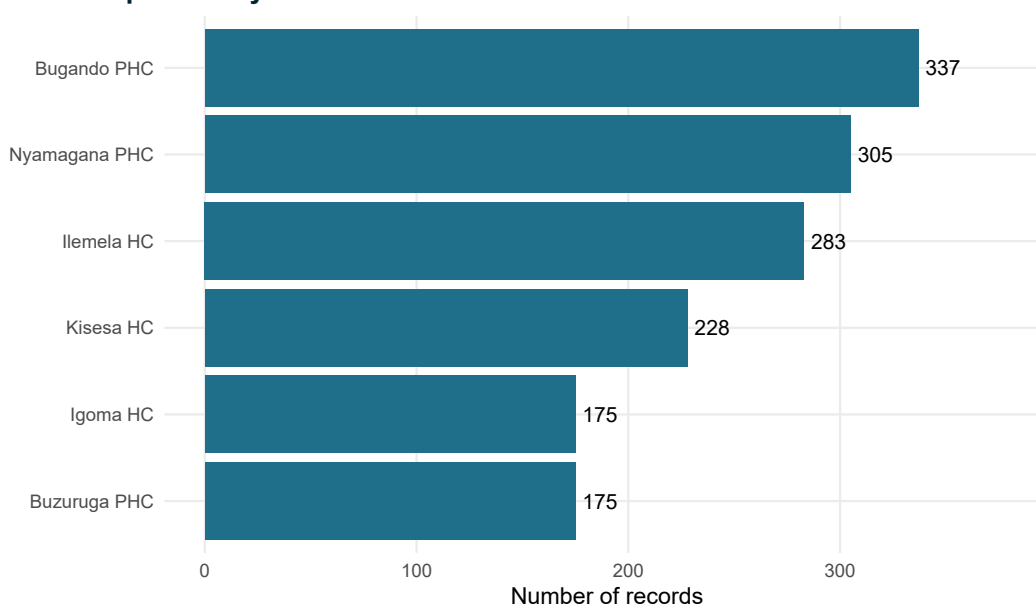


Figure 1.2: Number of records per primary healthcare facility in the raw case-study data. Bugando PHC and Nyamagana PHC contributed the most participants; Buzuruga PHC and Igoma HC the fewest.

```

    "enroll_date: 01/02/2024 and 2024-09-03",
    "facility names with leading/trailing spaces")
)
knitr::kable(problems,
  caption = "Data problems found by a first inspection.")

```

Table 1.7: Data problems found by a first inspection.

Problem	Evidence
Duplicate records	1,503 rows for 1,500 patients
Implausible values	age 0 and 200; SBP 0 and 700; weight 7 kg
Missing-value codes	cholesterol -99; glucose 999
Inconsistent spellings	sex: Female/female/F/f, Male/male/M/m
Mixed binary codes	treatment_uptake: Yes/Y/1, No/N/0
Mixed date formats	enroll_date: 01/02/2024 and 2024-09-03
Stray whitespace	facility names with leading/trailing spaces

Clinical interpretation

Every one of these problems would change a clinical conclusion if left in place. An SBP of 700 mmHg inflates the mean blood pressure; a cholesterol of -99 mmol/L pulls the mean cholesterol down; three duplicated patients are counted twice; and eight spellings of sex mean that "the proportion of women on treatment" cannot even be computed. A first inspection is therefore not optional housekeeping: it is part of the scientific method, and its findings should be documented in the analysis script.

1.11 The pipe and a first taste of dplyr

1.11.1 The pipe operator |>

Data analysis is a sequence of steps: take the data, keep certain rows, keep certain columns, compute something, sort the result. Written as nested function calls, the steps must be read from the inside out, which is the reverse of the order in which they happen:

```
head(arrange(select(htn, patient_id, age), desc(age)), 3)
```

```
#> # A tibble: 3 × 2
#>   patient_id age
#>   <chr>      <dbl>
#> 1 PHC-0174    200
#> 2 PHC-0144     95
#> 3 PHC-1062     92
```

The **pipe** operator |> (built into R since version 4.1) passes the result on its left as the first argument of the function on its right. The same computation becomes a readable recipe, read from top to bottom, with |> pronounced "and then":

```
htn |>
  select(patient_id, age) |>
  arrange(desc(age)) |>
  head(3)
```

```
#> # A tibble: 3 × 2
#>   patient_id age
#>   <chr>      <dbl>
#> 1 PHC-0174    200
#> 2 PHC-0144     95
#> 3 PHC-1062     92
```

"Take htn, and then select the patient identifier and age, and then arrange by age in descending order, and then show the first three rows." The result identifies the single patient recorded as 200 years old (PHC-0174), followed by two genuinely old but plausible patients aged 95 and 92: this is how you would find exactly which records need to be queried with the study site. In RStudio the shortcut **Ctrl+Shift+M** (**Cmd+Shift+M** on macOS) types the pipe. You will also see the older pipe %>% from the **magrittr** package in many books and online answers; for everyday use the two are interchangeable.

1.11.2 Five essential dplyr verbs

The **dplyr** package provides a small set of functions, often called *verbs*, each of which does one thing to a data frame. Every verb takes a data frame as its first argument and returns a new data frame, which is what makes them easy to chain with the pipe (Wickham et al. 2023). Column names are written without quotation marks.

select() keeps the named columns. A minus sign drops a column, and helper functions select columns by pattern:

Table 1.8: Five dplyr verbs introduced in this chapter.

Verb	What it does	Example
<code>select()</code>	Keep (or drop) columns	<code>select(htn, age, sex)</code>
<code>filter()</code>	Keep rows that meet a condition	<code>filter(htn, age >= 60)</code>
<code>arrange()</code>	Sort rows	<code>arrange(htn, desc(sbp_mmhg))</code>
<code>mutate()</code>	Create or modify columns	<code>mutate(htn, pp = sbp_mmhg - dbp_mmhg)</code>
<code>count()</code>	Count rows per category	<code>count(htn, facility)</code>

```
htn |>
  select(patient_id, ends_with("_mmhg")) |>
  head(3)
```

```
#> # A tibble: 3 × 3
#>   patient_id sbp_mmhg dbp_mmhg
#>   <chr>      <dbl>    <dbl>
#> 1 PHC-1224      140        92
#> 2 PHC-1169      185        91
#> 3 PHC-1391      147        85
```

`filter()` keeps the rows for which a condition is TRUE. Conditions use the comparison and logical operators of Section 1.5; several conditions separated by commas must all be true.

```
htn |>
  filter(facility == "Kisesa HC", age >= 70) |>
  select(patient_id, facility, age, sbp_mmhg) |>
  head(4)
```

```
#> # A tibble: 4 × 4
#>   patient_id facility    age sbp_mmhg
#>   <chr>      <chr>    <dbl>    <dbl>
#> 1 PHC-0112  Kisesa HC      86      131
#> 2 PHC-0035  Kisesa HC      81      174
#> 3 PHC-0379  Kisesa HC      72      135
#> 4 PHC-0737  Kisesa HC      80      154
```

The same idea finds the implausible blood-pressure values we saw in the summary:

```
htn |>
  filter(sbp_mmhg < 60 | sbp_mmhg > 260) |>
  select(patient_id, facility, sbp_mmhg, dbp_mmhg)
```

```
#> # A tibble: 2 × 4
#>   patient_id facility    sbp_mmhg dbp_mmhg
#>   <chr>      <chr>      <dbl>    <dbl>
#> 1 PHC-0768  Bugando PHC         0        73
#> 2 PHC-0360  Bugando PHC       700        81
```

Two records, both from Bugando PHC, have systolic pressures of 0 and 700 mmHg, values incompatible with life; their diastolic pressures (73 and 81 mmHg) look ordinary, which suggests a keying error in the systolic field only. Note that `filter()` drops rows where the condition is NA as

well as those where it is `FALSE`.

`arrange()` sorts rows, in ascending order by default or descending with `desc()`. We used it above to find the oldest recorded ages.

`mutate()` adds new columns computed from existing ones. Here we compute the mean arterial pressure for every patient at once, using the same formula as in Section 1.2:

```
htn |>
  mutate(map_mmhg = dbp_mmhg + (sbp_mmhg - dbp_mmhg) / 3,
         map_mmhg = round(map_mmhg, 1)) |>
  select(patient_id, sbp_mmhg, dbp_mmhg, map_mmhg) |>
  head(5)
```

```
#> # A tibble: 5 × 4
#>   patient_id sbp_mmhg dbp_mmhg map_mmhg
#>   <chr>      <dbl>    <dbl>    <dbl>
#> 1 PHC-1224      140        92      108
#> 2 PHC-1169      185        91     122.3
#> 3 PHC-1391      147        85     105.7
#> 4 PHC-0142      124       102     109.3
#> 5 PHC-0112      131        95      107
```

The formula written once for a single patient now runs on all 1,503 rows. Notice that `mutate()` does not change `htn` itself: the new column exists only in the result printed. To keep it, assign the result to an object (for example, `htn <- htn |> mutate(...)`). This is a general principle: dplyr verbs never modify their input in place.

`count()` tabulates the values of one or more variables and returns a tibble, which, unlike the output of `table()`, can be piped onwards:

```
htn |>
  count(residence, sort = TRUE)
```

```
#> # A tibble: 2 × 2
#>   residence     n
#>   <chr>    <int>
#> 1 Urban      799
#> 2 Rural      704
```

Slightly more patients live in urban (799) than in rural (704) areas; `sort = TRUE` lists the most frequent category first. Counting two variables gives a cross-tabulation in long format:

```
htn |>
  count(facility, residence) |>
  head(6)
```

```
#> # A tibble: 6 × 3
#>   facility residence     n
#>   <chr>      <chr>    <int>
#> 1 Bugando PHC Rural      147
#> 2 Bugando PHC Urban      190
#> 3 Buzuruga PHC Rural       75
#> 4 Buzuruga PHC Urban      100
```

```
#> 5 Igoma HC      Rural      92
#> 6 Igoma HC      Urban      83
```

We will use these verbs, together with `summarise()` and `group_by()`, throughout the rest of the book.

Tip

Build a pipeline one step at a time. Run the first line, check the result, add the next verb, run again. When a long pipeline gives an unexpected answer, delete steps from the end until the output makes sense again; the problem lies in the step you just removed.

1.12 Writing a reproducible script

1.12.1 Saving and organising a script

Save your script with *File > Save* (Ctrl+S or Cmd+S) into the `Scripts/` folder of your project, with a name that says what it does, such as `01_import_and_inspect.R`. Numbering scripts in the order they should run makes a multi-step analysis easy to follow. Save often; RStudio shows the file name in red with an asterisk while there are unsaved changes.

1.12.2 Comments

Comments explain *why* the code does what it does; the code itself shows *what*. Good comments record decisions ("exclude SBP > 260 mmHg as physiologically implausible, per protocol section 8.2"), the source of a formula, and anything a reader would otherwise have to guess. Section headings made of comment lines, such as `# 2. Import data ----`, are recognised by RStudio and appear in the document outline (the button at the top right of the script editor), making long scripts easy to navigate.

1.12.3 A template for an analysis script

Every analysis script in this book follows the same skeleton: a header saying what the script is for, who wrote it and when; the packages; the data; the analysis; and, where relevant, saving outputs. Here is a complete script for the work of this chapter. Because it writes nothing to disk, it is safe to run as many times as you like.

```
# =====
# Project: Determinants of hypertension treatment uptake
# Script:  01_import_and_inspect.R
# Purpose: Import the raw data and perform a first inspection
# Author:  <your name>           Date: <date>
# Input:   Data/hypertension_phc_raw.csv
# Output:  none (inspection only)
# =====

# 1. Packages ----
library(tidyverse)
```

```
# 2. Import data ----
htn <- read_csv("Data/hypertension_phc_raw.csv")

# 3. Inspect ----
dim(htn)                                # expect 1,500 patients
```

```
#> [1] 1503  36
```

```
n_distinct(htn$patient_id)              # number of unique patients
```

```
#> [1] 1500
```

```
summary(htn$age)                        # range check
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>    0.0   43.0   52.0   52.3   62.0   200.0
```

```
sum(htn$sbp_mmhg > 260 | htn$sbp_mmhg < 60, na.rm = TRUE) # implausible SBP
```

```
#> [1] 2
```

```
# 4. Notes for cleaning (Chapter 2) ----
# - 1,503 rows but 1,500 unique IDs: remove 3 duplicates
# - age 0 and 200; SBP 0 and 700: set to missing
```

The output confirms the findings of this chapter in four lines: 1,503 rows but only 1,500 distinct patient identifiers (the function `n_distinct()` counts unique values), an age range from 0 to 200, and two systolic readings outside a plausible range.

When an analysis does write files, such as a cleaned dataset or a figure, the code to save them belongs at the end of the script. For example (shown but not run here, so that nothing is written into the project):

```
write_csv(htn, "Outputs/htn_inspected.csv") # save a data frame as CSV
ggsave("Outputs/age_histogram.png", width = 7, height = 4.3) # save last plot
```

Good practice

A quick test of reproducibility: restart R (Ctrl+Shift+F10), then run the whole script from the top (Ctrl+Shift+Enter). If it runs without error and gives the same results, it is self-contained. If it fails, it depended on something you did by hand or on an object left over in memory. Make this test a habit before you share any result. For reports that combine text, code and output in one document, R Markdown and its successor Quarto take reproducibility one step further (Xie 2015).

1.13 Summary

This chapter has introduced the tools and habits on which the rest of the book depends. R is a free language for statistical computing, and RStudio is the environment in which we use it. Writing analyses as scripts, rather than typing in the console or clicking in menus, makes every result traceable to the data and the code that produced it, which is the essence of reproducible research.

We used R first as a calculator, paying attention to operator precedence, and computed BMI and mean arterial pressure. We stored values in objects with `<-`, met the main data types and saw that type errors, especially numbers stored as text and silent coercion to `NA`, are a common source of hidden mistakes. Vectors hold many values of one type; they can be indexed by position or by logical conditions, and arithmetic on them is vectorised. Missing values (`NA`) propagate through calculations unless removed deliberately with `na.rm = TRUE`. Functions take arguments by position or by name, and their help pages and error messages are there to be read. Packages extend R; they are installed once and loaded in every session, and the tidyverse provides a consistent set of tools for most of our work. RStudio Projects and relative paths make an analysis portable.

Finally, we imported the raw case-study data from both CSV and Excel files and inspected it with `dim()`, `glimpse()`, `summary()`, `table()` and two quick figures. Even this first look uncovered duplicate records, impossible values, missing-value codes, inconsistent spellings, mixed codings, mixed date formats and stray whitespace. Chapter 2 shows how to correct all of them in a documented, reproducible way.

Key points

- Write analysis code in a saved, commented script inside an RStudio Project; use the console only for exploration.
- Use `<-` for assignment and `==` for comparison; remember that R is case-sensitive and that text needs quotation marks.
- Every value has a type. Check types with `class()` or `glimpse()`; numbers stored as text and silent coercion to `NA` are common hidden errors.
- Vectors hold values of one type; operations on them are vectorised; `sum()` and `mean()` of a logical vector give a count and a proportion.
- `NA` marks a missing value and propagates through calculations; use `na.rm = TRUE` deliberately and always report how many values are missing.
- Install a package once with `install.packages()`; load it with `library()` at the top of every script.
- Use relative paths inside a Project; never `setwd()` to an absolute path.
- Import with `read_csv()` or `read_excel()`, then always inspect with `dim()`, `glimpse()`, `summary()` and `table()` before analysing.
- The pipe `|>` chains steps into a readable recipe; `select()`, `filter()`, `arrange()`, `mutate()` and `count()` cover a large share of everyday data handling.

1.14 Further reading

- Wickham et al. (2023), *R for Data Science* (2nd edition, free online): the standard introduction to the tidyverse; its chapters on workflow, data import and data transformation extend everything

in this chapter.

- Wilson et al. (2017), "Good enough practices in scientific computing": short, practical advice on organising projects, data and code that every researcher can adopt immediately.
- Broman and Woo (2018), "Data organization in spreadsheets": how to set up data-collection spreadsheets so that they can be imported and analysed without the problems seen in this chapter.
- Peng (2011), "Reproducible research in computational science": a two-page argument for why code and data should accompany published results.
- Wickham et al. (2019), "Welcome to the tidyverse": a brief overview of the design and components of the tidyverse.

1.15 Exercises

Exercise 1.1

R as a calculator. A patient weighs 82 kg, is 168 cm tall and has a blood pressure of 148/96 mmHg.

1. Store the four measurements in objects with sensible names.
2. Use them to compute the patient's BMI and mean arterial pressure, each rounded to one decimal place.
3. What happens if you forget the parentheses when converting height to metres? Explain the result using operator precedence.

Exercise 1.2

Vectors and logical comparisons. The diastolic pressures of eight patients seen in a morning clinic were 88, 92, 79, 101, 95, NA, 84 and 90 mmHg.

1. Store them in a vector `dbp` and find its length.
2. Compute the mean with and without `na.rm = TRUE`, and explain the difference.
3. How many readings are at least 90 mmHg, and what proportion of the *observed* readings is that?
4. Extract the third to fifth readings, and then all readings except the missing one.

Exercise 1.3

Data types and coercion. Consider the vector `glucose <- c("5.4", "6.1", "<2.0", "7.3", "999", "4.8")`, as it might arrive from a laboratory system.

1. What is its class, and why?
2. Convert it to numeric with `as.numeric()`. Which values become NA, and why?
3. Which remaining value is almost certainly not a real measurement? What would happen to the mean if it were left in?

Exercise 1.4

Importing data. Load the tidyverse and readxl. Import `Data/hypertension_phc_raw.csv` into an object `htn` and the data sheet of `Data/hypertension_phc_raw.xlsx` into `htn_xl`.

1. How many rows and columns does each have? How many rows would you expect, and why do they differ?
2. Confirm that the two objects have the same column names.
3. Use `n_distinct()` to count the unique values of `patient_id`.

Exercise 1.5

Inspecting structure. Use `glimpse(htn)` to answer the following.

1. List three variables stored as numeric and three stored as character.
2. Name two variables that are stored as character but should, after cleaning, be Yes/No factors. What values in them prevent R from treating them more simply?
3. Why is `enroll_date` not stored as a date?

Exercise 1.6

Spotting data problems. Use `summary()` and `table()` on the raw data.

1. Report the minimum and maximum of `sbp_mmhg`, `dbp_mmhg` and `height_cm`. Which are implausible?
2. Make a frequency table of `diabetes` and of `htn_diagnosed`. How many different codes are used in each?
3. How many values of `weight_kg` are missing?

Exercise 1.7

Using the pipe and dplyr. Starting from `htn`, write one pipeline that keeps patients at Nyamagana PHC, creates a column `map_mmhg` with the mean arterial pressure rounded to one decimal, keeps `patient_id`, `age`, `sbp_mmhg`, `dbp_mmhg` and `map_mmhg`, and sorts the result from the highest to the lowest MAP. Show the first five rows. Do the top values look plausible? Then use `count()` to show how many patients at each facility are recorded as having diabetes coded exactly "Yes".

Exercise 1.8

Challenge: a first figure. Make a histogram of systolic blood pressure for the raw data, using `ggplot()` and `geom_histogram()` as in Section 1.10. Then make a second histogram after keeping only values between 60 and 260 mmHg with `filter()`. How many records were removed, and how does the shape of the distribution change? Write two sentences describing the distribution of the plausible values.

Understanding and Cleaning Clinical Data

A clinical dataset arrives on your desk. It has the right number of columns, the variable names look sensible and the first few rows seem fine. It is tempting to go straight to the interesting part: the table of baseline characteristics, the comparison of treated and untreated patients, the regression model. Resist the temptation. Somewhere in those 1,500 rows there is a systolic blood pressure of 700 mmHg, a patient aged 200 years, a weight of 7 kg recorded for an adult, a diabetes status coded in six different ways and three patients who appear twice. None of these problems will stop R from producing a mean, a *p*-value or an odds ratio. All of them will make those numbers wrong. This chapter is about finding such problems systematically, deciding what to do about each one, and writing the decisions down as R code so that the journey from raw file to analysis-ready data is transparent and can be repeated by anyone, at any time, with exactly the same result. The clean dataset produced here, `analysis_data`, is the starting point for every remaining chapter of the book.

Learning objectives

- Explain why data cleaning is a scientific step that must be scripted, documented and reproducible, and describe the "cleaning contract" between raw data and analysis-ready data.
- Apply the principles of tidy data and classify clinical variables as continuous, discrete, nominal, ordinal, binary or date, choosing the appropriate R representation for each.
- Import a raw file while declaring missing-value codes explicitly, and demonstrate how undeclared codes such as `-99` and `999` distort summary statistics.
- Distinguish data missing completely at random, missing at random and missing not at random, explain why complete-case analysis can be biased, and produce a missing-data audit table and figure.
- Detect, inspect, remove and report duplicate records.
- Standardise inconsistent categories and stray whitespace with `str_trim()`, `str_to_lower()` and `case_when()`, and write a reusable helper function for Yes/No variables.
- Validate continuous measurements against plausible physiological ranges, set impossible values to missing and count what changed.
- Derive body mass index and clinical categories of BMI and blood pressure, and check derived variables against their definitions.

- Parse dates recorded in mixed formats with `lubridate`.
- Convert categorical variables to factors with deliberate level orders and reference categories, save the clean data and verify that the cleaning pipeline is reproducible.

2.1 Why cleaning matters

2.1.1 Garbage in, garbage out

Statistical methods take numbers as given. A *t*-test does not know that a systolic blood pressure of 700 mmHg is physiologically impossible; a logistic regression does not know that "Y", "yes" and "1" mean the same thing; a frequency table does not know that two rows describe the same patient. Every method in this book assumes that the data in front of it are a faithful record of what was measured. When that assumption fails, the analysis fails silently: the software produces output that looks every bit as authoritative as the output from clean data. This is the old principle of computing, *garbage in, garbage out*, and in clinical research its consequences are not merely embarrassing. A mean blood pressure inflated by a data-entry error can change the apparent prevalence of uncontrolled hypertension; a misclassified exposure can hide a real association or create a spurious one; a duplicated patient can make a rare adverse event look twice as common.

It is often said that data preparation takes most of the time in a real analysis, with figures such as 50 to 80 per cent commonly quoted. Whatever the exact share, the experience of most applied statisticians and epidemiologists is that cleaning, recoding and checking take far longer than fitting models. This is not wasted time. Cleaning is the stage at which you learn what the variables really mean, how they were collected, which values are suspicious and which questions the data can and cannot answer. A careful analyst comes out of the cleaning stage knowing the dataset intimately, and that knowledge pays off in every later step.

2.1.2 The cleaning contract

The central idea of this chapter is what we will call the **cleaning contract**. The contract has three clauses.

- 1. The raw data are never edited by hand.** The file that came from the data collection system (here, `hypertension_phc_raw.csv`) is treated as read-only. You do not open it in a spreadsheet and overwrite the 700 with 170, delete the duplicate rows or retype "F" as "Female". Manual edits leave no trace, cannot be reviewed and cannot be repeated when a corrected version of the raw file arrives.
- 2. Every change is made by code.** A cleaning script reads the raw file and applies each correction as an explicit, commented line of R. Anyone who reads the script can see exactly what was changed and why; anyone who runs it gets exactly the same result.
- 3. The output is a single, documented, analysis-ready file.** The script ends by saving the clean data (here, `analysis_data.rds`). Every analysis, in every later chapter, starts by loading this file and never by repeating or modifying the cleaning steps.

This separation of raw data, cleaning code and clean data is a cornerstone of reproducible research (Peng 2011). It is also the first recommendation of most guides to good practice in data management.

Broman and Woo (2018) give practical rules for organising data in spreadsheets so that they can be read reliably by software (one value per cell, consistent codes, no colour-coding as information, a separate data dictionary), and Wilson et al. (2017) recommend that raw data be saved in their original form, that cleaning be performed by scripts, and that the cleaned data be stored separately with a record of how they were produced. Following these principles costs a little discipline at the start and saves an enormous amount of confusion later.

Good practice

Keep a folder structure that makes the contract visible: raw files in one place (never modified), scripts in another, and derived data and outputs in a third. If you ever find yourself wanting to “just fix one value” in the raw file, write a line of code instead, with a comment explaining the source of the correction (for example, “value confirmed with the facility register on 12 March”).

2.1.3 The case-study data and the known problems

We work throughout with the raw file of the case study introduced in the Preface: a multicentre cross-sectional study of adults attending six primary healthcare facilities. The data are simulated for teaching, and some problems have been planted deliberately so that we can practise finding and fixing them. The data dictionary (`Data/data_dictionary.md`) lists the known problems: missing-value codes (blank, NA, 999, -99), three duplicate records, inconsistent spellings of sex, mixed codings of Yes/No variables, stray spaces in some text variables, impossible values of age, blood pressure, height and weight, a supplied BMI that contains errors, and enrolment dates recorded in three different formats. In real projects you will not be told the problems in advance; part of the skill is to look for them. We will therefore treat the dictionary as a check on our own detective work rather than as a list to be worked through blindly.

The cleaning steps in this chapter are applied in a fixed order, and the order matters. We import with missing-value codes declared, remove duplicates, standardise text, validate impossible values, derive new variables from the validated measurements, parse dates and finally set factor levels. At the end, we check that our result is identical to the file `Data/analysis_data.rds` that the remaining chapters use.

2.2 Tidy data and variable types

2.2.1 Tidy data

Before cleaning values, it is worth checking that the *shape* of the data is right. Wickham (2014) formalised a simple set of principles, known as **tidy data**, that describe the layout most statistical software expects:

1. each variable forms a column;
2. each observation forms a row;
3. each type of observational unit forms a table.

In a cross-sectional study such as ours, the observational unit is the patient, so a tidy dataset has one row per patient and one column per characteristic or measurement. Many real clinical datasets violate these principles in ways that are easy to recognise once you know the rules. A spreadsheet

with one column per clinic visit (`sbp_visit1`, `sbp_visit2`, `sbp_visit3`) stores a variable (visit) in the column names; a column such as `bp` containing "148/96" stores two variables in one cell; a sheet in which header rows for each facility are interleaved with patient rows mixes two observational units. Each of these needs reshaping before analysis, using tools such as `pivot_longer()` and `separate()` that are described in Wickham et al. (2023).

Our raw file is already tidy in shape: 36 columns, each a single variable, and (once duplicates are removed) one row per patient. The problems we face are problems of *values* and *types*, not of layout. That is typical of data exported from an electronic case report form, and it lets us concentrate on the content.

2.2.2 Types of clinical variable

Statistical methods depend on the kind of variable being analysed, so the first question to ask of every column is what sort of quantity it records. The usual classification is as follows (Kirkwood and Sterne 2003; Altman 1991).

- **Continuous** variables can, in principle, take any value in a range: age, blood pressure, height, weight, cholesterol. They are summarised with means and standard deviations or medians and interquartile ranges, and compared with *t*-tests, rank tests or linear regression.
- **Discrete** (count) variables take whole-number values: the number of comorbidities, the number of clinic visits in a year. They are numeric, but arithmetic on them must respect that "1.6 comorbidities" is an average, not a possible patient.
- **Nominal** categorical variables have categories with no natural order: facility, occupation, marital status. They are summarised with counts and percentages.
- **Ordinal** categorical variables have categories with a natural order but no fixed distance between them: education (none, primary, secondary, tertiary), physical activity (low, moderate, high). The order carries information that should not be thrown away.
- **Binary** variables are the special case of two categories: diabetes yes or no, treatment uptake yes or no. Binary outcomes are the domain of logistic regression (Chapter 5).
- **Dates and times** record when something happened: enrolment, diagnosis, death. They support arithmetic (time since diagnosis, follow-up time) only if stored as dates rather than as text.

R represents each kind of variable with a particular type, and part of cleaning is making sure each column ends up with the right one. The correspondence for the case study is summarised in the table below.

Two features of this table deserve comment. First, the identifier `patient_id` is stored as text even though it contains digits. Identifiers are labels, not quantities: it makes no sense to average them, and storing them as numbers would strip leading zeros (PHC-0011 is not the same as 11). Second, binary and categorical variables become **factors**. A factor stores the set of allowed categories (its *levels*) together with their order. That order determines how tables are laid out and, crucially, which category serves as the *reference* in regression models, a point we return to in Section 2.10.

2.2.3 What R guesses on import

When `read_csv()` imports a file, it inspects the first rows of each column and guesses a type. Numbers become doubles, everything else becomes character. Let us see what it guesses for the

Table 2.1: Kinds of clinical variable, examples from the case study and their R representation after cleaning.

Kind	Examples in the case study	R type after cleaning
Continuous	age, sbp_mmhg, height_cm, total_chol_mmol_l	numeric (double)
Discrete count	comorbidity_count, knowledge_score	numeric (double)
Nominal	facility, occupation, marital_status	factor
Ordinal	education, physical_activity, bmi_cat	factor (ordered or with ordered levels)
Binary	diabetes, treatment_uptake, sex	factor with two levels
Date	enroll_date	Date
Identifier	patient_id	character

raw file, importing it naively, that is, without telling R anything about the data.

```
library(tidyverse)
library(lubridate)

# Naive import: let read_csv() guess everything
raw_naive <- read_csv("Data/hypertension_phc_raw.csv")
dim(raw_naive)

#> [1] 1503 36
```

```
# How many columns of each type did read_csv() choose?
map_chr(raw_naive, \(x) class(x)[1]) |> table()

#>
#> character    numeric
#>          18         18
```

The file has 1,503 rows and 36 columns. Eighteen columns were read as numbers and eighteen as text. That is a reasonable first guess, but it is not the final answer. `enroll_date` was read as text because its values are written in several formats. The Yes/No variables were read as text because they mix words and digits. The categorical variables are text rather than factors, which is the tidyverse default and the correct one: we will decide the levels ourselves rather than let R choose them alphabetically. And the row count tells us something is wrong before we have looked at a single value: the study enrolled 1,500 patients, not 1,503.

2.3 Importing with explicit missing-value codes

2.3.1 Sentinel values

Data collection systems and the people who use them have many ways of saying "no value recorded". A cell can be left blank, filled with the letters NA, or filled with a **sentinel value**: a number that cannot be a real measurement and is used as a flag, typically 999, -99, -9 or 9999. Sentinels were common in older software that could not store an empty numeric cell, and they survive in many laboratory and registry systems today.

Sentinels are dangerous because they look like numbers. R will happily include a cholesterol of -99 mmol/L in a mean. Let us look at total cholesterol in the naive import.

```
# Total cholesterol as imported naively
summary(raw_naive$total_chol_mmol_l)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#>  -99.00   4.30   5.10   2.59   5.80   8.30    55
```

```
# How many values are the sentinel -99?
sum(raw_naive$total_chol_mmol_l == -99, na.rm = TRUE)
```

```
#> [1] 35
```

The minimum of -99 is impossible: a concentration cannot be negative. The naive summary reports 55 missing values (the blanks and NA strings that `read_csv()` recognises by default), but there are another 35 values coded -99, and they drag the mean far below the median. A summary like this should stop you in your tracks.

Before deciding which codes to treat as missing, it is good practice to scan all numeric columns for suspicious values, rather than relying on one column at a time. The next chunk counts, for every numeric column, how many values equal each of the common sentinels.

```
# Count candidate sentinel codes in every numeric column
sentinel_scan <- raw_naive |>
  select(where(is.numeric)) |>
  pivot_longer(everything(), names_to = "variable") |>
  filter(value %in% c(-99, -9, 999, 9999)) |>
  count(variable, value, name = "n")

knitr::kable(sentinel_scan,
  caption = "Sentinel codes in the numeric columns of the raw file.")
```

Table 2.2: Sentinel codes in the numeric columns of the raw file.

variable	value	n
bmi	999	22
fasting_glucose_mmol_l	999	40
total_chol_mmol_l	-99	35

Three variables contain sentinels: the supplied BMI and fasting glucose use 999, and total cholesterol uses -99. No column uses -9 or 9999. The scan also tells us something reassuring: there is no variable in which 999 could plausibly be a real value (a creatinine of 999 $\mu\text{mol/L}$, for example, is possible in severe renal failure, and would need much more careful handling).

Common mistake

Declaring a sentinel as missing applies to every column of the file. Before adding "999" to the list of missing codes, check that no variable could genuinely take that value. Creatinine, triglycerides in $\mu\text{mol/L}$, distances in metres and many laboratory counts can legitimately exceed 999. If a code is a sentinel in one column and a real value in another, convert it to NA only in the columns where it is a sentinel, for example with `na_if()` inside `mutate()`.

2.3.2 Declaring missing codes at import

The cleanest way to deal with sentinels is to declare them when the file is read, using the `na` argument of `read_csv()`. Any cell whose text exactly matches one of the strings becomes NA.

```
# Import, declaring every code that means "missing"
raw <- read_csv(
  "Data/hypertension_phc_raw.csv",
  na = c("", "NA", "999", "-99") # blank, "NA" and the two sentinels
)
dim(raw)
```

```
#> [1] 1503 36
```

```
summary(raw$total_chol_mmol_l)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#>      2.8     4.4     5.1     5.1     5.8     8.3     90
```

Cholesterol now ranges from 2.8 to 8.3 mmol/L, a clinically sensible range, its mean (5.1) equals its median (5.1), and the 90 missing values are counted honestly as missing. The comparison below shows the effect of the `na` argument on every column that has any missing values.

```
# Missing values per column: naive import versus declared codes
na_compare <- tibble(
  variable = names(raw),
  na_naive = colSums(is.na(raw_naive)),
  na_declared = colSums(is.na(raw))
) |>
  filter(na_declared > 0) |>
  mutate(gained = na_declared - na_naive)

knitr::kable(na_compare,
  caption = "Missing values before and after declaring codes.")
```

Table 2.3: Missing values before and after declaring codes.

variable	na_naive	na_declared	gained
education	30	30	0
weight_kg	45	45	0
bmi	38	60	22
smoking	45	45	0
total_chol_mmol_l	55	90	35
ldl_mmol_l	105	105	0
fasting_glucose_mmol_l	35	75	40
knowledge_score	45	45	0
distance_to_facility_km	60	60	0
adherence	995	995	0
bp_controlled	995	995	0

Read the table column by column. `na_naive` counts the values that `read_csv()` already recognised as missing (blanks and the string NA); `na_declared` counts the missing values once the sentinels are added; and `gained` is the difference. Total cholesterol gains 35 missing values, fasting glucose 40 and the supplied BMI 22: exactly the sentinel counts found in the scan. Every other column is unchanged. The very large counts for `adherence` and `bp_controlled` are not sentinels at all; we return to them in the next section.

Good practice

The `na` argument matches the *text* in the file. A sentinel written as `-99.0` or `999.00` would not match `"-99"` or `"999"`. After importing, always run `summary()` on the numeric columns and look at the minima and maxima: an impossible extreme value is often an undeclared code.

2.4 Missing data: concepts and an audit

Declaring missing values correctly is only the beginning. The harder question is what missing values *mean* for the analysis. Almost every clinical dataset has missing data, and the way they are handled can matter as much as the choice of statistical test.

2.4.1 Why values are missing: three mechanisms

Rubin (1976) introduced a classification of the processes, or **mechanisms**, that cause data to be missing. It is the foundation of modern thinking about missing data (Little and Rubin 2019), and every clinical researcher should know it.

- **Missing completely at random (MCAR).** The probability that a value is missing is unrelated to any characteristic of the patient, observed or unobserved. A blood sample dropped in the laboratory, a page of the form lost in transit, a randomly selected subset of patients not offered a test because of a shortage of reagent: in each case the patients with missing values are a random subsample of all patients. Under MCAR, analysing only the complete records loses precision but does not introduce bias.
- **Missing at random (MAR).** The probability that a value is missing depends on characteristics that *have been observed*, but not on the missing value itself once those characteristics are taken into account. Suppose cholesterol is measured less often at one facility because its laboratory is further away, or less often in younger patients because clinicians consider them at low risk. Missingness then depends on facility or age, both of which are recorded. Within each facility, or each age group, the patients with missing cholesterol are like those with observed cholesterol. The name is unfortunate: MAR does not mean "random" in the everyday sense, it means "random once we account for what we know".
- **Missing not at random (MNAR).** The probability that a value is missing depends on the value itself, even after accounting for the observed data. Patients with very high blood pressure may be more likely to miss a follow-up visit because they feel unwell; people who drink heavily may be more likely to leave the alcohol question blank; a laboratory analyser may fail to report glucose values above its measurement range. MNAR is the most troublesome mechanism, because the observed data cannot, on their own, tell us how the missing values differ.

The mechanism cannot usually be proved from the data. We can show that missingness depends on observed variables (evidence against MCAR), but we can never show from the observed data

alone that it does not also depend on the unobserved values. Judgements about MNAR must come from clinical knowledge of how the data were collected.

2.4.2 Why complete-case analysis can mislead

Most R functions, and most published analyses, deal with missing data by **complete-case analysis** (also called listwise deletion): any patient with a missing value in any variable used in the model is dropped. This is what `lm()` and `glm()` do by default, reporting "observations deleted due to missingness" in their output.

Complete-case analysis has two costs. The first is a loss of precision. If five covariates each have 5 per cent of values missing, independently, about 23 per cent of patients may be lost from a model that uses all five, and confidence intervals widen accordingly. The second cost is potential **bias**. Unless the data are MCAR, the complete cases are not a random sample of the study population. If cholesterol is missing more often in rural patients, and rural patients are less likely to take treatment, then an analysis restricted to patients with a cholesterol result over-represents urban patients and may give a misleading picture of treatment uptake. The direction and size of the bias depend on the mechanism and on the analysis, and in some situations complete-case analysis is unbiased even when data are not MCAR (for example, in regression when missingness depends only on the covariates) (Little and Rubin 2019). The important point is that it is an assumption, not a neutral default.

2.4.3 Multiple imputation: a glimpse ahead

When missing data are substantial and plausibly MAR, the recommended approach is often **multiple imputation** (Rubin 1976; Little and Rubin 2019). Instead of deleting incomplete records, each missing value is replaced by a value drawn from a model that predicts it from the other variables. This is done several times (say 20), producing several complete datasets that differ only in the imputed values. The analysis is performed on each dataset and the results are combined using rules that reflect both the within-dataset uncertainty and the extra uncertainty caused by not knowing the missing values. In R, the `mice` package implements multiple imputation by chained equations (Buuren and Groothuis-Oudshoorn 2011). Multiple imputation is powerful but not magic: it depends on a good imputation model, on the MAR assumption, and on careful reporting. Sterne et al. (2009) describe the most common pitfalls and give guidance on reporting. It is beyond the scope of this introductory book, but you should know that it exists and recognise when your analysis may need it.

Common mistake

Do not replace missing values with the mean, with zero or with "normal" values before analysis. Single mean imputation makes the data look more complete and less variable than they are, understates standard errors and can bias associations. Leave missing values as `NA` during cleaning, and make a deliberate, documented decision about how to handle them at the analysis stage.

2.4.4 A missing-data audit

The first practical step is an **audit**: a table showing, for every variable, how many values are missing and what percentage that represents. We write it once as a small function so that we can reuse it at the end of the chapter on the clean data.

```
# Count and percentage of missing values in every column
missing_audit <- function(data) {
  tibble(
    variable = names(data),
    n_missing = colSums(is.na(data)),
    pct = round(100 * n_missing / nrow(data), 1)
  ) |>
  arrange(desc(n_missing))
}

audit_raw <- missing_audit(raw)
audit_raw |>
  filter(n_missing > 0) |>
  knitr::kable(caption = "Missing values in the raw data (codes declared).")
```

Table 2.4: Missing values in the raw data (codes declared).

variable	n_missing	pct
adherence	995	66.2
bp_controlled	995	66.2
ldl_mmol_l	105	7.0
total_chol_mmol_l	90	6.0
fasting_glucose_mmol_l	75	5.0
bmi	60	4.0
distance_to_facility_km	60	4.0
weight_kg	45	3.0
smoking	45	3.0
knowledge_score	45	3.0
education	30	2.0

Eleven of the 36 columns have missing values. Two stand out: *adherence* and *bp_controlled* are missing for 995 of 1,503 records (66 per cent). Before panicking, check the data dictionary: *adherence* and *blood-pressure control* are only defined for patients who are *on treatment*. A patient who does not take antihypertensive medication cannot be adherent or non-adherent to it. These values are **structurally missing**, or “not applicable”, and are not a data-quality problem at all. We verify this formally at the end of the chapter.

The remaining variables have between 2 and 7 per cent missing. The laboratory values (LDL, total cholesterol, fasting glucose) have the most, which is typical: blood tests require a sample, a working laboratory and a result that finds its way back into the record. A figure makes the pattern easier to see.

```
audit_raw |>
  filter(n_missing > 0) |>
  mutate(
    type = if_else(variable %in% c("adherence", "bp_controlled"),
                  "Not applicable (untreated)", "Data missing"),
    variable = fct_reorder(variable, pct)
  ) |>
  ggplot(aes(x = pct, y = variable, fill = type)) +
  geom_col() +
```

```
geom_text(aes(label = paste0(pct, "%")), hjust = -0.15, size = 3) +
scale_fill_manual(values = c("#04242F", "#8FB8C4")) +
scale_x_continuous(limits = c(0, 75), expand = c(0, 0)) +
labs(x = "Missing (%)", y = NULL, fill = NULL,
     title = "Missing values by variable") +
theme(legend.position = "bottom")
```

Missing values by variable

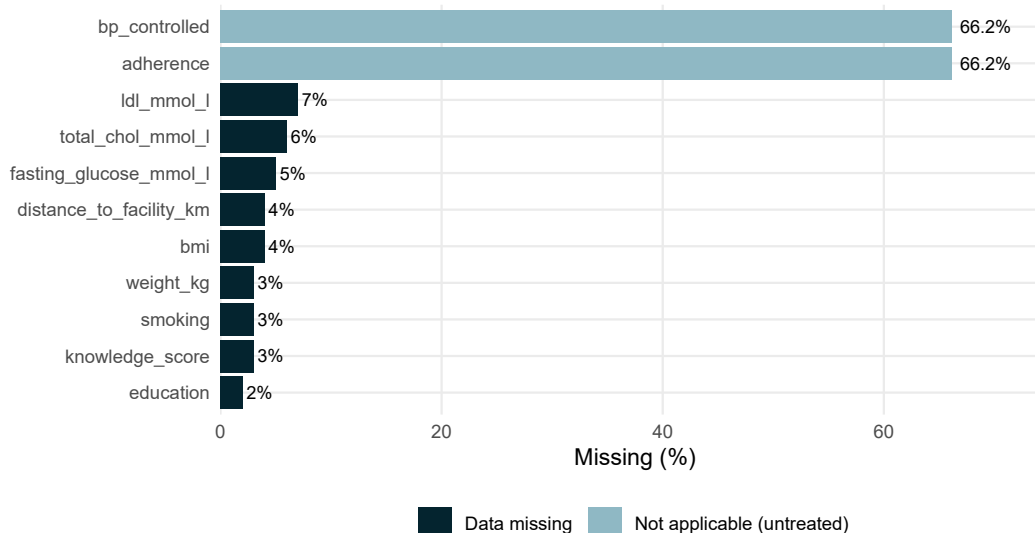


Figure 2.1: Percentage of missing values per variable in the raw data, after declaring missing-value codes. Structurally missing variables (defined only for treated patients) are shown in a lighter colour.

2.4.5 Exploring the mechanism

We cannot prove a missingness mechanism, but we can look for evidence against MCAR by asking whether missingness is related to observed characteristics. The next chunk compares the percentage of missing laboratory values across facilities.

```
# Percentage missing for three laboratory values, by facility
raw |>
mutate(facility = str_trim(facility)) |>
group_by(facility) |>
summarise(
  n = n(),
  chol_pct = round(100 * mean(is.na(total_chol_mmol_l)), 1),
  ldl_pct = round(100 * mean(is.na(ldl_mmol_l)), 1),
  gluc_pct = round(100 * mean(is.na(fasting_glucose_mmol_l)), 1)
) |>
knitr::kable(caption = "Percentage of missing laboratory values by facility.")
```

The percentages vary somewhat from facility to facility, but with 175 to 337 patients per facility, differences of a few percentage points are what chance alone produces. There is no facility where the laboratory results are systematically absent. In a real study, a facility with, say, 30 per cent missing cholesterol would prompt a conversation with the field team: was the laboratory closed for a month, or were the forms filled in differently? Such a pattern would suggest that missingness is MAR (dependent on facility) and that facility should be included in any analysis or imputation model that uses cholesterol.

Table 2.5: Percentage of missing laboratory values by facility.

facility	n	chol_pct	ldl_pct	gluc_pct
Bugando PHC	337	5.6	5.6	5.3
Buzuruga PHC	175	4.6	7.4	7.4
Igoma HC	175	6.3	6.9	6.3
Ilemela HC	283	5.3	7.8	3.2
Kisesa HC	228	9.2	7.0	3.9
Nyamagana PHC	305	5.2	7.5	4.9

Clinical interpretation

Ask *why* each value is missing before deciding what to do. A missing glucose result because the analyser was broken (MCAR), because only patients with symptoms were tested (MAR, if symptoms are recorded) and because very high values were not reported (MNAR) call for very different handling. The people who collected the data usually know the answer; ask them. Because these data are simulated, the patterns here illustrate the methods, not real clinical findings.

2.5 Duplicate records

2.5.1 Where duplicates come from

A **duplicate** is a record that appears more than once for what should be a single observational unit. In clinical data, duplicates arise when a form is entered twice (once by each of two data clerks, or once before and once after a correction), when files from two sources are combined and overlap, or when an export is run twice and appended. Duplicates inflate the sample size, give some patients double weight, and make standard errors too small. If the duplicated patients differ systematically from the rest (for example, if forms with problems were more likely to be re-entered), they can also introduce bias.

There are two kinds of duplicate to look for. **Exact duplicates** are rows identical in every column; they are almost certainly the same record entered twice and can be removed safely. **Key duplicates** share the same identifier but differ in some columns; they may be a re-entered record with a correction, two visits by the same patient, or two different patients given the same ID by mistake. Key duplicates need investigation, not automatic deletion.

2.5.2 Detecting duplicates

We start by comparing the number of rows with the number of distinct patient IDs, and then list the IDs that appear more than once.

```
nrow(raw) # number of rows
```

```
#> [1] 1503
```

```
n_distinct(raw$patient_id) # number of distinct patients
```

```
#> [1] 1500
```

```
# Which IDs appear more than once?
```

```
raw |>
  count(patient_id) |>
  filter(n > 1)
```

```
#> # A tibble: 3 × 2
#>   patient_id     n
#>   <chr>       <int>
#> 1 PHC-0011         2
#> 2 PHC-0251         2
#> 3 PHC-0881         2
```

There are 1,503 rows but only 1,500 distinct patient IDs, and three IDs (PHC-0011, PHC-0251 and PHC-0881) each appear twice. Next we ask whether these are exact duplicates.

```
# Number of rows identical to an earlier row in every column
sum(duplicated(raw))
```

```
#> [1] 3
```

`duplicated()` marks each row that is identical to a row that appeared earlier, so the sum, 3, is the number of surplus copies. Because the number of exact duplicates equals the number of surplus IDs, every repeated ID is an exact copy of another row: there are no conflicting key duplicates to investigate. It is still worth looking at the duplicated records before removing them.

```
# Show the duplicated records side by side (first few columns)
raw |>
  filter(patient_id %in% c("PHC-0011", "PHC-0251", "PHC-0881")) |>
  arrange(patient_id) |>
  select(patient_id, facility, enroll_date, age, sex, sbp_mmhg)
```

```
#> # A tibble: 6 × 6
#>   patient_id facility    enroll_date    age sex    sbp_mmhg
#>   <chr>       <chr>       <chr>      <dbl> <chr>    <dbl>
#> 1 PHC-0011 Ilemela HC  2024-12-01    51 Male     128
#> 2 PHC-0011 Ilemela HC  2024-12-01    51 Male     128
#> 3 PHC-0251 Bugando PHC  2024-01-28    51 Female   131
#> 4 PHC-0251 Bugando PHC  2024-01-28    51 Female   131
#> 5 PHC-0881 Kisesa HC   2024-02-25    45 Female   140
#> 6 PHC-0881 Kisesa HC   2024-02-25    45 Female   140
```

Each pair is identical, as expected. The `janitor` package provides `get_dupes()`, which produces the same listing in one call with a count column added, and is a convenient alternative for larger files.

2.5.3 Removing duplicates and checking the result

`distinct()` keeps the first occurrence of each unique row and drops the rest.

```

n_before <- nrow(raw)
raw <- distinct(raw)           # drop exact duplicate rows
n_after <- nrow(raw)

c(before = n_before, after = n_after, removed = n_before - n_after)

```

```

#> before after removed
#>    1503    1500      3

```

```

# Every row should now be a different patient
n_distinct(raw$patient_id) == nrow(raw)

```

```

#> [1] TRUE

```

Three rows were removed and 1,500 remain, one per patient. The final line is a simple but powerful check: it returns `TRUE` only if the patient ID is now a unique key.

Good practice

Always report duplicates. A sentence such as "Three records were exact duplicates of other records and were removed, leaving 1,500 participants" belongs in the methods section or in a flow diagram, as recommended by the STROBE guidelines for observational studies (Elm et al. 2007). If you find key duplicates with conflicting values, do not choose one at random: return to the source documents, and record which version was kept and why.

2.6 Inconsistent categories and whitespace

Categorical variables are where human data entry shows most clearly. Different clerks, different forms and different software versions produce different spellings of the same category: "Female", "female", "F" and "f"; "Yes", "Y" and "1". To a human reader they are obviously equivalent. To R they are different strings, and a frequency table or regression model will treat them as different categories.

2.6.1 Whitespace

The least visible problem is **whitespace**: spaces before or after a value, as in " Rural ". They are invisible in most printouts, but " Rural " is not equal to "Rural". The data dictionary warns that some values of facility, residence, education and occupation have stray spaces. Yet the frequency table of facility in our imported data looks clean.

```

table(raw$facility)

```

```

#>
#> Bugando PHC Buzuruga PHC Igoma HC Ilemela HC Kisesa HC
#>          336          175          175          282          227
#> Nyamagana PHC
#>          305

```

The reason is that `read_csv()` trims leading and trailing whitespace by default (its argument `trim_`

ws = TRUE). We can reveal the problem by importing with trimming switched off.

```
# Re-import WITHOUT trimming, only to see the hidden spaces
raw_untrimmed <- read_csv("Data/hypertension_phc_raw.csv",
  na = c("", "NA", "999", "-99"),
  trim_ws = FALSE)

# Number of values with leading or trailing spaces, per text column
raw_untrimmed |>
  select(facility, residence, education, occupation) |>
  summarise(across(everything(), \(x) sum(x != str_trim(x), na.rm = TRUE)))
```

```
#> # A tibble: 1 × 4
#>   facility residence education occupation
#>   <int>      <int>      <int>      <int>
#> 1      108      143      117      109
```

```
# What R sees without trimming
table(raw_untrimmed$residence)
```

```
#>
#> Rural   Urban   Rural   Urban
#>    68     75    636    724
```

Without trimming, `residence` has four categories instead of two, and 108 to 143 values in each of the four variables carry hidden spaces. Many import routes, including copying data from another R object, reading from a database or reading some text formats, do not trim automatically. We therefore apply `str_trim()` to every text column explicitly. It costs nothing when the data are already clean and protects the pipeline if the import route ever changes.

```
# Remove leading/trailing spaces from every character column
raw <- raw |>
  mutate(across(where(is.character), str_trim))
```

`across(where(is.character), str_trim)` reads as "for every column that is character, replace it with its trimmed version". It is the first of several uses of `across()` in this chapter: it lets one line of code do the same job on many columns.

2.6.2 Inconsistent spellings: sex

Now we tabulate `sex`, asking `table()` to show missing values too.

```
table(raw$sex, useNA = "ifany")
```

```
#>
#>   f      F female Female      m      M   male   Male
#>  74     73     68    670    40     39     42    494
```

There are eight spellings of two categories: full words in title case and lower case, and single letters in upper and lower case. The fix has two parts. First we convert the text to lower case with `str_to_lower()`, which collapses "Female", "female" into "female" and "F", "f" into "f". Then `case_when()` maps each group of spellings to a single clean label. `case_when()` evaluates its conditions

from top to bottom and returns the value of the first condition that is true; the final `TRUE ~` line is a catch-all for anything not recognised.

```
sex_before <- raw$sex # keep a copy to check the recoding

raw <- raw |>
  mutate(sex = case_when(
    str_to_lower(sex) %in% c("female", "f") ~ "Female",
    str_to_lower(sex) %in% c("male", "m") ~ "Male",
    TRUE ~ NA_character_ # anything unrecognised becomes missing
  ))

table(raw$sex, useNA = "ifany")
```

```
#>
#> Female   Male
#>    885    615
```

There are now 885 women and 615 men, with no missing values. The catch-all produced no NAs, which tells us that every original value was recognised. A recoding should always be checked by cross-tabulating the old values against the new.

```
table(before = sex_before, after = raw$sex, useNA = "ifany")
```

```
#>      after
#> before Female Male
#> f         74    0
#> F         73    0
#> female    68    0
#> Female   670    0
#> m          0   40
#> M          0   39
#> male       0   42
#> Male       0  494
```

Each original spelling maps to exactly one clean category, and nothing has been lost. This cross-tabulation is the best protection against a typo in a recoding rule (for example, writing "femle"), which would otherwise silently turn valid values into missing.

Common mistake

Ending `case_when()` with `TRUE ~ sex` (keep the original value) instead of `TRUE ~ NA_character_` hides problems: an unexpected spelling such as "FM" would pass through unchanged and appear as a third category in later tables. Mapping unrecognised values to NA and then checking that no new NAs appeared makes surprises visible.

2.6.3 A reusable helper for Yes/No variables

Five variables (diabetes, family_history_htn, health_insurance, htn_diagnosed and treatment_uptake) mix three codings of the same binary information.

```
table(raw$diabetes, useNA = "ifany")
```

```
#>
#>      0      1      N    No      Y    Yes
#> 229    7   143 1093     4    24
```

```
table(raw$treatment_uptake, useNA = "ifany")
```

```
#>
#>      0      1      N    No      Y    Yes
#> 134    79    95  763    52  377
```

Diabetes is recorded as 0/1, N/Y and No/Yes. Note that `read_csv()` read these columns as character because the words and the digits are mixed; had a column contained only 0 and 1 it would have been read as a number, which is another reason to convert values with `as.character()` before comparing them with text.

Rather than writing the same `case_when()` five times, we write a small **function**. Functions are the natural way to apply the same logic consistently: if the rule ever needs to change (say, a new code "U" for unknown appears), it changes in one place.

```
# Standardise any Yes/No coding to "Yes", "No" or NA
to_yesno <- function(x) {
  x <- str_to_lower(str_trim(as.character(x))) # text, no spaces, lower case
  case_when(
    x %in% c("yes", "y", "1", "true") ~ "Yes",
    x %in% c("no", "n", "0", "false") ~ "No",
    TRUE ~ NA_character_ # anything else -> missing
  )
}
```

```
# Test the helper on a small vector before using it on the data
to_yesno(c("Yes", " y", "1", "No", "N", "0", "TRUE", "maybe", NA))
```

```
#> [1] "Yes" "Yes" "Yes" "No"  "No"  "No"  "Yes" NA    NA
```

Testing a helper on a small, hand-made vector that includes awkward cases (a leading space, a logical-looking value, an unexpected word, a missing value) is a habit worth forming. The output is exactly as intended: four forms of yes, three of no, and NA for the unrecognised word and the missing value. Now we apply it to all five columns at once with `across()`.

```
binary_vars <- c("diabetes", "family_history_htn", "health_insurance",
                 "htn_diagnosed", "treatment_uptake")
binary_before <- raw |> select(all_of(binary_vars)) # copy for checking

raw <- raw |>
  mutate(across(all_of(binary_vars), to_yesno))

# Counts of each clean value in each binary variable
raw |>
  select(all_of(binary_vars)) |>
  pivot_longer(everything(), names_to = "variable") |>
  count(variable, value) |>
```

```
pivot_wider(names_from = value, values_from = n, values_fill = 0) |>
knitr::kable(caption = "Binary variables after standardisation.")
```

Table 2.6: Binary variables after standardisation.

variable	No	Yes
diabetes	1465	35
family_history_htn	940	560
health_insurance	988	512
htn_diagnosed	411	1089
treatment_uptake	992	508

Every binary variable now has only the values "No" and "Yes", and there is no NA column in the table, so every original code was recognised. Among the 1,500 patients, 1,089 have been diagnosed with hypertension and 508 are taking antihypertensive treatment. As a check on the mapping, the cross-tabulation for treatment uptake confirms that each original code went to the right place.

```
table(before = binary_before$treatment_uptake,
      after = raw$treatment_uptake)
```

R

```
#>      after
#> before No Yes
#>    0  134  0
#>    1    0 79
#>    N   95  0
#>   No  763  0
#>    Y    0 52
#>   Yes    0 377
```

Good practice

The `forcats` package (part of the `tidyverse`) offers `fct_recode()` and `fct_collapse()` for recoding factor levels, for example `fct_collapse(sex, Female = c("F", "f", "female", "Female"))`. They are convenient when the full list of spellings is known in advance. `case_when()` with `str_to_lower()` is more robust to spellings you did not anticipate, because it normalises the text before comparing it.

2.7 Validating impossible values

2.7.1 Plausible ranges

After missing codes and spellings, the next class of problem is values that are recorded as numbers but cannot be true. Some are obvious typing errors: a systolic blood pressure of 700 is probably 170 with a slipped digit, a height of 17 cm is probably 170 cm with a dropped zero. Others are placeholders (age 0 for "unknown") or unit confusion (weight in pounds in a column meant for kilograms).

The standard approach is to define, for each continuous variable, a **plausible range** based on physiology and on the study's eligibility criteria, and to flag every value outside it. The ranges should be wide enough to keep genuine extreme values (severe hypertension, very tall or very

thin patients are real and clinically important) and narrow enough to exclude values that cannot be true. Ideally they are written into the study's data management plan before the data are seen. The ranges used for the case study are shown below.

Table 2.7: Plausible ranges used to validate continuous measurements in adult primary-care patients.

Variable	Lower limit	Upper limit	Rationale
age (years)	18	110	Adults only (eligibility); oldest plausible age
sbp_mmhg (mmHg)	70	260	Below 70 is shock; above 260 almost never recorded
dbp_mmhg (mmHg)	40	150	Physiological limits for a seated adult
height_cm (cm)	120	210	Range of adult stature
weight_kg (kg)	30	200	Range of adult body weight in this population

These limits are judgements, and reasonable people might choose slightly different ones. What matters is that they are explicit, justified and applied by code, so that a reader can see them and a reviewer can ask for a sensitivity analysis with different limits.

2.7.2 Finding the offending values

Before changing anything, we count how many values fall outside each range and look at them.

```
# Count values outside each plausible range (NA values are not counted)
raw |>
  summarise(
    age      = sum(age < 18 | age > 110, na.rm = TRUE),
    sbp      = sum(sbp_mmhg < 70 | sbp_mmhg > 260, na.rm = TRUE),
    dbp      = sum(dbp_mmhg < 40 | dbp_mmhg > 150, na.rm = TRUE),
    height   = sum(height_cm < 120 | height_cm > 210, na.rm = TRUE),
    weight   = sum(weight_kg < 30 | weight_kg > 200, na.rm = TRUE)
  )
```

```
#> # A tibble: 1 × 5
#>   age    sbp    dbp height weight
#>   <int> <int> <int>   <int>   <int>
#> 1     2     2     1     1     1
```

```
# The records concerned
raw |>
  filter(age < 18 | age > 110 | sbp_mmhg < 70 | sbp_mmhg > 260 |
         dbp_mmhg < 40 | dbp_mmhg > 150 | height_cm < 120 |
         height_cm > 210 | weight_kg < 30 | weight_kg > 200) |>
  select(patient_id, age, sbp_mmhg, dbp_mmhg, height_cm, weight_kg)
```

```
#> # A tibble: 7 × 6
#>   patient_id age sbp_mmhg dbp_mmhg height_cm weight_kg
#>   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1 PHC-0174   200     130     83    154.4    77.9
#> 2 PHC-0689    34     130     87    170.7     7
#> 3 PHC-0768    32      0     73    158.3    81.4
#> 4 PHC-0664    52    102     61     17    63.9
#> 5 PHC-0360    34     700     81    169.2    64.9
#> 6 PHC-0806     0    157     91    166.3    85.8
```

```
#> 7 PHC-1342      54      106      5      150.2      40.1
```

Seven values in seven different patients are out of range: two ages (0 and 200), two systolic pressures (0 and 700), one diastolic pressure (5), one height (17 cm) and one weight (7 kg). These are exactly the problems listed in the data dictionary. Look at each row: apart from the single impossible value, every other measurement for these patients is plausible. That is typical of data-entry errors and supports the decision to remove only the impossible value, not the whole patient.

2.7.3 Setting impossible values to missing

What should replace an impossible value? It is tempting to "correct" 700 to 170, or 17 to 170. Resist: you do not know that the intended value was 170 rather than 107 or 160, and an invented value is worse than an honest blank. Unless the correct value can be confirmed from the source documents (the paper form, the clinic register), the safe choice is to set the value to missing. `if_else()` keeps the value when the condition is true and substitutes NA otherwise.

```
raw_before_validation <- raw      # keep a copy for the before/after figures

raw <- raw |>
  mutate(
    age      = if_else(age >= 18 & age <= 110, age, NA_real_),
    sbp_mmhg = if_else(sbp_mmhg >= 70 & sbp_mmhg <= 260, sbp_mmhg, NA_real_),
    dbp_mmhg = if_else(dbp_mmhg >= 40 & dbp_mmhg <= 150, dbp_mmhg, NA_real_),
    height_cm = if_else(height_cm >= 120 & height_cm <= 210,
                        height_cm, NA_real_),
    weight_kg = if_else(weight_kg >= 30 & weight_kg <= 200, weight_kg, NA_real_)
  )

summary(select(raw, age, sbp_mmhg, dbp_mmhg, height_cm, weight_kg))
```

```
#>      age      sbp_mmhg      dbp_mmhg      height_cm      weight_kg
#> Min.   :18.0   Min.    : 90   Min.    : 56   Min.    :141   Min.    : 34.6
#> 1st Qu.:43.0   1st Qu.:126   1st Qu.: 79   1st Qu.:158   1st Qu.: 60.9
#> Median :52.0   Median :139   Median : 86   Median :164   Median : 70.1
#> Mean   :52.3   Mean    :139   Mean    : 86   Mean    :164   Mean    : 70.9
#> 3rd Qu.:62.0   3rd Qu.:153   3rd Qu.: 93   3rd Qu.:169   3rd Qu.: 80.4
#> Max.   :95.0   Max.    :201   Max.    :121   Max.    :193   Max.    :132.8
#> NAs    :2     NAs     :2     NAs     :1     NAs     :46
```

```
# Number of values set to missing by validation
vars_checked <- c("age", "sbp_mmhg", "dbp_mmhg", "height_cm", "weight_kg")
n_invalid <- sum(is.na(raw[vars_checked])) -
  sum(is.na(raw_before_validation[vars_checked]))
n_invalid
```

```
#> [1] 7
```

Read the summary column by column. Age now runs from 18 to 95 years, systolic pressure from 90 to 201 mmHg, diastolic pressure from 56 to 121 mmHg, height from about 141 to 193 cm (`summary()` rounds to the significant digits it displays) and weight from 34.6 to 132.8 kg. Every minimum and

maximum is clinically believable. The NA's row shows the cost: age and systolic pressure have two missing values each (the two impossible values), diastolic pressure and height one each, and weight 46 (the 45 that were already missing plus the 7 kg value). The means have barely moved, except for systolic pressure, where the single value of 700 had pulled the mean up by a noticeable fraction of a millimetre of mercury.

Two details of the code are worth noting. `NA_real_` is the missing value of the numeric type; `if_else()` insists that both of its outcomes have the same type, so a plain NA (which is logical) would cause an error. And the condition is written so that a value that is *already* missing stays missing: `NA >= 18` is NA, and `if_else()` returns NA when its condition is NA.

Common mistake

Do not delete whole patients (rows) because one measurement is impossible. Removing the patient with a weight of 7 kg would also remove a valid age, blood pressure, diagnosis and treatment status, and every such deletion nudges the sample away from the population. Set the single impossible value to NA and keep the rest of the record.

2.7.4 Seeing the effect

Figures make the effect of validation obvious. The first compares the distribution of systolic blood pressure before and after validation.

```
bind_rows(
  tibble(stage = "Before validation", sbp = raw_before_validation$sbp_mmhg),
  tibble(stage = "After validation", sbp = raw$sbp_mmhg)
) |>
mutate(stage = fct_inorder(stage)) |>
ggplot(aes(x = sbp, y = stage)) +
  geom_boxplot(fill = "#8FB8C4", outlier.colour = "#B03A2E", na.rm = TRUE) +
  labs(x = "Systolic blood pressure (mmHg)", y = NULL,
       title = "Systolic blood pressure before and after validation")
```

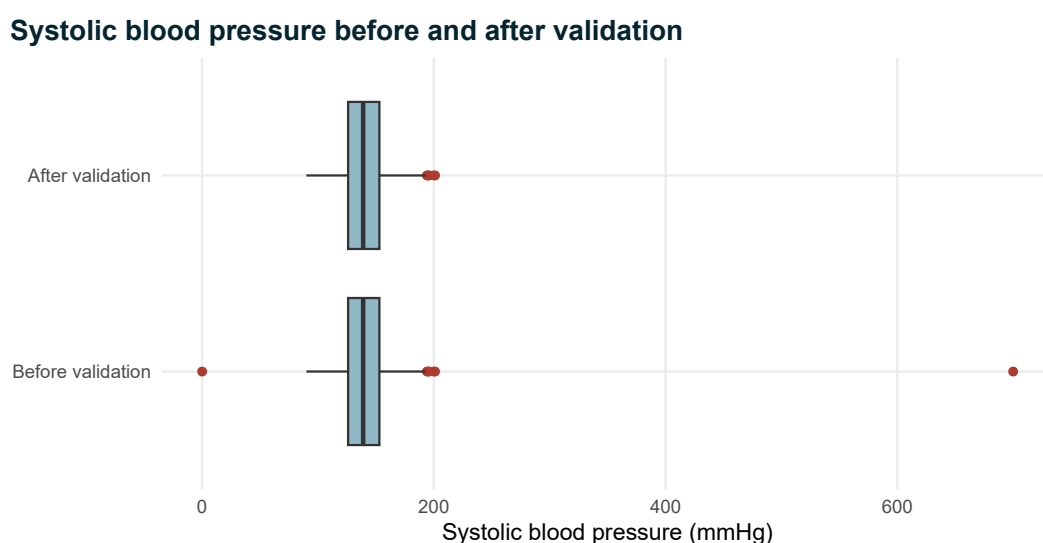


Figure 2.2: Systolic blood pressure before and after validation. Before validation, the values of 0 and 700 mmHg stretch the axis far beyond the plausible data; after validation the distribution is clinically sensible.

Before validation, a single value of 700 mmHg stretches the axis to more than three times the range

of the real data, so that the box containing the middle half of the patients occupies only a small part of the plot, and the value of 0 sits alone on the left. In a histogram or a figure for publication, the same two values would distort the scale even more. After validation, the box spans roughly 126 to 153 mmHg and the remaining outliers are genuine high readings, the patients with severe hypertension whom a hypertension study most needs to keep. The second figure shows the same idea for age, using histograms.

```

bind_rows(
  tibble(stage = "Before validation", age = raw_before_validation$age),
  tibble(stage = "After validation", age = raw$age)
) |>
mutate(stage = fct_inorder(stage)) |>
ggplot(aes(x = age)) +
  geom_histogram(binwidth = 5, fill = "#04242F", colour = "white",
    na.rm = TRUE) +
  facet_wrap(~ stage, ncol = 1, scales = "free_x") +
  labs(x = "Age (years)", y = "Number of patients",
    title = "Age before and after validation")

```

Age before and after validation

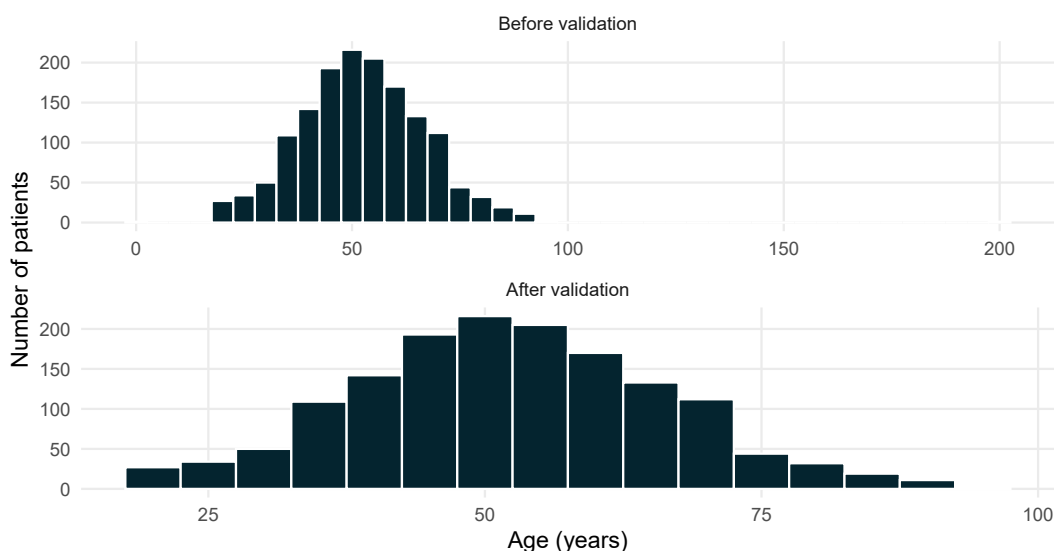


Figure 2.3: Distribution of age before and after validation. The impossible ages of 0 and 200 years appear as isolated bars before validation and are absent afterwards.

Clinical interpretation

A plausible range catches impossible values, not implausible combinations. A diastolic pressure higher than the systolic pressure, a 20-year-old with 110 months since diagnosis or a man recorded as pregnant are each within the range of the individual variables but impossible together. After range checks, think about **cross-variable consistency checks** suggested by the clinical meaning of the data (Exercise 2.4 asks you to try one).

2.8 Recoding and deriving variables

Many variables used in analysis are not measured directly but **derived** from other variables: body mass index from height and weight, mean arterial pressure from systolic and diastolic pressure, a

hypertension category from blood-pressure readings, an age group from age. A derived variable inherits every error in its inputs, so it must be computed *after* the inputs have been validated, and it must be checked against its definition.

2.8.1 Body mass index

Body mass index is weight in kilograms divided by the square of height in metres:

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height (m)}^2} = \frac{\text{weight (kg)}}{(\text{height (cm)}/100)^2}.$$

The raw file already contains a `bmi` column. Should we trust it? Let us compare it with BMI recomputed from the validated height and weight.

```
bmi_check <- raw |>
  mutate(bmi_recalc = round(weight_kg / (height_cm / 100)^2, 1),
         diff = bmi - bmi_recalc)

# Which BMI values are available in each version?
bmi_check |>
  count(supplied_missing = is.na(bmi), recalc_missing = is.na(bmi_recalc))
```

```
#> # A tibble: 4 × 3
#>   supplied_missing recalc_missing     n
#>   <lg1>           <lg1>         <int>
#> 1 FALSE          FALSE         1397
#> 2 FALSE          TRUE           43
#> 3 TRUE           FALSE          56
#> 4 TRUE           TRUE           4
```

```
# Where both exist, do they disagree by more than rounding?
bmi_check |>
  filter(abs(diff) > 0.1) |>
  select(patient_id, height_cm, weight_kg, bmi, bmi_recalc)
```

```
#> # A tibble: 1 × 5
#>   patient_id height_cm weight_kg   bmi bmi_recalc
#>   <chr>         <dbl>     <dbl> <dbl>   <dbl>
#> 1 PHC-0286     167.6       69.7   120     24.8
```

The first table cross-classifies the patients by whether each version of BMI is available. For 1,397 patients both exist. For 56 patients the supplied BMI is missing (a blank or the sentinel 999) although height and weight were recorded, so BMI can be recovered by recomputing it. For 43 patients the opposite holds: a BMI was supplied, but the weight or height behind it is missing or was set to missing during validation (including the patients with a weight of 7 kg and a height of 17 cm). A BMI that cannot be traced back to its measurements cannot be checked, so we do not keep it. The second table shows that, among the 1,397 patients with both versions, only one disagrees by more than rounding: a supplied BMI of 120 kg/m², an impossible value, for a patient whose height and weight give 24.8. The lesson is general: **never trust a supplied derived variable; recompute it from its validated inputs.**

```
raw <- raw |>
  mutate(bmi = round(weight_kg / (height_cm / 100)^2, 1))

summary(raw$bmi)
```

```
#>      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#>      15.0   23.2   26.3   26.4   29.5   42.5     47
```

The recomputed BMI ranges from 15.0 to 42.5 kg/m², with a median of 26.3 and 47 missing values: the 46 patients without a valid weight plus the one patient without a valid height.

2.8.2 BMI categories

For description and for some analyses, BMI is grouped into the categories defined by the World Health Organization.

Table 2.8: World Health Organization BMI categories for adults.

Category	BMI (kg/m ²)
Underweight	below 18.5
Normal weight	18.5 to 24.9
Overweight	25.0 to 29.9
Obese	30.0 or above

The function `cut()` divides a continuous variable into intervals defined by **break points**. With `breaks = c(-Inf, 18.5, 25, 30, Inf)` there are four intervals, and `labels` names them.

```
raw <- raw |>
  mutate(bmi_cat = cut(bmi,
    breaks = c(-Inf, 18.5, 25, 30, Inf),
    labels = c("Underweight", "Normal",
      "Overweight", "Obese")))

table(raw$bmi_cat, useNA = "ifany")
```

```
#>
#> Underweight      Normal Overweight      Obese      <NA>
#>           84         476         577         316         47
```

A derived category should always be checked against the variable it came from. The simplest check is the range of BMI within each category.

```
raw |>
  group_by(bmi_cat) |>
  summarise(n = n(), min_bmi = min(bmi), max_bmi = max(bmi)) |>
  knitr::kable(caption = "Range of BMI within each derived BMI category.")
```

Each category covers the expected part of the BMI scale and the categories do not overlap. Patients with missing BMI fall into the NA category, as they should: a missing input must give a missing category, never a default one.

Table 2.9: Range of BMI within each derived BMI category.

bmi_cat	n	min_bmi	max_bmi
Underweight	84	15.0	18.5
Normal	476	18.6	25.0
Overweight	577	25.1	30.0
Obese	316	30.1	42.5
NA	47	NA	NA

Common mistake

By default `cut()` builds intervals that are *closed on the right*: $(18.5, 25]$ includes 25 but not 18.5. A BMI of exactly 25.0 is therefore labelled "Normal" and a BMI of exactly 18.5 "Underweight", whereas the WHO definition puts 25.0 in "Overweight" and 18.5 in "Normal". With BMI rounded to one decimal place, a few patients sit exactly on a cut-point (in our data, 21 patients). The clean dataset used in this book keeps the default so that it matches the file used in later chapters, but in your own work add `right = FALSE` to `cut()` so that each interval includes its lower limit and the categories match the published definition exactly. Exercise 2.5 explores the difference.

2.8.3 Blood-pressure categories

Blood pressure is the central measurement of the case study. We derive a three-level category from the measured systolic and diastolic pressures. Following the conventional threshold of 140/90 mmHg for hypertension (World Health Organization 2021), a patient is classified as "Hypertension" if the systolic pressure is at least 140 mmHg *or* the diastolic pressure is at least 90 mmHg; as "Elevated" if, failing that, systolic is at least 130 or diastolic at least 80; and as "Normal" otherwise. Because `case_when()` stops at the first true condition, the order of the rules implements the "failing that" logic for us.

```

raw <- raw |>
  mutate(bp_category = case_when(
    is.na(sbp_mmhg) | is.na(dbp_mmhg) ~ NA_character_, # need both readings
    sbp_mmhg >= 140 | dbp_mmhg >= 90 ~ "Hypertension",
    sbp_mmhg >= 130 | dbp_mmhg >= 80 ~ "Elevated",
    TRUE ~ "Normal"
  ))

table(raw$bp_category, useNA = "ifany")

#>
#>      Elevated Hypertension      Normal      <NA>
#>          357          996          144          3

```

The first rule is easy to forget and essential. Without it, a patient with a missing systolic pressure and a diastolic pressure of 70 would fail the first two rules (because `NA >= 140` is `NA`, which `case_when()` treats as not true) and fall through to "Normal", a classification based on half the information. With it, the three patients with a missing reading are correctly given a missing category.

Almost two thirds of the patients (996 of 1,500) have a measured blood pressure in the hypertensive

range. This is not surprising: the study recruited adults attending primary care, many of whom are being followed for known hypertension. Let us check the derived variable against its definition, both numerically and graphically.

```
raw |>
  group_by(bp_category) |>
  summarise(n = n(),
            sbp_min = min(sbp_mmhg), sbp_max = max(sbp_mmhg),
            dbp_min = min(dbp_mmhg), dbp_max = max(dbp_mmhg)) |>
  knitr::kable(caption = "Blood pressure ranges within each derived category.")
```

Table 2.10: Blood pressure ranges within each derived category.

bp_category	n	sbp_min	sbp_max	dbp_min	dbp_max
Elevated	357	90	139	56	89
Hypertension	996	90	201	59	121
Normal	144	90	129	57	79
NA	3	NA	NA	NA	NA

In the "Normal" group, all systolic pressures are below 130 and all diastolic pressures below 80, exactly as the definition requires. In the "Elevated" group the maximum systolic is 139 and the maximum diastolic 89, so no hypertensive reading has slipped into it. The "Hypertension" group needs more careful reading: its minimum systolic pressure is 90 and its minimum diastolic pressure 59, which might look like an error. It is not, because the two minima come from different patients. Someone with a systolic pressure of 90 is in the group because of a diastolic pressure of at least 90, and someone with a diastolic pressure of 59 because of a systolic pressure of at least 140. Column-wise minima cannot check an "or" rule; a picture can. (Note also that the categories are listed alphabetically, because `bp_category` is still text; we fix the order in Section 2.10.)

```
raw |>
  filter(!is.na(bp_category)) |>
  mutate(bp_category = fct_relevel(bp_category, "Normal", "Elevated")) |>
  ggplot(aes(x = dbp_mmhg, y = sbp_mmhg, colour = bp_category)) +
  geom_point(alpha = 0.5, size = 1) +
  geom_hline(yintercept = c(130, 140), linetype = "dashed", colour = "grey40") +
  geom_vline(xintercept = c(80, 90), linetype = "dashed", colour = "grey40") +
  scale_colour_manual(values = c("#2E8B57", "#E0A800", "#B03A2E")) +
  labs(x = "Diastolic blood pressure (mmHg)",
       y = "Systolic blood pressure (mmHg)", colour = "Category",
       title = "Derived blood-pressure categories")
```

The points form three clean regions separated along the threshold lines: green in the lower-left corner (below 130 and below 80), amber in an L-shaped band and red everywhere above or to the right of the 140/90 lines. No point of one colour sits in another colour's region. This kind of picture is one of the quickest ways to catch an error in a derivation, such as & written where | was intended.

Clinical interpretation

`bp_category` describes a single measured blood pressure on the day of the survey, not a diagnosis. A patient on effective treatment may have a "Normal" reading and still have hypertension; a patient with "white-coat" hypertension may have a single high reading without

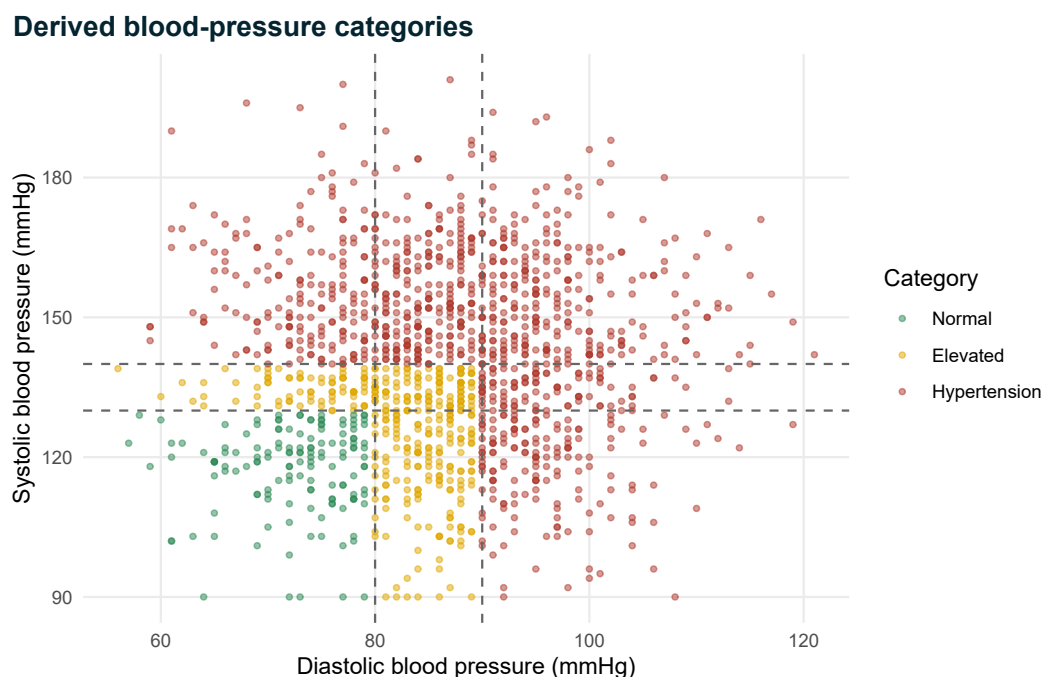


Figure 2.4: Systolic against diastolic blood pressure, coloured by the derived blood-pressure category. Dashed lines mark the 130/80 and 140/90 mmHg thresholds; the categories form the expected L-shaped regions.

the disease. The study's diagnosis variable is `htn_diagnosed`, and the primary analysis of treatment uptake is restricted to patients with `htn_diagnosed == "Yes"`. Keep the two concepts distinct when you describe results.

2.9 Dates

2.9.1 Why dates need parsing

Dates are deceptively difficult. The same day can be written as `2024-02-01`, `01/02/2024`, `02/01/2024`, `1-Feb-2024` or `1 February 2024`, and some of these are ambiguous: `01/02/2024` is 1 February in most of the world and 2 January in the United States. Text that looks like a date is not a date to R until it has been **parsed** into R's Date type, which stores the number of days since 1 January 1970. Only then can dates be sorted chronologically, subtracted to give intervals, or grouped by month.

Let us look at the formats in `enroll_date`. A quick way to see the patterns, rather than the values, is to replace every digit with 9.

```
head(raw$enroll_date, 8)
```

```
#> [1] "01/02/2024" "2024-09-03" "2024-11-26" "2024-10-23" "2024-06-03"
#> [6] "2024-04-22" "08/08/2024" "18/02/2024"
```

```
# Replace digits by 9 to reveal the formats used
raw |>
  mutate(pattern = str_replace_all(enroll_date, "[0-9]", "9"),
         pattern = str_replace(pattern, "[A-Za-z]{3}", "Mon")) |>
  count(pattern)
```

```
#> # A tibble: 3 × 2
#>   pattern      n
#>   <chr>    <int>
#> 1 99-Mon-9999    103
#> 2 99/99/9999    347
#> 3 9999-99-99   1050
```

Three formats are present: the international standard YYYY-MM-DD (1,050 records), the day-first format DD/MM/YYYY (347 records) and a format with an abbreviated month name, DD-Mon-YYYY (103 records). Because the study was conducted in a country that writes dates day first, we interpret 01/02/2024 as 1 February 2024.

2.9.2 Parsing with lubridate

The `lubridate` package makes parsing dates straightforward (Grolemund and Wickham 2011). Its functions are named after the order of the components: `ymd()` parses year-month-day, `dmy()` day-month-year, and so on. For a column that mixes formats, `parse_date_time()` accepts several **orders** and tries them in turn for each value. It returns a date-time, which `as_date()` converts to a plain date.

```
raw <- raw |>
  mutate(enroll_date = parse_date_time(
    enroll_date,
    orders = c("ymd", "dmy", "d-b-Y") # year-first, day-first, day-Mon-year
  ) |> as_date())

class(raw$enroll_date)
```

```
#> [1] "Date"
```

```
sum(is.na(raw$enroll_date)) # values that failed to parse
```

```
#> [1] 0
```

```
range(raw$enroll_date)
```

```
#> [1] "2024-01-08" "2024-12-02"
```

All 1,500 dates were parsed, none failed, and the column now has class `Date`. The range, 8 January to 2 December 2024, is consistent with a study conducted during a single calendar year. A date outside the study period (for example, in 2042 because of a typing error, or in 1924 because of a two-digit year) would show up immediately in this range and should be investigated.

Common mistake

Always check how ambiguous dates were interpreted. If `orders` had listed "mdy" before "dmy", then 01/02/2024 would have been read as 2 January, and dates such as 15/03/2024 (which cannot be month-first) would still be read correctly, so the error would be silent for most values. Decide the convention from knowledge of where and how the data were recorded, list that order explicitly, and check the range and a few individual values afterwards.

2.9.3 Using dates

Once parsed, dates behave like numbers on a time scale. The figure below counts enrolments by month, using `floor_date()` to round each date down to the first day of its month.

```
raw |>
  mutate(month = floor_date(enroll_date, "month")) |>
  count(month) |>
  ggplot(aes(x = month, y = n)) +
  geom_col(fill = "#04242F") +
  scale_x_date(date_labels = "%b", date_breaks = "1 month") +
  labs(x = "Month of enrolment (2024)", y = "Patients enrolled",
       title = "Recruitment over time")
```

Recruitment over time

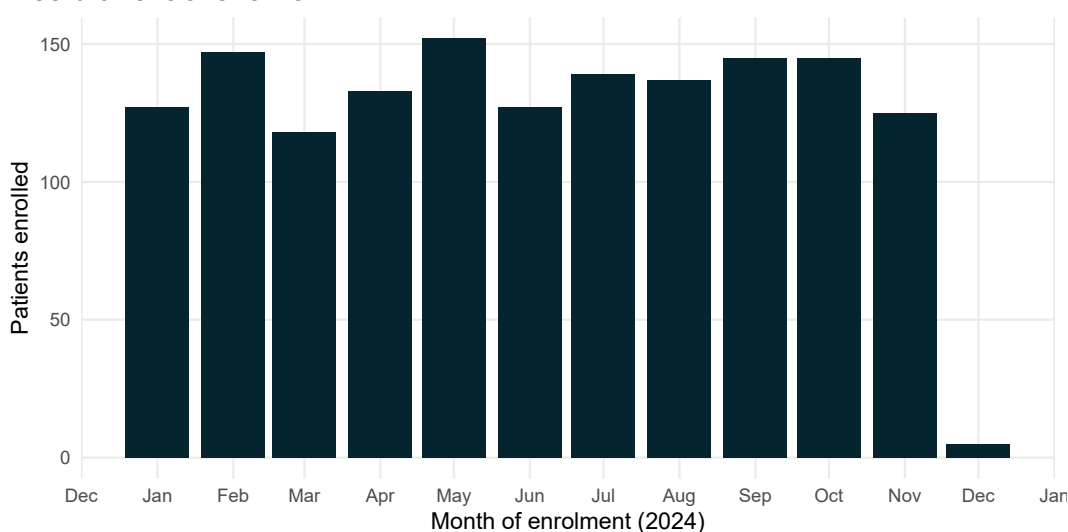


Figure 2.5: Number of patients enrolled per month in 2024. Between 118 and 152 patients were enrolled in each month from January to November; recruitment ended in the first days of December.

Recruitment was steady from January to November, and the handful of December enrolments reflects the end of recruitment on 2 December rather than a problem. A plot like this is a useful data-quality check in its own right: a month with no enrolments, or a sudden spike, might reflect a real event (a facility closure, a recruitment drive) or a problem with the dates.

2.10 Factors, level order and reference levels

2.10.1 Why factors

The final structural step is to convert categorical variables from text to **factors**. A factor stores a set of permitted categories, called **levels**, in a specified order. Three things depend on that order:

1. **Tables and figures.** Frequency tables and bar charts list categories in level order. Left to itself, R orders levels alphabetically, which gives "Heavy", "Moderate", "None" for alcohol and "Current", "Former", "Never" for smoking: orders that make tables harder to read.
2. **Ordinal information.** For education and physical activity the order is part of the meaning. An **ordered factor** records that None < Primary < Secondary < Tertiary, so that R can sort and compare categories correctly.
3. **Reference levels in regression.** When a factor is used as a predictor in a regression model, the first level becomes the **reference category**, and each coefficient compares another level with it. When a binary factor is the outcome of a logistic regression, R models the probability of the *second* level. Getting the order wrong produces odds ratios that are the reciprocal of the ones intended.

2.10.2 Setting factor levels

We now create the final analysis dataset. For nominal variables with no natural order and no obvious reference (facility, occupation, marital_status), alphabetical order is acceptable. For every other categorical variable, we state the levels explicitly. For the binary variables, "No" comes first so that it is the reference and models estimate the odds of "Yes". For smoking and alcohol the "unexposed" category (Never, None) is first, and for blood pressure the "Normal" category is first.

```
analysis_data <- raw |>
  mutate(
    facility      = factor(facility),
    sex           = factor(sex, levels = c("Female", "Male")),
    residence     = factor(residence, levels = c("Rural", "Urban")),
    education     = factor(education,
                          levels = c("None", "Primary",
                                     "Secondary", "Tertiary"),
                          ordered = TRUE),
    occupation    = factor(occupation),
    marital_status = factor(marital_status),
    physical_activity = factor(physical_activity,
                              levels = c("Low", "Moderate", "High"),
                              ordered = TRUE),
    smoking       = factor(smoking,
                          levels = c("Never", "Former", "Current")),
    alcohol       = factor(alcohol,
                          levels = c("None", "Moderate", "Heavy")),
    bp_category   = factor(bp_category,
                          levels = c("Normal", "Elevated",
                                     "Hypertension")),
    # Binary predictors and outcomes: reference level "No" first
    health_insurance = factor(health_insurance, levels = c("No", "Yes")),
    family_history_htn = factor(family_history_htn, levels = c("No", "Yes")),
    diabetes         = factor(diabetes, levels = c("No", "Yes")),
    htn_diagnosed     = factor(htn_diagnosed, levels = c("No", "Yes")),
```

```

    treatment_uptake = factor(treatment_uptake, levels = c("No", "Yes")),
    adherence        = factor(na_if(adherence, ""),
                              levels = c("Poor", "Good")),
    bp_controlled     = factor(na_if(bp_controlled, ""),
                              levels = c("No", "Yes"))
  )

glimpse(analysis_data)

#> Rows: 1,500
#> Columns: 38
#> $ patient_id      <chr> "PHC-1224", "PHC-1169", "PHC-1391", "PHC-01...
#> $ facility        <fct> Igoma HC, Kisesa HC, Bugando PHC, Ilemela H...
#> $ enroll_date     <date> 2024-02-01, 2024-09-03, 2024-11-26, 2024-1...
#> $ age             <dbl> 74, 56, 54, 33, 86, 70, 45, 59, 46, 61, 70,...
#> $ sex            <fct> Female, Female, Female, Female, Male, Male,...
#> $ residence       <fct> Urban, Urban, Urban, Urban, Urban, Urban, U...
#> $ education       <ord> Primary, Primary, None, Primary, Primary, P...
#> $ occupation      <fct> Trader, Professional, Farmer, Farmer, Unemp...
#> $ marital_status  <fct> Married, Married, Single, Single, Married, ...
#> $ health_insurance <fct> Yes, No, No, Yes, No, No, No, No, No, No, Y...
#> $ height_cm       <dbl> 164.6, 160.0, 152.8, 166.2, 182.0, 175.6, 1...
#> $ weight_kg       <dbl> 61.3, 68.3, 49.9, 78.5, 71.6, 97.4, 106.7, ...
#> $ bmi            <dbl> 22.6, 26.7, 21.4, 28.4, 21.6, 31.6, 38.9, 2...
#> $ smoking        <fct> Former, Current, Never, Never, Former, Neve...
#> $ alcohol        <fct> None, Moderate, Moderate, None, Heavy, Heav...
#> $ physical_activity <ord> Low, Low, Moderate, Low, High, Moderate, Mo...
#> $ family_history_htn <fct> Yes, Yes, Yes, Yes, No, No, No, No, Yes, No...
#> $ diabetes        <fct> No, No, No, No, No, No, No, No, No, No, No,...
#> $ sbp_mmhg        <dbl> 140, 185, 147, 124, 131, 153, 162, 108, 152...
#> $ dbp_mmhg        <dbl> 92, 91, 85, 102, 95, 95, 100, 100, 87, 69, ...
#> $ total_chol_mmol_l <dbl> NA, 5.4, 5.4, 5.9, 4.8, 3.6, 5.3, 2.9, 6.0,...
#> $ hdl_mmol_l       <dbl> 1.93, 0.70, 1.67, 0.65, 1.69, 1.20, 1.83, 1...
#> $ ldl_mmol_l       <dbl> 3.5, NA, 2.3, 4.1, 2.4, 1.4, 2.2, NA, 3.4, ...
#> $ triglycerides_mmol_l <dbl> 1.0, 2.1, 2.7, 1.7, 2.0, 1.3, 1.0, 1.5, 2.5...
#> $ fasting_glucose_mmol_l <dbl> 6.6, 5.5, 6.0, 5.2, 5.1, 5.8, 5.5, 4.1, 6.2...
#> $ creatinine_umol_l <dbl> 61, 66, 75, 86, 87, 72, 103, 57, 98, 76, 99...
#> $ sodium_mmol_l    <dbl> 134, 134, 139, 140, 142, 138, 141, 141, 140...
#> $ potassium_mmol_l <dbl> 4.1, 3.6, 4.4, 4.4, 4.2, 4.0, 4.5, 5.0, 4.7...
#> $ knowledge_score  <dbl> 14, 7, 9, 12, NA, 12, 17, 8, 13, NA, 11, 6,...
#> $ distance_to_facility_km <dbl> 2.9, 10.5, 2.4, 2.2, 5.4, 6.7, 0.2, 0.4, 5...
#> $ comorbidity_count <dbl> 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0...
#> $ htn_diagnosed    <fct> Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes, Yes...
#> $ months_since_diagnosis <dbl> 95, 87, 116, 31, 80, 28, 6, 96, 114, 53, 39...
#> $ treatment_uptake <fct> Yes, No, No, Yes, No, No, Yes, No, No, Yes,...
#> $ adherence        <fct> Poor, NA, NA, Good, NA, NA, Good, NA, NA, P...
#> $ bp_controlled    <fct> No, NA, NA, No, NA, NA, No, NA, NA, Yes, Ye...
#> $ bmi_cat          <fct> Normal, Overweight, Normal, Overweight, Nor...
#> $ bp_category      <fct> Hypertension, Hypertension, Hypertension, H...

```

`glimpse()` now shows the final structure: 1,500 rows and 38 columns (the original 36 plus `bmi_cat` and `bp_category`). Identifiers are character, measurements are double, `enroll_date` is a date, and every categorical variable is a factor, shown as `<fct>` or, for the two ordered factors, `<ord>`.

Common mistake

When you give `factor()` a `levels` argument, any value that does not match one of the levels *exactly* becomes `NA`, without a warning. If `residence` still contained " Urban" with a leading space, or "urban" in lower case, those patients would silently lose their residence. This is why trimming and recoding come before factor conversion, and why the final audit compares missing counts before and after.

2.10.3 Checking levels

It is worth printing the levels of the key variables to confirm the order.

```
levels(analysis_data$treatment_uptake)
```

```
#> [1] "No" "Yes"
```

```
levels(analysis_data$education)
```

```
#> [1] "None" "Primary" "Secondary" "Tertiary"
```

```
levels(analysis_data$bp_category)
```

```
#> [1] "Normal" "Elevated" "Hypertension"
```

```
is.ordered(analysis_data$education)
```

```
#> [1] TRUE
```

The outcome has "No" as its first level, so "Yes" is the event modelled; education is an ordered factor running from "None" to "Tertiary"; and blood-pressure categories run from "Normal" to "Hypertension".

2.10.4 Why the reference level matters

A short demonstration shows the consequence of the reference level for a regression model. Chapter 5 covers logistic regression properly; here we only look at the names and values of the coefficients. We model treatment uptake by sex among diagnosed patients, first with "Female" as the reference and then with "Male".

```
diagnosed <- filter(analysis_data, htn_diagnosed == "Yes")
```

```
# Reference = Female (the first level)
```

```
fit_f <- glm(treatment_uptake ~ sex, family = binomial, data = diagnosed)
round(exp(coef(fit_f)), 2)
```

```
#> (Intercept)    sexMale
#>          1.00          0.73
```

```
# Reference = Male, using relevel()
fit_m <- glm(treatment_uptake ~ relevel(sex, ref = "Male"),
             family = binomial, data = diagnosed)
round(exp(coef(fit_m)), 2)
```

```
#>               (Intercept) relevel(sex, ref = "Male")Female
#>                   0.73                        1.37
```

In the first model the coefficient is labelled `sexMale`: it is the odds ratio for men compared with women. In the second it compares women with men, and its value is the reciprocal of the first (one divided by the other). Both models describe exactly the same data; they answer the comparison in opposite directions. In a paper, an odds ratio reported without its reference category is uninterpretable, and an odds ratio computed with an unintended reference can be reported the wrong way round. Setting reference levels deliberately during cleaning, and stating them in every table, prevents both errors.

Good practice

Choose as the reference the category that makes the comparison most natural to the reader: the unexposed group (never smokers, no diabetes), the most common group, or the group with the lowest level of an ordinal variable. Avoid a very small reference category, because every comparison with it will be imprecise.

2.11 Final validation, saving and the cleaning log

2.11.1 A final missing-data audit

With all the cleaning done, we repeat the missing-data audit on the analysis dataset.

```
missing_audit(analysis_data) |>
  filter(n_missing > 0) |>
  knitr::kable(caption = "Missing values in the clean analysis dataset.")
```

Compare this with the audit of the raw data. The structurally missing variables `adherence` and `bp_controlled` now have 992 missing values instead of 995, because three duplicate records have gone. BMI has 47 missing values instead of 60, because recomputing it recovered values that were missing only in the supplied column. Weight has gained one missing value (the 7 kg record), and age, systolic pressure, diastolic pressure and height have gained the values set to missing during validation. The derived variables `bmi_cat` and `bp_category` have exactly as many missing values as their inputs imply. No variable has gained missing values unexpectedly, which confirms that the factor conversion did not lose any data.

2.11.2 Checking structural missingness

We claimed that `adherence` and `bp_controlled` are missing exactly for patients who are not on treatment. That is a testable claim.

```
analysis_data |>
  count(treatment_uptake,
        adherence_missing = is.na(adherence),
```

Table 2.11: Missing values in the clean analysis dataset.

variable	n_missing	pct
adherence	992	66.1
bp_controlled	992	66.1
ldl_mmol_l	105	7.0
total_chol_mmol_l	90	6.0
fasting_glucose_mmol_l	75	5.0
distance_to_facility_km	60	4.0
bmi	47	3.1
bmi_cat	47	3.1
weight_kg	46	3.1
smoking	45	3.0
knowledge_score	45	3.0
education	30	2.0
bp_category	3	0.2
age	2	0.1
sbp_mmhg	2	0.1
height_cm	1	0.1
dbp_mmhg	1	0.1

```
control_missing = is.na(bp_controlled))
```

```
#> # A tibble: 2 × 4
#>   treatment_uptake adherence_missing control_missing    n
#>   <fct>          <lgl>              <lgl>          <int>
#> 1 No            TRUE              TRUE           992
#> 2 Yes           FALSE             FALSE           508
```

All 992 untreated patients have both variables missing, and all 508 treated patients have both recorded. The missingness is entirely structural, and analyses of adherence and blood-pressure control will be restricted to treated patients, where these variables are complete.

2.11.3 Automated checks

The checks we have made by eye can also be written as **assertions**: statements that must be true of the clean data. `stopifnot()` does nothing if all its conditions are true and stops with an error if any is false. Putting assertions at the end of a cleaning script means that if a future version of the raw file introduces a new problem, the script fails loudly instead of producing quietly wrong data.

```
ad <- analysis_data                                # short name for the checks
stopifnot(
  nrow(ad) == 1500,                                # one row per patient
  n_distinct(ad$patient_id) == nrow(ad),           # IDs are unique
  all(between(ad$age, 18, 110), na.rm = TRUE),     # ages in range
  all(between(ad$sbp_mmhg, 70, 260), na.rm = TRUE), # SBP in range
  all(levels(ad$treatment_uptake) == c("No", "Yes")), # reference is "No"
  !anyNA(ad$sex),                                   # sex fully recoded
)
```

```
!anyNA(ad$enroll_date),                                # every date parsed
inherits(ad$enroll_date, "Date")
)
"All checks passed"
```

```
#> [1] "All checks passed"
```

2.11.4 Saving the clean data

The clean data are saved in two formats. The `.rds` format is R's own: it stores a single R object exactly, including factor levels, their order, ordered factors and dates. Reloading it with `readRDS()` gives back the identical object. A `.csv` copy is convenient for colleagues who use other software, but it stores only text, so factor levels and date types must be re-declared when it is read back. In this book the project's copy of the clean data already exists in `Data/analysis_data.rds`, so we demonstrate saving to R's temporary directory. In your own cleaning script, you would save to your project's data folder, as in the commented lines below.

```
# In a real project:
#   saveRDS(analysis_data, "Data/analysis_data.rds")
#   write_csv(analysis_data, "Data/analysis_data.csv")

# Here we save to a temporary folder to leave the project's files untouched
out_rds <- file.path(tempdir(), "analysis_data.rds")
out_csv <- file.path(tempdir(), "analysis_data.csv")
saveRDS(analysis_data, out_rds)
write_csv(analysis_data, out_csv)

# Reload and confirm nothing was lost in the round trip
reloaded <- readRDS(out_rds)
identical(reloaded, analysis_data)
```

```
#> [1] TRUE
```

`identical()` returns `TRUE`: the saved and reloaded objects are the same in every detail. The same would not be true of the CSV, whose factors would come back as text.

Good practice

Every later analysis script should begin with `analysis_data <- readRDS("Data/analysis_data.rds")` and nothing else from this chapter. If you discover a new data problem during analysis, fix it in the cleaning script, rerun the script to regenerate the clean file, and rerun the analyses. Never patch the clean data inside an analysis script.

2.11.5 Is the pipeline reproducible?

The ultimate test of the cleaning contract is reproducibility: does running our code on the raw file give exactly the dataset that the rest of the book uses?

```
reference <- readRDS("Data/analysis_data.rds")
```

R

```
dim(analysis_data)
```

```
#> [1] 1500 38
```

```
dim(reference)
```

R

```
#> [1] 1500 38
```

```
all.equal(analysis_data, reference)
```

R

```
#> [1] TRUE
```

Both datasets have 1,500 rows and 38 columns, and `all.equal()` reports TRUE: every value, every type and every factor level is the same. Starting from a messy raw file and following the steps of this chapter, anyone with R can regenerate the analysis dataset exactly. That is what reproducibility means in practice.

2.11.6 The cleaning log

Finally, a **cleaning log** records what was done and how many values each step changed. It is the summary a reviewer, a co-author or your future self will read first, and it supplies the numbers needed for the methods section of a paper.

```
cleaning_log <- tribble(
  ~step, ~action, ~records_affected,
  "1. Import", "Declared ' ', 'NA', '999', '-99' as missing",
  sum(na_compare$gained),
  "2. Duplicates", "Removed exact duplicate rows",
  n_before - n_after,
  "3. Whitespace", "Trimmed spaces in text columns (hidden at import)",
  sum(raw_untrimmed$facility != str_trim(raw_untrimmed$facility),
  na.rm = TRUE),
  "4. Sex", "Recoded 8 spellings to Female/Male",
  sum(sex_before != as.character(analysis_data$sex)),
  "5. Validation", "Set out-of-range age, BP, height, weight to NA",
  n_invalid,
  "6. BMI", "Recomputed from validated height and weight",
  sum(is.na(bmi_check$bmi) & !is.na(bmi_check$bmi_recalc)),
  "7. Dates", "Parsed 3 date formats; failures",
  sum(is.na(analysis_data$enroll_date))
)

knitr::kable(cleaning_log,
  col.names = c("Step", "Action", "Records affected"),
  caption = "Cleaning log for the case-study data.")
```

R

Each line of the log corresponds to a section of this chapter. Step 3 counts the facility values that carried hidden spaces (the other text columns had similar numbers). Step 4 counts the 336 values of sex whose spelling changed. Step 6 counts the patients whose BMI was missing in the supplied

Table 2.12: Cleaning log for the case-study data.

Step	Action	Records affected
1. Import	Declared "", 'NA', '999', '-99' as missing	97
2. Duplicates	Removed exact duplicate rows	3
3. Whitespace	Trimmed spaces in text columns (hidden at import)	108
4. Sex	Recoded 8 spellings to Female/Male	336
5. Validation	Set out-of-range age, BP, height, weight to NA	7
6. BMI	Recomputed from validated height and weight	56
7. Dates	Parsed 3 date formats; failures	0

column but could be recovered from height and weight. Every number in the log is computed by code rather than typed by hand, so the log updates itself if the raw data change and the script is run again.

2.12 Summary

Data cleaning turns a raw file, with all the imperfections of real data collection, into an analysis-ready dataset. It is part of the science, not a chore to be rushed, and it must be done by code so that every decision is visible and repeatable. In this chapter we imported the raw case-study file while declaring the missing-value codes, which recovered 97 hidden missing values and removed impossible negative cholesterol values from the mean. We met the three mechanisms of missing data, MCAR, MAR and MNAR, saw why complete-case analysis is an assumption rather than a neutral default, and produced a missing-data audit that separated genuinely missing values from structurally missing ones. We removed three duplicate records, trimmed hidden spaces, collapsed eight spellings of sex and three codings of five Yes/No variables, and set seven impossible measurements to missing using explicit plausible ranges. We recomputed BMI from validated inputs, derived BMI and blood-pressure categories and checked them against their definitions, parsed three date formats, and set factor levels and reference categories deliberately. Finally we audited the result, wrote assertions, saved the data and showed that our pipeline reproduces the book's analysis dataset exactly.

Key points

- Never edit raw data by hand. A cleaning script that reads the raw file and saves a clean file is the reproducible record of every decision.
- Declare missing-value codes such as 999 and -99 at import with `na = c(...)`, after checking that no variable could genuinely take those values.
- Missing data are MCAR, MAR or MNAR. Complete-case analysis is unbiased under MCAR but can be biased otherwise; multiple imputation is the standard method under MAR. Distinguish structurally missing ("not applicable") values from genuinely missing ones.
- Detect duplicates by comparing rows with distinct IDs; remove exact duplicates with `distinct()`, investigate conflicting ones, and report what was removed.
- Standardise text with `str_trim()` and `str_to_lower()`, recode with `case_when()` (map-

ping unrecognised values to NA), and use helper functions with `across()` for repeated tasks. Always cross-tabulate old against new values.

- Validate continuous variables against explicit plausible ranges and set impossible values, not whole patients, to NA.
- Recompute derived variables from validated inputs and check every derived category against its definition.
- Parse dates with `lubridate`, deciding ambiguous orders from knowledge of the data, and check the resulting range.
- Set factor levels and reference categories deliberately: they determine table layout and the direction of every regression comparison.
- End with an audit, assertions, a saved `.rds` file and a cleaning log.

2.13 Further reading

- Wickham (2014): the paper that defined tidy data; short, clear and the basis of the tidyverse approach to data organisation used throughout this book.
- Broman and Woo (2018): practical, widely cited rules for organising data in spreadsheets, invaluable if you are involved in designing data collection rather than only analysing the result.
- Wilson et al. (2017): a concise set of "good enough" practices for data management, code and project organisation that any research group can adopt.
- Sterne et al. (2009): an accessible guide for clinical researchers to the potential and pitfalls of multiple imputation, with recommendations for reporting missing data.
- Little and Rubin (2019): the standard reference on the theory of missing data, for readers who want to go beyond the introduction given here.

2.14 Exercises

Exercise 2.1

Missing-value codes. Import `Data/hypertension_phc_raw.csv` twice: once with `read_csv()` defaults and once with `na = c("", "NA", "999", "-99")`.

1. For fasting glucose, report the mean, median and maximum under each import. Which summary is most affected by the sentinel, and why?
2. How many missing values does `fasting_glucose_mmol_l` have under each import?
3. Suppose a laboratory sent creatinine values in which 999 was a genuine measurement. How would you treat 999 as missing in glucose but not in creatinine?

Exercise 2.2

Duplicates. Using the raw import (with missing codes declared):

1. Use `count()` to find the patient IDs that occur more than once, and `janitor::get_dupes(patient_id)` to list their records.
2. Confirm that the duplicated records are exact copies by comparing `sum(duplicated(raw))` with the number of surplus IDs.
3. Write one sentence, suitable for the methods section of a paper, that reports how duplicates were handled.

Exercise 2.3

Standardising categories. Starting again from the raw import:

1. Tabulate `health_insurance` and `htn_diagnosed` with `useNA = "ifany"`. How many different codes does each use?
2. Apply the `to_yn()` helper to both variables and cross-tabulate old against new values to check the mapping.
3. What proportion of patients in the clean data have health insurance, and what proportion have been diagnosed with hypertension?

Exercise 2.4

Cross-variable consistency. Load the clean data with `analysis_data <- readRDS("Data/analysis_data.rds")`.

1. Count the patients whose diastolic pressure is greater than or equal to their systolic pressure. Is that physiologically possible?
2. Pulse pressure is systolic minus diastolic pressure. Summarise it, and count patients with a pulse pressure below 20 mmHg.
3. Check another consistency rule from the data dictionary: patients not diagnosed with hypertension should have `months_since_diagnosis` equal to 0, and adherence should be recorded only for treated patients. Do the data satisfy both rules?

Exercise 2.5

BMI and cut-points. Using `analysis_data`:

1. Recreate `bmi_cat` with `cut()` and `right = FALSE`, so that each interval includes its lower limit as in the WHO definition.
2. Cross-tabulate the two versions. How many patients change category, and which BMI values are responsible?
3. Create an age-group variable with the groups 18–39, 40–59 and 60 or over, and check it by summarising the minimum and maximum age in each group.

Exercise 2.6

Dates. Starting from the raw file:

1. Use `str_detect()` to count how many `enroll_date` values are in each of the three formats.
2. Parse the dates using the orders `c("ymd", "mdy", "d-b-Y")`, that is, with month-first instead of day-first for the slash format. How many dates fail to parse, and how many parsed dates differ from those in `analysis_data`?
3. Using `analysis_data`, add a column `enroll_quarter` with `quarter(enroll_date)` and count the patients enrolled in each quarter.

Exercise 2.7

Exploring the missingness mechanism. Using `analysis_data`:

1. Create an indicator `chol_missing` that is `TRUE` when `total_chol_mmol_l` is missing. Compare the percentage with missing cholesterol between patients diagnosed and not diagnosed with hypertension, and between rural and urban residents.
2. Compare the mean age of patients with and without missing cholesterol.
3. On the basis of these results, is MCAR plausible for cholesterol? Can these data tell you whether cholesterol is MNAR? Explain.

Exercise 2.8

Challenge: a cleaning function. Write a function `clean_htn(path)` that takes the path of the raw CSV file, performs all the cleaning steps of this chapter in the same order, and returns the clean data.

1. Run `clean_htn("Data/hypertension_phc_raw.csv")` and use `all.equal()` to compare the result with `readRDS("Data/analysis_data.rds")`.
2. Add at least three `stopifnot()` assertions inside the function, just before it returns, so that it fails if the raw data ever contain a new problem.
3. Why is a function like this more useful than a long script when a corrected raw file arrives?

Descriptive Statistics, Tables and Figures

Every clinical paper, however sophisticated its later analyses, begins with the same humble paragraph: *who were the patients?* Before a reader can judge whether a regression model, an odds ratio or a p-value means anything, they need to know how old the participants were, how many were women, how high their blood pressure was, how many had diabetes and how much information was missing. That paragraph, and the "Table 1" that accompanies it, is the foundation on which every inferential claim rests. If the description is wrong, or careless, or hides the shape of the data, nothing that follows can be trusted.

This chapter teaches you to describe a clinical dataset properly, both with numbers and with pictures. We start from the clean analysis file produced in Chapter 2 and work through the theory of summary statistics (what a mean, a standard deviation or an interquartile range actually measures, and when each is the right choice), then show how to compute them in R, how to build a manuscript-ready Table 1 with **gtsummary**, and how to draw honest, publication-quality figures with **ggplot2**. Along the way we meet the standard error and the confidence interval, the bridge from describing a sample to making statements about a population, which is the subject of Chapter 4.

Learning objectives

- Explain why careful description precedes any statistical test, and what a "Table 1" must contain.
- Match each type of variable (continuous, discrete, binary, nominal, ordinal) to appropriate numerical and graphical summaries.
- Define and compute the mean, median, mode, range, variance, standard deviation, quantiles, interquartile range and coefficient of variation, and explain their properties.
- Judge the shape of a distribution (symmetry, skewness, normality) and decide between mean (SD) and median (IQR).
- Avoid the `na.rm` trap and report the number of observations behind every summary.
- Produce grouped summaries with `dplyr::group_by()` and `summarise()`.
- Compute and interpret the standard error and 95% confidence interval for a mean and for a proportion.
- Build frequency tables and cross-tabulations, and choose correctly between row and column percentages.
- Build a manuscript Table 1 with `gtsummary::tbl_summary()`.

- Draw histograms, density plots, bar charts, box and violin plots, scatter plots and faceted plots with `ggplot2`, following principles of good scientific graphics.
- Export tables and figures for a manuscript.

3.1 Why describe before testing?

Descriptive statistics summarise the data you actually have. They make no claim about a wider population and involve no hypothesis; their job is to *show*. Inferential statistics (confidence intervals, hypothesis tests, regression models) go one step further and use the sample to say something about the population it came from. It is tempting to rush to the inferential step, because that is where the “results” seem to live. Experienced analysts do the opposite: they spend a large share of their time on description, for at least four reasons.

- 1. Description is the first result.** Reporting guidelines for observational research require it. Item 13 of the STROBE statement asks authors to report the numbers of individuals at each stage of the study, and item 14 asks for the characteristics of study participants (demographic, clinical, social) together with the number of participants with missing data for each variable of interest (Elm et al. 2007). In practice this becomes the first paragraph of the Results section and Table 1 of the paper.
- 2. Description reveals problems.** A maximum age of 200, a systolic blood pressure of 0, a laboratory value of 999 that is really a missing-value code: these are discovered by summarising, not by testing. Chapter 2 cleaned the obvious errors, but describing the clean data is the final check that the cleaning worked.
- 3. Description guides the choice of analysis.** Whether a variable is symmetric or skewed, whether a category is so rare that it must be merged with another, whether a predictor has 30% missing values: these facts decide which tests and models are appropriate later (Altman 1991; Kirkwood and Sterne 2003).
- 4. Description lets readers judge generalisability.** A treatment-uptake rate in primary-care patients aged 50 on average, mostly women and mostly uninsured, may not apply to a hospital clinic of younger insured men. Readers can only judge this if you tell them who was studied.

Clinical interpretation

Think of Table 1 as the patient list a clinician would want before reading your conclusions. It answers “are these patients like mine?” A trial or survey whose participants are poorly described cannot be applied safely to practice, however small its p-values.

3.1.1 Setting up

As in every chapter, we load the packages and read the clean analysis file saved at the end of Chapter 2. Because the primary outcome, `treatment_uptake`, is only meaningful for patients who have been diagnosed with hypertension, we also create the analysis population `diagnosed` now and use it whenever the outcome is involved.

```

library(tidyverse) # dplyr, ggplot2, tidyr, readr, ...
library(gtsummary) # Table 1 and other summary tables
library(janitor)    # tabyl() and adorn_*() for frequency tables
library(knitr)      # kable() for simple tables
library(scales)     # percent formatting for axes and labels
library(patchwork)  # combine several ggplots into one figure

# Read the analysis-ready data created in Chapter 2
analysis_data <- readRDS("Data/analysis_data.rds")

# Analysis population for the primary outcome: diagnosed hypertensives
diagnosed <- analysis_data |>
  filter(htn_diagnosed == "Yes")

# One colour defined once and reused in every figure
teal <- "#0D7377"

c(all_patients = nrow(analysis_data), diagnosed = nrow(diagnosed))

```

```

#> all_patients    diagnosed
#>           1500         1089

```

The full dataset contains 1500 adults attending six primary healthcare facilities, of whom 1089 had been diagnosed with hypertension. Keep both numbers in mind: the first describes everyone enrolled, the second is the denominator for statements about treatment uptake. A common source of confusion in manuscripts is a percentage whose denominator is never stated; we will be explicit throughout.

Common mistake

The data are simulated for teaching. Every number in this chapter illustrates a method; none of them is a real clinical finding about hypertension care in any country or facility.

3.2 Types of variables and the summaries that match them

The right summary depends on the kind of variable. Chapter 2 introduced variable types from the point of view of storage in R (numeric, character, factor). Here we look at them from the point of view of *measurement*, which is what decides the statistics (Altman 1991; Kirkwood and Sterne 2003, ch. 2).

- **Numerical (quantitative) variables** take numbers whose differences are meaningful.
 - *Continuous* variables can, in principle, take any value in a range: age, systolic blood pressure (SBP), body mass index (BMI), cholesterol, glucose, distance to the facility.
 - *Discrete* variables are counts: number of comorbidities, number of visits. They are often summarised like continuous variables when they take many values, and like categorical variables when they take only a few.
- **Categorical (qualitative) variables** place each person in one of a set of groups.
 - *Binary* (dichotomous) variables have two categories: diabetes yes/no, treatment uptake yes/no.

- *Nominal* variables have more than two categories with no natural order: facility, occupation, marital status.
- *Ordinal* variables have ordered categories: education (none < primary < secondary < tertiary), physical activity (low < moderate < high), blood-pressure category.

Table 3.1: Variable types in the case study and the summaries and graphs that suit them.

Type	Examples in the case study	Numerical summary	Graph
Continuous, symmetric	age, sbp_mmhg, bmi	mean (SD)	histogram, density, box plot
Continuous, skewed	distance_to_facility_km	median (IQR)	histogram, box plot
Discrete count	comorbidity_count	median (IQR) or n (%)	bar chart
Binary	diabetes, treatment_uptake	n (%)	bar chart
Nominal	facility, occupation	n (%)	bar chart (sorted)
Ordinal	education, bp_category	n (%), sometimes median	bar chart (in order)

Two warnings follow from this table. First, a number is not always a numerical variable: a patient identifier or a facility code stored as 1, 2, 3 is categorical, and its mean is meaningless. Second, continuous variables are sometimes *grouped* into categories, as BMI is grouped into `bmi_cat` and blood pressure into `bp_category`. Grouping helps clinical communication, because clinicians think in terms of “obese” or “hypertensive”, but it discards information, so a good Table 1 often shows both the continuous measurement and the clinical category.

```
# How R stores each variable: numeric (dbl), factor (fct), ordered (ord)
analysis_data |>
  select(age, sbp_mmhg, comorbidity_count, diabetes, facility, education) |>
  glimpse()
```

```
#> Rows: 1,500
#> Columns: 6
#> $ age          <dbl> 74, 56, 54, 33, 86, 70, 45, 59, 46, 61, 70, 56, 4...
#> $ sbp_mmhg     <dbl> 140, 185, 147, 124, 131, 153, 162, 108, 152, 127,...
#> $ comorbidity_count <dbl> 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0...
#> $ diabetes     <fct> No, No, No, No, No, No, No, No, No, No, No, No, N...
#> $ facility     <fct> Igoma HC, Kisesa HC, Bugando PHC, Ilemela HC, Kis...
#> $ education    <ord> Primary, Primary, None, Primary, Primary, Primary...
```

`glimpse()` confirms that the continuous variables are stored as numbers (`<dbl>`), the nominal and binary variables as factors (`<fct>`) and education as an *ordered* factor (`<ord>`), exactly as set up in Chapter 2. Storage that matches the measurement scale is what allows R, and **gtsummary** in particular, to pick sensible summaries automatically.

3.3 Measures of central tendency

A measure of central tendency (or *location*) answers the question “what is a typical value?”. Three are in common use.

3.3.1 The mean

The arithmetic mean of n observations $x_1, x_2, \dots, x_n$ is their sum divided by n :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

The mean has attractive mathematical properties. It uses every observation; it is the balance point of the data, in the sense that the deviations $x_i - \bar{x}$ sum to zero; and it is the value that minimises the sum of squared deviations $\sum (x_i - c)^2$. Its sampling behaviour is well understood, which is why most of classical statistics (t-tests, analysis of variance, linear regression) is built on means. Its weakness is the other side of the same coin: because every observation contributes in proportion to its size, a single extreme value can pull the mean a long way.

3.3.2 The median

The median is the middle value when the observations are sorted from smallest to largest. If $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$ denote the ordered values, then

$$\text{median} = \begin{cases} x_{((n+1)/2)} & \text{if } n \text{ is odd,} \\ \frac{1}{2} (x_{(n/2)} + x_{(n/2+1)}) & \text{if } n \text{ is even.} \end{cases}$$

Half of the observations lie below the median and half above. Because only the *ranks* of the observations matter, not their exact magnitudes, the median is **robust**: making the largest value ten times larger leaves the median unchanged. The median minimises the sum of absolute deviations $\sum |x_i - c|$, and it is the 50th centile, a link we exploit when we discuss quantiles below.

3.3.3 The mode

The mode is the most frequent value. It is rarely useful for continuous measurements (with enough decimal places every value is unique) but it is the natural "typical value" for a nominal variable: the modal occupation, the modal facility. A distribution with two clear peaks is called *bimodal*, which often signals that two different groups of patients have been mixed together. Base R has no function for the statistical mode (the function `mode()` returns the storage type of an object, a classic source of confusion), so we write a small one.

```
# Statistical mode: the most frequent value(s) of a vector
stat_mode <- function(x) {
  counts <- table(x) # frequency of each value (NA dropped)
  names(counts)[counts == max(counts)] # value(s) with the highest count
}
```

```
stat_mode(analysis_data$education) # modal education level
```

```
#> [1] "Primary"
```

```
stat_mode(analysis_data$occupation) # modal occupation
```

```
#> [1] "Farmer"
```

```
stat_mode(analysis_data$sbp_mmhg) # most common recorded SBP value(s)
```

```
#> [1] "143"
```

Primary education and farming are the modal categories, and these are useful facts about the population. The modal SBP of 143 mmHg is much less useful: with hundreds of possible values, which one happens to occur most often is largely a matter of chance, and it would change with a different sample. For continuous measurements the mode is mainly a tool for spotting oddities. In real clinical data, for example, a modal SBP of 120 or 140 mmHg and an excess of readings ending in 0 or 5 would reveal *digit preference*, the tendency of observers to round manual blood-pressure readings, which is a recognised source of measurement error.

3.3.4 Robustness: what one wrong value does

The difference between the mean and the median is easiest to see with a tiny example. The seven SBP readings below are plausible. We then replace the last one with 700 mmHg, the kind of data-entry error we removed in Chapter 2.

```
sbp_small <- c(128, 132, 135, 140, 141, 145, 150)
sbp_error  <- c(128, 132, 135, 140, 141, 145, 700) # one typing error

c(mean = mean(sbp_small), median = median(sbp_small))
```

```
#>   mean median
#> 138.7  140.0
```

```
c(mean = mean(sbp_error), median = median(sbp_error))
```

```
#>   mean median
#> 217.3  140.0
```

One wrong value moved the mean from about 139 mmHg to about 217 mmHg, a value that does not describe any of the seven patients, while the median stayed at 140 mmHg. This is the practical meaning of robustness, and it is why summaries of *uncleaned* data can be wildly misleading. In clean data the same logic applies to genuine extreme values: a few patients who live 40 km from a clinic pull the mean distance upwards even though most live much closer.

3.3.5 Central tendency in the case-study data

```
# Mean and median of four clinical measurements (missing values removed)
analysis_data |>
  summarise(
    across(c(age, sbp_mmhg, bmi, distance_to_facility_km),
      list(mean = ~ mean(.x, na.rm = TRUE),
           median = ~ median(.x, na.rm = TRUE)))
```

```

) |>
pivot_longer(everything(),
              names_to = c("variable", ".value"),
              names_pattern = "(.*)_(mean|median)") |>
kable(digits = 2, caption = "Mean and median of four continuous variables.")

```

Table 3.2: Mean and median of four continuous variables.

variable	mean	median
age	52.28	52.00
sbp_mmhg	139.29	139.00
bmi	26.35	26.30
distance_to_facility_km	7.82	6.45

For age, SBP and BMI the mean and the median are almost identical (52.3 versus 52 years, 139.3 versus 139 mmHg, 26.35 versus 26.3 kg/m²). That agreement is a first hint that these distributions are roughly symmetric. Distance to the facility is different: the mean of 7.8 km is well above the median of 6.45 km, the signature of a *right-skewed* distribution with a long tail of patients who live far away. We return to this in Section 3.5.

3.4 Measures of spread

Two groups of patients can share the same mean SBP of 140 mmHg while one group ranges from 130 to 150 mmHg and the other from 100 to 200 mmHg. Clinically these are very different populations. Measures of *spread* (dispersion, variability) quantify how far observations typically lie from the centre.

3.4.1 The range

The range is the difference between the largest and smallest values, $x_{(n)} - x_{(1)}$. In practice it is more informative to report the two extremes themselves ("ages ranged from 18 to 95 years"), which also serves as a data check. The range depends on only two observations, so it is extremely sensitive to outliers and it tends to increase with sample size: the more patients you enrol, the more likely you are to meet an extreme one. It is therefore a poor summary of variability, but a good check of plausibility.

3.4.2 Variance and standard deviation

The variance is the average squared deviation from the mean. For a sample it is computed as

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

and the **standard deviation** (SD) is its square root:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Squaring the deviations does two things: it stops positive and negative deviations from cancelling, and it gives large deviations more weight than small ones. Taking the square root brings the result back to the original units, so the SD of SBP is in mmHg, which is why the SD rather than the variance is reported. The divisor $n - 1$ rather than n is the *degrees of freedom*: once the mean has been estimated from the data, only $n - 1$ deviations are free to vary (the last is fixed because the deviations sum to zero). Dividing by $n - 1$ makes s^2 an unbiased estimate of the population variance σ^2 (Kirkwood and Sterne 2003, ch. 4). R's `var()` and `sd()` use $n - 1$.

A useful way to think about the SD: it is roughly the typical distance of a patient's value from the mean. For data that are approximately normally distributed it has a precise interpretation, given in Section 3.5.

3.4.3 Quantiles, quartiles and the interquartile range

A **quantile** divides ordered data in given proportions. The p -th quantile $Q(p)$ is a value below which a proportion p of the observations lie. Quantiles expressed in hundredths are called *centiles* (or percentiles), in quarters *quartiles*, in fifths *quintiles* (Altman and Bland 1994). The three quartiles are

- $Q_1 = Q(0.25)$, the lower quartile (25th centile);
- $Q_2 = Q(0.50)$, the median;
- $Q_3 = Q(0.75)$, the upper quartile (75th centile).

The **interquartile range** is $IQR = Q_3 - Q_1$, the width of the interval that contains the middle half of the data. Like the median it depends only on ranks, so it is robust to extreme values. In papers it is usually better to report the two quartiles than their difference, for example "median 6.5 km (IQR 4.0 to 10.3)", because the pair also shows asymmetry: if the median is much closer to Q_1 than to Q_3 , the distribution is skewed to the right.

There are several conventions for computing a quantile when $p(n - 1)$ is not a whole number. R offers nine (`type = 1` to `9` in `quantile()`); the default, `type 7`, places $Q(p)$ at position $h = (n - 1)p + 1$ in the ordered data and interpolates linearly between the two neighbouring observations. With hundreds of observations the conventions agree to within rounding, but with a handful of values they differ, which explains why SPSS, Stata and R can give slightly different quartiles for the same small sample.

Together, the minimum, Q_1 , median, Q_3 and maximum form the **five-number summary**, which is exactly what a box plot draws.

3.4.4 The coefficient of variation

The SD is expressed in the units of the variable, so the SD of SBP (mmHg) cannot be compared directly with the SD of cholesterol (mmol/L). The **coefficient of variation** (CV) removes the units by expressing the SD as a percentage of the mean:

$$CV = 100 \times \frac{s}{\bar{x}} \%$$

The CV is widely used in laboratory medicine to describe the precision of an assay (an assay with a CV of 3% is more reproducible than one with 10%) and to compare the relative variability of

different measurements. It is only meaningful for variables measured on a ratio scale with a true zero and positive values; it is meaningless for temperature in degrees Celsius or for a score that can be negative.

3.4.5 Spread in R

```
age <- analysis_data$age
range(age, na.rm = TRUE)      # minimum and maximum
```

```
#> [1] 18 95
```

```
var(age, na.rm = TRUE)      # variance (years squared)
```

```
#> [1] 198.7
```

```
sd(age, na.rm = TRUE)      # standard deviation (years)
```

```
#> [1] 14.1
```

```
IQR(age, na.rm = TRUE)      # Q3 - Q1
```

```
#> [1] 19
```

```
quantile(age, probs = c(0, 0.25, 0.5, 0.75, 1), na.rm = TRUE)
```

```
#>   0%   25%   50%   75%  100%
#>   18   43   52   62   95
```

```
fivenum(age)                # Tukey's five-number summary
```

```
#> [1] 18 43 52 62 95
```

Reading the output line by line: ages run from 18 to 95 years, which is plausible for an adult primary-care population and confirms the cleaning in Chapter 2. The variance is about 199 *years squared*, a unit nobody can interpret, which is why we report its square root, the SD of about 14.1 years. The middle half of patients were aged between 43 and 62 years, an IQR of 19 years. `quantile()` with the five probabilities and `fivenum()` give the same numbers here; `fivenum()` uses a slightly different rule (Tukey's hinges) that can differ from `quantile()` in small samples, and it removes missing values automatically.

A compact way to see several variables at once is a long summary table. We reshape the data so that each row is one measurement of one patient, then summarise by variable.

```

continuous_vars <- c("age", "sbp_mmhg", "dbp_mmhg", "bmi",
                    "total_chol_mmol_l", "fasting_glucose_mmol_l",
                    "distance_to_facility_km")

spread_table <- analysis_data |>
  select(all_of(continuous_vars)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "value") |>
  group_by(variable) |>
  summarise(
    n      = sum(!is.na(value)),          # observations actually used
    mean   = mean(value, na.rm = TRUE),
    sd     = sd(value, na.rm = TRUE),
    median = median(value, na.rm = TRUE),
    iqr    = IQR(value, na.rm = TRUE),
    cv_pct = 100 * sd / mean              # coefficient of variation
  )

spread_table |>
  kable(digits = 2,
        caption = "Centre and spread of seven continuous variables.")

```

Table 3.3: Centre and spread of seven continuous variables.

variable	n	mean	sd	median	iqr	cv_pct
age	1498	52.28	14.10	52.00	19.0	26.96
bmi	1453	26.35	4.87	26.30	6.3	18.48
dbp_mmhg	1499	86.00	10.67	86.00	14.0	12.40
distance_to_facility_km	1440	7.82	6.09	6.45	6.3	77.82
fasting_glucose_mmol_l	1425	5.45	0.99	5.40	1.2	18.16
sbp_mmhg	1498	139.29	19.74	139.00	27.0	14.17
total_chol_mmol_l	1410	5.11	0.98	5.10	1.4	19.16

Several lessons are visible in this one table. The column *n* differs between variables: total cholesterol is available for 1,410 patients but age for 1,498, so every summary rests on a different denominator, which a manuscript must report. The SD of SBP is about 20 mmHg and that of diastolic blood pressure (DBP) about 11 mmHg; relative to their means these give similar CVs (14% and 12%). Distance to the facility has a CV near 78%: patients differ enormously in how far they travel, a fact with obvious implications for access to care.

Good practice

Always report the number of non-missing observations alongside a summary statistic. "Mean total cholesterol 5.1 mmol/L (SD 1.0; *n* = 1,410)" tells the reader that 90 patients are not represented, and invites the question of whether they differ from the rest.

3.5 The shape of a distribution

The mean and SD are a sufficient description of a distribution only if its shape is known. For a symmetric, bell-shaped distribution they say almost everything; for a skewed one they can mislead. Before choosing a summary, look at the shape.

3.5.1 Skewness

A distribution is **symmetric** if its left and right halves are mirror images, as for a normal distribution, where the mean, median and mode coincide. It is **right-skewed** (positively skewed) if it has a long tail to the right, as for incomes, hospital lengths of stay, laboratory values such as triglycerides or C-reactive protein, and distance to a facility. In a right-skewed distribution the few large values pull the mean above the median. **Left-skewed** distributions, with a long tail to the left, are rarer in medicine; gestational age at birth is a classic example.

Skewness can be quantified by the sample skewness coefficient,

$$g_1 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^{3/2}},$$

which is zero for a symmetric distribution, positive for right skew and negative for left skew. Cubing the deviations keeps their sign, so a long right tail produces a large positive sum. As a rough guide, values between -0.5 and 0.5 indicate approximate symmetry and values beyond ± 1 marked skew, although a picture is always more informative than a single coefficient.

```
# Sample skewness coefficient g1 (no extra package needed)
skewness <- function(x) {
  x <- x[!is.na(x)]
  m <- mean(x)
  mean((x - m)^3) / mean((x - m)^2)^1.5
}

analysis_data |>
  summarise(across(all_of(continuous_vars), skewness)) |>
  pivot_longer(everything(), names_to = "variable", values_to = "skewness") |>
  arrange(desc(skewness)) |>
  kable(digits = 2, caption = "Sample skewness of seven continuous variables.")
```

Table 3.4: Sample skewness of seven continuous variables.

variable	skewness
distance_to_facility_km	1.51
fasting_glucose_mmol_l	0.57
total_chol_mmol_l	0.10
age	0.07
dbp_mmhg	0.05
bmi	0.04
sbp_mmhg	0.03

Distance to the facility has a skewness of about 1.5, clearly right-skewed. Fasting glucose shows mild right skew (about 0.6), as glucose usually does: a minority of patients with undiagnosed or poorly controlled diabetes produce a tail of high values. The remaining variables have skewness close to zero and can reasonably be treated as symmetric.

3.5.2 The normal distribution

The **normal** (Gaussian) distribution is the familiar symmetric bell curve. It is completely determined by two parameters, its mean μ and standard deviation σ , and its probability density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Many biological measurements in healthy populations, such as height or blood pressure, are approximately normal, and the normal distribution plays a central role in statistics because sample means are approximately normally distributed even when the individual observations are not (the central limit theorem, used in Section 3.8). For a normal distribution, a fixed proportion of values lies within any given number of SDs of the mean (Altman and Bland 1995b):

- about 68% of values lie within $\mu \pm 1\sigma$;
- about 95% within $\mu \pm 2\sigma$ (more precisely $\mu \pm 1.96\sigma$);
- about 99.7% within $\mu \pm 3\sigma$.

This is the **68–95–99.7 rule**. It explains why mean and SD are such an efficient summary of normal data: "mean SBP 139 mmHg (SD 20)" immediately tells a reader that about 95% of patients had SBP between roughly 100 and 179 mmHg. The interval $\bar{x} \pm 1.96s$ is called a *95% reference range* and is the basis of many laboratory "normal ranges" (Altman 1991, ch. 14).

We can check the rule empirically for SBP by counting the proportion of patients within one, two and three SDs of the mean.

```

sbp <- na.omit(analysis_data$sbp_mmhg) # drop the 2 missing values
m_sbp <- mean(sbp)
s_sbp <- sd(sbp)

tibble(k = 1:3) |>
  mutate(
    lower = m_sbp - k * s_sbp,
    upper = m_sbp + k * s_sbp,
    observed = map_dbl(k, ~ mean(abs(sbp - m_sbp) <= .x * s_sbp)),
    expected = c(0.683, 0.954, 0.997) # theoretical normal values
  ) |>
  kable(digits = 3,
    caption = "Proportion of SBP values within k SDs of the mean.")

```

Table 3.5: Proportion of SBP values within k SDs of the mean.

k	lower	upper	observed	expected
1	119.56	159.0	0.674	0.683
2	99.82	178.8	0.961	0.954
3	80.09	198.5	0.999	0.997

The observed proportions (about 0.67, 0.96 and 0.999) are very close to the normal expectations, so the mean and SD summarise SBP well. The figure below makes the same point visually: the histogram of SBP, scaled to a density, is overlaid with a normal curve that has the same mean and SD as the data.

```

ggplot(analysis_data, aes(x = sbp_mmhg)) +
  # histogram on the density scale so that a curve can be overlaid
  geom_histogram(aes(y = after_stat(density)), binwidth = 5,
    fill = teal, colour = "white", alpha = 0.7, na.rm = TRUE) +
  # kernel density estimate: a smoothed version of the histogram
  geom_density(linewidth = 0.9, colour = "grey20", na.rm = TRUE) +
  # theoretical normal curve with the sample mean and SD
  stat_function(fun = dnorm, args = list(mean = m_sbp, sd = s_sbp),
    linetype = "dashed", linewidth = 0.9, colour = "#C0392B") +
  geom_vline(xintercept = 140, linetype = "dotted") +
  labs(title = "Systolic blood pressure is close to normally distributed",
    x = "Systolic blood pressure (mmHg)", y = "Density")

```

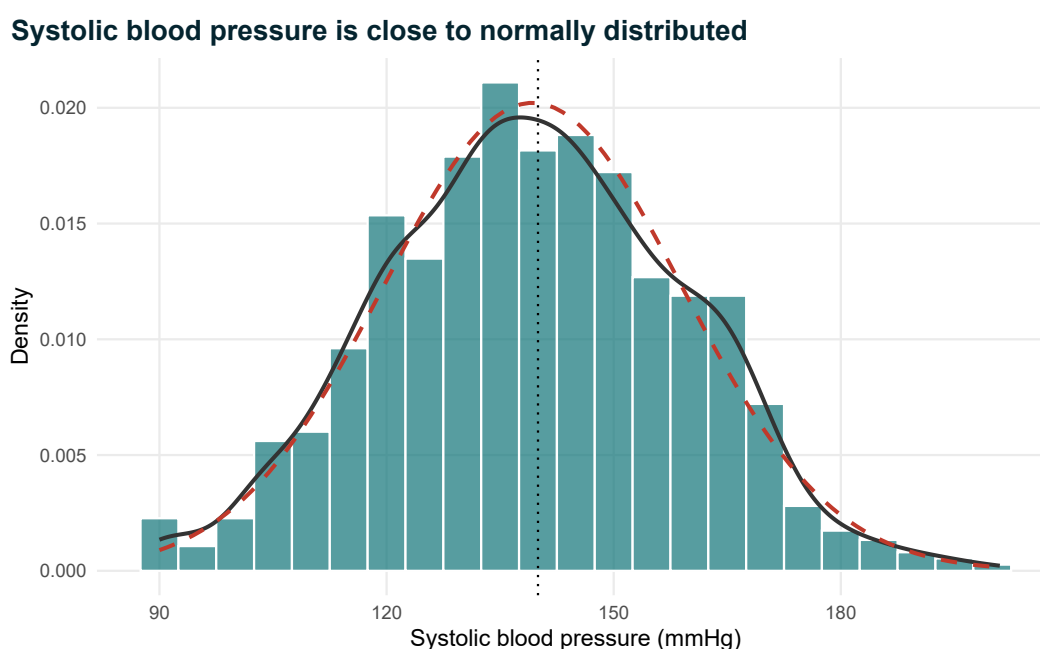


Figure 3.1: Distribution of systolic blood pressure in 1,498 patients: histogram (5 mmHg bins) with a smoothed density estimate (dark line) and the normal curve with the same mean and SD (dashed); the dotted line marks the 140 mmHg diagnostic threshold.

Three layers are drawn on the same axes. The bars are the histogram; the solid line is a *kernel density estimate*, which you can think of as a histogram smoothed so that it does not depend on where the bin edges fall; the dashed line is the ideal normal curve. The closeness of the solid and dashed curves is a visual check of normality. The dotted vertical line at 140 mmHg, the conventional threshold for hypertension (World Health Organization 2021), shows that a little over half of the patients had a raised reading on the day of the survey. Formal tests and normal quantile-quantile plots are introduced in Chapter 4.

3.5.3 When to report median (IQR) rather than mean (SD)

Distance to the facility is a different story. The next figure shows its histogram with the mean and median marked.

```

dist_summary <- analysis_data |>
  summarise(mean = mean(distance_to_facility_km, na.rm = TRUE),
    median = median(distance_to_facility_km, na.rm = TRUE))

```

```
ggplot(analysis_data, aes(x = distance_to_facility_km)) +
  geom_histogram(binwidth = 1, boundary = 0, fill = teal,
                 colour = "white", na.rm = TRUE) +
  geom_vline(xintercept = dist_summary$mean, linewidth = 0.9) +
  geom_vline(xintercept = dist_summary$median, linewidth = 0.9,
             linetype = "dashed") +
  # text labels placed to the right of the two lines
  annotate("text", x = dist_summary$mean + 0.8, y = c(150, 135), hjust = 0,
          label = c(sprintf("median = %.2f km (dashed)", dist_summary$median),
                    sprintf("mean = %.2f km (solid)", dist_summary$mean))) +
  labs(title = "Distance to the facility has a long right tail",
       x = "Distance to facility (km)", y = "Number of patients")
```

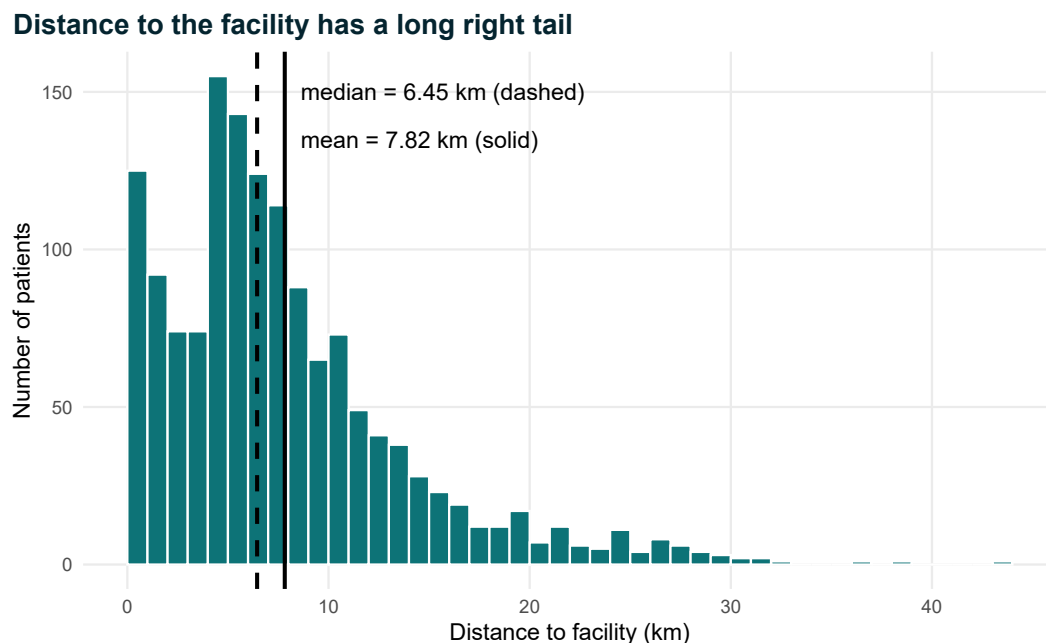


Figure 3.2: Distance from home to the health facility ($n = 1,440$) is right-skewed: most patients live within 10 km but a long tail travel much further, so the mean (solid line) lies to the right of the median (dashed line).

Most patients live within about 10 km, but a few travel 30 km or more. Those few pull the mean to 7.8 km, above the median of 6.45 km. Moreover, the SD (6.1 km) is almost as large as the mean, so "mean 7.8 km (SD 6.1)", read as a normal distribution, would imply that about one patient in ten lives at a *negative* distance (zero is 1.3 SDs below the mean), which is impossible. The mean and SD are simply the wrong language for this shape. A good practical rule is: **if the SD is more than about half the mean for a variable that cannot be negative, the distribution is skewed** (Altman 1991). For such variables report the median with the interquartile range: "median distance 6.5 km (IQR 4.0 to 10.3)".

In summary, the choice is guided by shape, not by habit:

- **Mean (SD)** for approximately symmetric distributions (age, SBP, DBP, BMI and cholesterol in this dataset).
- **Median (IQR)** for skewed distributions, for distributions with outliers, for discrete variables with few values, and for ordinal scores.
- When in doubt, report both, or show the distribution in a figure.

Common mistake

Do not confuse the SD with the standard error (SE). The SD describes how much *patients* vary; the SE (Section 3.8) describes how precisely a *mean* has been estimated. Reporting “mean $\pm$ SE” in a descriptive table makes the data look far less variable than they are, because the SE shrinks as the sample grows while the SD does not (Altman and Bland 1994).

3.6 Missing values and the `na.rm` trap

Real clinical data are incomplete, and R is deliberately cautious about missing values. In R a missing value is NA (“not available”), and the rule is simple: *any calculation involving an unknown value gives an unknown result*. If one patient’s SBP is missing, R cannot know the sum of all SBPs, and therefore cannot know the mean.

```
sum(is.na(analysis_data$sbp_mmhg))           # how many SBP values are missing? R
#> [1] 2
```

```
mean(analysis_data$sbp_mmhg)                 # the trap: one NA makes it NA R
#> [1] NA
```

```
mean(analysis_data$sbp_mmhg, na.rm = TRUE)    # the fix: remove NAs first R
#> [1] 139.3
```

Two patients have no SBP recorded, so `mean()` returns NA. Adding `na.rm = TRUE` (“NA remove”) tells R to drop the missing values and compute the mean of the remaining 1,498. The same argument exists for `median()`, `sd()`, `var()`, `quantile()`, `IQR()`, `min()`, `max()`, `range()` and `sum()`.

The trap matters most for variables with substantial missingness. Total cholesterol has 90 missing values.

```
chol <- analysis_data$total_chol_mmol_l      R

c(n_total   = length(chol),
  n_missing = sum(is.na(chol)),
  pct_miss  = round(100 * mean(is.na(chol)), 1),
  mean      = round(mean(chol, na.rm = TRUE), 2))

#>   n_total n_missing pct_miss   mean
#>   1500.00    90.00    6.00    5.11
```

Note the idiom `mean(is.na(x))`: `is.na()` returns TRUE/FALSE, R treats TRUE as 1 and FALSE as 0, so the mean of a logical vector is the *proportion* of TRUE values. Here 6% of cholesterol values are missing. The mean of 5.1 mmol/L describes the 1,410 patients who were tested, not all 1,500.

Common mistake

`na.rm = TRUE` makes the error message disappear; it does not make the missing-data problem disappear. Removing missing values silently changes the denominator, and if the patients without a cholesterol result differ from those with one (perhaps the sickest patients were sent straight to hospital before blood was drawn), the mean is biased. Always count the missing values, report them, and think about why they are missing (Rubin 1976; Little and Rubin 2019).

Tip

Inside `summarise()` it is good practice to compute `n = sum(!is.na(x))` next to every mean, as in the spread table of Section 3.4. Then the denominator travels with the statistic and cannot be forgotten when the table is copied into a manuscript.

3.7 Grouped summaries

Descriptive statistics become most informative when we compare groups: treated versus untreated patients, women versus men, one facility versus another. The **dplyr** verbs `group_by()` and `summarise()` do this in two steps: `group_by()` splits the data into groups and `summarise()` computes one row of statistics per group (Wickham et al. 2023).

3.7.1 By treatment uptake

We first write a tiny helper that formats a mean and SD as a single text string, the format used in published tables. Then we summarise five variables by treatment uptake among diagnosed patients.

```
# Format "mean (SD)" with one decimal place
mean_sd <- function(x) {
  sprintf("%.1f (%.1f)", mean(x, na.rm = TRUE), sd(x, na.rm = TRUE))
}

diagnosed |>
  group_by(treatment_uptake) |>
  summarise(
    n          = n(),                                # patients in each group
    Age        = mean_sd(age),
    SBP        = mean_sd(sbp_mmhg),
    DBP        = mean_sd(dbp_mmhg),
    BMI        = mean_sd(bmi),
    Knowledge  = mean_sd(knowledge_score)
  ) |>
  kable(caption = paste("Mean (SD) of five variables by treatment uptake",
                        "among diagnosed patients."))
```

Of the 1,089 diagnosed patients, 581 were not on treatment and 508 were. Treated patients were on average about five years older and had a slightly higher mean knowledge score (by about one point on the 0–20 scale). Their mean SBP was also *higher*, which may seem surprising if treatment lowers blood pressure. In a cross-sectional study, however, we see each patient once: patients with the highest pressures are the ones most likely to be started on treatment, and treatment does not bring every patient to target. Descriptive comparisons cannot separate cause and effect; they

Table 3.6: Mean (SD) of five variables by treatment uptake among diagnosed patients.

treatment_uptake	n	Age	SBP	DBP	BMI	Knowledge
No	581	51.1 (13.4)	142.2 (19.8)	88.8 (10.6)	26.8 (4.9)	9.9 (3.4)
Yes	508	56.3 (14.1)	148.1 (18.3)	88.2 (10.7)	26.9 (4.7)	10.9 (3.4)

generate questions for the analyses of Chapters 4 and 5.

3.7.2 By facility

```

diagnosed |>
  group_by(facility) |>
  summarise(
    n          = n(),
    treated    = sum(treatment_uptake == "Yes"),
    pct_treated = 100 * mean(treatment_uptake == "Yes"),
    median_dist = median(distance_to_facility_km, na.rm = TRUE),
    pct_insured = 100 * mean(health_insurance == "Yes")
  ) |>
  arrange(desc(pct_treated)) |>
  kable(digits = 1,
        caption = "Uptake, distance and insurance by facility (diagnosed).")

```

Table 3.7: Uptake, distance and insurance by facility (diagnosed).

facility	n	treated	pct_treated	median_dist	pct_insured
Nyamagana PHC	220	118	53.6	6.4	36.8
Bugando PHC	259	123	47.5	6.5	34.0
Kisesa HC	154	71	46.1	6.2	33.1
Ilemela HC	192	87	45.3	7.2	35.9
Buzuruga PHC	134	59	44.0	5.5	26.1
Igoma HC	130	50	38.5	5.8	30.8

Treatment uptake ranges from about 54% at Nyamagana PHC to about 39% at Igoma HC. Sorting by the statistic of interest (`arrange(desc(...))`) makes the ranking obvious at a glance. Whether this spread is larger than chance alone would produce is a question for confidence intervals (Section 3.8) and formal tests (Chapter 4). Note again the idiom `mean(treatment_uptake == "Yes")`: the comparison returns TRUE/FALSE, and its mean is a proportion.

3.7.3 By two grouping variables

`group_by()` accepts several variables. Here we look at mean SBP by sex and treatment uptake among diagnosed patients.

```

diagnosed |>
  group_by(sex, treatment_uptake) |>
  summarise(n = n(),
            mean_sbp = mean(sbp_mmhg, na.rm = TRUE),
            sd_sbp   = sd(sbp_mmhg, na.rm = TRUE),

```

```
.groups = "drop") |> # remove grouping after summarising
kable(digits = 1, caption = "SBP (mmHg) by sex and treatment uptake.")
```

Table 3.8: SBP (mmHg) by sex and treatment uptake.

sex	treatment_uptake	n	mean_sbp	sd_sbp
Female	No	315	141.1	19.8
Female	Yes	314	149.1	17.9
Male	No	266	143.4	19.8
Male	Yes	194	146.6	18.9

The pattern seen overall (higher SBP among treated patients) holds in both women and men, although the gap is larger in women (about 8 mmHg) than in men (about 3 mmHg). So the overall difference is not simply produced by a different mix of men and women in the two groups, but its size may depend on sex, a hint of what epidemiologists call *effect modification*. The argument `.groups = "drop"` returns an ungrouped result, which avoids surprises if the table is processed further.

Good practice

Use `across()` to apply the same summary to many columns, as in Sections 3.3 and 3.4, rather than copying and pasting a line per variable. Fewer lines mean fewer typing errors, and adding a variable becomes a one-word change.

3.8 From description to inference: standard error and confidence interval

So far we have described the sample. But the purpose of most clinical studies is to learn about a *population*: all adults attending primary care in the region, not just the 1,500 who were enrolled. If we drew a different sample of 1,500, we would obtain a slightly different mean SBP. How much would it vary? The answer is the **standard error**, and it leads directly to the **confidence interval**, the most useful single tool of statistical inference (Altman 1991; Kirkwood and Sterne 2003).

3.8.1 Standard error of a mean

Imagine repeating the study many times, each time computing the sample mean $\bar{x}$. The means would form their own distribution, the *sampling distribution of the mean*. The central limit theorem says that, for reasonably large samples, this distribution is approximately normal, centred on the population mean μ , with standard deviation

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}}.$$

This standard deviation of the sampling distribution is the **standard error of the mean**. It decreases with the square root of the sample size: four times as many patients halve the SE. The SD, by contrast, does not shrink with sample size, because patients do not become more alike as more are enrolled.

3.8.2 Confidence interval for a mean

Because $\bar{x}$ is approximately normal with SD equal to the SE, about 95% of samples give a mean within 1.96 SE of μ . Turning this around gives the 95% **confidence interval** (CI)

$$\bar{x} \pm t_{n-1, 0.975} \times \frac{s}{\sqrt{n}},$$

where $t_{n-1, 0.975}$ is the 97.5th centile of Student's t distribution with $n - 1$ degrees of freedom (Student 1908). For large n it is very close to 1.96; for small samples it is larger, widening the interval to allow for the uncertainty in s .

```
R
n_sbp <- length(sbp)                # 1,498 non-missing values
se_sbp <- s_sbp / sqrt(n_sbp)        # standard error of the mean
t_crit <- qt(0.975, df = n_sbp - 1)  # about 1.96 for large n

c(mean = m_sbp, sd = s_sbp, se = se_sbp, t = t_crit,
  lower = m_sbp - t_crit * se_sbp, upper = m_sbp + t_crit * se_sbp)
```

```
#>      mean      sd      se      t    lower    upper
#> 139.2924 19.7355  0.5099  1.9615 138.2922 140.2926
```

```
R
# The same interval from t.test(), which we meet properly in Chapter 4
t.test(sbp)$conf.int
```

```
#> [1] 138.3 140.3
#> attr(,"conf.level")
#> [1] 0.95
```

The mean SBP is 139.3 mmHg. The SD of 19.7 mmHg describes how much individual patients vary; the SE of only 0.51 mmHg describes how precisely the population mean has been estimated from 1,498 patients. The 95% CI runs from about 138.3 to 140.3 mmHg. `t.test()` gives the same interval without the arithmetic.

Clinical interpretation

The 95% CI says that the data are consistent with a population mean SBP anywhere between about 138 and 140 mmHg. Strictly, "95%" refers to the method: if the study were repeated many times, 95% of intervals computed this way would contain the true mean. The CI is about the *mean*; it does not say that 95% of patients have SBP in this range. For that we need the reference range, $\text{mean} \pm 1.96 \text{ SD}$, which is roughly 101 to 178 mmHg.

3.8.3 Confidence interval for a proportion

The same logic applies to a proportion. If x of n patients have a characteristic, the sample proportion is $\hat{p} = x/n$, its standard error is

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}},$$

and the simple (Wald) 95% CI is

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

The Wald interval works well when $n\hat{p}$ and $n(1 - \hat{p})$ are both at least about 10. For small samples or proportions near 0 or 1 it can be too narrow or even extend below 0; the **Wilson score interval** behaves much better and is what `prop.test()` reports (Agresti 2013, ch. 1). With large samples the two agree closely.

```
x_treated <- sum(diagnosed$treatment_uptake == "Yes") # treated patients
n_diag    <- nrow(diagnosed)                        # all diagnosed
p_hat     <- x_treated / n_diag
se_p      <- sqrt(p_hat * (1 - p_hat) / n_diag)

# Wald interval, computed by hand
round(100 * c(p = p_hat, lower = p_hat - 1.96 * se_p,
              upper = p_hat + 1.96 * se_p), 1)
```

```
#>      p lower upper
#> 46.6  43.7  49.6
```

```
# Wilson score interval (correct = FALSE turns off the continuity correction)
round(100 * prop.test(x_treated, n_diag, correct = FALSE)$conf.int, 1)
```

```
#> [1] 43.7 49.6
#> attr(,"conf.level")
#> [1] 0.95
```

Among the 1,089 diagnosed patients, 508 (46.6%) were on antihypertensive treatment. The 95% CI is about 43.7% to 49.6% by either method. In words: the data are consistent with a population treatment-uptake rate somewhere in the mid-40s, and clearly below one half. Fewer than half of diagnosed patients being on treatment is the kind of “treatment gap” reported in many real surveys of hypertension care (NCD Risk Factor Collaboration (NCD-RisC) 2021; Mills et al. 2020), although, again, these are simulated data.

3.8.4 Confidence intervals by group

Confidence intervals become especially informative when groups are compared. The code below computes the uptake proportion and its Wald 95% CI for each facility, and the figure that follows plots them.

```
facility_ci <- diagnosed |>
  group_by(facility) |>
  summarise(n = n(), treated = sum(treatment_uptake == "Yes")) |>
  mutate(p      = treated / n,
         se      = sqrt(p * (1 - p) / n),
         lower   = p - 1.96 * se,
         upper   = p + 1.96 * se)

facility_ci |>
  mutate(across(c(p, lower, upper), ~ 100 * .x)) |>
```

```
select(facility, n, treated, pct = p, lower, upper) |>
kable(digits = 1, caption = "Treatment uptake (%) with 95% CI by facility.")
```

Table 3.9: Treatment uptake (%) with 95% CI by facility.

facility	n	treated	pct	lower	upper
Bugando PHC	259	123	47.5	41.4	53.6
Buzuruga PHC	134	59	44.0	35.6	52.4
Igoma HC	130	50	38.5	30.1	46.8
Ilemela HC	192	87	45.3	38.3	52.4
Kisesa HC	154	71	46.1	38.2	54.0
Nyamagana PHC	220	118	53.6	47.0	60.2

```
ggplot(facility_ci,
  aes(x = p, y = fct_reorder(facility, p))) + # order facilities by p
  geom_vline(xintercept = p_hat, linetype = "dashed", colour = "grey40") +
  geom_pointrange(aes(xmin = lower, xmax = upper),
    colour = teal, linewidth = 0.8) +
  scale_x_continuous(labels = label_percent(), limits = c(0.25, 0.70)) +
  labs(title = "Treatment uptake varies between facilities",
    x = "Diagnosed patients on treatment (95% CI)", y = NULL)
```

Treatment uptake varies between facilities

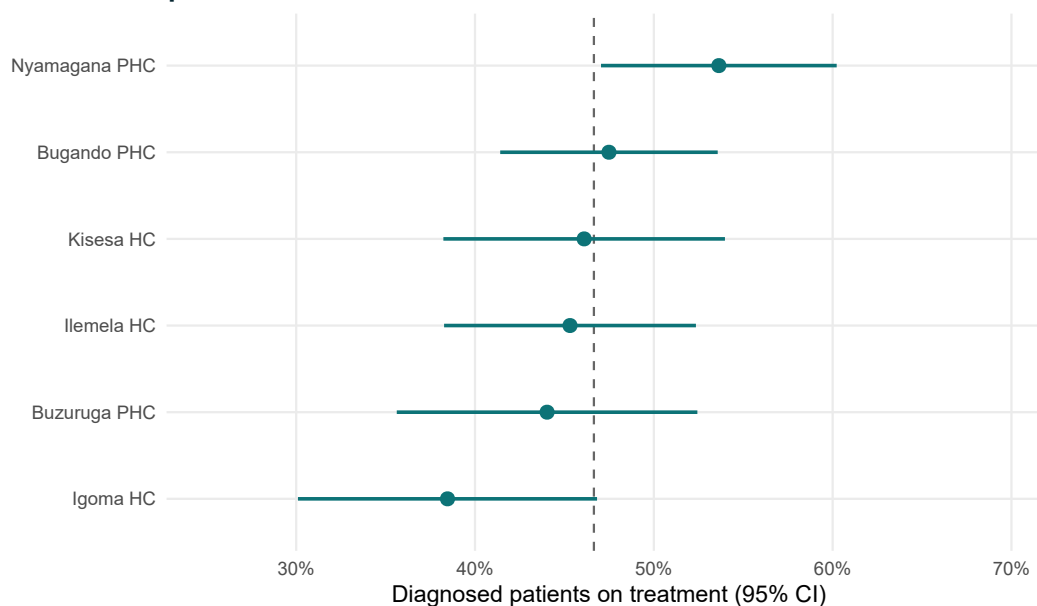


Figure 3.3: Proportion of diagnosed hypertensive patients on treatment at each facility, with 95% confidence intervals; the dashed line marks the overall proportion (46.6%). Facilities are ordered by uptake.

Each point is a facility's uptake and the horizontal bar its 95% CI. The intervals are wide, about ± 6 to ± 8 percentage points, because each facility contributes only 130 to 260 diagnosed patients, compared with ± 3 points for all 1,089 together. Igoma HC sits lowest and Nyamagana PHC highest, and their intervals (30.1% to 46.8% and 47.0% to 60.2%) only just fail to overlap; the four other facilities are consistent with the overall rate. Overlap of intervals is *not* a formal test (two intervals can overlap slightly even when the groups differ significantly), and picking out the two extremes from six facilities exaggerates the contrast, so we defer the comparison to the chi-squared test in

Chapter 4. A plot of estimates with intervals, ordered by size, is far more informative than a bar chart of percentages, because it shows the uncertainty as well as the estimate.

3.9 Describing categorical variables

For categorical variables the natural summaries are **frequencies** (counts) and **relative frequencies** (proportions or percentages). The only real decisions are what denominator to use and how to treat missing values.

3.9.1 Frequency tables with base R

`table()` counts each category and `prop.table()` turns counts into proportions.

```
table(analysis_data$sex) # counts
```

R

```
#>
#> Female    Male
#>      885    615
```

```
round(100 * prop.table(table(analysis_data$sex)), 1) # percentages
```

R

```
#>
#> Female    Male
#>      59    41
```

The sample contains 885 women (59.0%) and 615 men (41.0%). A preponderance of women is typical of primary-care attenders in many settings, and it is exactly the kind of fact that Table 1 must report, because it affects how far the results generalise.

3.9.2 Making missing values visible

By default `table()` silently drops NA. For a variable with missing values this changes the denominator without telling you.

```
table(analysis_data$education) # NA silently dropped
```

R

```
#>
#>      None    Primary Secondary Tertiary
#>      270      608      424      168
```

```
table(analysis_data$education, useNA = "ifany") # NA shown if present
```

R

```
#>
#>      None    Primary Secondary Tertiary    <NA>
#>      270      608      424      168      30
```

Thirty patients have no education recorded. The first table hides them, so percentages computed from it would refer to 1,470 patients while appearing to describe all 1,500. The argument `useNA =`

"ifany" adds an <NA> column whenever missing values exist (use "always" to show the column even when the count is zero).

Good practice

In a manuscript, percentages for a categorical variable are usually computed among patients with known values, with the number missing reported separately ("education was missing for 30 patients (2.0%)"). Whichever convention you use, state it in the table footnote, and never let missing values vanish unannounced.

3.9.3 Tidy frequency tables with `count()`

`dplyr::count()` returns a data frame rather than a table object, so the result can be piped into further steps, for example to add a percentage column.

```
analysis_data |>
  count(bp_category) |> # NA kept as its own row
  mutate(percent = round(100 * n / sum(n), 1))
```

```
#> # A tibble: 4 × 3
#>   bp_category      n percent
#>   <fct>      <int>   <dbl>
#> 1 Normal        144     9.6
#> 2 Elevated       357    23.8
#> 3 Hypertension   996    66.4
#> 4 <NA>           3     0.2
```

`count()` keeps NA as a separate row by default. Of 1,500 patients, 996 (66.4%) had a reading in the hypertensive range, 357 (23.8%) were in the elevated range and 144 (9.6%) were normal; three could not be classified because a blood-pressure value was missing.

3.9.4 Polished frequency tables with `janitor`

The **janitor** package provides `tabyl()`, which produces frequency tables with counts, percentages and (when present) a separate "valid percent" excluding missing values, and a family of `adorn_*`() functions that format them for presentation.

```
analysis_data |>
  tabyl(education) |>
  adorn_totals("row") |> # add a Total row
  adorn_pct_formatting(digits = 1) # format proportions as percentages
```

```
#>   education      n percent valid_percent
#>   None       270   18.0%      18.4%
#>   Primary    608   40.5%      41.4%
#>   Secondary  424   28.3%      28.8%
#>   Tertiary   168   11.2%      11.4%
#>   <NA>        30    2.0%         -
#>   Total    1500  100.0%     100.0%
```

The column `percent` uses all 1,500 patients as denominator; `valid_percent` excludes the 30 missing values. Primary education was most common (40.5% of all patients, 41.4% of those with

known education). Because education is an *ordered* factor, the categories appear in their natural order rather than alphabetically, one of the benefits of setting factor levels in Chapter 2.

3.9.5 Cross-tabulations

A **cross-tabulation** (contingency table) counts patients by two categorical variables at once. Among diagnosed patients, let us cross-tabulate diabetes against treatment uptake.

```
xtab <- table(Diabetes = diagnosed$diabetes,
              Treated  = diagnosed$treatment_uptake)
xtab
```

```
#>      Treated
#> Diabetes No Yes
#>      No  574 487
#>      Yes   7  21
```

```
addmargins(xtab) # add row and column totals
```

```
#>      Treated
#> Diabetes No Yes Sum
#>      No  574 487 1061
#>      Yes   7  21  28
#>      Sum 581 508 1089
```

Each cell counts the patients with a given combination: for example, 21 diagnosed patients had diabetes *and* were on treatment. `addmargins()` adds the totals: 28 diagnosed patients had diabetes, 508 were treated, and the grand total is 1,089.

3.9.6 Row or column percentages?

A two-way table can be turned into percentages in three ways, controlled by the `margin` argument of `prop.table()`:

- `margin = 1`: **row percentages**, each row sums to 100%;
- `margin = 2`: **column percentages**, each column sums to 100%;
- no margin: percentages of the grand total.

```
round(100 * prop.table(xtab, margin = 1), 1) # row %: within diabetes group
```

```
#>      Treated
#> Diabetes No Yes
#>      No  54.1 45.9
#>      Yes  25.0 75.0
```

```
round(100 * prop.table(xtab, margin = 2), 1) # column %: within uptake group
```

```
#>           Treated
#> Diabetes    No   Yes
#>         No  98.8 95.9
#>         Yes   1.2  4.1
```

The two tables answer different questions. With diabetes in the rows, the **row percentages** say that 75.0% of diagnosed patients *with diabetes* were on treatment, compared with 45.9% of those without diabetes. The **column percentages** say that 4.1% of *treated* patients had diabetes, compared with 1.2% of untreated patients.

Which to report depends on the question. If treatment uptake is the outcome and diabetes a possible determinant, as in our study, the natural comparison is the proportion *treated* within each diabetes group, which here is the row percentage. The rule is to compute percentages **within the categories of the explanatory variable**, so that each percentage answers "of patients with this characteristic, what proportion had the outcome?" (Altman 1991; Kirkwood and Sterne 2003). In a Table 1 laid out by outcome group (columns = treated and untreated), the convention is the reverse: column percentages, describing the make-up of each group. Both are correct for their purpose; the mistake is to compute one and describe it in words appropriate to the other.

Common mistake

"4.1% of diabetic patients were treated" is a misreading of the column percentage above, which actually says that 4.1% of treated patients were diabetic. Before writing a sentence about a percentage, say aloud what the denominator is: "of the __, what percentage __?"

Note also the small numbers: only 28 diagnosed patients have diabetes, so the 75.0% rests on 21 out of 28 patients and has a wide confidence interval. Small cells are a warning sign for later analyses (Chapter 4 explains why the chi-squared test may be replaced by Fisher's exact test in this situation).

3.9.7 Cross-tabulations with janitor

`tabyl()` also builds two-way tables, and the adorn functions add totals, percentages and the underlying counts in one readable display.

```
diagnosed |>
  tabyl(diabetes, treatment_uptake) |>
  adorn_totals(c("row", "col")) |>
  adorn_percentages("row") |>           # row percentages
  adorn_pct_formatting(digits = 1) |>
  adorn_ns() |>                         # append the counts in brackets
  adorn_title("combined")               # label rows and columns together
```

```
#> diabetes/treatment_uptake      No      Yes      Total
#>               No 54.1% (574) 45.9% (487) 100.0% (1,061)
#>               Yes 25.0%   (7) 75.0%  (21) 100.0%   (28)
#>               Total 53.4% (581) 46.6% (508) 100.0% (1,089)
```

Each cell now shows the row percentage followed by the count, for example "75.0% (21)", which is the format many journals prefer: the percentage for comparison, the count so the reader can see how many patients it represents.

3.10 Building Table 1 with gtsummary

3.10.1 What belongs in Table 1

Table 1 of a clinical paper describes the characteristics of the study participants. Under STROBE it should present demographic, clinical and social characteristics, information on exposures and potential confounders, and the number of participants with missing data for each variable (Elm et al. 2007). In our cross-sectional study of treatment uptake, a typical Table 1 would show:

- **demographic** variables: age, sex, residence, education, health insurance;
- **behavioural** variables: smoking, alcohol, physical activity;
- **clinical** variables: BMI, blood pressure, diabetes, family history;
- **access and knowledge**: distance to the facility, knowledge score;

usually for the whole sample and by the main grouping variable, here treatment uptake among diagnosed patients. Each continuous variable is summarised by mean (SD) or median (IQR) according to its distribution, and each categorical variable by n (%).

Building such a table by hand, cell by cell, is slow and error-prone: one wrong copy-paste and a published percentage is wrong. The **gtsummary** package automates the whole process (Sjoberg et al. 2021). It inspects each variable, chooses a sensible summary, formats the numbers, and produces a table that can be exported to Word, HTML or PDF.

3.10.2 A first Table 1

```
diagnosed |>
  select(age, sex, residence, health_insurance, diabetes,
         treatment_uptake) |>
  tbl_summary(by = treatment_uptake) |> # one column per uptake group
  as_kable(caption = "A first Table 1: default gtsummary output.")
```

Table 3.10: A first Table 1: default gtsummary output.

Characteristic	No N = 581	Yes N = 508
age	51 (41, 60)	56 (47, 66)
Unknown	1	0
sex		
Female	315 (54%)	314 (62%)
Male	266 (46%)	194 (38%)
residence		
Rural	306 (53%)	195 (38%)
Urban	275 (47%)	313 (62%)
health_insurance	162 (28%)	202 (40%)
diabetes	7 (1.2%)	21 (4.1%)

With one line of real work, `tbl_summary()` has produced a usable table. Reading it:

- The column headers give the group sizes: 581 untreated and 508 treated diagnosed patients.

- Age, a continuous variable, is summarised by its *median* (*Q1*, *Q3*), the `gtsummary` default for continuous variables, because the median and IQR are safe whatever the shape.
- Sex and residence have two levels, so both levels are listed with *n* (%).
- Health insurance and diabetes are Yes/No variables, so `gtsummary` shows only the "Yes" row (a *dichotomous* summary), which is the conventional and more compact display.
- Row labels are the raw variable names (`age`, `health_insurance`), and the one patient with missing age appears in a row labelled "Unknown". Both need tidying for publication.
- Percentages are rounded to whole numbers, except below 10%, where one decimal is shown.
- The percentages are **column percentages**: they describe the make-up of each group. The figure for health insurance in the "No" column is the proportion of the 581 *untreated* patients who were insured, not the proportion of insured patients who were untreated.

The last point deserves care, and we return to it after customising the table.

3.10.3 Customising statistics, labels and missing values

A table for a manuscript needs readable labels, the statistic that suits each variable, a consistent number of decimal places and an explicit treatment of missing data.

```

table1 <- diagnosed |>
  select(age, sex, residence, education, health_insurance,
         bmi, sbp_mmhg, diabetes, knowledge_score,
         distance_to_facility_km, treatment_uptake) |>
  tbl_summary(
    by = treatment_uptake,
    # symmetric variables: mean (SD); skewed distance: median (IQR)
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      distance_to_facility_km ~ "{median} ({p25}, {p75})",
      all_categorical() ~ "{n} ({p}%)"
    ),
    digits = list(all_continuous() ~ 1, all_categorical() ~ c(0, 1)),
    label = list(
      age ~ "Age, years",
      sex ~ "Sex",
      residence ~ "Residence",
      education ~ "Education",
      health_insurance ~ "Health insurance",
      bmi ~ "BMI, kg/m^2",
      sbp_mmhg ~ "Systolic BP, mmHg",
      diabetes ~ "Diabetes",
      knowledge_score ~ "Knowledge score (0-20)",
      distance_to_facility_km ~ "Distance to facility, km"
    ),
    missing = "ifany",          # show a Missing row only where needed
    missing_text = "Missing"
  ) |>
  add_overall(last = FALSE) |> # Overall column first
  modify_header(label = "**Characteristic**")

table1 |>

```

```
as_kable(caption = paste("Characteristics of diagnosed hypertensive patients",
                          "by treatment uptake."))
```

Table 3.11: Characteristics of diagnosed hypertensive patients by treatment uptake.

Characteristic	Overall N = 1,089	No N = 581	Yes N = 508
Age, years	53.5 (14.0)	51.1 (13.4)	56.3 (14.1)
Missing	1	1	0
Sex			
Female	629 (57.8%)	315 (54.2%)	314 (61.8%)
Male	460 (42.2%)	266 (45.8%)	194 (38.2%)
Residence			
Rural	501 (46.0%)	306 (52.7%)	195 (38.4%)
Urban	588 (54.0%)	275 (47.3%)	313 (61.6%)
Education			
None	201 (18.9%)	126 (22.1%)	75 (15.2%)
Primary	444 (41.7%)	252 (44.2%)	192 (38.8%)
Secondary	292 (27.4%)	137 (24.0%)	155 (31.3%)
Tertiary	128 (12.0%)	55 (9.6%)	73 (14.7%)
Missing	24	11	13
Health insurance	364 (33.4%)	162 (27.9%)	202 (39.8%)
BMI, kg/m ²	26.8 (4.8)	26.8 (4.9)	26.9 (4.7)
Missing	33	19	14
Systolic BP, mmHg	144.9 (19.4)	142.2 (19.8)	148.1 (18.3)
Missing	1	0	1
Diabetes	28 (2.6%)	7 (1.2%)	21 (4.1%)
Knowledge score (0-20)	10.4 (3.5)	9.9 (3.4)	10.9 (3.4)
Missing	29	16	13
Distance to facility, km	6.3 (3.8, 10.2)	6.8 (4.3, 10.7)	5.8 (3.1, 9.6)
Missing	47	20	27

This is a table that could go into a manuscript with little further editing. Points to note in the code and the output:

- `statistic` takes a list of *formulas*: the left side selects variables (with helpers such as `all_continuous()` and `all_categorical()`, or by name), the right side is a template in which `{mean}`, `{sd}`, `{median}`, `{p25}`, `{p75}`, `{n}` and `{p}` are replaced by the computed values. The specific rule for distance overrides the general rule for continuous variables.
- `digits` sets one decimal place for continuous summaries, and integer counts with one decimal place for percentages.
- `label` replaces variable names with readable labels including units.
- `missing = "ifany"` adds a "Missing" row under each variable that has missing values, as STROBE requires. Missing values are *not* included in the denominators of the percentages.
- `add_overall()` adds a column for all diagnosed patients, here placed first.

Reading the content, treated patients were older on average (56.3 versus 51.1 years), more often women (61.8% versus 54.2%), more often urban residents (61.6% versus 47.3%), better educated, more often insured (39.8% versus 27.9%) and more often diabetic, scored about one point higher on the knowledge scale, and lived somewhat closer to the facility (median 5.8 versus 6.8 km). Their mean SBP was higher, for the reasons discussed in Section 3.7, while BMI was almost identical in the two groups. These are *descriptive* differences; whether they are larger than chance, and whether they persist after adjustment for one another, are questions for Chapters 4 and 5.

Clinical interpretation

The percentages in each column describe that group. "Diabetes: 21 (4.1%)" in the treated column means that 4.1% of treated patients had diabetes. It does *not* mean that 4.1% of diabetic patients were treated (that figure is 75%, from the row percentages in Section 3.9). If your question is "how does uptake differ by diabetes status?", report the row percentages in the text, or use `tbl_summary(by = diabetes, percent = "row")`.

3.10.4 Should Table 1 contain p-values?

`gtsummary` can add a column of p-values with `add_p()`, which chooses a test for each variable (a Wilcoxon rank-sum or t-test for continuous variables, a chi-squared or Fisher's exact test for categorical ones). Whether it *should* is debated.

In a **randomised trial**, Table 1 compares groups formed by chance. Any baseline difference is, by definition, due to chance, so a significance test asks a question whose answer is already known; the CONSORT guidance and many statisticians therefore advise against p-values in a trial's baseline table (Altman 1991). What matters is whether baseline imbalances are large enough to be *clinically* important, which a p-value cannot tell you.

In an **observational study** like ours, comparing groups defined by the outcome is a genuine (unadjusted) analysis of association, and p-values are more defensible. Even so, a Table 1 p-value column invites readers to treat " $p < 0.05$ " as a verdict on each variable, encourages multiple testing, and confuses statistical with clinical significance (Wasserstein and Lazar 2016; Greenland et al. 2016). A sound compromise, followed in this book, is to keep Table 1 descriptive and present tests and effect estimates with confidence intervals in a separate analytic table (Chapters 4 and 5). If a journal requires p-values, `gtsummary` adds them in one line, and the tests are explained in Chapter 4:

```
diagnosed |>
  select(age, sex, diabetes, treatment_uptake) |>
  tbl_summary(by = treatment_uptake,
              statistic = all_continuous() ~ "{mean} ({sd})",
              label = list(age ~ "Age, years", sex ~ "Sex",
                           diabetes ~ "Diabetes")) |>
  add_p(test = list(age ~ "t.test")) |> # Welch t-test for age
  as_kable(caption = "Table 1 with a p-value column (use with caution).")
```

Common mistake

A non-significant p-value in Table 1 does not show that the groups are "comparable", and a significant one does not show that a variable is a confounder. Absence of evidence is not evidence of absence (Altman and Bland 1995a). Judge imbalance by the size of the

Table 3.12: Table 1 with a p-value column (use with caution).

Characteristic	No N = 581	Yes N = 508	p-value
Age, years	51 (13)	56 (14)	<0.001
Unknown	1	0	
Sex			0.011
Female	315 (54%)	314 (62%)	
Male	266 (46%)	194 (38%)	
Diabetes	7 (1.2%)	21 (4.1%)	0.002

difference, and handle confounding with adjustment (Chapter 5).

3.11 Figures with ggplot2

Tables give exact numbers; figures show patterns, shapes and outliers that tables hide. A good figure is often the most-read part of a paper. In R the standard tool is **ggplot2** (Wickham 2016), which we have already used for the histograms above.

3.11.1 The grammar of graphics

ggplot2 implements a *grammar of graphics*: instead of choosing from a menu of chart types, you describe a plot as a combination of independent components (Wickham 2016):

- **data**: the data frame to plot;
- **aesthetic mappings** (`aes()`): which variables are mapped to which visual properties, such as position on the x and y axes, colour, fill, size or shape;
- **geometric objects** (`geom_*()`): what is drawn, such as points, bars, lines, boxes;
- **statistical transformations** (`stat_*()`): computations applied before drawing, such as the binning in a histogram or the fitted line in a smoother;
- **scales** (`scale_*()`): how data values are translated into visual values, including axis breaks and labels and colour palettes;
- **facets** (`facet_wrap()`, `facet_grid()`): small multiples, one panel per subgroup;
- **coordinates and themes**: the coordinate system and the non-data appearance.

A plot is built by adding layers with `+`. Every plot in this chapter follows the same template:

```
ggplot(data = <DATA>, aes(x = <X>, y = <Y>, fill = <GROUP>)) +
  geom_<TYPE>(<fixed settings such as colour or alpha>) +
  labs(title = "...", x = "... (units)", y = "...") +
  theme_minimal()
```

R

A distinction that confuses beginners: a property *mapped* to a variable goes inside `aes()` (`aes(fill = treatment_uptake)` colours bars by group), whereas a property *set* to a constant goes outside (`fill = teal` colours every bar teal).

3.11.2 Principles of good scientific graphs

A few principles, drawn from Weissgerber et al. (2015) and Wickham (2016), will improve almost every figure:

1. **Show the data.** Wherever sample sizes allow, show individual observations or their full distribution, not just a summary.
2. **Match the figure to the variable types:** histogram or density for one continuous variable, bar chart for one categorical variable, box, violin or dot plot for a continuous variable by group, scatter plot for two continuous variables.
3. **Label axes with units** and write a caption that can be understood without the text.
4. **Order categories meaningfully:** ordinal variables in their natural order, nominal variables by frequency or by the statistic of interest.
5. **Use colour with purpose,** to encode a variable, not to decorate; prefer palettes that remain distinguishable in greyscale and for colour-blind readers.
6. **Avoid chart junk:** three-dimensional effects, heavy gridlines and dual axes add ink without information.

3.11.3 Bar charts for categorical variables

`geom_bar()` counts the rows in each category itself, so it is given the raw data, not a pre-computed table. To show percentages we compute them first and use `geom_col()`, which draws bars of a given height.

```

edu_pct <- analysis_data |>
  filter(!is.na(education)) |>
  count(education) |>
  mutate(pct = n / sum(n))

ggplot(edu_pct, aes(x = education, y = pct)) +
  geom_col(fill = teal, width = 0.7) +
  geom_text(aes(label = percent(pct, accuracy = 0.1)), # label each bar
            vjust = -0.4, size = 3.5) +
  scale_y_continuous(labels = label_percent(),
                     expand = expansion(mult = c(0, 0.08))) +
  labs(title = "Most participants had primary or secondary education",
       x = "Highest education level", y = "Patients (%)")

```

Because `education` is an ordered factor, the bars appear in their natural order, from none to tertiary. Labelling each bar with its percentage spares the reader from estimating values from the axis. The caption states the denominator and what was excluded.

To compare the distribution of a categorical variable *between groups*, a 100% stacked bar chart is a compact choice. The next figure shows BMI category by treatment uptake among diagnosed patients.

```

diagnosed |>
  filter(!is.na(bmi_cat)) |>
  ggplot(aes(x = treatment_uptake, fill = bmi_cat)) +
  geom_bar(position = "fill", width = 0.6) + # "fill" = 100% stacked

```

Most participants had primary or secondary education

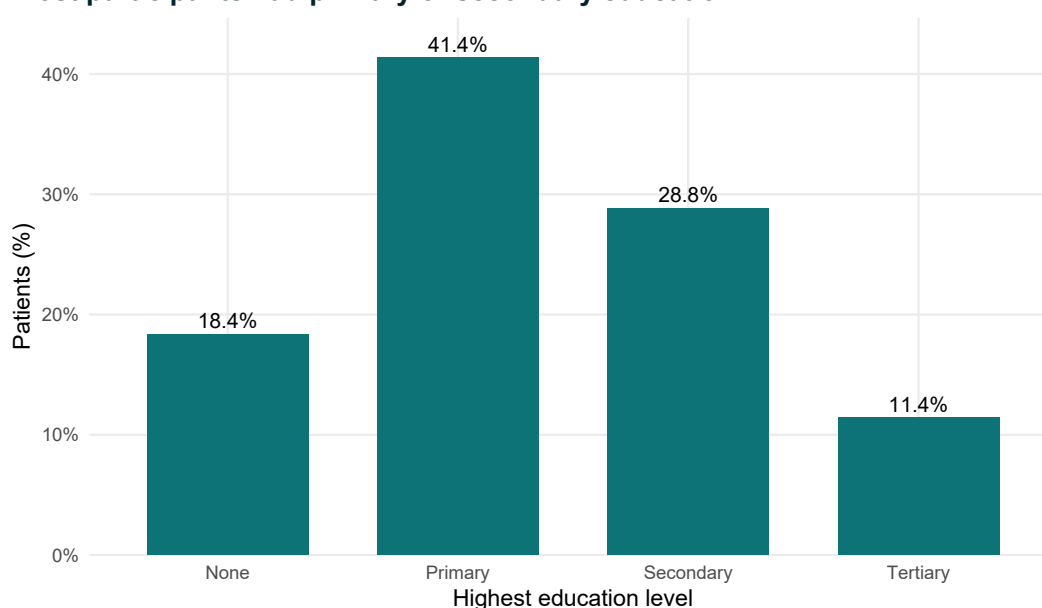


Figure 3.4: Highest level of education among all 1,500 participants; bars are in the natural order of the categories and labelled with the percentage of patients with known education (30 missing values not shown).

```
scale_y_continuous(labels = label_percent()) +
scale_fill_brewer(palette = "BuGn") + # sequential palette, ordered variable
labs(title = "BMI category by treatment uptake",
     x = "On antihypertensive treatment", y = "Patients (%)",
     fill = "BMI category")
```

position = "fill" rescales each bar to 100%, so the bars compare *proportions*, not counts. A sequential colour palette (light to dark) mirrors the order of the BMI categories. The two bars look very similar: the BMI profile of treated and untreated patients is much the same.

3.11.4 Box plots, violin plots and why bar charts of means hide data

A very common figure in clinical papers is a bar chart of group means with error bars (the so-called "dynamite plot"). Weissgerber et al. (2015) showed that such charts are widespread and misleading: many different distributions (symmetric, skewed, bimodal, with outliers, or with very unequal sample sizes) can produce the same bar. The bar also suggests that values lie between zero and the mean, which is meaningless for a measurement like blood pressure. The figure below puts the two approaches side by side for the same data.

```
# Panel A: bar of means +/- SE (the plot to avoid)
sbp_means <- diagnosed |>
  group_by(treatment_uptake) |>
  summarise(mean = mean(sbp_mmhg, na.rm = TRUE),
            se = sd(sbp_mmhg, na.rm = TRUE) / sqrt(sum(!is.na(sbp_mmhg))))

p_bar <- ggplot(sbp_means, aes(x = treatment_uptake, y = mean)) +
  geom_col(fill = "grey70", width = 0.6) +
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se), width = 0.2) +
  labs(title = "A. Bar of means ( $\pm$  SE)",
       x = "On treatment", y = "Systolic BP (mmHg)")
```

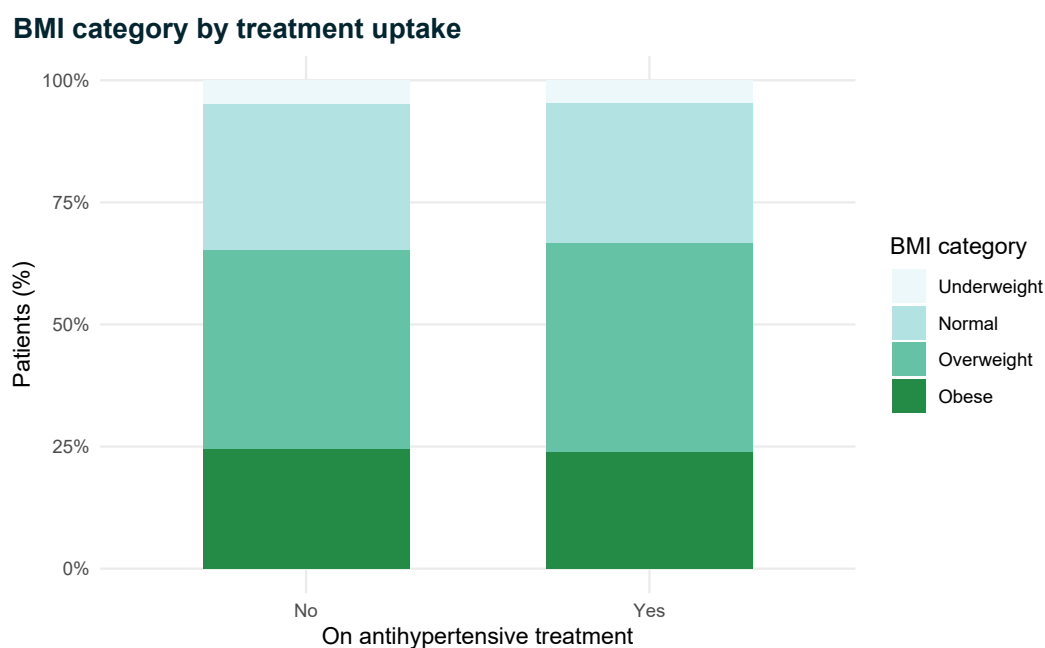


Figure 3.5: Distribution of BMI category among diagnosed patients not on treatment and on treatment; each bar sums to 100% of patients with a known BMI.

```
# Panel B: violin + box + jittered points (show the data)
p_violin <- ggplot(diagnosed, aes(x = treatment_uptake, y = sbp_mmhg)) +
  geom_violin(fill = teal, alpha = 0.25, colour = NA, na.rm = TRUE) +
  geom_jitter(width = 0.15, alpha = 0.25, size = 0.8, na.rm = TRUE) +
  geom_boxplot(width = 0.18, outlier.shape = NA, fill = "white",
               alpha = 0.8, na.rm = TRUE) +
  labs(title = "B. Violin, box and points",
       x = "On treatment", y = "Systolic BP (mmHg)")

p_bar + p_violin      # patchwork places the plots side by side
```

Panel A suggests two neatly separated groups. Panel B tells the truth: the two distributions overlap almost completely, with SBP in both groups ranging from about 100 to 200 mmHg; the difference in means of about 6 mmHg is small compared with the spread among patients. Each element of panel B carries information:

- the **violin** is a mirrored density estimate, showing the shape of the distribution;
- the **box plot** shows the median (thick line), the quartiles (box edges, so the box height is the IQR) and the *whiskers*, which extend to the most extreme values within $1.5 \times \text{IQR}$ of the box; points beyond the whiskers are conventionally flagged as potential outliers (we suppress them with `outlier.shape = NA` because the jittered points already show every patient);
- the **jittered points** are the individual patients, spread sideways at random so that they do not overlap.

The + between two saved plots is provided by **patchwork**, which composes several ggplots into one figure.

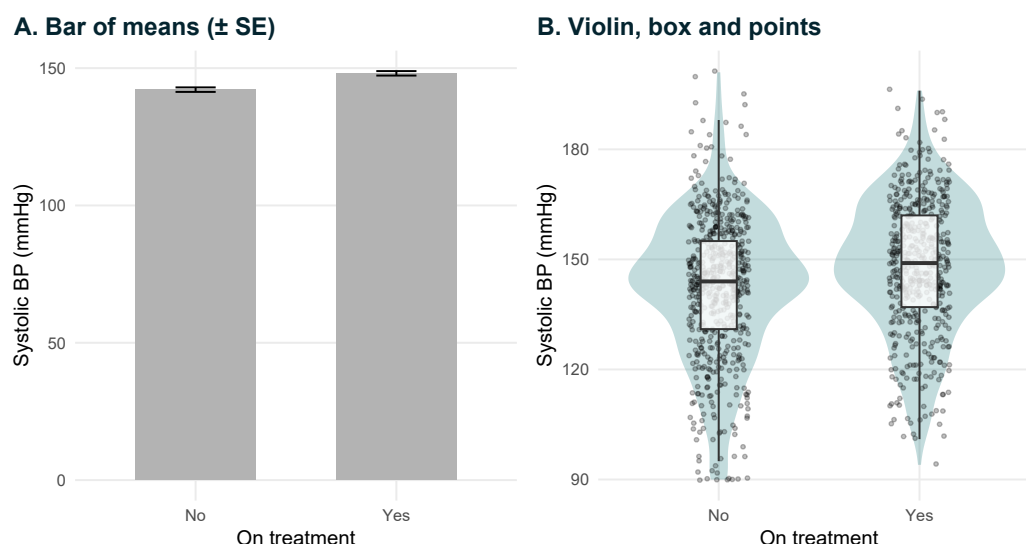


Figure 3.6: The same SBP data for diagnosed patients shown two ways: (A) a bar chart of means with standard-error bars hides the distribution; (B) a violin plot with a box plot and individual patients (jittered points) shows the full spread, the overlap between groups and any outliers.

Good practice

For a continuous outcome compared between groups, prefer a box plot, violin plot or dot plot, and add the individual points when there are fewer than a few hundred per group (Weissgerber et al. 2015). Keep bar charts for counts and percentages, where the bar's length from zero really does represent the quantity.

3.11.5 Scatter plots for two continuous variables

The relationship between two continuous variables is shown with a scatter plot. Here we plot SBP against BMI for all patients and add a fitted straight line.

```
ggplot(analysis_data, aes(x = bmi, y = sbp_mmhg)) +  
  # transparency (alpha) lets dense, overlapping regions show up darker  
  geom_point(alpha = 0.25, colour = teal, na.rm = TRUE) +  
  # straight-line (linear) trend with its 95% confidence band  
  geom_smooth(method = "lm", formula = y ~ x, colour = "grey15",  
              na.rm = TRUE) +  
  labs(title = "Higher BMI goes with higher systolic blood pressure",  
       x = expression("Body mass index (kg/m^2)"),  
       y = "Systolic BP (mmHg)")
```

Each point is a patient. Transparency ($\alpha = 0.25$) makes dense regions darker, so overplotting does not hide where most patients lie. The line, fitted by least squares with `method = "lm"`, rises from left to right: on average, patients with higher BMI have higher SBP. The grey band is the 95% confidence interval for the *mean* SBP at each BMI, narrow because there are many patients. The cloud of points around the line is wide, so BMI explains only a modest part of the variation in SBP between individuals. Measuring and testing this association (correlation) is covered in Chapter 4, and the regression line itself in Chapter 5.

Higher BMI goes with higher systolic blood pressure



Figure 3.7: Systolic blood pressure against body mass index in 1,451 patients with both measurements; each point is one patient, and the line is a least-squares linear fit with its 95% confidence band.

Clinical interpretation

The positive slope is consistent with the well-established link between excess body weight and raised blood pressure (Mills et al. 2020). In a cross-sectional study it shows association only: it cannot show that reducing BMI would lower SBP in these patients, and other factors such as age may contribute to both.

3.11.6 Faceting: small multiples

When a comparison must be repeated across subgroups, **facets** draw the same plot once per subgroup, on common axes, so that the panels can be compared directly. The figure below shows the age distribution of treated and untreated diagnosed patients separately in each facility.

```
ggplot(diagnosed, aes(x = age, fill = treatment_uptake,
                      colour = treatment_uptake)) +
  geom_density(alpha = 0.35, na.rm = TRUE) +
  facet_wrap(~ facility, ncol = 3) + # one panel per facility
  scale_fill_manual(values = c(No = "grey60", Yes = teal)) +
  scale_colour_manual(values = c(No = "grey40", Yes = teal)) +
  labs(title = "Treated patients tend to be older in every facility",
       x = "Age (years)", y = "Density",
       fill = "On treatment", colour = "On treatment") +
  theme(legend.position = "bottom")
```

In every panel the teal curve (treated) is shifted to the right of the grey one (untreated); the difference in mean age ranges from about 3 years at Kisesa HC, where the curves almost coincide, to about 7 years at Igoma HC. The age difference seen overall is therefore not produced by one unusual facility but is consistent in direction across sites. Faceting is a simple and powerful way to check whether a pattern is general or driven by a subgroup, and it anticipates the idea of stratified analysis and effect modification.

Treated patients tend to be older in every facility

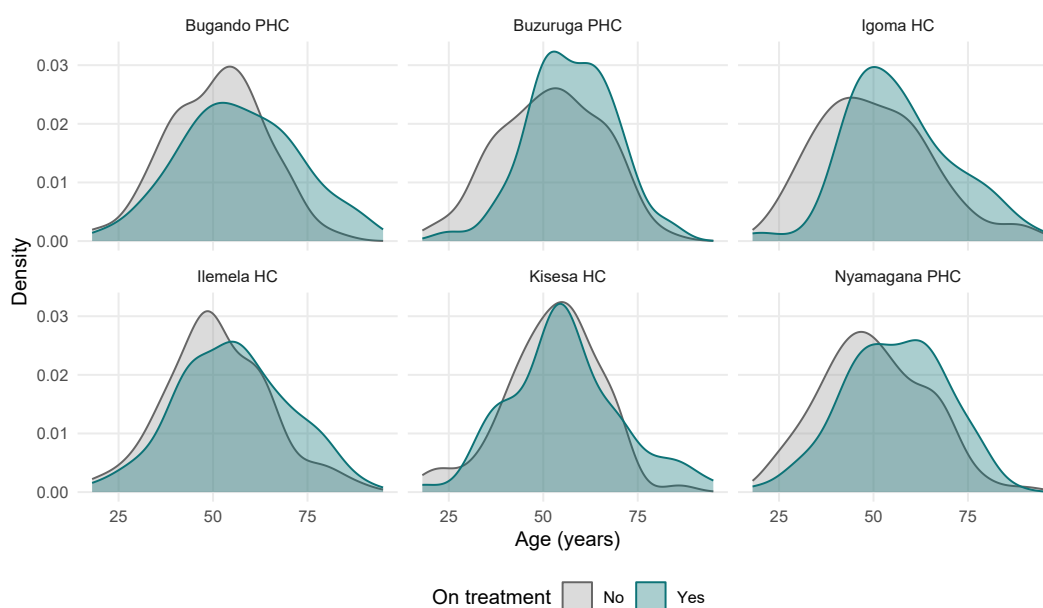


Figure 3.8: Age distribution of diagnosed patients by treatment uptake, shown separately for each of the six facilities (density curves on common axes); treated patients tend to be older in every facility.

3.12 Exporting tables and figures

A manuscript, a report to a ministry or a slide for a meeting needs files, not R output. The principle is the same as for data cleaning: the file should be produced by code, so that it can be regenerated identically whenever the data or the analysis change.

3.12.1 Figures

`ggsave()` saves a plot to a file whose format is chosen from the extension (`.png`, `.pdf`, `.tiff`, `.svg`). Journals typically ask for at least 300 dots per inch (dpi) for raster images, or a vector format such as PDF. Here we save to R's temporary directory; in your own project you would use a folder such as `outputs/`.

```
p_scatter <- ggplot(analysis_data, aes(x = bmi, y = sbp_mmhg)) +
  geom_point(alpha = 0.25, colour = teal, na.rm = TRUE) +
  labs(x = "BMI (kg/m^2)", y = "Systolic BP (mmHg)")

out_png <- file.path(tempdir(), "fig_sbp_bmi.png")
ggsave(out_png, plot = p_scatter, width = 7, height = 5, dpi = 300)
file.exists(out_png)
```

```
#> [1] TRUE
```

Always pass the plot object with `plot =` rather than relying on "the last plot displayed", and set width and height explicitly (in inches by default) so that text sizes are consistent across figures.

3.12.2 Tables

A `gtsummary` table can be converted to a **flextable**, which writes native Word tables that co-authors can edit, or to a **gt** table for HTML. Simple data frames can be written to CSV for colleagues who use other software.

```

out_docx <- file.path(tempdir(), "table1.docx")
table1 |>
  as_flex_table() |>                # gtsummary -> flextable
  flextable::save_as_docx(path = out_docx) # write a Word document

out_csv <- file.path(tempdir(), "spread_table.csv")
write_csv(spread_table, out_csv)      # plain CSV for any software

file.exists(c(out_docx, out_csv))

#> [1] TRUE TRUE

```

In a real project the same code would write to your output folder, for example:

```

# Inside your own analysis project (not run here)
ggsave("outputs/figure2_sbp_bmi.pdf", plot = p_scatter, width = 7, height = 5)
table1 |>
  as_flex_table() |>
  flextable::save_as_docx(path = "outputs/table1.docx")
table1 |> as_gt() |> gt::gtsave("outputs/table1.html")

```

Good practice

Never retype numbers from R output into a manuscript by hand. Export tables directly, and for numbers in the text use inline R code in R Markdown (as this book does), so that every number updates automatically when the data change (Xie 2015; Peng 2011).

3.13 Summary

Description is the first and most fundamental step of any clinical analysis. In this chapter we described the case-study population numerically and graphically. We matched summaries to variable types; defined the mean, median and mode, and saw how a single error moves the mean but not the median; measured spread with the range, variance, SD, quantiles, IQR and CV; examined shape through skewness and the normal distribution's 68–95–99.7 rule, and used it to choose between mean (SD) and median (IQR). We met the `na.rm` trap and the more important problem it hides, the silently changing denominator. We produced grouped summaries, computed standard errors and confidence intervals for a mean and a proportion, built frequency tables and cross-tabulations with the correct percentages, assembled a manuscript Table 1 with `gtsummary`, and drew honest figures with `ggplot2` before exporting them.

Key points

- Describe before you test: the description of participants is the first result of every study and Table 1 of every paper (STROBE items 13 and 14).

- Use mean (SD) for symmetric continuous variables and median (IQR) for skewed ones; check the shape with a histogram, not by habit.
- The SD describes variation between patients; the SE describes the precision of a mean. Never use the SE to describe a sample.
- `na.rm = TRUE` removes missing values from a calculation; always count and report them.
- A 95% CI ($\bar{x} \pm 1.96$ SE, or $\hat{p} \pm 1.96$ SE) expresses the precision of an estimate and is more informative than the estimate alone.
- Make missing values visible in frequency tables (`useNA = "ifany"`, `tabyl()`).
- Compute percentages within categories of the explanatory variable, and always state the denominator; know whether you are reading row or column percentages.
- `gtsummary::tbl_summary()` builds a reproducible Table 1; keep it descriptive and present tests and effect estimates separately.
- Show the data: prefer box, violin and dot plots to bar charts of means, and label axes with units.
- Export tables and figures by code (`ggsave()`, `as_flex_table()`), never by retyping.

3.14 Further reading

- Altman (1991), chapters 2–4: a classic, clear account of types of data, summary statistics and their presentation, written for medical researchers.
- Kirkwood and Sterne (2003), chapters 2–6: concise treatment of summarising numerical and categorical data, the normal distribution, standard errors and confidence intervals.
- Weissgerber et al. (2015): a short, persuasive paper on why bar charts of means mislead, with practical alternatives.
- Sjöberg et al. (2021): the paper describing `gtsummary`, with examples of the tables it can produce.
- Wickham (2016): the book on `ggplot2` by its author, explaining the grammar of graphics in depth.

3.15 Exercises

Exercise 3.1

Using `analysis_data`, compute the mean, SD, median and IQR of `age` and of `distance_to_facility_km`, remembering `na.rm = TRUE`. How many values are missing for each? For each variable, decide whether mean (SD) or median (IQR) is the better summary, and justify your choice using the numbers.

Exercise 3.2

For `ldl_mmol_l` (LDL cholesterol), show what `mean()` returns without `na.rm = TRUE`. Then report the number and percentage of missing values, the mean and SD among patients with

a result, and the coefficient of variation. Write one sentence reporting the result as it would appear in a paper.

Exercise 3.3

Produce frequency tables (counts and percentages) of `smoking` and `bp_category` for all patients, making missing values visible. Use both `table(..., useNA = "ifany")` and `janitor::tabyl()`. What percentage of patients are current smokers (a) among all patients and (b) among those with known smoking status?

Exercise 3.4

Among diagnosed patients (`htn_diagnosed == "Yes"`), cross-tabulate `health_insurance` by `treatment_uptake`. Compute both row and column percentages. What proportion of *insured* diagnosed patients are on treatment, and what proportion of *uninsured*? Which percentage answers the question "is insurance associated with uptake?", and why?

Exercise 3.5

Among diagnosed patients, compute the mean SBP with its standard error and 95% confidence interval separately for treated and untreated patients, using `group_by()` and `summarise()`. Then compute the proportion on treatment among rural and among urban diagnosed patients, each with a 95% CI. Explain in words the difference between the SD and the SE in your output.

Exercise 3.6

Build a Table 1 for diagnosed patients stratified by `treatment_uptake` with `gtsummary::tbl_summary()`, including `age`, `sex`, `residence`, `education`, `health_insurance`, `family_history_htn`, `knowledge_score` and `comorbidity_count`. Use mean (SD) for age and knowledge score, median (IQR) for comorbidity count, readable labels, an overall column and a missing-data row where needed. Name two characteristics that differ between the groups and say whether the differences are clinically plausible.

Exercise 3.7

Draw (a) a box plot with jittered points of `knowledge_score` by `treatment_uptake` among diagnosed patients, and (b) a bar chart of the percentage of diagnosed patients on treatment by `education`, in the natural order of education levels. Give both figures informative titles, axis labels and a caption-style sentence describing what they show.

Exercise 3.8

(Challenge.) For each facility, compute among diagnosed patients the number of patients, the percentage on treatment with its 95% Wilson CI (`prop.test()`), the median distance to the facility and the percentage insured. Present the result as a table sorted by uptake, and draw a scatter plot of facility uptake (y) against median distance (x), with points labelled by facility name. Does facility-level uptake appear related to distance? Why should conclusions from six facilities be drawn with great caution?

Common Statistical Tests in Medical Research

In Chapter 3 we described our 1,500 patients: their ages, their blood pressures, how many had health insurance and how many had taken up antihypertensive treatment. Description is essential, but it rarely answers the question a clinician or a health planner actually asks. They want to know whether patients who take up treatment are *different* from those who do not: older, better informed, closer to the clinic, more often insured. In our sample the treated patients are on average about five years older than the untreated. Is that a real feature of hypertensive patients attending primary care, or could a difference of that size easily have arisen by chance, because we happened to recruit these particular 1,089 diagnosed patients and not others? Statistical tests, together with their close relatives, confidence intervals, are the tools that let us move from *describing a sample* to *making cautious statements about the population it came from*. This chapter explains the logic behind them, shows the handful of tests that cover most questions in medical research, and, just as importantly, explains how to read and report their results without falling into the traps that have made the p-value one of the most misunderstood numbers in science.

Learning objectives

- Explain the difference between a population and a sample, and why estimates vary from sample to sample (the standard error).
- Describe the logic of a hypothesis test: null and alternative hypotheses, test statistic, null distribution and p-value.
- State correctly what a p-value is and what it is not, and explain Type I and Type II errors, power and confidence intervals.
- Define the analytic population explicitly and justify it.
- Assess the normality of a continuous variable with a histogram, a Q–Q plot and the Shapiro–Wilk test, and explain why formal normality tests mislead in large samples.
- Compare a continuous variable between two groups (Student’s and Welch’s t-tests, Wilcoxon rank-sum test) and between more than two groups (one-way ANOVA, Kruskal–Wallis test), with appropriate post-hoc comparisons.
- Test the association between two categorical variables (chi-square and Fisher’s exact tests) and estimate the risk difference, relative risk and odds ratio with confidence intervals.
- Measure the association between two continuous variables with Pearson’s and Spearman’s correlation coefficients.

- Choose an appropriate test for a research question, recognise when single-variable tests are not enough, and report results in words with an estimate, a confidence interval and a p-value.

We use the same packages as in the previous chapters and reload the clean data saved at the end of Chapter 2. Every chapter starts from that saved file; nothing is re-cleaned by hand.

```
library(tidyverse) # dplyr for data handling, ggplot2 for figures
library(broom)    # tidy() turns test and model output into data frames
library(patchwork) # places several ggplot2 panels side by side
library(gtsummary) # summary tables with p-values

course_teal <- "#0D7377" # the accent colour used for the book's figures
course_amber <- "#E8A33D" # a second colour for two-group figures

analysis_data <- readRDS("Data/analysis_data.rds")
dim(analysis_data)
```

```
#> [1] 1500 38
```

4.1 From description to inference

4.1.1 Populations, samples, parameters and statistics

Almost every clinical study observes a **sample** of people in order to learn about a larger **population**. Our study enrolled adults attending six primary healthcare facilities; the population of interest is, broadly, adults attending primary care facilities like these. The quantities that describe the population, such as the true proportion of diagnosed hypertensive patients who are on treatment or the true mean age of those who are, are called **parameters**. They are fixed but unknown. The corresponding quantities calculated from our data, such as the observed proportion on treatment, are called **statistics** or **estimates**. We use Greek letters for parameters and Roman letters for estimates: μ is a population mean and $\bar{x}$ the sample mean that estimates it; π is a population proportion and p the sample proportion (Kirkwood and Sterne 2003; Altman 1991).

Statistical inference is the business of using the estimates to say something about the parameters, together with an honest statement of how uncertain we are. It rests on a crucial assumption: that the sample can be regarded as (approximately) a random sample from the population. No statistical test can rescue a sample that is badly unrepresentative. If, for example, only the most motivated patients agreed to take part, the proportion on treatment in our sample would overestimate the proportion in the population, and a p-value would do nothing to warn us. The tests in this chapter deal with **random error** (the play of chance); they do not deal with **systematic error** (bias in selection, measurement or confounding), which must be handled through study design and, as Chapter 5 shows, partly through statistical adjustment (Rothman et al. 2008).

4.1.2 Sampling variation and the standard error

Imagine repeating our study many times, each time recruiting a different random set of patients. Each repetition would give a slightly different mean systolic blood pressure. This **sampling variation** is unavoidable, and its size depends on two things: how variable the measurement is

between individuals, and how many individuals we measure. The distribution of an estimate across hypothetical repetitions of the study is called its **sampling distribution**, and the standard deviation of that sampling distribution is called the **standard error** (SE). For a sample mean the standard error is

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}},$$

where σ is the standard deviation of the measurement in the population, estimated by the sample standard deviation s , and n is the sample size. For a proportion p the standard error is $\sqrt{p(1-p)/n}$. The distinction between the two "standard" quantities is important and frequently confused in published papers: the **standard deviation** describes how much *individuals* vary, whereas the **standard error** describes how precisely we have estimated a *summary* such as a mean (Altman 1991; Bland 2015). The standard deviation does not shrink as we recruit more patients; the standard error does, in proportion to $1/\sqrt{n}$. To halve the standard error we need four times as many patients.

We can see sampling variation directly with a small simulation. We treat the 1,088 diagnosed patients with a recorded systolic blood pressure as if they were the whole population, draw 2,000 random samples of 25 patients and 2,000 samples of 100 patients, and compute the mean systolic blood pressure of each sample.

```

dx_all <- filter(analysis_data, htn_diagnosed == "Yes")
sbp_pop <- na.omit(dx_all$sbp_mmhg) # our pretend "population"

set.seed(2026) # makes the simulation reproducible
sim_means <- tibble(
  n = rep(c(25, 100), each = 2000),
  sample_mean = c(replicate(2000, mean(sample(sbp_pop, 25))),
                  replicate(2000, mean(sample(sbp_pop, 100))))
)

# Spread of the 2,000 sample means versus the formula sigma / sqrt(n)
sim_means |>
  group_by(n) |>
  summarise(mean_of_means = mean(sample_mean),
            sd_of_means = sd(sample_mean),
            formula_se = sd(sbp_pop) / sqrt(first(n)))

#> # A tibble: 2 × 4
#>       n mean_of_means sd_of_means formula_se
#>   <dbl>      <dbl>      <dbl>      <dbl>
#> 1    25      145.        3.80        3.87
#> 2   100      145.        1.80        1.94

```

The population mean systolic blood pressure is 144.9 mmHg and its standard deviation is 19.4 mmHg. The output shows three things. First, the average of the 2,000 sample means (`mean_of_means`) is essentially equal to the population mean whatever the sample size: the sample mean is an **unbiased** estimator. Second, the sample means vary much less with samples of 100 than with samples of 25: the standard deviation of the simulated means (`sd_of_means`) is roughly halved when n is quadrupled. Third, that empirical spread agrees closely with the formula $\sigma/\sqrt{n}$ (`formula_se`); it is slightly smaller for $n = 100$ only because we are sampling without replacement from a

finite "population" of 1,088 patients. In a real study we have only one sample, but the formula lets us estimate the standard error from that one sample. Figure 4.1 shows the two sampling distributions.

```
ggplot(sim_means, aes(x = sample_mean)) +  
  geom_histogram(binwidth = 1, fill = course_teal, colour = "white") +  
  geom_vline(xintercept = mean(sbp_pop), linetype = "dashed") +  
  facet_wrap(~ n, labeller = label_both) +  
  labs(x = "Mean systolic blood pressure in the sample (mmHg)",  
       y = "Number of samples")
```

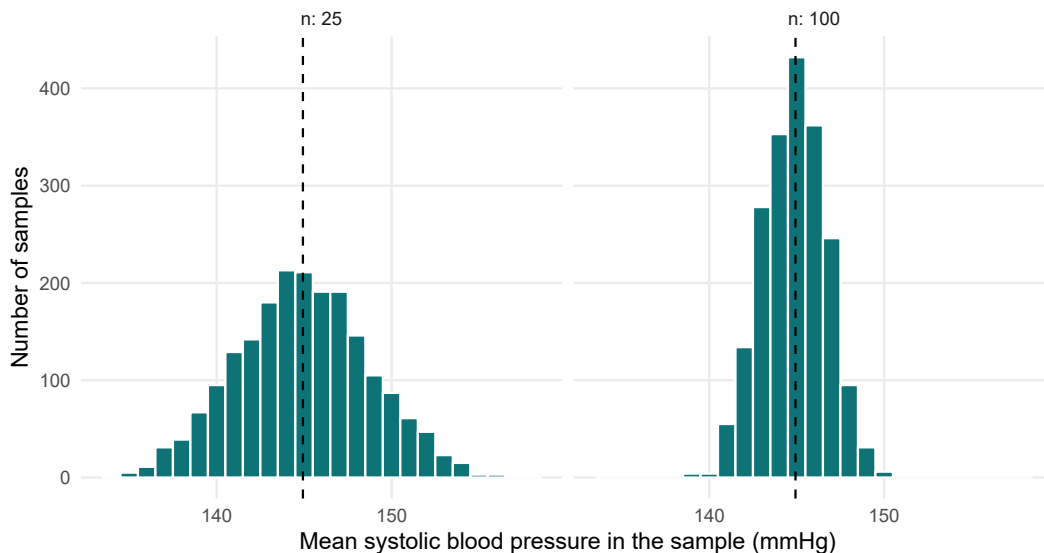


Figure 4.1: Sampling distribution of the mean systolic blood pressure in 2,000 random samples of 25 and of 100 diagnosed patients. Larger samples give means that cluster more tightly around the population mean (dashed line).

Notice also that both histograms look like the familiar bell-shaped normal curve, even though systolic blood pressure in individuals is slightly skewed. This is the **central limit theorem** at work: whatever the shape of the distribution of individual values, the sampling distribution of a mean (or a proportion) becomes approximately normal as the sample size grows (Kirkwood and Sterne 2003; Lumley et al. 2002). The theorem is the reason why so many tests based on the normal distribution work well in medium and large samples, and we return to it when we discuss normality in Section 4.3.

4.1.3 The logic of a hypothesis test

A hypothesis test asks a narrow question: *if there were truly no difference (or no association) in the population, how surprising would our data be?* The procedure has five steps (Kirkwood and Sterne 2003; Altman 1991).

1. State a **null hypothesis**, H_0 , usually of "no effect": for example, the mean distance to the facility is the same in patients who take up treatment and in those who do not, $H_0 : \mu_{\text{Yes}} = \mu_{\text{No}}$.
2. State the **alternative hypothesis**, H_1 , usually that there is some difference in either direction, $H_1 : \mu_{\text{Yes}} \neq \mu_{\text{No}}$ (a *two-sided* alternative).
3. Compute a **test statistic** from the data: a single number that measures how far the data

depart from what H_0 predicts, typically in units of standard error,

$$\text{test statistic} = \frac{\text{observed estimate} - \text{value under } H_0}{\text{standard error of the estimate}}.$$

4. Work out the **null distribution**: the distribution the test statistic would have if H_0 were true. For many tests this is a known theoretical distribution (normal, t , χ^2 or F).
5. Compare the observed statistic with the null distribution to obtain the **p-value**.

The null distribution is the conceptual heart of the test, and it can be built without any theory at all. Suppose that treatment uptake had nothing to do with distance to the facility. Then the labels "Yes" and "No" attached to the patients would be arbitrary, and shuffling them at random should produce differences in mean distance similar to the one we observed. This is a **permutation test**. We shuffle the labels 5,000 times and record the difference in mean distance each time.

```
perm_data <- filter(dx_all, !is.na(distance_to_facility_km))

# Observed difference in mean distance, No minus Yes (km)
obs_diff <- with(perm_data,
  mean(distance_to_facility_km[treatment_uptake == "No"]) -
  mean(distance_to_facility_km[treatment_uptake == "Yes"]))
obs_diff

#> [1] 1.0777
```

```
set.seed(2026)
perm_diffs <- replicate(5000, {
  shuffled <- sample(perm_data$treatment_uptake) # break any real link
  mean(perm_data$distance_to_facility_km[shuffled == "No"]) -
  mean(perm_data$distance_to_facility_km[shuffled == "Yes"])
})

# Two-sided p-value: proportion of shuffles at least as extreme as observed
mean(abs(perm_diffs) >= abs(obs_diff))

#> [1] 0.0034
```

Untreated patients lived on average about 1.08 km further from the facility than treated patients. Among 5,000 random shufflings, a difference at least that large in either direction occurred only a handful of times, so the permutation p-value is about 0.003. Figure 4.2 shows the null distribution and where the observed difference falls.

```
tibble(diff = perm_diffs) |>
  mutate(extreme = abs(diff) >= abs(obs_diff)) |>
  ggplot(aes(x = diff, fill = extreme)) +
  geom_histogram(binwidth = 0.05, colour = "white", linewidth = 0.1) +
  geom_vline(xintercept = obs_diff, linewidth = 0.8) +
  geom_vline(xintercept = -obs_diff, linetype = "dashed") +
  scale_fill_manual(values = c(`FALSE` = "grey70", `TRUE` = "darkred"),
    guide = "none") +
  labs(x = "Difference in mean distance under H0 (km)",
    y = "Number of shuffles")
```

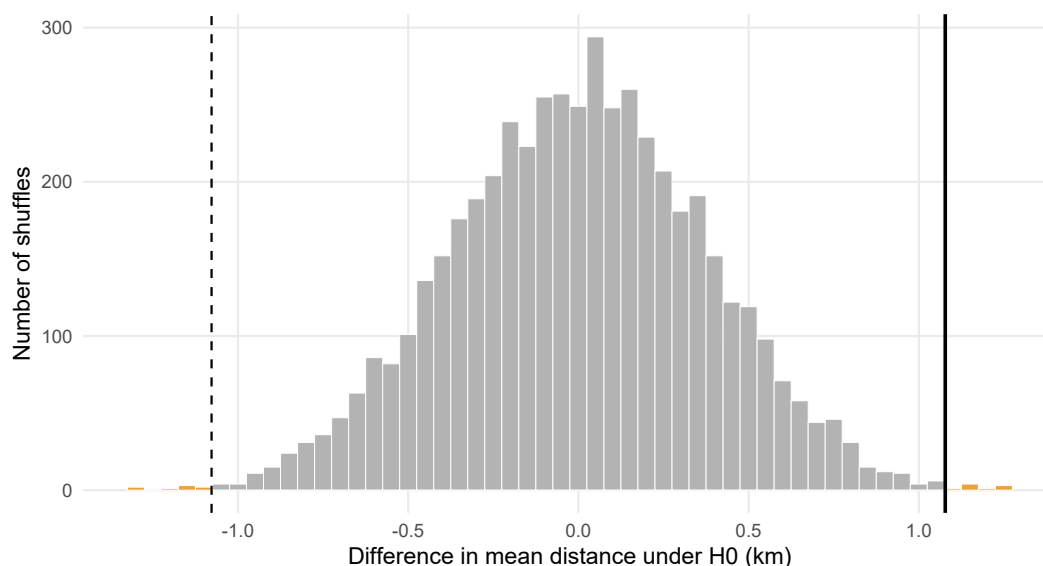


Figure 4.2: Permutation null distribution of the difference in mean distance to the facility (untreated minus treated) when treatment labels are shuffled at random. The solid line marks the observed difference and the dashed line its mirror image; the shaded tails contain the shuffles at least as extreme as the data.

The grey histogram is what chance alone produces: differences centred on zero, mostly within about ± 0.7 km. Our observed difference of 1.08 km sits far out in the right tail. Either we have witnessed a rare event, or the null hypothesis is wrong. The t-test of Section 4.4 reaches the same conclusion using a theoretical t distribution instead of shuffling, and gives almost exactly the same p-value (0.003).

4.1.4 What a p-value is, and what it is not

The **p-value** is the probability, *calculated assuming that the null hypothesis (and every other assumption of the test) is true*, of obtaining a test statistic at least as extreme as the one observed. A small p-value means that the data would be unusual if H_0 were true; it is evidence against H_0 . The smaller the p-value, the stronger the evidence (Kirkwood and Sterne 2003).

In 2016 the American Statistical Association issued an unusual public statement on p-values because their misuse had become so widespread (Wasserstein and Lazar 2016). Its six principles, together with the longer catalogue of misinterpretations by Greenland et al. (2016), can be summarised as follows.

- A p-value indicates how incompatible the data are with a specified statistical model (the null hypothesis plus the test's assumptions).
- A p-value is **not** the probability that the null hypothesis is true, nor the probability that the results arose "by chance alone". A p-value of 0.003 does not mean there is a 0.3% chance that distance is unrelated to uptake.
- Scientific conclusions and clinical decisions should not be based only on whether a p-value passes a threshold such as 0.05.
- Proper inference requires full reporting and transparency: running many tests and reporting only the "significant" ones invalidates the p-values.

- A p-value does **not** measure the size of an effect or the importance of a result. A tiny, clinically irrelevant difference can have a very small p-value in a large study.
- By itself a p-value is not a good measure of evidence about a hypothesis; it should be accompanied by an estimate of the effect and its confidence interval.

Common mistake

Writing "p = 0.003, so there is a 99.7% probability that the difference is real" is wrong. The p-value is computed *assuming* the null hypothesis is true, so it cannot be the probability that the null hypothesis is true. Equally wrong is "p = 0.40, so there is no difference": a large p-value only says the data are compatible with no difference; they may also be compatible with a large one.

4.1.5 Significance level, Type I and Type II errors, and power

Many studies turn the p-value into a decision by comparing it with a pre-chosen **significance level** α , conventionally 0.05: if $p < \alpha$ the result is called "statistically significant" and H_0 is "rejected". The threshold is a convention, not a law of nature, and there is nothing materially different between $p = 0.049$ and $p = 0.051$ (Wasserstein and Lazar 2016). Thinking in terms of decisions does, however, clarify the two kinds of mistake a test can make (Table 4.1).

Table 4.1: The four possible outcomes of a hypothesis test.

Decision from the test	H_0 actually true	H_0 actually false
Reject H_0 ("significant")	Type I error (false positive); probability α	Correct; probability $1 - \beta = \text{power}$
Do not reject H_0	Correct; probability $1 - \alpha$	Type II error (false negative); probability β

A **Type I error** is declaring a difference when none exists; by construction its probability is α for a single test. A **Type II error** is failing to detect a difference that does exist; its probability is called β . The **power** of a study, $1 - \beta$, is the probability that it will detect a true effect of a specified size. Power increases with the size of the true effect, with the sample size and with the significance level, and decreases as the variability of the outcome increases (Altman 1991; Kirkwood and Sterne 2003). Clinical studies are usually designed to have 80% or 90% power for the smallest effect that would be clinically important. R's `power.t.test()` performs the standard calculation for comparing two means.

```
# Patients needed per group to detect a 5 mmHg difference in mean SBP,
# with SD 19 mmHg (as in our data), two-sided alpha = 0.05, 80% power
power.t.test(delta = 5, sd = 19, sig.level = 0.05, power = 0.80)
```

```
#>
#>      Two-sample t test power calculation
#>
#>              n = 227.64
#>            delta = 5
#>              sd = 19
#>    sig.level = 0.05
#>        power = 0.8
#> alternative = two.sided
#>
```

```
#> NOTE: n is number in *each* group
```

To detect a 5 mmHg difference in mean systolic blood pressure, given a standard deviation of 19 mmHg, with 80% power at the 5% level, about 228 patients are needed in each group (R reports a fractional n , which we round up). Our study has more than 500 patients per group, so it has high power for differences of that size, which also means it will flag rather small differences as statistically significant.

4.1.6 Confidence intervals and p-values

A **confidence interval** (CI) gives a range of values for the population parameter that are compatible with the data. For an estimate whose sampling distribution is approximately normal, the 95% confidence interval is

$$\text{estimate} \pm 1.96 \times \text{SE}(\text{estimate}).$$

The interpretation is about the procedure: if we repeated the study many times and computed a 95% CI each time, about 95% of those intervals would contain the true parameter (Altman 1991; Kirkwood and Sterne 2003). Informally, the interval shows the range of population values with which our data are reasonably consistent.

Confidence intervals and two-sided p-values are two views of the same calculation. If the 95% CI for a difference excludes zero (or the 95% CI for a ratio excludes one), then $p < 0.05$, and vice versa. But the interval carries far more information than the p-value: it shows the **size** of the effect, its **direction**, and the **precision** with which it has been estimated. A wide interval warns that the study cannot pin down the effect; a narrow interval that lies entirely within a range of clinically unimportant values tells us that any effect is too small to matter. For this reason medical journals and reporting guidelines ask authors to report effect estimates with confidence intervals, and to treat p-values as secondary (Altman 1991; Elm et al. 2007).

4.1.7 Statistical significance versus clinical significance

"Statistically significant" means only that the data are hard to reconcile with the null hypothesis. It does not mean that the effect is large, important or useful. In a large study, a difference in mean systolic blood pressure of 1 mmHg may be statistically significant but is of no clinical consequence for an individual patient. Conversely, in a small study, a clinically important reduction of 10 mmHg may fail to reach significance because the confidence interval is wide. The question "is it significant?" should always be followed by "how big is it, and does that size matter for patients?" Answering the second question requires clinical knowledge, not statistics: for example, knowing that a sustained 10 mmHg fall in systolic pressure substantially reduces cardiovascular risk (World Health Organization 2021).

4.1.8 Absence of evidence is not evidence of absence

A non-significant result is often misreported as showing that "there is no difference" or "the treatment has no effect". Altman and Bland (1995a) called this one of the most common errors in medical research. A large p-value means only that the study did not find convincing evidence of a difference; it may simply have been too small, or the outcome too variable, to detect one. The confidence

interval reveals which situation we are in. If the interval is narrow and centred near zero, we can reasonably conclude that any difference is small. If it is wide and includes clinically important values, the study is inconclusive and the honest summary is "we cannot tell".

Good practice

Report every comparison as an **estimate**, its **95% confidence interval** and the **exact p-value** (for example $p = 0.003$, not " $p < 0.05$ " or "NS"), and interpret the estimate in clinical terms. Very small p-values may be given as $p < 0.001$.

4.2 Defining the analytic population

Before any test, we must decide exactly *whose* data answer the question. The primary outcome of the study is **treatment uptake**: whether a patient is currently taking antihypertensive medication. That outcome is only meaningful for patients who have been diagnosed with hypertension. A patient who has never been told that their blood pressure is raised cannot have "failed to take up" treatment; their outcome is not "No", it is undefined. Including them would mix two different questions (who gets diagnosed, and who is treated once diagnosed) and would bias every comparison. The **analytic population** for uptake is therefore the patients with `htn_diagnosed == "Yes"`, and we define it once, explicitly, and use the same object throughout the chapter.

```
# The analytic population: patients already diagnosed with hypertension
dx <- analysis_data |>
  filter(htn_diagnosed == "Yes")

nrow(dx)                                # number of patients analysed
```

```
#> [1] 1089
```

```
dx |> count(treatment_uptake) |>
  mutate(percent = round(100 * n / sum(n), 1))
```

```
#> # A tibble: 2 × 3
#>   treatment_uptake     n percent
#>   <fct>           <int>   <dbl>
#> 1 No              581     53.4
#> 2 Yes              508     46.6
```

Of the 1,500 patients, 1,089 had been diagnosed with hypertension, and 508 of them (46.6%) were on treatment. Every result in the rest of this chapter refers to these 1,089 patients. Note that some variables have missing values within this group (for example 47 patients have no recorded distance to the facility); R's test functions drop missing values silently, so the number of patients contributing to each test can be slightly smaller than 1,089. A good report states the number analysed for every result.

Common mistake

Running the uptake analysis on all 1,500 patients. The 411 undiagnosed patients would all appear as "No" for treatment, which artificially lowers uptake and distorts every comparison

(for instance, any characteristic linked to *being diagnosed* would appear to be linked to *being treated*). Define the analytic population first, write it down in the methods, and use it consistently.

The data are simulated for teaching. The associations they contain were built in to illustrate methods; the numbers in this chapter should not be read as real clinical findings about hypertension care.

4.3 Checking the normality assumption

Several classical tests, called **parametric** tests, are derived under the assumption that the data (or, more precisely, the residuals or the values within each group) follow a **normal distribution**: the t-test, analysis of variance and Pearson's correlation coefficient. Their **non-parametric** counterparts (the Wilcoxon rank-sum test, the Kruskal–Wallis test and Spearman's correlation) replace the observed values by their ranks and do not need that assumption (Altman 1991; Bland 2015). Deciding between them requires a look at the shape of the data.

The normal distribution is symmetric and bell-shaped, and is fully described by its mean μ and standard deviation σ : about 68% of values lie within one standard deviation of the mean and 95% within 1.96 standard deviations (Altman and Bland 1995b). We examine three continuous variables in the analytic population: age, systolic blood pressure and distance to the facility.

4.3.1 Look first: histograms

```

p_age <- ggplot(dx, aes(x = age)) +
  geom_histogram(binwidth = 4, fill = course_teal, colour = "white") +
  labs(x = "Age (years)", y = "Patients")
p_sbp <- ggplot(dx, aes(x = sbp_mmhg)) +
  geom_histogram(binwidth = 6, fill = course_teal, colour = "white") +
  labs(x = "Systolic BP (mmHg)", y = NULL)
p_dist <- ggplot(dx, aes(x = distance_to_facility_km)) +
  geom_histogram(binwidth = 1.5, fill = course_teal, colour = "white") +
  labs(x = "Distance (km)", y = NULL)

p_age + p_sbp + p_dist      # patchwork places the three panels side by side

```

A histogram shows the overall shape. Age and systolic blood pressure are roughly symmetric and unimodal; systolic pressure has a slightly longer right tail. Distance to the facility is clearly **right-skewed**: most patients live within about 10 km but a few live more than 25 km away. For a skewed variable the mean is pulled towards the long tail (here the mean distance is 7.7 km but the median only 6.3 km), and the median and interquartile range are better summaries (Chapter 3).

4.3.2 Q–Q plots

A **quantile–quantile (Q–Q) plot** is a sharper tool. It plots the ordered data values against the values we would expect if the data came from a normal distribution with the same mean and standard deviation. If the data are normal, the points lie close to a straight line. Systematic curvature indicates skewness; points that peel away at both ends in opposite directions indicate heavier (or lighter) tails than the normal distribution (Altman and Bland 1995b; Bland 2015).

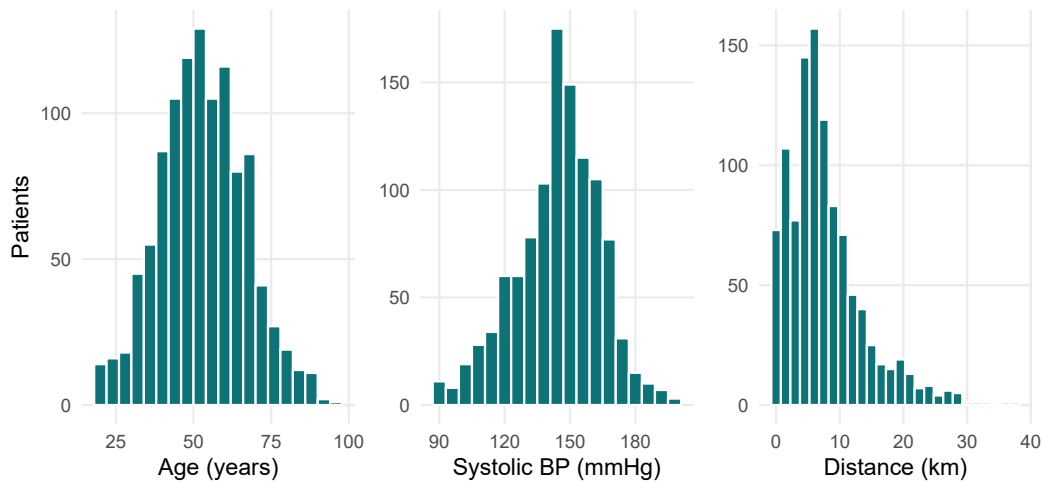


Figure 4.3: Histograms of age, systolic blood pressure and distance to the facility among diagnosed patients. Age and systolic blood pressure are roughly symmetric; distance is strongly right-skewed.

```

qq_panel <- function(var, label) {
  ggplot(dx, aes(sample = .data[[var]])) +
    stat_qq(size = 0.6, alpha = 0.5, colour = course_teal) +
    stat_qq_line(colour = "black") +
    labs(title = label, x = "Theoretical quantiles", y = "Sample quantiles")
}

qq_panel("age", "Age") +
  qq_panel("sbp_mmhg", "Systolic BP") +
  qq_panel("distance_to_facility_km", "Distance")

```

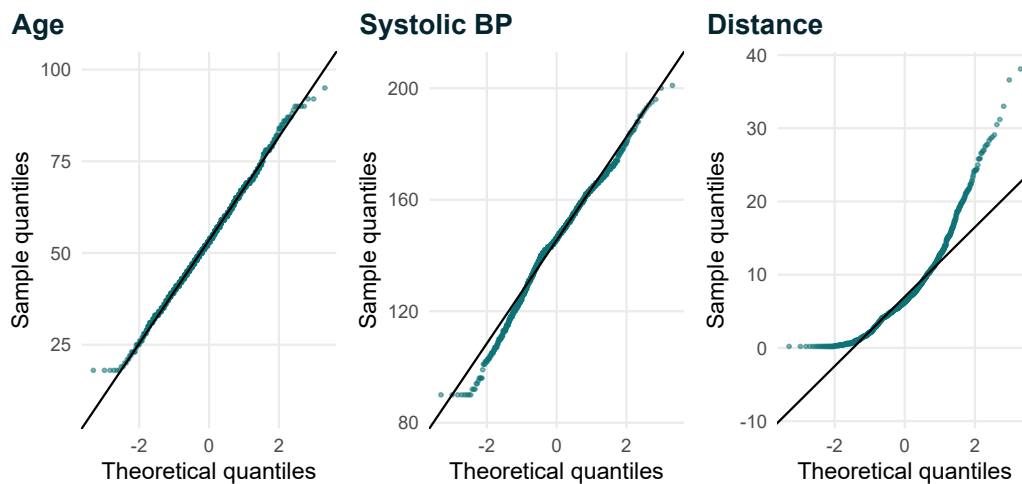


Figure 4.4: Normal Q-Q plots for age, systolic blood pressure and distance to the facility. Points close to the reference line indicate approximate normality; the curve for distance shows marked right skew.

The points for age follow the line almost perfectly, apart from a short horizontal run at the lower end: the study enrolled adults only, so the left tail is cut off at 18 years. Systolic blood pressure follows the line well through the middle of the distribution, with a horizontal run at the bottom (a floor of very low readings) and a gentle S-shape; these are modest departures that affect only the extreme few per cent of patients. Distance curves upwards in a banana shape: the largest

distances are far greater than a normal distribution would produce, the signature of right skew.

4.3.3 Formal tests of normality: Shapiro–Wilk

The **Shapiro–Wilk test** (Shapiro and Wilk 1965) tests the null hypothesis that the data come from a normal distribution. Its statistic W compares an optimally weighted combination of the ordered observations with the sample variance; W is close to 1 for normal data and smaller for non-normal data. In R, `shapiro.test()` accepts between 3 and 5,000 observations.

```
shapiro.test(dx$age)
```

R

```
#>
#>      Shapiro-Wilk normality test
#>
#> data:  dx$age
#> W = 0.997, p-value = 0.056
```

```
shapiro.test(dx$sbp_mmhg)
```

R

```
#>
#>      Shapiro-Wilk normality test
#>
#> data:  dx$sbp_mmhg
#> W = 0.988, p-value = 7.5e-08
```

```
shapiro.test(dx$distance_to_facility_km)
```

R

```
#>
#>      Shapiro-Wilk normality test
#>
#> data:  dx$distance_to_facility_km
#> W = 0.888, p-value <2e-16
```

Reading the output: each block names the test and the data, then gives W and the p-value. For age $W = 0.997$ and $p = 0.056$, so there is no strong evidence against normality. For distance $W = 0.888$ and $p < 2.2 \times 10^{-16}$ (R's way of saying "smaller than the smallest number it reports"), confirming the obvious skew. The interesting case is systolic blood pressure: $W = 0.988$, very close to 1, yet $p = 7.5 \times 10^{-8}$. The test "rejects normality" decisively for a variable whose histogram and Q–Q plot look nearly normal.

4.3.4 Why tests of normality mislead in large samples

This is the general problem with normality tests. Like every test, the Shapiro–Wilk test gains power as the sample grows. With more than a thousand observations it detects departures from normality that are far too small to matter for any practical purpose. With a small sample the opposite happens: the test has little power and can fail to detect serious skewness. So the test is most likely to reject normality precisely when normality matters least (large samples) and to miss non-normality when it matters most (small samples).

The reason normality matters least in large samples is the central limit theorem. The t-test and

ANOVA compare *means*, and the sampling distribution of a mean is close to normal in large samples whatever the distribution of the individual values, as Figure 4.1 showed. Lumley et al. (2002) reviewed the evidence and showed that in public health data sets of the usual size the t-test and linear regression give valid results even for markedly skewed outcomes; what matters more is that the mean is a sensible summary of the question. The normality assumption therefore deserves most attention when samples are small (say fewer than 30 per group) or when the data are extremely skewed or contain gross outliers.

```
# The same test on a random sample of only 50 ages: much less power
set.seed(1)
shapiro.test(sample(dx$age, 50))
```

R

```
#>
#>          Shapiro-Wilk normality test
#>
#> data:  sample(dx$age, 50)
#> W = 0.97, p-value = 0.23
```

Good practice

Decide on parametric versus non-parametric methods by **looking** (histogram, Q–Q plot), by considering the **sample size** and by asking whether the **mean** is a meaningful summary of the question. Do not let a Shapiro–Wilk p-value make the decision on its own. In large samples, roughly symmetric data can be analysed with t-tests and ANOVA; strongly skewed data, ordinal scores and small samples point to rank-based methods (or to a transformation, such as the logarithm, before a parametric analysis).

4.4 Comparing two groups: a continuous outcome

Our first substantive question is whether patients who take up treatment differ in age from those who do not. The outcome of the comparison is a continuous variable (age) and the grouping variable has two categories (uptake Yes or No).

4.4.1 Independent and paired samples

The right test depends on how the two sets of measurements were obtained.

- **Independent samples:** the two groups contain different, unrelated individuals, such as treated and untreated patients. Each patient contributes one measurement.
- **Paired samples:** each observation in one group is linked to a specific observation in the other, such as blood pressure measured in the same patients before and after starting treatment, or in matched case–control pairs. The pairs are analysed through their *differences*, which removes the between-person variation and usually gives much more precision.

For paired data, the **paired t-test** is simply a one-sample t-test on the within-pair differences d_i , with statistic $t = \bar{d}/(s_d/\sqrt{n})$ on $n - 1$ degrees of freedom, where n is the number of pairs; its rank-based counterpart is the **Wilcoxon signed-rank test** (Altman 1991; Kirkwood and Sterne 2003). In R they are `t.test(x, y, paired = TRUE)` and `wilcox.test(x, y, paired = TRUE)`.

Our cross-sectional study measures each patient once, so all comparisons in this chapter are between independent groups. Analysing paired data as if they were independent (or the reverse) is a serious error that gives wrong standard errors.

4.4.2 Student's t-test

The **two-sample t-test** was derived by W. S. Gosset, writing as "Student" while working for the Guinness brewery (Student 1908). For two independent groups with means $\bar{x}_1$, $\bar{x}_2$, standard deviations s_1 , s_2 and sizes n_1 , n_2 , it assumes that the outcome is normally distributed in each group *with the same variance*. The common variance is estimated by pooling the two groups,

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2},$$

and the test statistic is

$$t = \frac{\bar{x}_1 - \bar{x}_2}{s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}},$$

which under $H_0 : \mu_1 = \mu_2$ follows a t distribution with $n_1 + n_2 - 2$ degrees of freedom. The t distribution looks like the normal distribution but has heavier tails, allowing for the extra uncertainty from estimating the standard deviation; with more than a few dozen degrees of freedom it is practically identical to the normal. The 95% confidence interval for the difference in means is $(\bar{x}_1 - \bar{x}_2) \pm t_{0.975, n_1+n_2-2} \times s_p \sqrt{1/n_1 + 1/n_2}$.

4.4.3 Welch's t-test

If the two groups have different variances, the pooled standard error is wrong, especially when the groups also differ in size. Welch (1947) proposed using the separate variances,

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}},$$

compared with a t distribution whose degrees of freedom are given by the Welch–Satterthwaite approximation,

$$\nu \approx \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}},$$

which is usually not a whole number. Welch's test loses very little when the variances are in fact equal and protects against error when they are not, so R uses it by default and it is the sensible routine choice.

4.4.4 The t-test in R

First, a numerical summary of age by group, so that we know what the test is comparing.

```
dx |>
  group_by(treatment_uptake) |>
  summarise(n      = sum(!is.na(age)),
            mean    = mean(age, na.rm = TRUE),
            sd      = sd(age, na.rm = TRUE))
```

```
#> # A tibble: 2 × 4
#>   treatment_uptake     n  mean    sd
#>   <fct>           <int> <dbl> <dbl>
#> 1 No              580  51.1  13.4
#> 2 Yes              508  56.3  14.1
```

Treated patients are on average about 5 years older (56.3 versus 51.1 years), and the standard deviations are similar (13.4 and 14.1 years). Now the test. The formula `age ~ treatment_uptake` reads "age by treatment uptake".

```
t.test(age ~ treatment_uptake, data = dx)
```

```
#>
#>      Welch Two Sample t-test
#>
#> data:  age by treatment_uptake
#> t = -6.18, df = 1050, p-value = 8.9e-10
#> alternative hypothesis: true difference in means between group No and group
#>   Yes is not equal to 0
#> 95 percent confidence interval:
#>  -6.8184 -3.5338
#> sample estimates:
#>  mean in group No mean in group Yes
#>           51.103           56.280
```

Reading the output line by line:

- **Welch Two Sample t-test:** R used Welch's version (unequal variances allowed).
- **data: age by treatment_uptake:** the outcome and the grouping variable.
- **t = -6.18:** the test statistic. The difference in means is about 6.2 standard errors away from zero. It is negative because R subtracts in the order of the factor levels, "No" minus "Yes", and the "No" group is younger.
- **df = 1050:** the Welch–Satterthwaite degrees of freedom, rounded for printing (the exact value is 1049.6; Welch degrees of freedom are usually not whole numbers).
- **p-value = 8.9e-10:** 8.9×10^{-10} , i.e. $p < 0.001$. If mean age were truly identical in the two populations, a difference this large would almost never occur by chance.
- **alternative hypothesis:** confirms a two-sided test of whether the difference (No minus Yes) is not equal to 0.
- **95 percent confidence interval: -6.82 -3.53:** the difference in mean age (No minus Yes) is estimated to lie between -6.8 and -3.5 years. Equivalently, treated patients are between 3.5

and 6.8 years older on average.

- **sample estimates:** the two group means, 51.1 and 56.3 years.

For comparison, Student's version with a pooled variance is requested with `var.equal = TRUE`.

```
t.test(age ~ treatment_uptake, data = dx, var.equal = TRUE) R

#>
#>          Two Sample t-test
#>
#> data:  age by treatment_uptake
#> t = -6.21, df = 1086, p-value = 7.7e-10
#> alternative hypothesis: true difference in means between group No and group
#>   Yes is not equal to 0
#> 95 percent confidence interval:
#>  -6.8125 -3.5397
#> sample estimates:
#>  mean in group No mean in group Yes
#>           51.103           56.280
```

With large groups and similar standard deviations the two versions agree to the first decimal place: $t = -6.21$ on 1,086 degrees of freedom, the same confidence interval and the same conclusion. They diverge only when variances and group sizes differ substantially, and in that situation Welch's test is the trustworthy one.

The function `broom::tidy()` turns the test result into a one-row data frame, which is convenient for building tables of results.

```
t.test(age ~ treatment_uptake, data = dx) |> R
  tidy() |>
  select(estimate, estimate1, estimate2, statistic, p.value,
         conf.low, conf.high)

#> # A tibble: 1 × 7
#>   estimate estimate1 estimate2 statistic  p.value conf.low conf.high
#>   <dbl>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>    <dbl>
#> 1    -5.18       51.1       56.3     -6.18 8.92e-10    -6.82    -3.53
```

4.4.5 Effect size: the difference in means

The most useful effect size for a continuous outcome measured in clinically meaningful units is simply the **difference in means with its 95% confidence interval**: treated patients were on average 5.2 years older (95% CI 3.5 to 6.8 years). This is directly interpretable. Standardised effect sizes, such as Cohen's d (the difference in means divided by the pooled standard deviation, here about $5.2/13.7 \approx 0.38$), are useful for comparing outcomes measured on different scales or for sample size planning, but they are less transparent to clinical readers and should supplement, not replace, the raw difference (Kirkwood and Sterne 2003).

Clinical interpretation

Among diagnosed hypertensive patients, those on treatment were on average 5.2 years older than those not on treatment (95% CI 3.5 to 6.8 years; $p < 0.001$). Older patients are more likely to be in regular contact with health services and to have accumulated cardiovascular risk, so this pattern is plausible. It is an association in a cross-sectional sample, not evidence that ageing *causes* uptake, and other factors that change with age may explain part of it (Section 4.9).

4.4.6 The Wilcoxon rank-sum (Mann–Whitney) test

When the outcome is skewed, ordinal, or the samples are small, a rank-based test is preferable. The **Wilcoxon rank-sum test**, which is equivalent to the **Mann–Whitney U test** (Mann and Whitney 1947), pools the two groups, ranks all observations from smallest to largest (tied values share the average rank), and compares the ranks between groups. If R_1 is the sum of the ranks in group 1, the statistic is

$$W = R_1 - \frac{n_1(n_1 + 1)}{2},$$

which counts the number of pairs (one patient from each group) in which the group 1 value exceeds the group 2 value. Under H_0 that the two distributions are identical, W has a known distribution with mean $n_1 n_2 / 2$; for large samples a normal approximation is used. Because it uses only ranks, the test is unaffected by the size of extreme values. It tests whether values in one group tend to be larger than in the other; it is a test of medians only if the two distributions have the same shape (Altman 1991; Bland 2015).

Distance to the facility is strongly skewed, so we compare it between uptake groups with the rank-sum test. First the medians and quartiles:

```
dx |>
  group_by(treatment_uptake) |>
  summarise(n      = sum(!is.na(distance_to_facility_km)),
            median = median(distance_to_facility_km, na.rm = TRUE),
            q1     = quantile(distance_to_facility_km, 0.25, na.rm = TRUE),
            q3     = quantile(distance_to_facility_km, 0.75, na.rm = TRUE))
```

```
#> # A tibble: 2 × 5
#>   treatment_uptake     n median    q1    q3
#>   <fct>          <int> <dbl> <dbl> <dbl>
#> 1 No             561    6.8    4.3   10.7
#> 2 Yes            481    5.8    3.1    9.6
```

```
wilcox.test(distance_to_facility_km ~ treatment_uptake, data = dx,
             conf.int = TRUE)
```

```
#>
#>      Wilcoxon rank sum test with continuity correction
#>
#> data:  distance_to_facility_km by treatment_uptake
#> W = 151283, p-value = 0.00073
```

```
#> alternative hypothesis: true location shift is not equal to 0
#> 95 percent confidence interval:
#>  0.39997 1.50000
#> sample estimates:
#> difference in location
#>                0.99998
```

Reading the output:

- **Wilcoxon rank sum test with continuity correction:** R used the normal approximation (with a small continuity correction) because the samples are large and there are tied values.
- **W = 151283:** the rank-sum statistic for the first group ("No"). Its absolute size is not interpretable on its own.
- **p-value = 0.00073:** strong evidence that distances tend to differ between the groups.
- **95 percent confidence interval: 0.40 1.50 and difference in location 1.00** (printed as 0.39997, 1.50000 and 0.99998, because the estimate is found numerically): because we asked for `conf.int = TRUE`, R also reports the **Hodges–Lehmann estimate**, the median of all pairwise differences between a "No" patient and a "Yes" patient. Untreated patients typically live about 1.0 km further away (95% CI 0.4 to 1.5 km).

The medians tell the same story: 6.8 km (IQR 4.3 to 10.7) among untreated and 5.8 km (IQR 3.1 to 9.6) among treated patients. The rank-based Hodges–Lehmann estimate is the natural effect size to report alongside this test. For comparison, the Welch t-test on the same skewed variable:

```
t.test(distance_to_facility_km ~ treatment_uptake, data = dx) |>
  tidy() |>
  select(estimate, statistic, p.value, conf.low, conf.high)
```

```
#> # A tibble: 1 × 5
#>   estimate statistic p.value conf.low conf.high
#>   <dbl>      <dbl>   <dbl>   <dbl>   <dbl>
#> 1     1.08      2.95 0.00326    0.361    1.79
```

The t-test estimates a difference in *means* of 1.08 km (95% CI 0.36 to 1.79 km, $p = 0.003$), the same p-value as the permutation test of Section 4.1. With more than 500 patients per group the central limit theorem protects the t-test even for a skewed variable, and both tests reach the same conclusion. The choice between them is therefore partly about what we want to describe: a difference in *means* (relevant, for example, to the average travel burden) or a typical *shift* in the distribution.

```
# The rank-sum test for age, for comparison with the t-test
wilcox.test(age ~ treatment_uptake, data = dx)
```

```
#>
#>      Wilcoxon rank sum test with continuity correction
#>
#> data:  age by treatment_uptake
#> W = 117011, p-value = 4.5e-09
#> alternative hypothesis: true location shift is not equal to 0
```

For age, which is roughly normal, the rank-sum test ($p = 4.5 \times 10^{-9}$) agrees with the t-test. When the parametric and non-parametric tests agree, report the one that matches your summary statistics (means with the t-test, medians with the rank-sum test). When they disagree, look again at the data: usually skewness or outliers are driving the t-test.

Common mistake

Reporting only "Mann–Whitney $p < 0.001$ " without any effect size. A rank test answers "is there a difference?", not "how big is it?". Report the medians (and IQRs) of both groups and, ideally, the Hodges–Lehmann difference with its confidence interval.

4.5 Comparing more than two groups

Knowledge of hypertension, measured by a score from 0 to 20, may differ by education. With four education levels (None, Primary, Secondary, Tertiary) there are six possible pairs of groups. Running six t-tests would inflate the chance of at least one false-positive result (Section 4.5.5). **One-way analysis of variance** (ANOVA) instead asks a single question: are all the group means equal?

4.5.1 The idea of analysis of variance

ANOVA, developed by R. A. Fisher, compares two estimates of variability (Altman 1991; Kirkwood and Sterne 2003). Suppose there are k groups, group j has n_j observations with mean $\bar{x}_j$, the overall mean is $\bar{x}$ and the total number of observations is N . The total variability of the observations around the overall mean is split into two parts:

$$\underbrace{\sum_j \sum_i (x_{ij} - \bar{x})^2}_{\text{total SS}} = \underbrace{\sum_j n_j (\bar{x}_j - \bar{x})^2}_{\text{between-groups SS}} + \underbrace{\sum_j \sum_i (x_{ij} - \bar{x}_j)^2}_{\text{within-groups SS}}.$$

Each **sum of squares** (SS) is divided by its degrees of freedom to give a **mean square** (MS): $MS_{\text{between}} = SS_{\text{between}} / (k - 1)$ and $MS_{\text{within}} = SS_{\text{within}} / (N - k)$. The within-groups mean square estimates the variance of individuals around their group mean, whatever the truth about the group means. The between-groups mean square estimates the same variance *if* H_0 is true, but is inflated if the group means differ. Their ratio is the **F statistic**,

$$F = \frac{MS_{\text{between}}}{MS_{\text{within}}},$$

which is close to 1 under H_0 and is compared with an F distribution on $k - 1$ and $N - k$ degrees of freedom. With two groups, one-way ANOVA is identical to Student's t-test ($F = t^2$). The assumptions are those of Student's t-test extended to k groups: independent observations, approximately normal values within each group (less important in large samples), and similar variances across groups.

4.5.2 Look at the groups first

```
dx |>
  filter(!is.na(education), !is.na(knowledge_score)) |>
  group_by(education) |>
  summarise(n = n(), mean = mean(knowledge_score),
            sd = sd(knowledge_score), median = median(knowledge_score))
```

```
#> # A tibble: 4 × 5
#>   education      n mean   sd median
#>   <ord>      <int> <dbl> <dbl> <dbl>
#> 1 None         197 10.4  3.15    11
#> 2 Primary      432  9.85  3.37    10
#> 3 Secondary    285 10.8  3.64    11
#> 4 Tertiary     122 11.3  3.41    11
```

```
dx |>
  filter(!is.na(education), !is.na(knowledge_score)) |>
  ggplot(aes(x = education, y = knowledge_score)) +
  geom_jitter(width = 0.2, height = 0.2, alpha = 0.15, size = 0.8,
             colour = course_teal) +
  geom_boxplot(fill = NA, outlier.shape = NA, width = 0.5) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 3,
              fill = course_amber) +
  labs(x = "Highest education", y = "Knowledge score (0-20)")
```

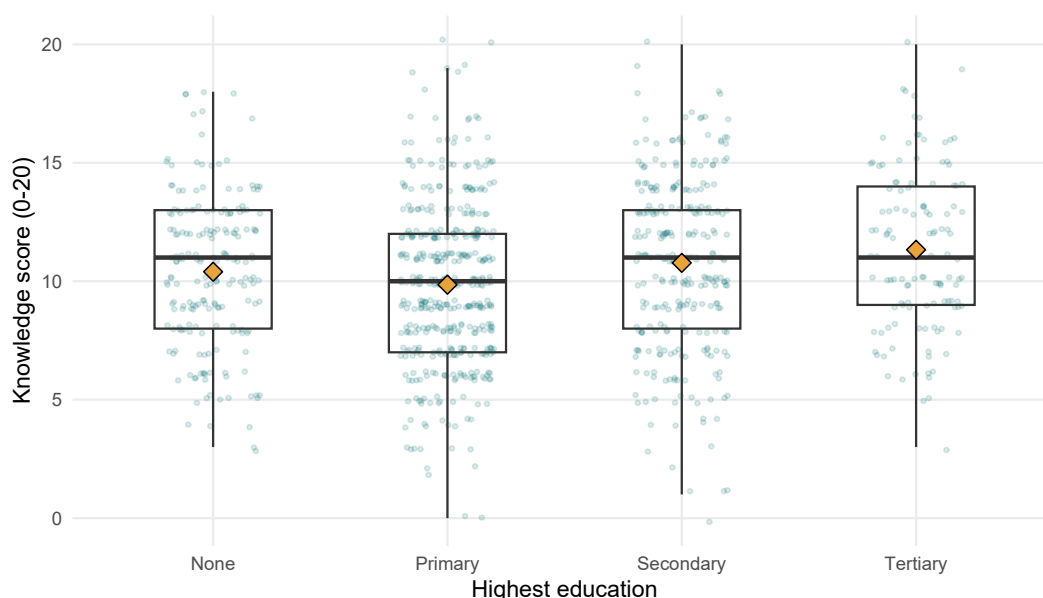


Figure 4.5: Hypertension knowledge score by highest education level among diagnosed patients. Boxes show the median and interquartile range; points are individual patients (jittered); the diamond marks the group mean.

Showing the individual points as well as the boxes, as recommended by Weissgerber et al. (2015), lets the reader see the sample size and the spread in each group. Mean knowledge scores range from 9.9 (Primary) to 11.3 (Tertiary), with standard deviations of 3.2 to 3.6 in every group: the differences between groups are small compared with the variation within them, but the groups are large.

4.5.3 One-way ANOVA in R

`aov()` fits the analysis of variance; `summary()` prints the ANOVA table.

```
aov_know <- aov(knowledge_score ~ education, data = dx)
summary(aov_know)
```

```
#>               Df Sum Sq Mean Sq F value    Pr(>F)
#> education      3      274      91.2     7.84 3.5e-05 ***
#> Residuals    1032    12008      11.6
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> 53 observations deleted due to missingness
```

Reading the ANOVA table:

- **education** row: the between-groups line. **Df = 3** ($k - 1 = 4 - 1$), **Sum Sq = 274** (between-groups SS), **Mean Sq = 91.2** ($274/3$).
- **Residuals** row: the within-groups line. **Df = 1032** ($N - k = 1036 - 4$), **Sum Sq = 12008**, **Mean Sq = 11.6**, the pooled within-group variance (its square root, 3.4, is the pooled standard deviation).
- **F value = 7.84**: $91.2/11.6$. The variation between education groups is almost eight times what we would expect from within-group variation alone.
- **Pr(>F) = 3.5e-05**: the p-value, $p < 0.001$. There is strong evidence that mean knowledge differs between at least some education levels.
- The stars and "Signif. codes" are a legend for the p-value; they add nothing and are best ignored.
- **53 observations deleted due to missingness**: patients missing either education or the knowledge score were dropped, so $N = 1,036$. Always note this line.

4.5.4 Checking the assumptions

The equal-variance assumption can be examined informally by comparing the standard deviations (here 3.2 to 3.6, very similar) or with **Levene's test**, available in the `car` package. If variances clearly differ, **Welch's ANOVA** (`oneway.test()`) generalises Welch's t-test to several groups.

```
# Levene's test: H0 = equal variances in all groups.
# education is an ordered factor; car wants a plain factor here.
car::leveneTest(knowledge_score ~ factor(education, ordered = FALSE),
  data = dx)
```

```
#> Levene's Test for Homogeneity of Variance (center = median)
#>      Df F value Pr(>F)
#> group    3     1.88  0.13
#>      1032
```

```
# Welch's ANOVA does not assume equal variances
oneway.test(knowledge_score ~ education, data = dx)
```

```
#>
#>      One-way analysis of means (not assuming equal variances)
#>
#> data:  knowledge_score and education
#> F = 7.74, num df = 3, denom df = 405, p-value = 4.9e-05
```

Levene's test gives $p = 0.13$: no evidence that the variances differ. Welch's ANOVA gives $F = 7.74$ on 3 and 404.5 degrees of freedom, $p < 0.001$, essentially the same answer as the classical ANOVA. The knowledge score is a bounded count from 0 to 20 and is not exactly normal, but with more than 100 patients per group the central limit theorem makes this unimportant; we confirm with the Kruskal–Wallis test below.

4.5.5 Post-hoc comparisons and multiple testing

A significant F test says that the group means are not all equal, but not which groups differ. To find out we compare pairs of groups, and here the **multiple testing** problem appears. If each test has a 5% chance of a false positive, then with m independent tests the chance of at least one false positive when all null hypotheses are true is $1 - 0.95^m$: 26% for six tests and 64% for twenty. Searching through many comparisons and reporting the significant ones is a reliable way to produce false findings.

The simplest remedy is the **Bonferroni correction** (Bland and Altman 1995): to keep the overall (family-wise) error rate at α , test each of the m comparisons at level α/m , or equivalently multiply each p-value by m (capping at 1). The method is simple and always valid but conservative, especially when m is large or the tests are correlated. **Tukey's honestly significant difference** (HSD) method is designed specifically for all pairwise comparisons after ANOVA and gives simultaneous confidence intervals for every pair of differences (Altman 1991).

TukeyHSD(aov_know)

R

```
#> Tukey multiple comparisons of means
#> 95% family-wise confidence level
#>
#> Fit: aov(formula = knowledge_score ~ education, data = dx)
#>
#> $education
#>      diff      lwr      upr    p adj
#> Primary-None -0.54685 -1.301448 0.20775 0.24398
#> Secondary-None 0.37442 -0.438846 1.18769 0.63668
#> Tertiary-None 0.92685 -0.084373 1.93808 0.08591
#> Secondary-Primary 0.92127 0.251447 1.59110 0.00237
#> Tertiary-Primary 1.47370 0.573791 2.37361 0.00016
#> Tertiary-Secondary 0.55243 -0.397215 1.50208 0.43971
```

Each row of the Tukey output is one pairwise comparison. **diff** is the difference in mean knowledge score (the first-named group minus the second); **lwr** and **upr** are the limits of a 95% confidence interval adjusted for making six comparisons; **p adj** is the adjusted p-value. Two comparisons stand out: Secondary versus Primary (difference 0.92 points, 95% CI 0.25 to 1.59, adjusted $p = 0.002$) and Tertiary versus Primary (1.47 points, 0.57 to 2.37, $p < 0.001$). The other intervals include zero. Interestingly, the "None" group scores slightly higher than the "Primary" group, so the pattern is not a smooth gradient; with simulated data we should not over-interpret this.

The Bonferroni-adjusted pairwise t-tests give the same picture:

```
pairwise.t.test(dx$knowledge_score, dx$education,
               p.adjust.method = "bonferroni")
```

R

```
#>
#>      Pairwise comparisons using t tests with pooled SD
#>
#> data:  dx$knowledge_score and dx$education
#>
#>      None   Primary Secondary
#> Primary  0.375 -         -
#> Secondary 1.000 0.003   -
#> Tertiary  0.111 2e-04   0.808
#>
#> P value adjustment method: bonferroni
```

This output is a triangle of adjusted p-values, one for each pair. Again only Secondary–Primary ($p = 0.003$) and Tertiary–Primary ($p < 0.001$) are below 0.05.

Good practice

Decide on the comparisons you care about *before* looking at the data, keep their number small, and say in the methods how multiplicity was handled. A few pre-specified comparisons need little or no correction; an exploratory trawl through dozens of variables needs a correction and should be described as exploratory. Do not run post-hoc tests after a non-significant F test in search of something significant.

4.5.6 The Kruskal–Wallis test

The rank-based counterpart of one-way ANOVA is the **Kruskal–Wallis test** (Kruskal and Wallis 1952). All N observations are ranked together; if $\bar{R}_j$ is the mean rank in group j , the statistic

$$H = \frac{12}{N(N+1)} \sum_{j=1}^k n_j \left(\bar{R}_j - \frac{N+1}{2} \right)^2$$

measures how far the mean ranks of the groups are from the overall mean rank $(N+1)/2$. Under H_0 that all groups come from the same distribution, H follows approximately a χ^2 distribution with $k - 1$ degrees of freedom. With two groups it is equivalent to the Wilcoxon rank-sum test.

```
kruskal.test(knowledge_score ~ education, data = dx)
```

R

```
#>
#>      Kruskal-Wallis rank sum test
#>
#> data:  knowledge_score by education
#> Kruskal-Wallis chi-squared = 24.4, df = 3, p-value = 2e-05
```

The output gives the statistic (**Kruskal-Wallis chi-squared = 24.4**), its degrees of freedom (**df = 3**) and the p-value (2.0×10^{-5}). The rank-based analysis agrees with the ANOVA. If pairwise follow-up is needed after a Kruskal–Wallis test, use pairwise Wilcoxon tests with a correction, for

```
example pairwise.wilcox.test(x, g, p.adjust.method = "bonferroni").
```

Clinical interpretation

Hypertension knowledge differed by education ($F = 7.8$ on 3 and 1,032 df, $p < 0.001$), mainly because patients with secondary or tertiary education scored about 1 to 1.5 points higher (out of 20) than those with primary education. The differences are real in a statistical sense but modest in size relative to the spread of scores (SD about 3.4): education explains only a small part of why patients differ in knowledge. Health education needs to reach patients at every education level.

4.6 Two categorical variables

The study's central questions concern a binary outcome, treatment uptake, and categorical determinants such as health insurance, residence and diabetes. Is uptake associated with having health insurance?

4.6.1 Contingency tables and the right percentages

Data on two categorical variables are summarised in a **contingency table** (cross-tabulation) of counts. With the potential determinant in the rows and the outcome in the columns, the **row percentages** answer the clinically natural question: what proportion of insured, and of uninsured, patients take up treatment?

```
tab_ins <- table(Insurance = dx$health_insurance,
                 Uptake    = dx$treatment_uptake)
addmargins(tab_ins) # counts with totals
```

```
#>      Uptake
#> Insurance No  Yes Sum
#>      No   419 306 725
#>      Yes  162 202 364
#>      Sum  581 508 1089
```

```
# Row percentages: uptake within each insurance group
round(100 * prop.table(tab_ins, margin = 1), 1)
```

```
#>      Uptake
#> Insurance No  Yes
#>      No   57.8 42.2
#>      Yes  44.5 55.5
```

Among the 364 insured patients 55.5% were on treatment, compared with 42.2% of the 725 uninsured. Figure 4.6 shows the same comparison graphically, together with education.

```
bar_data <- bind_rows(
  dx |> count(group = health_insurance, treatment_uptake) |>
    mutate(variable = "Health insurance"),
  dx |> filter(!is.na(education)) |>
    count(group = as.character(education), treatment_uptake) |>
```

```

mutate(variable = "Education")
) |>
mutate(group = factor(group, levels = c("No", "Yes", "None", "Primary",
                                         "Secondary", "Tertiary")),
       variable = factor(variable, levels = c("Health insurance",
                                             "Education")))

ggplot(bar_data, aes(x = group, y = n, fill = treatment_uptake)) +
  geom_col(position = "fill", width = 0.7) +
  geom_text(aes(label = n), position = position_fill(vjust = 0.5),
           colour = "white", size = 3) +
  facet_grid(~ variable, scales = "free_x", space = "free_x") +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(values = c(No = "grey60", Yes = course_teal),
                   name = "Treatment uptake") +
  labs(x = NULL, y = "Percentage of patients")

```

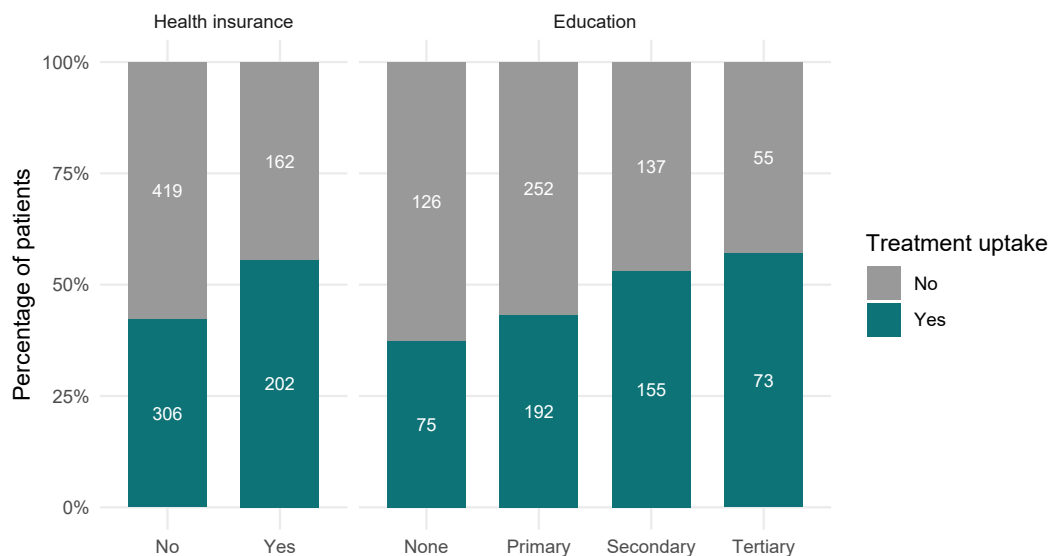


Figure 4.6: Percentage of diagnosed patients on antihypertensive treatment by health insurance status and by education level. Each bar sums to 100%; numbers inside the bars are patient counts.

4.6.2 Pearson's chi-square test

If insurance and uptake were **independent**, the proportion taking up treatment would be the same among insured and uninsured patients, equal to the overall proportion. The **expected count** in the cell in row i and column j would then be

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{\text{grand total}}.$$

Pearson's chi-square statistic (Pearson 1900) sums the squared discrepancies between observed counts O_{ij} and expected counts, scaled by the expected counts:

$$X^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}.$$

Under H_0 of independence, X^2 approximately follows a χ^2 distribution with $(r - 1)(c - 1)$ degrees of freedom, where r and c are the numbers of rows and columns; a 2×2 table has one degree of freedom. Large values of X^2 indicate that the observed table is far from what independence predicts (Kirkwood and Sterne 2003; Agresti 2013). The calculation is simple enough to do by hand, which makes the logic transparent.

```
observed <- tab_ins
expected <- outer(rowSums(observed), colSums(observed)) / sum(observed)
round(expected, 1)
```

```
#>      No   Yes
#> No  386.8 338.2
#> Yes 194.2 169.8
```

```
x2 <- sum((observed - expected)^2 / expected) # Pearson's statistic
x2
```

```
#> [1] 17.192
```

```
pchisq(x2, df = 1, lower.tail = FALSE) # p-value from chi-square(1)
```

```
#> [1] 3.3789e-05
```

Under independence we would expect 169.8 insured patients on treatment; we observed 202. The statistic is 17.2 on one degree of freedom, $p = 3.4 \times 10^{-5}$. The built-in function does the same, with one refinement:

```
chisq.test(tab_ins)
```

```
#>
#>      Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab_ins
#> X-squared = 16.7, df = 1, p-value = 4.5e-05
```

- **Pearson's Chi-squared test with Yates' continuity correction:** for 2×2 tables R by default applies **Yates' correction**, which subtracts 0.5 from each $|O - E|$ before squaring. It makes the test slightly conservative.
- **X-squared = 16.7, df = 1:** the corrected statistic, a little smaller than our 17.2.
- **p-value = 4.5e-05:** strong evidence that uptake and insurance are not independent.

With large samples, corrected and uncorrected tests agree closely; `chisq.test(tab_ins, correct = FALSE)` reproduces our hand calculation. Neither the statistic nor the p-value tells us the *direction* or the *size* of the association: for those we need the row percentages and the measures of effect in Section 4.6.4.

4.6.3 The expected-count assumption and Fisher's exact test

The chi-square distribution is a large-sample approximation. A widely used rule, due to Cochran, is that it is adequate when no expected count is below 1 and no more than 20% of expected counts are below 5; for a 2×2 table, all expected counts should be at least 5 (Kirkwood and Sterne 2003; Agresti 2013). Note that the rule concerns *expected* counts, not observed counts. When it fails, **Fisher's exact test** computes the exact probability of the observed table, and of all tables more extreme, given the row and column totals, using the hypergeometric distribution. It makes no large-sample approximation and is valid for any sample size (Agresti 2013).

Diabetes provides a useful example, because only 28 diagnosed patients have diabetes.

```
tab_diab <- table(Diabetes = dx$diabetes, Uptake = dx$treatment_uptake)
addmargins(tab_diab)
```

```
#>           Uptake
#> Diabetes    No  Yes  Sum
#>      No   574  487 1061
#>      Yes     7   21   28
#>      Sum   581  508 1089
```

```
chisq.test(tab_diab)$expected      # check the expected counts
```

```
#>           Uptake
#> Diabetes    No    Yes
#>      No  566.062 494.938
#>      Yes  14.938  13.062
```

The smallest expected count is 13.1, so the chi-square test is in fact acceptable here, but two observed cells are small (7 and 21), and Fisher's test is a natural check.

```
fisher.test(tab_diab)
```

```
#>
#>           Fisher's Exact Test for Count Data
#>
#> data:  tab_diab
#> p-value = 0.0033
#> alternative hypothesis: true odds ratio is not equal to 1
#> 95 percent confidence interval:
#>  1.4313 9.9226
#> sample estimates:
#> odds ratio
#>      3.5321
```

- **p-value = 0.0033**: strong evidence of association between diabetes and uptake.
- **odds ratio 3.53 with 95 percent confidence interval 1.43 to 9.92**: Fisher's test reports a *conditional maximum likelihood* estimate of the odds ratio, which can differ slightly from the simple cross-product ratio described below. The interval is wide because only 28 patients have diabetes.

In R, `fisher.test()` also works for tables larger than 2×2 , although it can be slow for large tables; `simulate.p.value = TRUE` gives a Monte Carlo approximation.

Common mistake

Applying the "at least 5 per cell" rule to the *observed* counts. A cell with an observed count of 2 is fine if its expected count is, say, 8. Check `chisq.test(tab)$expected`, and switch to Fisher's exact test when expected counts are small.

4.6.4 Measures of effect from a 2×2 table

A test answers "is there an association?" Clinical readers want to know "how strong?". Write the 2×2 table as

	Outcome Yes	Outcome No	Total
Exposed	a	b	$n_1 = a + b$
Unexposed	c	d	$n_0 = c + d$

with risks (proportions with the outcome) $p_1 = a/n_1$ in the exposed and $p_0 = c/n_0$ in the unexposed. Three measures are in common use (Kirkwood and Sterne 2003; Agresti 2013; Bland and Altman 2000).

- The **risk difference** $RD = p_1 - p_0$, with standard error $\sqrt{p_1(1 - p_1)/n_1 + p_0(1 - p_0)/n_0}$. It is on the absolute scale and is the most useful measure for planning services (how many more patients per 100 would be treated).
- The **relative risk** (risk ratio) $RR = p_1/p_0$. Its confidence interval is computed on the log scale, $SE(\ln RR) = \sqrt{1/a - 1/n_1 + 1/c - 1/n_0}$.
- The **odds ratio** $OR = \frac{a/b}{c/d} = \frac{ad}{bc}$, the ratio of the odds of the outcome in the two groups, with $SE(\ln OR) = \sqrt{1/a + 1/b + 1/c + 1/d}$.

For the ratio measures the 95% CI is $\exp\{\ln(\text{ratio}) \pm 1.96 \times SE\}$, which is asymmetric around the estimate on the original scale. "No association" corresponds to $RD = 0$, $RR = 1$ and $OR = 1$. We compute all three for health insurance (exposed = insured) and uptake.

```
# Cells named as in the table above (we avoid calling a cell "c",
# which would mask R's c() function)
a <- tab_ins["Yes", "Yes"]; b <- tab_ins["Yes", "No"] # insured
cc <- tab_ins["No", "Yes"]; dd <- tab_ins["No", "No"] # uninsured
n1 <- a + b; n0 <- cc + dd
p1 <- a / n1; p0 <- cc / n0

rd <- p1 - p0
se_rd <- sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)

rr <- p1 / p0
se_log_rr <- sqrt(1 / a - 1 / n1 + 1 / cc - 1 / n0)

or <- (a * dd) / (b * cc)
```

```
se_log_or <- sqrt(1 / a + 1 / b + 1 / cc + 1 / dd)

effects <- tibble(
  measure = c("Risk difference", "Relative risk", "Odds ratio"),
  estimate = c(rd, rr, or),
  lower = c(rd - 1.96 * se_rd, exp(log(rr) - 1.96 * se_log_rr),
            exp(log(or) - 1.96 * se_log_or)),
  upper = c(rd + 1.96 * se_rd, exp(log(rr) + 1.96 * se_log_rr),
            exp(log(or) + 1.96 * se_log_or))
)
knitr::kable(effects, digits = 3,
  caption = "Association between health insurance and treatment uptake:
  three measures of effect with 95% confidence intervals.")
```

Table 4.2: Association between health insurance and treatment uptake: three measures of effect with 95% confidence intervals.

measure	estimate	lower	upper
Risk difference	0.133	0.070	0.195
Relative risk	1.315	1.160	1.490
Odds ratio	1.707	1.325	2.201

Reading the table: insured patients were 13.3 percentage points more likely to be on treatment (95% CI 7.0 to 19.5); their probability of uptake was 1.31 times that of uninsured patients (95% CI 1.16 to 1.49); and their odds of uptake were 1.71 times as high (95% CI 1.32 to 2.20). All three intervals exclude the null value, consistent with the chi-square p-value.

Why are there three measures, and why is the odds ratio (1.71) so much further from 1 than the relative risk (1.31)? Odds and risks are close only when the outcome is rare (say below 10%). Treatment uptake is common (47%), so here the odds ratio considerably exaggerates the relative risk if it is mistakenly read as one (Zhang and Yu 1998; Bland and Altman 2000). The odds ratio is nonetheless important: it is the measure that can be estimated from case-control studies, it is symmetric (the odds ratio for "not taking up treatment" is simply $1/1.71$), and it is what **logistic regression** estimates, which is why it dominates the analysis of binary outcomes (Section 4.9 and Chapter 5).

Clinical interpretation

In this (simulated) sample, 55.5% of insured and 42.2% of uninsured diagnosed patients were on treatment, a difference of 13 percentage points (95% CI 7 to 20; $p < 0.001$). If the association were causal, roughly 13 more patients in every 100 would be treated if uninsured patients achieved the uptake of insured patients. That is a large difference in public health terms, but insurance status is linked to other characteristics (income, education, residence), so this crude comparison may partly reflect those factors.

4.6.5 Larger tables

The chi-square test applies equally to larger tables. For education (four levels) by uptake:

```
tab_edu <- table(Education = dx$education, Uptake = dx$treatment_uptake)
round(100 * prop.table(tab_edu, margin = 1), 1)
```

R

```
#>           Uptake
#> Education      No  Yes
#>   None        62.7 37.3
#>   Primary     56.8 43.2
#>   Secondary   46.9 53.1
#>   Tertiary    43.0 57.0
```

```
chisq.test(tab_edu)
```

R

```
#>
#>           Pearson's Chi-squared test
#>
#> data:  tab_edu
#> X-squared = 19.5, df = 3, p-value = 0.00022
```

Uptake rises from 37.3% among patients with no formal education to 57.0% among those with tertiary education (Figure 4.6). The chi-square statistic is 19.5 on $(4 - 1)(2 - 1) = 3$ degrees of freedom, $p < 0.001$. Yates' correction is not used for tables larger than 2×2 . Note that this test ignores the *ordering* of education: it would give the same result if the rows were shuffled. When the categories are ordered, a test for trend (for example the chi-square test for trend, or logistic regression with education as a numerical score; see Chapter 5) is more powerful and answers the more specific question of a gradient (Altman 1991; Agresti 2013).

4.6.6 Producing the tests for a whole table

In a manuscript, the comparison of characteristics between outcome groups is often presented in one table. The `gtsummary` package (Sjoberg et al. 2021) adds p-values with `add_p()`, choosing a test for each variable. It is important to know which tests it uses: by default the Wilcoxon rank-sum test for continuous variables, and the chi-square test (or Fisher's exact test when expected counts are small) for categorical variables.

```
dx |>
  select(treatment_uptake, age, distance_to_facility_km,
         health_insurance, residence, diabetes) |>
  tbl_summary(by = treatment_uptake, missing = "no",
             label = list(age ~ "Age (years)",
                          distance_to_facility_km ~ "Distance (km)",
                          health_insurance ~ "Health insurance",
                          residence ~ "Residence",
                          diabetes ~ "Diabetes")) |>
  add_p() |>
  as_kable(caption = "Characteristics of diagnosed patients by treatment uptake,
with p-values from gtsummary's default tests.")
```

R

Common mistake

Putting p-values in a table of baseline characteristics of a randomised trial. In a trial any baseline differences are due to chance by design, so testing them is meaningless; the p-values in a table like this one are appropriate only in observational studies, where they describe associations, and even then the estimates matter more than the p-values.

Table 4.3: Characteristics of diagnosed patients by treatment uptake, with p-values from gtsummary's default tests.

Characteristic	No N = 581	Yes N = 508	p-value
Age (years)	51 (41, 60)	56 (47, 66)	<0.001
Distance (km)	6.8 (4.3, 10.7)	5.8 (3.1, 9.6)	<0.001
Health insurance	162 (28%)	202 (40%)	<0.001
Residence			<0.001
Rural	306 (53%)	195 (38%)	
Urban	275 (47%)	313 (62%)	
Diabetes	7 (1.2%)	21 (4.1%)	0.002

4.7 Correlation between two continuous variables

Is systolic blood pressure related to body mass index? When both variables are continuous and neither is clearly the "outcome", we can measure the strength of their association with a **correlation coefficient**.

4.7.1 Pearson's correlation coefficient

Pearson's product-moment correlation coefficient for n pairs (x_i, y_i) is

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}.$$

It ranges from -1 (perfect negative straight-line relationship) through 0 (no *linear* relationship) to $+1$ (perfect positive straight-line relationship). The test of $H_0 : \rho = 0$ uses $t = r\sqrt{(n-2)/(1-r^2)}$ on $n-2$ degrees of freedom, and the confidence interval is computed through Fisher's z transformation. Pearson's r measures only *linear* association, is sensitive to outliers, and its test assumes that at least one of the variables is approximately normally distributed (Altman 1991; Bland 2015).

4.7.2 Spearman's rank correlation coefficient

Spearman's rank correlation coefficient r_s (often written ρ) is Pearson's coefficient computed on the ranks of x and y rather than their values (Spearman 1904). Without ties it simplifies to

$$r_s = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)},$$

where d_i is the difference between the ranks of x_i and y_i . It measures **monotonic** association (one variable tends to increase as the other increases, not necessarily in a straight line), is robust to outliers and suits ordinal or skewed data.

4.7.3 Look at the scatter plot

Always plot the data before computing a correlation. A single number cannot distinguish a genuine linear relationship from a curve, a cluster of outliers or two separate subgroups.

```
ggplot(dx, aes(x = bmi, y = sbp_mmhg)) +
  geom_point(alpha = 0.3, size = 1, colour = "teal") +
  geom_smooth(method = "lm", formula = y ~ x, colour = "black") +
  labs(x = "Body mass index (kg/m²)", y = "Systolic BP (mmHg)")
```

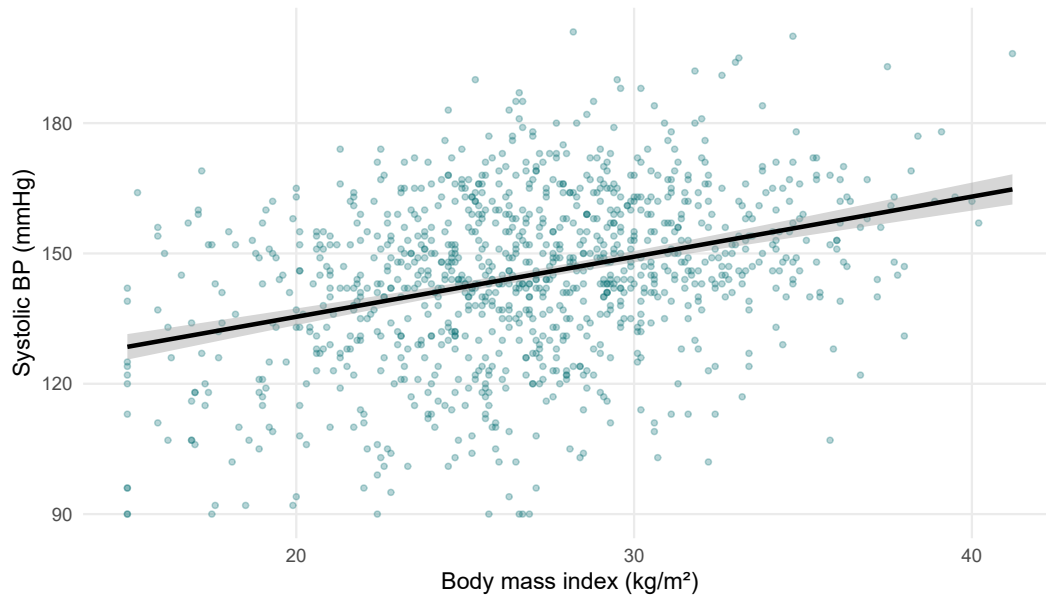


Figure 4.7: Systolic blood pressure against body mass index among diagnosed patients, with a least-squares straight line and its 95% confidence band.

The cloud of points slopes upwards: patients with higher BMI tend to have higher systolic pressure, but there is a great deal of scatter around the line.

4.7.4 Correlation in R

```
cor.test(dx$bmi, dx$sbp_mmhg, method = "pearson")
```

```
#>
#>      Pearson's product-moment correlation
#>
#> data:  dx$bmi and dx$sbp_mmhg
#> t = 11.8, df = 1053, p-value <2e-16
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  0.28742 0.39407
#> sample estimates:
#>      cor
#> 0.34185
```

Reading the output:

- **t = 11.8, df = 1053:** the test statistic for $H_0 : \rho = 0$; 1,055 patients had both measurements ($n - 2 = 1053$).
- **p-value < 2.2e-16:** overwhelming evidence that the correlation is not zero.
- **95 percent confidence interval: 0.29 0.39:** the plausible range for the population correlation.

- **cor 0.34185:** Pearson's $r = 0.34$, a moderate positive linear association.

```
cor.test(dx$bmi, dx$sbp_mmhg, method = "spearman", exact = FALSE)
```

R

```
#>
#>      Spearman's rank correlation rho
#>
#> data:  dx$bmi and dx$sbp_mmhg
#> S = 1.35e+08, p-value <2e-16
#> alternative hypothesis: true rho is not equal to 0
#> sample estimates:
#>      rho
#> 0.31132
```

Spearman's $r_s = 0.31$ is similar, as expected when the relationship is roughly linear and free of extreme outliers. The s statistic is the sum of squared rank differences; `exact = FALSE` asks for the usual large-sample p -value (the exact one is not available when there are ties). R does not give a confidence interval for Spearman's coefficient.

The square of Pearson's coefficient, $r^2 = 0.34^2 \approx 0.12$, is the proportion of the variance of one variable that is "explained" by a straight-line relationship with the other: BMI accounts for about 12% of the variation in systolic pressure in this sample. Rough verbal labels (below 0.3 weak, 0.3 to 0.7 moderate, above 0.7 strong) are sometimes used, but the clinical context matters more than the label.

Consider also the correlation between age and systolic blood pressure.

```
cor.test(dx$age, dx$sbp_mmhg) |> tidy() |>
  select(estimate, statistic, p.value, conf.low, conf.high)
```

R

```
#> # A tibble: 1 × 5
#>   estimate statistic    p.value conf.low conf.high
#>   <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1    0.142      4.72 0.00000263  0.0832    0.200
```

Here $r = 0.14$ with $p < 0.001$: highly "significant", yet age explains only about 2% of the variation in systolic pressure (0.14^2). With a thousand patients, almost any non-zero correlation is statistically significant; the size of r and its confidence interval are what matter.

Common mistake

Interpreting a correlation as evidence of causation, or a significant p -value as evidence of a strong relationship. Correlation measures association only: BMI and blood pressure may be correlated because both are influenced by diet, physical activity and age. A correlation also says nothing about agreement: two blood pressure devices can be highly correlated yet systematically disagree by 10 mmHg (Altman 1991; Bland 2015).

4.8 Matching the question to the test

The choice of method follows from the research question and from the types of the variables involved, not from which test gives the smallest p -value. The table below summarises the methods of this chapter.

Table 4.4: Choosing a statistical test for a comparison or association between two variables.

Question	Outcome type	Parametric method	Non-parametric / exact	Effect measure
Two independent groups differ?	Continuous	Welch t-test, <code>t.test()</code>	Wilcoxon rank-sum, <code>wilcox.test()</code>	Difference in means (or Hodges–Lehmann shift) with CI
Two paired measurements differ?	Continuous	Paired t-test, <code>t.test(paired = TRUE)</code>	Wilcoxon signed-rank	Mean difference with CI
More than two groups differ?	Continuous	One-way ANOVA, <code>aov()</code> ; Welch ANOVA, <code>oneway.test()</code>	Kruskal–Wallis, <code>kruskal.test()</code>	Pairwise differences (Tukey, Bonferroni)
Two categorical variables associated?	Categorical	Chi-square, <code>chisq.test()</code>	Fisher's exact, <code>fisher.test()</code>	RD, RR, OR with CI
Two continuous variables associated?	Continuous	Pearson, <code>cor.test()</code>	Spearman, <code>cor.test(method = "spearman")</code>	Correlation coefficient with CI
Binary outcome and one or more predictors	Binary	Logistic regression, <code>glm(family = binomial)</code>	—	Odds ratio with CI

Three practical questions guide the choice:

1. **What type is the outcome?** Continuous, binary or categorical, ordinal, time-to-event.
2. **How many groups, and are they independent or paired?**
3. **Are the assumptions of the parametric test reasonable?** Consider shape, sample size and whether the mean is a sensible summary.

4.9 A bridge to regression: why one variable at a time is not enough

Every test in this chapter examines **two variables at a time**. In observational research this is a severe limitation, because the groups being compared usually differ in many ways at once. Diabetic patients in our data are older than non-diabetic patients (mean age 59.7 versus 53.4 years), and older patients are more likely to be on treatment. So when we see that uptake is higher among diabetic patients, how much is due to diabetes and how much to age? A variable such as age, which is associated with both the exposure and the outcome and is not on the causal pathway between them, is a **confounder** (Rothman et al. 2008; Greenland et al. 1999). Two-variable tests cannot separate its effect from that of the exposure. Stratifying (repeating the test within age groups) quickly runs out of data as the number of confounders grows. **Regression models** solve the problem by estimating the effect of each variable while holding the others constant.

For a binary outcome the standard model is **logistic regression**, introduced fully in Chapter 5 (Hosmer et al. 2013). It models the log-odds of the outcome as a linear function of predictors,

$$\ln\left(\frac{\pi}{1-\pi}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k,$$

where π is the probability of uptake. For a single binary predictor, e^{β_1} is exactly the odds ratio of the 2×2 table. We can confirm this with health insurance.

```
m_ins <- glm(treatment_uptake ~ health_insurance, data = dx,
             family = binomial)
tidy(m_ins, exponentiate = TRUE, conf.int = TRUE) |>
  select(term, estimate, p.value, conf.low, conf.high)
```

```
#> # A tibble: 2 × 5
#>   term                estimate    p.value  conf.low  conf.high
#>   <chr>                <dbl>      <dbl>    <dbl>    <dbl>
#> 1 (Intercept)          0.730 0.0000292    0.630    0.846
#> 2 health_insuranceYes  1.71 0.0000363    1.33    2.20
```

```
# The cross-product odds ratio from the 2x2 table, for comparison
or
```

```
#> [1] 1.7074
```

The argument `family = binomial` tells `glm()` that the outcome is binary; because the reference level of `treatment_uptake` is "No", the model describes the odds of "Yes". `exponentiate = TRUE` converts the coefficients from log-odds to odds ratios. The row `health_insuranceYes` gives an odds ratio of 1.707, identical to the ad/bc we computed by hand. Its 95% CI (1.33 to 2.20) is a *profile-likelihood* interval, which `glm()` uses by default; it is almost identical to our log-scale (Wald) interval of 1.32 to 2.20. The p-value (3.6×10^{-5}) is very close to the chi-square test's. The (Intercept) row is the odds of uptake among the uninsured ($306/419 = 0.73$). Logistic regression with one predictor is therefore just another way of analysing a 2x2 table; its power is that further predictors can be added. A glimpse of what this achieves for diabetes:

```
crude <- glm(treatment_uptake ~ diabetes, data = dx, family = binomial)
adjusted <- glm(treatment_uptake ~ diabetes + age, data = dx,
               family = binomial)

# Odds ratio for diabetes before and after allowing for age
exp(c(crude = coef(crude)[["diabetesYes"]],
      age_adjusted = coef(adjusted)[["diabetesYes"]]))

#>      crude age_adjusted
#>      3.5359      3.0995
```

The crude odds ratio for diabetes is 3.54; after allowing for age it falls to 3.10. Part of the apparent effect of diabetes was really the effect of being older. Chapter 5 develops this properly: choosing confounders, fitting and checking multivariable models, and reporting adjusted odds ratios.

4.10 Reporting results correctly

A statistical result is only useful if readers can understand it. The guidance of the STROBE statement for observational studies (Elm et al. 2007), and of the statistical texts cited throughout this chapter, comes down to a few rules.

- **Describe before testing.** Give the summary statistics for each group (means and SDs, medians and IQRs, or numbers and percentages), with the number of patients analysed.

- **Lead with the estimate and its confidence interval**, in the units of the outcome, and give the exact p-value as supporting information.
- **Name the test** in the methods, and say how assumptions were checked and how multiple comparisons were handled.
- **Interpret the size** of the effect in clinical terms, and do not interpret a non-significant result as proof of no effect.
- **Do not over-state.** Cross-sectional associations are not causal effects, and crude associations may be confounded.

Here is how the main results of this chapter might appear in a results section.

Of 1,089 patients with diagnosed hypertension, 508 (46.6%) were taking antihypertensive treatment. Patients on treatment were older than those not on treatment (mean 56.3 vs 51.1 years; difference 5.2 years, 95% CI 3.5 to 6.8; Welch t-test $p < 0.001$) and lived closer to the facility (median 5.8 vs 6.8 km; Hodges–Lehmann difference 1.0 km, 95% CI 0.4 to 1.5; Wilcoxon rank-sum $p < 0.001$). Uptake was higher among insured than uninsured patients (55.5% vs 42.2%; risk difference 13.3 percentage points, 95% CI 7.0 to 19.5; odds ratio 1.71, 95% CI 1.32 to 2.20; $\chi^2 = 16.7$, 1 df, $p < 0.001$).

Good practice

Avoid "there was a significant difference ($p < 0.05$)" with no numbers. Write instead what the difference was, how precisely it was estimated, and how strong the evidence is: "treated patients were 5.2 years older on average (95% CI 3.5 to 6.8; $p < 0.001$)".

4.11 Summary

This chapter moved from describing data to drawing inferences about the population the data came from. Every estimate is subject to sampling variation, quantified by its standard error. A hypothesis test measures how compatible the data are with a null hypothesis of no effect, summarised by the p-value; a confidence interval shows the range of effect sizes compatible with the data and is usually more informative. For a continuous outcome in two independent groups we use Welch's t-test or the Wilcoxon rank-sum test; for several groups, one-way ANOVA or the Kruskal–Wallis test, followed if needed by pairwise comparisons that account for multiple testing. For two categorical variables we use the chi-square test or Fisher's exact test and estimate the risk difference, relative risk or odds ratio. For two continuous variables we use Pearson's or Spearman's correlation, after plotting the data. All of these examine two variables at a time and cannot deal with confounding, which is the task of the regression models in Chapter 5.

Key points

- Define the analytic population explicitly; for treatment uptake, analyse only diagnosed patients (`htn_diagnosed == "Yes"`, $n = 1,089$).
- The standard error measures the precision of an estimate and shrinks with $\sqrt{n}$; the standard deviation measures variation between individuals.
- A p-value is the probability of data at least as extreme as those observed *if the null hypothesis were true*. It is not the probability that the null hypothesis is true and it is not a

measure of effect size.

- Report estimates with 95% confidence intervals and exact p-values; distinguish statistical from clinical significance; absence of evidence is not evidence of absence.
- Judge normality by histograms and Q–Q plots, sample size and the question, not by a Shapiro–Wilk p-value alone; the central limit theorem protects tests of means in large samples.
- Two groups: `Welch t.test()` or `wilcox.test()`. More than two: `aov()` or `kruskal.test()`, with Tukey or Bonferroni post-hoc comparisons.
- Two categorical variables: `chisq.test()` (check expected counts) or `fisher.test()`; report RD, RR or OR with CIs. With a common outcome the OR is further from 1 than the RR.
- Correlation (`cor.test()`, Pearson or Spearman) measures association, not causation or agreement; always plot first.
- Single-variable tests cannot handle confounding; the odds ratio from a one-predictor logistic regression equals the 2×2 table odds ratio, and adding predictors gives adjusted estimates (Chapter 5).

4.12 Further reading

- Kirkwood and Sterne (2003), *Essential Medical Statistics*, chapters on confidence intervals, hypothesis tests, comparison of means and proportions: a clear and rigorous treatment of everything in this chapter, written for medical researchers.
- Altman (1991), *Practical Statistics for Medical Research*: the classic applied text, with particularly good guidance on choosing tests, checking assumptions and presenting results.
- Greenland et al. (2016), "Statistical tests, P values, confidence intervals, and power: a guide to misinterpretations": a list of 25 common misinterpretations, essential reading before writing any results section.
- Wasserstein and Lazar (2016), the ASA statement on p-values: a short, authoritative statement of what p-values can and cannot tell you.
- Agresti (2013), *Categorical Data Analysis*: the reference for contingency tables, exact tests and measures of association, for readers who want the theory in depth.

4.13 Exercises

All exercises use the diagnosed patients, `dx <- filter(analysis_data, htn_diagnosed == "Yes")`, unless stated otherwise. Worked solutions are given in the appendix.

Exercise 4.1

Load `Data/analysis_data.rds` and create the analytic population `dx`.

1. How many patients are in `dx`, and what percentage of them are on treatment?
2. Compute the percentage on treatment among *all* 1,500 patients. Explain in two sen-

tences why this figure should not be reported as the uptake of treatment.

3. Using `dx`, compute the mean, standard deviation and standard error of the knowledge score. Explain the difference between the standard deviation and the standard error.

Exercise 4.2

Assess whether body mass index (`bmi`) is approximately normally distributed in `dx`.

1. Draw a histogram and a normal Q–Q plot of `bmi`.
2. Run `shapiro.test()` on `bmi`. What does the p-value say, and how much weight should you give it with this sample size?
3. Based on your assessment, choose a test to compare BMI between patients who did and did not take up treatment, run it and interpret the result.

Exercise 4.3

Compare systolic blood pressure (`sbp_mmhg`) between patients on treatment and those not on treatment.

1. Give the mean and SD in each group.
2. Run Welch's t-test and report the difference in means with its 95% confidence interval and the p-value in one sentence.
3. Is the size of the difference clinically important? Treated patients have *higher* blood pressure: suggest a reason why this is plausible in a cross-sectional study.

Exercise 4.4

Does systolic blood pressure differ across BMI categories (`bmi_cat`: Underweight, Normal, Overweight, Obese)?

1. Produce a boxplot of `sbp_mmhg` by `bmi_cat` and a table of group means and SDs.
2. Fit a one-way ANOVA and read the ANOVA table: degrees of freedom, F statistic and p-value. How many patients were excluded because of missing values?
3. Run the Kruskal–Wallis test as a check.
4. Use Bonferroni-adjusted pairwise t-tests to identify which categories differ.

Exercise 4.5

Is treatment uptake associated with place of residence (`residence`)?

1. Build the 2×2 table and the row percentages of uptake within each residence group.
2. Run the chi-square test and check the expected counts.
3. Taking urban residence as the "exposure", compute the risk difference, relative risk and odds ratio with 95% confidence intervals.
4. Explain why the odds ratio is further from 1 than the relative risk.

Exercise 4.6

Return to diabetes and treatment uptake.

1. Compute the odds ratio for uptake (diabetic vs non-diabetic) from the 2×2 table as ad/bc , and with a logistic regression `glm(treatment_uptake ~ diabetes, family = binomial)`. Do they agree?
2. Compare these with the odds ratio reported by `fisher.test()`. Why is it slightly different?
3. Why is the confidence interval for this odds ratio so much wider than the one for health insurance?

Exercise 4.7

Is the hypertension knowledge score related to age or to distance from the facility?

1. Draw a scatter plot of `knowledge_score` against age.
2. Compute Pearson's and Spearman's correlation coefficients with `cor.test()` and interpret them, including the confidence interval for Pearson's r .
3. Repeat for `knowledge_score` and `distance_to_facility_km`. Which coefficient is more appropriate for distance, and why?
4. A colleague writes "knowledge is not related to age ($p = 0.73$), so age-targeted education is unnecessary". Comment on this statement.

Exercise 4.8

Multiple testing in practice. Consider the eight categorical variables `sex`, `residence`, `education`, `occupation`, `marital_status`, `smoking`, `alcohol` and `facility`.

1. For each, run a chi-square test of association with `treatment_uptake` and collect the eight p-values in a data frame (hint: use `lapply()` or `purrr::map()` and `chisq.test(table(...))$p.value`).
2. Adjust the p-values with `p.adjust()` using the Bonferroni and Holm methods. Which associations remain below 0.05?
3. Assuming all eight null hypotheses were true and the tests independent, what is the probability of at least one p-value below 0.05? What does this imply for a paper that reports only the significant results from such a screen?

Regression Modelling and Reporting Results

In Chapter 4 we asked one question at a time. Is treatment uptake more common among patients with health insurance? Do urban and rural patients differ? Each test compared groups defined by a single variable. Clinical reality is rarely that tidy. Patients with health insurance may also be younger, better educated or live closer to a clinic, and each of these characteristics may itself influence whether a person with hypertension starts treatment. If we want to know whether insurance is associated with uptake *over and above* these other factors, we need a method that considers several variables at once. That method is regression.

Regression is the workhorse of clinical and epidemiological research. It appears in almost every observational study published in medical journals, and it is the natural end point of the analysis we have been building since Chapter 1: we imported and cleaned the hypertension data, described the sample, tested simple hypotheses, and now we bring the determinants of treatment uptake together in a single model. This chapter explains how regression models work, how to fit and read them in R, how to check that they are reasonable, and how to turn the output into the tables, figures and sentences that appear in a published paper. It finishes with the practices that make the whole analysis reproducible, so that another person (or you, a year from now) can obtain exactly the same results.

Learning objectives

- Explain the three purposes of regression modelling (description, prediction and explanation) and why the purpose shapes the model.
- Fit and interpret a multiple linear regression for a continuous outcome, including coefficients, R^2 and residual checks.
- Convert between probabilities, odds and log-odds, and explain why logistic regression models the log-odds of a binary outcome.
- Fit a multivariable logistic regression with `glm()`, read its `summary()` output line by line, and report odds ratios with 95% confidence intervals.
- Distinguish crude from adjusted estimates, recognise confounding, and avoid the "Table 2 fallacy".
- Build a model from clinical reasoning rather than automated selection, and check events per variable, collinearity, linearity, discrimination, calibration and influential observations.
- Produce a publication-ready regression table and a forest plot, and write a Results paragraph that follows the STROBE recommendations.

- Organise an analysis so that it can be reproduced from raw data to report.

5.1 Why regression?

5.1.1 From one variable at a time to many

The tests in Chapter 4 answer questions of the form “is the outcome associated with *this* variable?”. A regression model answers a richer question: “how is the outcome associated with this variable, *given* the values of these other variables?” It does this by writing the expected value of the outcome (or a transformation of it) as a function of several predictors simultaneously. Each coefficient in the model then describes the association between one predictor and the outcome while the other predictors are held fixed. This “holding fixed” is what epidemiologists call *adjustment*, and it is the main reason regression is used in observational research (Vittinghoff et al. 2012; Kirkwood and Sterne 2003).

Regression also provides more than a p-value. It gives an *estimate* of the size of each association, with a confidence interval describing its precision, and it can produce predictions for individual patients. As Chapter 4 emphasised, an estimate with its interval is far more informative than a statement that a test was “significant” (Altman 1991; Wasserstein and Lazar 2016).

5.1.2 Description, prediction and explanation

Before fitting any model you should be clear about what it is *for*, because the purpose determines which variables to include and how to judge the result (Vittinghoff et al. 2012; Harrell 2015).

- **Description.** The aim is to summarise how an outcome varies with several characteristics in a population, for example how the prevalence of treatment uptake differs by age, sex and residence. The model is a compact, multivariable summary; no causal claim is made.
- **Prediction.** The aim is to estimate, as accurately as possible, the outcome for a new individual, for instance the probability that a newly diagnosed patient will start treatment. Any variable that improves predictive accuracy is welcome, whether or not it is a cause. Models are judged by discrimination and calibration in *new* data, and great care is needed to avoid overfitting (Harrell 2015).
- **Explanation (aetiology).** The aim is to estimate the effect of a particular exposure on the outcome, for example the effect of health insurance on uptake. Variables are included because they are *confounders* of that exposure–outcome relationship, and the choice is guided by subject-matter knowledge, often summarised in a causal diagram (Rothman et al. 2008; Greenland et al. 1999).

Our case study, *Determinants of hypertension treatment uptake among adults attending primary healthcare facilities*, sits between description and explanation. We want to know which patient and access characteristics are independently associated with uptake, so that services can be targeted. This chapter therefore fits a single, pre-specified multivariable model and interprets its coefficients as *adjusted associations*. We shall return repeatedly to the danger of reading them as causal effects.

Good practice

Write down the purpose of your model, and the list of variables you will include, in your analysis plan *before* you look at the outcome data. A prediction model, a descriptive model and an aetiological model built from the same data set can legitimately look quite different.

5.1.3 A family of regression models

All the models in this chapter share one structure: a *linear predictor*, which is a weighted sum of the explanatory variables,

$$\eta = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k,$$

linked to the expected value of the outcome by a function chosen to suit the type of outcome. These are the *generalised linear models* (Agresti 2013). The type of outcome determines the model, as summarised below.

Table 5.1: Choosing a regression model by the type of outcome.

Outcome type	Example in the study	Model	Effect measure	R function
Continuous	Systolic blood pressure	Linear	Difference in means	<code>lm()</code>
Binary	Treatment uptake (Yes/No)	Logistic	Odds ratio	<code>glm(family = binomial)</code>
Count	Number of comorbidities	Poisson	Rate ratio	<code>glm(family = poisson)</code>
Time to event	Time to treatment start	Cox	Hazard ratio	<code>survival::coxph()</code>

This chapter covers the first two rows in detail. Once you understand linear and logistic regression, the others follow the same logic.

5.2 Setting up the analysis

As in every chapter, we start a fresh R session by loading the packages and the clean analysis data set created in Chapter 2.

```
library(tidyverse) # dplyr, ggplot2, tidyr, ...
library(broom)     # tidy() and augment() for model objects
library(gtsummary) # publication-ready regression tables
library(car)       # vif() for collinearity
library(pROC)      # ROC curves and the AUC
library(patchwork) # combining ggplot2 panels
library(scales)    # axis formatting
library(knitr)     # kable() tables

analysis_data <- readRDS("Data/analysis_data.rds")
```

The primary outcome, `treatment_uptake`, only makes sense for people who already know they have hypertension: a person who has never been diagnosed cannot "take up" treatment. The analytic population is therefore the patients with `htn_diagnosed == "Yes"`.

```

dx <- analysis_data |>
  filter(htn_diagnosed == "Yes") |>
  # education is stored as an ORDERED factor (None < Primary < ...).
  # For regression we convert it to an ordinary factor so that each level
  # is compared with the reference level "None" (explained below).
  mutate(education = factor(education, ordered = FALSE))

nrow(dx)                                # size of the analytic population

#> [1] 1089

```

```

table(dx$treatment_uptake)              # the outcome distribution

#>
#>   No Yes
#> 581 508

```

```

levels(dx$treatment_uptake)             # first level = reference ("No")

#> [1] "No"  "Yes"

```

Of the 1,089 diagnosed patients, 508 (46.6%) are on antihypertensive treatment. The first level of the outcome factor is "No", so the models below estimate the odds of *uptake* ("Yes"), which is the event of interest. If the levels were the other way round, every odds ratio in this chapter would be inverted.

Common mistake

Ordered factors are convenient for tables and plots, but R codes them in regression with *orthogonal polynomial contrasts*. The output then contains terms called `education.L`, `education.Q` and `education.C` (linear, quadratic and cubic trends) instead of one comparison per level. Their odds ratios are not "Primary versus None" or "per level of education", and they are very easy to misreport. Unless you specifically want a trend test, convert ordered factors with `factor(x, ordered = FALSE)` before modelling, as we did above.

5.3 Linear regression for a continuous outcome

Linear regression is the simplest regression model and the foundation for everything else, so we begin with it, using systolic blood pressure (SBP) as the outcome. Because SBP is measured in everyone, we use the full sample of 1,500 adults for this section.

5.3.1 The simple linear regression model

With one predictor x (say body mass index, BMI), the simple linear regression model for patient i is

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2).$$

Here β_0 is the *intercept* (the mean of y when $x = 0$), β_1 is the *slope* (the change in the mean of

y for a one-unit increase in x), and ε_i is the random error: the part of patient i 's blood pressure not explained by BMI. The errors are assumed to be independent and normally distributed with constant variance σ^2 .

The coefficients are estimated by *ordinary least squares*: we choose $\hat{\beta}_0$ and $\hat{\beta}_1$ to make the sum of squared vertical distances between the observed points and the fitted line as small as possible,

$$\text{minimise } \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2,$$

which gives the closed-form solution

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

The slope is therefore closely related to the Pearson correlation of Chapter 4: $\hat{\beta}_1 = r s_y / s_x$ (Altman 1991; Bland 2015).

```
# Simple linear regression: SBP (mmHg) on BMI (kg/m^2)
lm_simple <- lm(sbp_mmhg ~ bmi, data = analysis_data)
coef(lm_simple)
```

```
#> (Intercept)      bmi
#>      101.467      1.435
```

The fitted line is $\widehat{\text{SBP}} = 101.5 + 1.44 \times \text{BMI}$. Each additional kg/m^2 of BMI is associated with a mean SBP about 1.44 mmHg higher. The intercept is the predicted SBP at a BMI of zero, which is biologically impossible; it is needed to position the line but has no clinical interpretation. This is common, and it is one reason some analysts *centre* continuous predictors (subtract a typical value such as the mean) before fitting.

5.3.2 Multiple linear regression

Blood pressure rises with age, and BMI may also differ by age, so we add age to the model. With k predictors the model becomes

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_k x_{ki} + \varepsilon_i,$$

or, in matrix notation, $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, with least-squares solution $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. You will never compute this by hand, but the formula makes one point clear: each coefficient is estimated using all the predictors together, so its value depends on which other variables are in the model. Each β_j is the expected difference in y between two people who differ by one unit in x_j but have *the same values* of all the other predictors (Vittinghoff et al. 2012).

```
lm_sbp <- lm(sbp_mmhg ~ age + bmi, data = analysis_data)
summary(lm_sbp)
```

```
#>
#> Call:
#> lm(formula = sbp_mmhg ~ age + bmi, data = analysis_data)
```

```
#>
#> Residuals:
#>   Min       1Q   Median       3Q      Max
#> -53.66 -12.69  -0.51   12.50   55.50
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  87.4657     3.1789   27.51 < 2e-16 ***
#> age          0.2593     0.0336    7.71 2.3e-14 ***
#> bmi          1.4510     0.0975   14.87 < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 18.1 on 1446 degrees of freedom
#> (51 observations deleted due to missingness)
#> Multiple R-squared:  0.161,    Adjusted R-squared:  0.159
#> F-statistic: 138 on 2 and 1446 DF,  p-value: <2e-16
```

Let us read this output from top to bottom.

- **Call** repeats the model formula and data, which is useful when you have fitted many models.
- **Residuals** summarises the differences between observed and fitted SBP. The median is close to zero and the quartiles are roughly symmetric (about ± 12.6 mmHg), which is what we expect if the errors are symmetric around zero.
- **Coefficients** is the heart of the output. For each term there is the estimate, its standard error (SE), the t statistic (estimate divided by SE) and the two-sided p -value for the null hypothesis that the true coefficient is zero.
 - (Intercept) 87.5 mmHg is the predicted SBP for a person aged 0 with a BMI of 0, again not clinically meaningful.
 - age 0.26: holding BMI fixed, each additional year of age is associated with a mean SBP 0.26 mmHg higher; over ten years that is about 2.6 mmHg.
 - bmi 1.45: among people of the same age, each additional kg/m^2 is associated with a mean SBP 1.45 mmHg higher; a 5 kg/m^2 difference corresponds to about 7.3 mmHg.
- **Residual standard error** (18.1 mmHg) estimates σ , the typical distance of an observation from its fitted value. It is large relative to the coefficients: age and BMI explain only part of the variation in blood pressure.
- **(51 observations deleted due to missingness)** reminds us that `lm()` silently drops every row with a missing value in any model variable. The model was fitted to 1,449 people, not 1,500.
- **Multiple R-squared** (0.16) is the proportion of the variance of SBP explained by the model, $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$. The **adjusted** R^2 applies a small penalty for the number of predictors.
- **F-statistic** tests the null hypothesis that *all* the slope coefficients are zero. With $p < 2.2 \times 10^{-16}$ we conclude that age and BMI together explain some variation in SBP.

Clinical interpretation

An R^2 of 0.16 does not mean the model is useless. In an explanatory analysis we care about the *coefficients*: a difference of about 7 mmHg in mean SBP for 5 kg/m² of BMI, independent of age, is clinically important at the population level. A low R^2 simply tells us that individual blood pressure depends on many other things (genetics, salt intake, medication, measurement error) that are not in the model.

To make the coefficients easier to quote, we can ask for confidence intervals and rescale age to a more meaningful unit.

```
tidy(lm_sbp, conf.int = TRUE) |>
  select(term, estimate, conf.low, conf.high, p.value) |>
  mutate(across(estimate:conf.high, \(x) round(x, 3)),
         p.value = signif(p.value, 2))
```

```
#> # A tibble: 3 × 5
#>   term      estimate conf.low conf.high  p.value
#>   <chr>      <dbl>    <dbl>    <dbl>    <dbl>
#> 1 (Intercept)  87.5      81.2     93.7 2.20e-134
#> 2 age          0.259     0.193     0.325 2.3 e- 14
#> 3 bmi          1.45      1.26     1.64 1.10e- 46
```

```
# Per 10 years of age: multiply the age coefficient and its CI by 10
age10 <- 10 * c(coef(lm_sbp)["age"], confint(lm_sbp)["age", ])
round(age10, 2)
```

```
#>   age 2.5 % 97.5 %
#> 2.59 1.93 3.25
```

We would report: "Each 10-year increase in age was associated with a 2.6 mmHg higher mean SBP (95% CI 1.9 to 3.3), adjusted for BMI."

5.3.3 Assumptions and residual checks

The inferences from a linear model (standard errors, confidence intervals, p-values) rely on four assumptions, conveniently remembered as **LINE** (Vittinghoff et al. 2012; Kirkwood and Sterne 2003):

1. **Linearity**: the mean of y changes linearly with each continuous predictor (given the others).
2. **Independence**: the errors of different patients are independent. This is a property of the study design; repeated measurements on the same patient, or patients clustered within facilities, can violate it.
3. **Normality**: the errors are approximately normally distributed. With large samples the central limit theorem makes the coefficients approximately normal even if the errors are not, so this matters most in small samples and for prediction intervals.
4. **Equal variance (homoscedasticity)**: the spread of the errors does not change with the fitted value.

We check these assumptions with the *residuals*, $e_i = y_i - \hat{y}_i$. Two plots are standard: residuals against fitted values (for linearity and equal variance) and a normal Q-Q plot of the residuals (for normality). `broom::augment()` returns the fitted values and residuals in a data frame that is convenient for `ggplot2`.

```
lm_aug <- augment(lm_sbp) # one row per patient used in the model

p_resfit <- ggplot(lm_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.25, colour = "#0D7377") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_smooth(method = "loess", se = FALSE, colour = "#E07A1F") +
  labs(title = "Residuals vs fitted",
       x = "Fitted SBP (mmHg)", y = "Residual (mmHg)")

p_qq <- ggplot(lm_aug, aes(sample = .std.resid)) +
  stat_qq(alpha = 0.25, colour = "#0D7377") +
  stat_qq_line(linetype = "dashed") +
  labs(title = "Normal Q-Q plot",
       x = "Theoretical quantiles", y = "Standardised residual")

p_resfit + p_qq
```

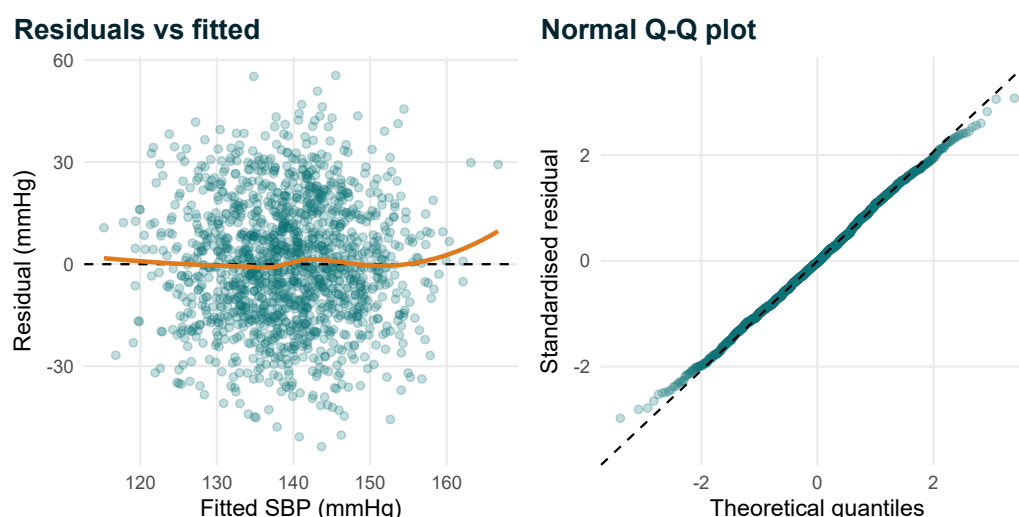


Figure 5.1: Residual diagnostics for the linear regression of systolic blood pressure on age and BMI. Left: residuals against fitted values, with a smooth curve; right: normal Q-Q plot of the residuals.

In the left panel the points form a horizontal band of roughly constant width centred on zero, and the smooth curve is close to flat apart from a slight upturn at the highest fitted values, where there are few patients and the smoother is unstable. There is no convincing sign of curvature (which would suggest a missing non-linear term) or of a funnel shape (which would suggest unequal variance). In the right panel the points follow the reference line closely, with only slight departures in the extreme tails. The assumptions look reasonable.

Good practice

Look at residual plots, do not test them. Formal tests of normality reject trivial departures in large samples and miss important ones in small samples (Altman and Bland 1995b). If a plot shows a curved pattern, consider adding a non-linear term (for example a quadratic or spline

in age); if it shows a funnel, consider transforming the outcome or using robust standard errors.

5.4 Binary outcomes: probability, odds and log-odds

Our primary outcome, treatment uptake, is binary: each patient is either on treatment ($y = 1$) or not ($y = 0$). Linear regression is not appropriate here. A straight line in x can predict "probabilities" below 0 or above 1, the variance of a binary variable depends on its mean (so the equal variance assumption fails), and the errors cannot be normal. We need a scale on which a linear model makes sense. That scale is the log-odds.

5.4.1 Three ways to express the same risk

Let p be the probability that a patient takes up treatment. The *odds* of uptake are the probability of uptake divided by the probability of no uptake,

$$\text{odds} = \frac{p}{1-p}, \quad \text{and conversely} \quad p = \frac{\text{odds}}{1 + \text{odds}}.$$

Odds are familiar from betting: odds of 3 ("3 to 1") mean the event is three times as likely to happen as not, which is a probability of $3/4 = 0.75$. While a probability is confined to $[0, 1]$, odds range from 0 to infinity. Taking the natural logarithm gives the *log-odds* or *logit*,

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right),$$

which ranges over the whole real line, from $-\infty$ to $+\infty$, and is symmetric around zero: a probability of 0.5 has odds 1 and log-odds 0.

Table 5.2: The same risk expressed as a probability, odds and log-odds.

Probability p	Odds $p/(1-p)$	Log-odds	Interpretation
0.10	0.11	-2.20	1 success for every 9 failures
0.25	0.33	-1.10	1 for every 3
0.50	1.00	0.00	Even chances
0.75	3.00	1.10	3 for every 1
0.90	9.00	2.20	9 for every 1

5.4.2 The odds ratio

To compare two groups we take the ratio of their odds. If p_1 is the probability of uptake among insured patients and p_0 among uninsured patients, the *odds ratio* (OR) is

$$\text{OR} = \frac{p_1/(1-p_1)}{p_0/(1-p_0)}.$$

An OR of 1 means no association; above 1, higher odds of the outcome in the first group; below 1, lower odds (Bland and Altman 2000). Let us compute it by hand for health insurance.

```
tab_ins <- table(Insurance = dx$health_insurance,
                 Uptake    = dx$treatment_uptake)
tab_ins
```

```
#>           Uptake
#> Insurance  No Yes
#>           No 419 306
#>           Yes 162 202
```

```
# Probability, odds and log-odds of uptake in each insurance group
ins_summary <- dx |>
  group_by(health_insurance) |>
  summarise(n = n(),
            p_uptake = mean(treatment_uptake == "Yes")) |>
  mutate(odds = p_uptake / (1 - p_uptake),
         log_odds = log(odds))
kable(ins_summary, digits = 3,
      caption = "Uptake by health insurance status (diagnosed patients).")
```

Table 5.3: Uptake by health insurance status (diagnosed patients).

health_insurance	n	p_uptake	odds	log_odds
No	725	0.422	0.730	-0.314
Yes	364	0.555	1.247	0.221

```
# The crude odds ratio: odds among insured / odds among uninsured
or_ins <- ins_summary$odds[2] / ins_summary$odds[1]
round(or_ins, 2)
```

```
#> [1] 1.71
```

Among uninsured patients 42.2% are on treatment (odds 0.73); among insured patients 55.5% (odds 1.25). The odds of uptake are 1.71 times higher among the insured. Notice the last column of the table: the *difference* in log-odds, $0.221 - (-0.314) = 0.535$, is the log of the odds ratio. This is the key to logistic regression: a difference on the log-odds scale is a log odds ratio, and exponentiating it gives the odds ratio.

5.4.3 Odds ratios versus relative risks

Clinicians usually think in terms of risks, and the *relative risk* (risk ratio, RR) p_1/p_0 is the more intuitive measure. Here the RR is $0.555 / 0.422 = 1.31$, noticeably smaller than the OR of 1.71. The two measures are close only when the outcome is rare (say, below 10%) in both groups, because then $1 - p \approx 1$ and the odds approximate the risk (Bland and Altman 2000; Rothman et al. 2008). When the outcome is common, as here (almost half of patients are on treatment), the OR is always further from 1 than the RR.

Zhang and Yu (1998) proposed a simple conversion from an OR to an approximate RR, given the risk p_0 in the reference group:

$$RR \approx \frac{OR}{(1 - p_0) + p_0 \times OR}.$$

With $p_0 = 0.422$ and $OR = 1.71$, the formula returns 1.31, which reproduces the crude RR exactly. For adjusted odds ratios the conversion is only approximate, and other methods (such as log-binomial or Poisson regression with robust standard errors) are preferred if an adjusted RR is the target.

Common mistake

Do not describe an odds ratio of 2.0 as "twice the risk" or "twice as likely" when the outcome is common. With an uptake of about 40% in the reference group, an OR of 2.0 corresponds to a risk ratio of only about 1.4. Say "twice the *odds*", and, if your readers need risks, report predicted probabilities or a risk ratio as well.

Why, then, do we use odds ratios at all? There are three good reasons. First, the logistic model on which they are based always produces valid probabilities between 0 and 1. Second, the OR is symmetric: the OR for uptake is the reciprocal of the OR for non-uptake, which is not true of the RR. Third, the OR can be estimated from case-control studies, where risks cannot (Rothman et al. 2008; Hosmer et al. 2013).

5.5 The logistic regression model

5.5.1 The model equation

Logistic regression models the log-odds of the outcome as a linear function of the predictors (Hosmer et al. 2013; Agresti 2013):

$$\text{logit}(p_i) = \log\left(\frac{p_i}{1 - p_i}\right) = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \dots + \beta_k x_{ki},$$

where $p_i = \Pr(y_i = 1 \mid x_{1i}, \dots, x_{ki})$ is the probability of uptake for patient i . Solving for p_i gives the *logistic function*,

$$p_i = \frac{\exp(\beta_0 + \beta_1 x_{1i} + \dots + \beta_k x_{ki})}{1 + \exp(\beta_0 + \beta_1 x_{1i} + \dots + \beta_k x_{ki})},$$

an S-shaped curve that always lies between 0 and 1. There is no error term in the equation: the randomness lies in the outcome itself, which is assumed to follow a Bernoulli distribution with probability p_i .

The figure below shows the two faces of the model for a single predictor, age. On the log-odds scale (left) the relationship is a straight line; on the probability scale (right) the same model is a gentle S-shaped curve.

```
glm_age <- glm(treatment_uptake ~ age, data = dx, family = binomial)

# Predictions over a wide age range, on both scales
age_grid <- tibble(age = seq(15, 100, by = 1))
age_grid <- age_grid |>
```

R

```
mutate(log_odds = predict(glm_age, newdata = age_grid, type = "link"),
       prob = predict(glm_age, newdata = age_grid, type = "response"))

p_link <- ggplot(age_grid, aes(age, log_odds)) +
  geom_line(linewidth = 1, colour = "#0D7377") +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  labs(title = "Log-odds scale", x = "Age (years)",
       y = "Log-odds of uptake")

p_prob <- ggplot(age_grid, aes(age, prob)) +
  geom_line(linewidth = 1, colour = "#0D7377") +
  geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey50") +
  scale_y_continuous(labels = percent, limits = c(0, 1)) +
  labs(title = "Probability scale", x = "Age (years)",
       y = "Probability of uptake")

p_link + p_prob
```

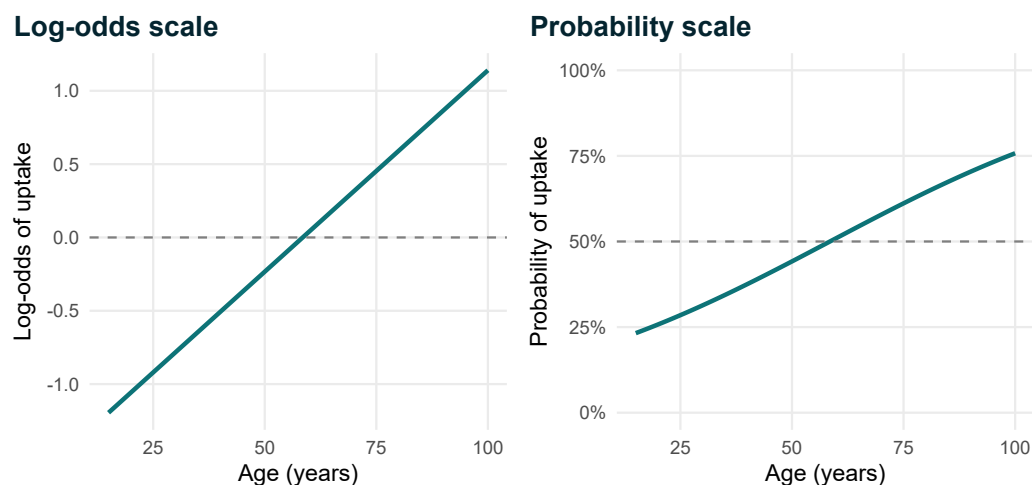


Figure 5.2: A logistic regression of treatment uptake on age, shown on the log-odds scale (left), where the model is linear, and on the probability scale (right), where it is S-shaped.

5.5.2 Estimation by maximum likelihood

Least squares is not appropriate for a binary outcome. Instead the coefficients are estimated by *maximum likelihood*: we choose the values of β that make the observed data most probable. For patient i the probability of the observed outcome is p_i if $y_i = 1$ and $1 - p_i$ if $y_i = 0$, which can be written compactly as $p_i^{y_i} (1 - p_i)^{1-y_i}$. Assuming independent patients, the likelihood is the product over all patients, and it is easier to work with its logarithm:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)],$$

where each p_i depends on β through the logistic function. There is no closed-form solution; R maximises ℓ numerically with an algorithm called *iteratively reweighted least squares* (also known as Fisher scoring), which usually converges in a handful of iterations (Agresti 2013; Hosmer et al. 2013).

Two quantities derived from the likelihood appear in the R output and are used for testing.

- The **deviance** of a model is $D = -2\ell(\hat{\beta})$ (relative to a saturated model that fits every observation perfectly). Smaller deviance means better fit. The *null deviance* is the deviance of a model with only an intercept; the *residual deviance* is that of the fitted model.
- The **likelihood-ratio (LR) test** compares two *nested* models, where the smaller model is a special case of the larger one. The difference in deviance, $D_{\text{small}} - D_{\text{large}}$, follows approximately a χ^2 distribution with degrees of freedom equal to the number of extra parameters in the larger model.

Each individual coefficient is also accompanied by a **Wald test**, $z = \hat{\beta}_j / \text{SE}(\hat{\beta}_j)$, compared with the standard normal distribution. Wald and LR tests usually agree in large samples; when they differ (small samples, sparse categories, very large effects) the LR test is more reliable (Hosmer et al. 2013; Agresti 2013).

5.5.3 Interpreting the coefficients as odds ratios

Consider two patients who are identical in every predictor except x_1 , which is one unit higher for the second. Subtracting their linear predictors, every term cancels except one:

$$\text{logit}(p_{\text{second}}) - \text{logit}(p_{\text{first}}) = \beta_1.$$

A difference in log-odds is the log of an odds ratio, so

$$\text{OR}_{x_1} = \exp(\beta_1).$$

This is the central result for interpretation (Hosmer et al. 2013): $\exp(\beta_j)$ **is the odds ratio for a one-unit increase in x_j , holding the other predictors fixed**. Because the model is linear on the log-odds scale, this OR is the same whatever the starting value of x_j and whatever the values of the other predictors (unless an interaction is included; see Section 5.7).

How "one unit" is defined depends on the type of predictor.

- **Binary predictor** (for example diabetes, coded No = 0, Yes = 1). $\exp(\beta)$ is the OR comparing patients with diabetes to those without, the reference category.
- **Categorical predictor with several levels** (for example education: None, Primary, Secondary, Tertiary). R creates one indicator ("dummy") variable for each non-reference level. Each $\exp(\beta)$ compares that level with the reference level (None), not with the level below it.
- **Continuous predictor** (for example age in years). $\exp(\beta)$ is the OR per one-unit increase (per year). A one-year difference is rarely clinically interesting, so we usually report the OR for a meaningful increment c , such as 10 years:

$$\text{OR}_{c \text{ units}} = \exp(c \beta) = [\exp(\beta)]^c,$$

with confidence interval $\exp(c \hat{\beta} \pm 1.96 c \text{SE}(\hat{\beta}))$. Note that the OR for 10 years is the per-year OR raised to the power 10, not multiplied by 10.

5.5.4 Assumptions of logistic regression

Logistic regression makes fewer assumptions than linear regression, but they are not trivial (Hosmer et al. 2013; Vittinghoff et al. 2012):

- 1. Correct link and linearity in the logit.** Each continuous predictor is linearly related to the *log-odds* of the outcome (not to the probability). This can be checked by adding non-linear terms.
- 2. Independence.** Outcomes of different patients are independent given the predictors. Clustering of patients within facilities is a potential violation that we discuss in Section 5.8.
- 3. No perfect separation.** No predictor (or combination) perfectly predicts the outcome. If, say, every patient with diabetes were on treatment, the maximum likelihood estimate of the diabetes coefficient would be infinite; R warns that "fitted probabilities numerically 0 or 1 occurred" and reports a huge coefficient and standard error.
- 4. Adequate sample size.** Maximum likelihood estimates are only approximately unbiased and normal in large samples; the number of *events* relative to the number of parameters matters (Section 5.8).

There is no assumption that the predictors are normally distributed, and no assumption of equal variance.

5.6 Fitting the logistic model in R

5.6.1 A first model with one predictor

Logistic regression is fitted in R with `glm()`, the function for generalised linear models, by specifying `family = binomial` (the logit link is the default). The outcome can be a factor, in which case the first level is treated as 0 (failure) and every other level as 1 (success). We start with the crude model for health insurance and check that it reproduces the odds ratio we computed by hand.

```
glm_ins <- glm(treatment_uptake ~ health_insurance,
              data = dx, family = binomial(link = "logit"))
coef(glm_ins)           # log-odds scale
```

```
#>      (Intercept) health_insuranceYes
#>      -0.3143      0.5350
```

```
exp(coef(glm_ins))      # odds scale
```

```
#>      (Intercept) health_insuranceYes
#>      0.7303      1.7074
```

The intercept, -0.314, is the log-odds of uptake in the reference group (uninsured); exponentiated, it is the odds 0.73 that we saw in the table. The coefficient for `health_insuranceYes`, 0.535, is the difference in log-odds between insured and uninsured patients, and its exponential, 1.71, is exactly the crude OR computed by hand. A one-predictor logistic regression is simply a re-expression of a 2×2 table.

5.6.2 The multivariable model

Now we fit the main model of the study. Its predictors were chosen *before* the analysis on clinical and public-health grounds (Section 5.8 explains why this matters):

- demographic characteristics that commonly confound health-service use: age, sex, education and residence;
- clinical factors that increase contact with health services: diabetes and a family history of hypertension;
- the study's stated determinants: health insurance (financial access), hypertension knowledge score (awareness) and distance to the facility (geographical access).

```
model_full <- glm(
  treatment_uptake ~ age + sex + education + residence + diabetes +
    family_history_htn + health_insurance + knowledge_score +
    distance_to_facility_km,
  data = dx,
  family = binomial(link = "logit")
)

summary(model_full)
```

```
#>
#> Call:
#> glm(formula = treatment_uptake ~ age + sex + education + residence +
#>   diabetes + family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km, family = binomial(link = "logit"),
#>   data = dx)
#>
#> Coefficients:
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)      -3.75688    0.44473   -8.45 < 2e-16 ***
#> age                0.03019    0.00507    5.96 2.6e-09 ***
#> sexMale           -0.30160    0.13956   -2.16 0.03069 *
#> educationPrimary    0.33065    0.19540    1.69 0.09061 .
#> educationSecondary  0.65087    0.20939    3.11 0.00188 **
#> educationTertiary   0.89294    0.25704    3.47 0.00051 ***
#> residenceUrban      0.62486    0.14595    4.28 1.9e-05 ***
#> diabetesYes         1.26938    0.47394    2.68 0.00740 **
#> family_history_htnYes 0.64480    0.14472    4.46 8.4e-06 ***
#> health_insuranceYes  0.71753    0.14779    4.85 1.2e-06 ***
#> knowledge_score     0.09348    0.02053    4.55 5.3e-06 ***
#> distance_to_facility_km -0.01859    0.01222   -1.52 0.12798
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#> Null deviance: 1369.1 on 991 degrees of freedom
#> Residual deviance: 1221.9 on 980 degrees of freedom
#> (97 observations deleted due to missingness)
#> AIC: 1246
#>
#> Number of Fisher Scoring iterations: 4
```

5.6.3 Reading the summary output line by line

This output contains a great deal of information, and it is worth reading slowly.

- **Call.** The model formula, the family and the data set. Always check that the formula is what you intended.
- **Coefficients.** One row per estimated parameter, on the **log-odds scale**.
 - Estimate is $\hat{\beta}_j$. A positive value means higher odds of uptake as the predictor increases (or in that category compared with the reference); a negative value means lower odds.
 - Std. Error is the standard error of $\hat{\beta}_j$, a measure of its precision. Note how large it is for `diabetesYes` (0.474) compared with the other binary predictors (about 0.14–0.15). Only 28 diagnosed patients have diabetes, so this coefficient is estimated imprecisely.
 - `z value` is the Wald statistic, Estimate / Std. Error.
 - `Pr(>|z|)` is the two-sided Wald p-value for the null hypothesis that $\beta_j = 0$ (OR = 1), adjusted for the other variables.
- **Reading individual rows.**
 - (Intercept) `−3.77` is the log-odds of uptake for a hypothetical patient with every predictor at zero or its reference level: aged 0, female, no education, rural, no diabetes, no family history, uninsured, knowledge score 0, living 0 km from the facility. It anchors the model but has no clinical meaning.
 - `age 0.030`: each additional year of age increases the log-odds of uptake by 0.030, other variables held fixed.
 - `sexMale −0.302`: men have lower log-odds of uptake than women of the same age, education, residence and so on. The suffix `Male` tells you that `Female` is the reference.
 - `educationPrimary, educationSecondary, educationTertiary`: each compares that level with no formal education. The coefficients increase steadily (0.33, 0.65, 0.89), suggesting a gradient.
 - `distance_to_facility_km −0.019`: lower odds with greater distance, but the p-value (0.128) indicates that the data are compatible with no association once the other variables are accounted for.
- **Signif. codes.** The stars are a legend for p-value thresholds. They encourage dichotomous thinking and should never be reported in a paper (Wasserstein and Lazar 2016; Greenland et al. 2016).
- **Dispersion parameter.** For a binomial model it is fixed at 1; this line matters only for other families.
- **Null deviance** (1369.1 on 991 degrees of freedom) is the deviance of an intercept-only model; **residual deviance** (1221.9 on 980 df) is the deviance after adding the 11 predictor terms. The drop of 147.2 on 11 df is the LR statistic for the null hypothesis that *none* of the predictors is associated with uptake, which is overwhelmingly rejected.
- **(97 observations deleted due to missingness).** This line is easy to overlook and very important. `glm()` uses complete cases only: any patient with a missing value in *any* model variable is dropped. The model was fitted to 992 of the 1,089 diagnosed patients (degrees of freedom 991 + 1). This number must appear in your paper.

- **AIC** (Akaike information criterion), $AIC = D + 2q$ where q is the number of parameters, balances fit against complexity. It is useful for comparing models fitted to the *same* observations; on its own it has no interpretation.
- **Number of Fisher Scoring iterations** (4) shows that the algorithm converged quickly. A large number (say, more than 20) is a warning sign, often of separation.

```
nobs(model_full) # observations used
```

R

```
#> [1] 992
```

```
table(model_full$y) # outcome among those used (0/1)
```

R

```
#>
#>  0  1
#> 535 457
```

Common mistake

Reporting the sample size of the data set rather than of the model. Here 97 diagnosed patients (8.9%) were excluded from the model because of missing knowledge score, distance, education or age. Always report the number analysed, describe the missing data, and consider whether a complete-case analysis could be biased. When missingness is substantial, multiple imputation is usually preferable (Sterne et al. 2009; Little and Rubin 2019).

5.6.4 Odds ratios with confidence intervals

The log-odds coefficients are needed for the mathematics, but readers need odds ratios. `broom::tidy()` with `exponentiate = TRUE` and `conf.int = TRUE` exponentiates the estimates and their confidence limits in one step.

```
or_table <- tidy(model_full, exponentiate = TRUE, conf.int = TRUE)
```

R

```
or_table |>
  select(term, OR = estimate, lower = conf.low, upper = conf.high,
         p = p.value) |>
  mutate(across(c(OR, lower, upper), \(x) round(x, 2)),
         p = format.pval(p, digits = 2, eps = 0.001))
```

```
#> # A tibble: 12 × 5
#>   term                OR lower upper p
#>   <chr>             <dbl> <dbl> <dbl> <chr>
#> 1 (Intercept)      0.02  0.01  0.06 <0.001
#> 2 age              1.03  1.02  1.04 <0.001
#> 3 sexMale          0.74  0.56  0.97 0.0307
#> 4 educationPrimary 1.39  0.95  2.05 0.0906
#> 5 educationSecondary 1.92  1.27  2.9  0.0019
#> 6 educationTertiary 2.44  1.48  4.06 <0.001
#> 7 residenceUrban    1.87  1.4  2.49 <0.001
#> 8 diabetesYes       3.56  1.46  9.61 0.0074
#> 9 family_history_htnYes 1.91  1.44  2.53 <0.001
```

```
#> 10 health_insuranceYes      2.05  1.54  2.74 <0.001
#> 11 knowledge_score          1.1   1.06  1.14 <0.001
#> 12 distance_to_facility_km  0.98  0.96  1.01 0.1280
```

The confidence limits returned by `tidy()` come from `confint()`, which for a `glm` uses the *profile likelihood*. These intervals are slightly asymmetric on the log scale and are more accurate than the Wald intervals $\exp(\hat{\beta} \pm 1.96 \text{ SE})$, particularly for imprecise estimates such as the diabetes OR (Hosmer et al. 2013). Notice also that a standard error cannot simply be exponentiated: the `std.` error column of `tidy()` stays on the log-odds scale even when `exponentiate = TRUE`.

Each odds ratio is interpreted as an *adjusted* association, holding the other variables in the model fixed.

- **Age**, OR 1.03 per year (95% CI 1.02 to 1.04): each additional year is associated with 3% higher odds of uptake.
- **Sex**, OR 0.74 (0.56 to 0.97): men have about 26% lower odds of uptake than women.
- **Education**: compared with no formal education, the ORs are 1.39 (0.95 to 2.05) for primary, 1.92 (1.27 to 2.90) for secondary and 2.44 (1.48 to 4.06) for tertiary education, a clear gradient.
- **Urban residence**, OR 1.87 (1.40 to 2.49) compared with rural.
- **Diabetes**, OR 3.56 (1.46 to 9.61): the largest OR, but also by far the widest interval, reflecting the small number of patients with diabetes.
- **Family history of hypertension**, OR 1.91 (1.44 to 2.53).
- **Health insurance**, OR 2.05 (1.54 to 2.74).
- **Knowledge score**, OR 1.10 per point (1.06 to 1.14).
- **Distance to facility**, OR 0.98 per km (0.96 to 1.01), $p = 0.13$.

Clinical interpretation

The width of a confidence interval tells you how much the data constrain the estimate. The diabetes interval runs from a modest 1.5-fold to an almost 10-fold increase in odds: we can be fairly confident the association is positive, but we know little about its size. In contrast, the interval for health insurance (1.5 to 2.7) is narrow enough to be useful for planning. A large point estimate with a wide interval is weaker evidence than a moderate estimate with a narrow one. (The data are simulated for teaching; these results illustrate methods, not real clinical findings.)

5.6.5 Odds ratios for clinically meaningful increments

The per-unit ORs for age, knowledge and distance are close to 1 simply because one year, one point or one kilometre is a small change. We rescale them with $\exp(c \hat{\beta})$, applying the same multiplication to the confidence limits on the log scale.

```
# Increments that are meaningful for each continuous predictor
increments <- tibble(
  term = c("age", "knowledge_score", "distance_to_facility_km"),
  unit = c("10 years", "5 points", "5 km"),
```

R

```

  c      = c(10, 5, 5)
)

# Profile-likelihood CIs on the log-odds scale, then rescale by c
ci_log <- confint(model_full)

rescaled <- increments |>
  mutate(beta = coef(model_full)[term],
         OR   = exp(c * beta),
         lower = exp(c * ci_log[term, 1]),
         upper = exp(c * ci_log[term, 2])) |>
  select(term, unit, OR, lower, upper)

kable(rescaled, digits = 2,
      caption = "Adjusted odds ratios for clinically meaningful increments.")

```

Table 5.4: Adjusted odds ratios for clinically meaningful increments.

term	unit	OR	lower	upper
age	10 years	1.35	1.23	1.50
knowledge_score	5 points	1.60	1.31	1.96
distance_to_facility_km	5 km	0.91	0.81	1.03

Expressed this way the results become much easier to communicate: a 10-year difference in age is associated with 1.35 times the odds of uptake, a 5-point higher knowledge score with 1.6 times the odds, and living 5 km further away with 0.91 times the odds (95% CI 0.81 to 1.03). The increment should be chosen for clinical relevance (often the interquartile range or a familiar unit such as a decade), stated clearly, and decided before seeing the results.

5.6.6 Predicted probabilities

Odds ratios are relative measures. To show what the model implies in absolute terms, we can compute predicted probabilities for representative patients with `predict(type = "response")`. Here we compare two 55-year-old women from a rural area with primary education, no diabetes, no family history, an average knowledge score of 10 and living 6 km from the facility, who differ only in health insurance.

```

profiles <- tibble(
  age = 55, sex = "Female", education = "Primary", residence = "Rural",
  diabetes = "No", family_history_htn = "No",
  health_insurance = c("No", "Yes"),
  knowledge_score = 10, distance_to_facility_km = 6
)

profiles |>
  mutate(p_uptake = predict(model_full, newdata = profiles,
                           type = "response")) |>
  select(health_insurance, p_uptake)

```

```

#> # A tibble: 2 × 2
#>   health_insurance p_uptake
#>   <chr>           <dbl>

```

```
#> 1 No 0.280
#> 2 Yes 0.444
```

The predicted probability of uptake rises from 28% to 44%, an absolute difference of 16 percentage points. The odds ratio is the same for every profile, but the absolute difference depends on the baseline probability, which is why predicted probabilities (or risk differences) are a valuable complement to ORs when communicating with clinicians and policy makers.

5.6.7 Testing a categorical predictor with several levels

Education enters the model as three indicator variables, each with its own Wald p-value. These answer three separate questions (is Primary different from None? Secondary from None? Tertiary from None?) but not the overall question "is education associated with uptake?". For that we need a single test of the null hypothesis that all three coefficients are zero, and the likelihood-ratio test is the natural choice (Hosmer et al. 2013).

`drop1()` removes each term in turn (all levels of a factor together) and compares the deviance with that of the full model.

```
drop1(model_full, test = "LRT")
```

R

```
#> Single term deletions
#>
#> Model:
#> treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km
#>
#>               Df Deviance   AIC   LRT Pr(>Chi)
#> <none>                1222 1246
#> age                  1    1259 1281 37.4  9.8e-10 ***
#> sex                  1    1227 1249  4.7  0.0303 *
#> education            3    1238 1256 16.3  0.0010 **
#> residence            1    1240 1262 18.6  1.6e-05 ***
#> diabetes            1    1230 1252  8.0  0.0047 **
#> family_history_htn   1    1242 1264 20.2  7.0e-06 ***
#> health_insurance     1    1246 1268 24.1  9.3e-07 ***
#> knowledge_score      1    1243 1265 21.4  3.7e-06 ***
#> distance_to_facility_km 1    1224 1246  2.3  0.1261
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The row for education shows that removing it increases the deviance by 16.3 on 3 degrees of freedom, $p = 0.001$: education as a whole is associated with uptake. The other rows give LR tests for each one-parameter term; their p-values are very close to the Wald p-values in `summary()`, as expected in a sample of this size.

The same test can be obtained by fitting the model without education and comparing the two nested models directly with `anova()`. There is a trap here. Education has 24 missing values among the diagnosed patients, so a model *without* education, fitted to `dx`, would include patients who were excluded from `model_full`. The two deviances would then refer to different patients and could not be compared (R stops with the error "models were not all fitted to the same size of dataset"). We therefore extract the complete-case data used by the full model with `model.frame()` and fit the smaller model to exactly those patients.

```
cc <- model.frame(model_full)      # the 992 patients used by model_full
nrow(cc)
```

```
#> [1] 992
```

```
model_no_edu <- update(model_full, . ~ . - education, data = cc)
lrt_edu <- anova(model_no_edu, model_full, test = "LRT")
attr(lrt_edu, "heading") <- NULL   # omit the long model formulas
lrt_edu
```

```
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      983      1238
#> 2      980      1222  3      16.3    0.001 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The LR statistic (the difference in deviance, 16.3 on 3 df) and its p-value are identical to the `drop1()` result, which performs the same comparison internally.

Good practice

When you compare nested models, make sure both were fitted to exactly the same observations. The simplest way is to create the complete-case data set once, with `model.frame()` of the largest model or with `drop_na()` on all the candidate variables, and to fit every model to it.

5.7 Confounding: crude versus adjusted estimates

5.7.1 What is confounding?

A *confounder* is a variable that distorts the observed association between an exposure and an outcome because it is associated with both. The classical criteria are that a confounder (1) is a cause, or a proxy for a cause, of the outcome; (2) is associated with the exposure in the source population; and (3) is not an effect of the exposure, that is, it does not lie on the causal pathway from exposure to outcome (Rothman et al. 2008). The *crude* (unadjusted) estimate mixes the effect of the exposure with that of the confounder; the *adjusted* estimate, from a model that includes the confounder, attempts to separate them.

5.7.2 Causal diagrams in words

Causal diagrams, or directed acyclic graphs (DAGs), make these ideas precise (Greenland et al. 1999). A DAG draws each variable as a node and each assumed direct causal effect as an arrow. Consider the question "is distance to the facility associated with treatment uptake?". Plausible assumptions are:

- *Distance* → *Uptake*: patients who live far from a clinic face travel costs and time, which may reduce uptake.
- *Residence* → *Distance*: rural patients tend to live further from the facility.

- *Residence* → *Uptake*: independently of distance, urban areas may have more pharmacies, more health information and different patterns of care.

Residence therefore opens a "back-door path" from distance to uptake ($\text{Distance} \leftarrow \text{Residence} \rightarrow \text{Uptake}$) that is not part of the effect of distance. To estimate the association of distance with uptake that is not due to residence, the back-door path must be blocked by adjusting for residence. In contrast, a variable caused by the exposure and lying on the path to the outcome (a *mediator*) should not be adjusted for if we want the total effect, and adjusting for a common *effect* of exposure and outcome (a *collider*) can create bias where none existed (Greenland et al. 1999; VanderWeele 2019).

VanderWeele (2019) offers a practical rule for situations in which the full causal structure is uncertain, the *modified disjunctive cause criterion*: adjust for any pre-exposure variable that is a cause of the exposure, the outcome or both; exclude known instrumental variables (causes of the exposure only); and include good proxies for unmeasured common causes.

5.7.3 A worked example: distance and residence

The data agree with the first two arrows: rural patients in the model sample live on average 9.6 km from their facility, urban patients 6.0 km. We now compare the crude odds ratio for distance with the odds ratio adjusted for residence alone and for all the variables in the full model. To make the comparison fair, all three models are fitted to the same 992 complete cases.

```
# cc: the complete-case data used by the full model (created above)
with(cc, tapply(distance_to_facility_km, residence, mean))
```

```
#> Rural Urban
#> 9.591 6.022
```

```
m_crude <- glm(treatment_uptake ~ distance_to_facility_km,
               data = cc, family = binomial)
m_res   <- glm(treatment_uptake ~ distance_to_facility_km + residence,
               data = cc, family = binomial)

# Extract the OR for distance (per 5 km) and its 95% CI from each model
dist_or <- function(model, label) {
  b <- coef(model)["distance_to_facility_km"]
  ci <- confint(model)["distance_to_facility_km", ]
  p <- summary(model)$coefficients["distance_to_facility_km", 4]
  tibble(model = label, OR_5km = exp(5 * b),
         lower = exp(5 * ci[1]), upper = exp(5 * ci[2]), p = p)
}

conf_tab <- bind_rows(
  dist_or(m_crude, "Crude"),
  dist_or(m_res, "Adjusted for residence"),
  dist_or(model_full, "Fully adjusted")
)
kable(conf_tab, digits = 3,
      caption = "OR for uptake per 5 km of distance: crude and adjusted.")
```

The crude analysis suggests that each additional 5 km is associated with 15% lower odds of uptake,

Table 5.5: OR for uptake per 5 km of distance: crude and adjusted.

model	OR_5km	lower	upper	p
Crude	0.850	0.762	0.947	0.003
Adjusted for residence	0.913	0.815	1.021	0.111
Fully adjusted	0.911	0.807	1.026	0.128

with a confidence interval that excludes 1 ($p = 0.0034$). After adjusting for residence alone, the OR moves towards 1 (0.91), the interval includes 1, and the p-value rises to 0.11. Expressed on the log-odds scale, the coefficient has shrunk by about 44%. Much of the crude association between distance and uptake was due to residence: rural patients live further away *and*, for reasons other than distance, are less likely to be on treatment. Adjusting for the remaining variables changes the distance OR very little further.

A common rule of thumb flags a variable as a confounder if adjusting for it changes the coefficient of the exposure by more than 10% (the *change-in-estimate* criterion). It is a useful screening tool, but it should not replace causal reasoning: a mediator or a collider can also change an estimate, and adjusting for it would introduce bias (Rothman et al. 2008; VanderWeele 2019).

Clinical interpretation

The crude analysis might have led a health planner to conclude that distance itself is a major barrier to treatment. The adjusted analysis suggests a more nuanced picture: rural residence is associated with lower uptake, and distance carries little additional information once residence is known. Interventions targeting rural patients (for example outreach or community dispensing) might therefore matter more than simply reducing travel distance. In the real world this would need confirmation from better designed studies; these data are simulated.

5.7.4 Not every change is confounding: non-collapsibility

Compare the crude and adjusted ORs for two other variables in the same 992 patients.

```
crude_or <- function(var) {
  f <- reformulate(var, response = "treatment_uptake")
  exp(coef(glm(f, data = cc, family = binomial)))[2]
}
tibble(variable = c("family_history_htn", "health_insurance"),
       crude_OR = c(crude_or("family_history_htn"),
                    crude_or("health_insurance")),
       adjusted_OR = exp(coef(model_full)[c("family_history_htnYes",
                                             "health_insuranceYes")))) |>
  kable(digits = 2, caption = "Crude and adjusted ORs for two predictors.")
```

Table 5.6: Crude and adjusted ORs for two predictors.

variable	crude_OR	adjusted_OR
family_history_htn	1.65	1.91
health_insurance	1.67	2.05

Both odds ratios move *away* from 1 after adjustment, from about 1.65 to about 2.0, although family history and insurance are only weakly related to the other predictors in these data. Only part of this change is confounding. The rest is a mathematical property of the odds ratio called *non-collapsibility*: when other variables that strongly predict the outcome are added to a logistic model, the OR for a given exposure generally moves away from the null *even in the complete absence of confounding* (Rothman et al. 2008; Vittinghoff et al. 2012). The adjusted (*conditional*) OR compares patients with identical values of all the other predictors, whereas the crude (*marginal*) OR averages over the population, and with odds ratios these two quantities differ. Risk differences and risk ratios do not have this property. Exercise 5.4 shows how to separate the two components by standardisation. The practical lesson is that a change between crude and adjusted ORs is not, by itself, proof of confounding; your causal reasoning must come first.

5.7.5 The Table 2 fallacy

It is tempting to interpret *every* adjusted OR in a multivariable model as the effect of that variable, as we did, with caveats, in the list above. Westreich and Greenland (2013) called this the *Table 2 fallacy* (since the main multivariable results are often in a paper's second table). The problem is that a model built to estimate the effect of one exposure is adjusted for that exposure's confounders, which are generally not the right set of adjustment variables for each of the other predictors.

Take education and knowledge score. Education plausibly *causes* better knowledge of hypertension, which in turn increases uptake. In our model, which includes knowledge, the education OR estimates the association of education with uptake *among patients with the same knowledge score*: the part of the education effect that does not operate through knowledge (the *direct effect*). The total effect of education would be estimated by a model that omits its mediators. Similarly, the OR for age is adjusted for diabetes, which is itself partly a consequence of age.

Common mistake

Interpreting each coefficient in a "determinants" model as a causal effect of equal standing. In a descriptive multivariable model, the ORs are *mutually adjusted associations*. If one particular exposure is of causal interest, build the model for that exposure (using a causal diagram) and report only its OR as the main result (Westreich and Greenland 2013).

5.7.6 Interaction (effect modification)

So far we have assumed that the OR for each predictor is the same at every value of the other predictors. *Interaction*, or *effect modification*, occurs when the association of one variable with the outcome differs according to the level of another (Rothman et al. 2008; Kirkwood and Sterne 2003). For example, the association of diabetes with uptake might be stronger in younger patients, for whom a diabetes diagnosis brings regular clinic contact. An interaction is modelled by adding the product of the two variables:

$$\text{logit}(p) = \beta_0 + \beta_1 \text{ age} + \beta_2 \text{ diabetes} + \beta_3 (\text{age} \times \text{diabetes}) + \dots$$

The OR for diabetes is then $\exp(\beta_2 + \beta_3 \times \text{age})$, which changes with age. We test $\beta_3 = 0$ with an LR test comparing the models with and without the product term.

```
model_int <- update(model_full, . ~ . + diabetes:age)
anova(model_full, model_int, test = "LRT")
```

R

```
#> Analysis of Deviance Table
#>
#> Model 1: treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km
#> Model 2: treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km + age:diabetes
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      980      1222
#> 2      979      1221  1      1.32      0.25
```

The interaction term reduces the deviance by only 1.3 on 1 df ($p = 0.25$). There is no evidence that the diabetes OR varies with age, so we keep the simpler model without the interaction. Interactions should be specified in advance on clinical grounds and tested sparingly: tests of interaction have low power, and searching many possible interactions produces spurious findings.

Common mistake

When an interaction term *is* kept in a model, the "main effect" coefficients change meaning. In the model above, `diabetesYes` would be the log OR for diabetes at age 0, a meaningless value. Report the OR for one variable at chosen values of the other (for example at ages 40, 55 and 70), or centre the continuous variable before forming the product.

5.8 Building the model

5.8.1 Pre-specification versus automated selection

How should the predictors in a multivariable model be chosen? In an explanatory or descriptive analysis the answer is: from subject-matter knowledge, decided before looking at the association of each candidate with the outcome, and documented in a statistical analysis plan (Vittinghoff et al. 2012; Harrell 2015; Heinze et al. 2018). This is how `model_full` was built.

The alternative, still common in clinical papers, is to let an algorithm choose: *forward selection* adds variables one at a time, *backward elimination* starts with all candidates and removes the least significant, and *stepwise* methods alternate between the two. A variant is the popular two-step approach of screening variables with univariable tests (for example keeping those with $p < 0.2$) and then running backward elimination. These procedures have well-documented problems (Harrell 2015; Heinze et al. 2018):

- **Biased estimates.** Variables are selected *because* their estimated effects happen to be large in this sample, so the selected coefficients are biased away from zero.
- **Invalid inference.** Standard errors, confidence intervals and p-values of the final model ignore the selection process; intervals are too narrow and p-values too small.
- **Instability.** Small changes to the data (a different sample, a few extra patients) often produce a different set of "selected" variables.

- **Dropped confounders.** An important confounder may be removed because its own p-value is above 0.05, reintroducing the bias that adjustment was meant to remove.
- **Univariable screening is misleading.** A variable can be unassociated with the outcome on its own but important after adjustment (and vice versa), so crude p-values are a poor guide to what belongs in a multivariable model.

R's `step()` function performs stepwise selection by AIC. For completeness, here is what it does with our model.

```
model_step <- step(model_full, direction = "both", trace = 0)
attr(terms(model_step), "term.labels")      # the variables it kept
```

```
#> [1] "age"          "sex"
#> [3] "education"    "residence"
#> [5] "diabetes"     "family_history_htn"
#> [7] "health_insurance" "knowledge_score"
#> [9] "distance_to_facility_km"
```

```
AIC(model_full, model_step)
```

```
#>           df  AIC
#> model_full 12 1246
#> model_step 12 1246
```

In this data set `step()` keeps every variable, including distance with its p-value of 0.13, because removing distance would slightly increase the AIC. On a different data set it might drop several variables. Our decision does not depend on what `step()` says: the pre-specified model is the main analysis.

Good practice

If you do use data-driven selection (for example in a prediction model), use methods designed for it, such as penalised regression, report the procedure fully, and assess the stability of the selection with bootstrap resampling (Heinze et al. 2018; Harrell 2015). Never present a model chosen by stepwise selection as if it had been specified in advance.

5.8.2 How many predictors can the data support?

Every parameter in a logistic model "uses up" information. The relevant amount of information is not the total sample size but the number of patients in the *less frequent* outcome category, often called the number of events. Peduzzi et al. (1996) showed in simulations that with fewer than about 10 *events per variable* (EPV, strictly per estimated parameter) the coefficients of a logistic model can be biased, their variances poorly estimated, and confidence intervals unreliable. The "10 EPV" rule is a rough guide rather than a law, and prediction models in particular may need considerably more (Harrell 2015), but it is a useful sanity check.

```
n_events <- min(table(model_full$y))      # less frequent category
n_parameters <- length(coef(model_full)) - 1 # excluding the intercept
c(events = n_events, parameters = n_parameters,
```

```
EPV = round(n_events / n_parameters, 1))
```

```
#>      events parameters      EPV
#>      457.0         11.0      41.5
```

With 457 events (patients on treatment) and 11 parameters, the EPV is about 42, well above

1. The model is not overfitted. The imprecision of the diabetes estimate

comes from a different problem: there are only 28 diagnosed patients with diabetes, and the precision of a binary predictor's OR depends on the number of patients in its smallest cell, not on the EPV of the whole model.

5.8.3 Collinearity

Collinearity (or multicollinearity) occurs when predictors are strongly correlated with each other. The model then struggles to separate their individual contributions, standard errors become inflated, and estimates can change wildly when a variable is added or removed. The *variance inflation factor* (VIF) for predictor j is

$$\text{VIF}_j = \frac{1}{1 - R_j^2},$$

where R_j^2 is the R^2 from regressing x_j on all the other predictors. A VIF of 1 means no collinearity; values above 5, and especially above 10, indicate a problem (Vittinghoff et al. 2012). For factors with several levels `car::vif()` reports the *generalised* VIF (GVIF) and $\text{GVIF}^{1/(2 \text{ df})}$, whose square is comparable with the usual VIF thresholds.

```
car::vif(model_full)
```

R

```
#>               GVIF Df GVIF^(1/(2*Df))
#> age              1.022  1          1.011
#> sex              1.006  1          1.003
#> education        1.048  3          1.008
#> residence         1.116  1          1.056
#> diabetes          1.019  1          1.009
#> family_history_htn 1.029  1          1.015
#> health_insurance  1.038  1          1.019
#> knowledge_score    1.034  1          1.017
#> distance_to_facility_km 1.097  1          1.047
```

All values are close to 1 (the largest GVIF, for residence, is about 1.12). The predictors carry largely separate information, and collinearity is not a concern. In clinical data, collinearity typically arises from including two measures of the same thing (weight and BMI, systolic and mean arterial pressure) or a variable together with a transformation of it.

5.8.4 Missing data and clustering

Two further design issues affect this model. First, as noted, 97 patients were excluded because of missing data. A complete-case analysis is unbiased if missingness does not depend on the

outcome given the predictors, but can be biased otherwise; with about 9% of patients excluded it is reasonable to report the complete-case analysis as primary and, in a full study, to check it with multiple imputation (Sterne et al. 2009; Little and Rubin 2019).

Second, patients were recruited from six facilities. Patients attending the same facility share staff, drug supplies and local policies, so their outcomes may be correlated, violating the independence assumption. Ignoring such clustering usually makes standard errors too small. Options include adding facility as a fixed effect (feasible with only six facilities; see Exercise 5.6), using cluster-robust standard errors, or fitting a mixed model with a random facility effect; the last two are beyond the scope of this book (Vittinghoff et al. 2012).

5.9 Checking the model

A fitted model should be checked before its results are reported. For logistic regression the main checks concern the form of the continuous predictors, how well the model separates patients with and without the outcome (*discrimination*), how closely predicted probabilities match observed proportions (*calibration*), and whether a few observations unduly influence the results (Hosmer et al. 2013; Harrell 2015).

5.9.1 Linearity of continuous predictors in the logit

The model assumes that age, knowledge score and distance each have a linear relationship with the log-odds of uptake. A simple check is to add a quadratic term for each and test it with an LR test. If the quadratic term is unnecessary, the linear form is adequate.

```

model_quad <- update(model_full, . ~ . + I(age^2) + I(knowledge_score^2) +
                      I(distance_to_facility_km^2))
anova(model_full, model_quad, test = "LRT")
R

#> Analysis of Deviance Table
#>
#> Model 1: treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km
#> Model 2: treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km + I(age^2) + I(knowledge_score^2) +
#>   I(distance_to_facility_km^2)
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      980      1222
#> 2      977      1219  3      2.71      0.44

```

The three quadratic terms together reduce the deviance by only a small amount ($p = 0.44$), so there is no evidence of curvature and the linear terms are retained. More flexible checks use restricted cubic splines (Harrell 2015).

5.9.2 Discrimination: the ROC curve and the AUC

For each patient the model produces a predicted probability of uptake. If we classify a patient as a "predicted taker" whenever this probability exceeds a threshold t , we can compute the *sensitivity* (the proportion of actual takers classified correctly) and the *specificity* (the proportion of non-takers

classified correctly). The *receiver operating characteristic* (ROC) curve plots sensitivity against 1 – specificity for every possible threshold.

The *area under the ROC curve* (AUC, also called the *c* statistic) summarises discrimination in a single number. Hanley and McNeil (1982) showed that it has a simple interpretation: the AUC is the probability that a randomly chosen patient *with* the outcome has a higher predicted probability than a randomly chosen patient *without* it. An AUC of 0.5 means the model discriminates no better than tossing a coin, and 1.0 means perfect separation. Common descriptive labels are 0.7–0.8 “acceptable”, 0.8–0.9 “excellent” and above 0.9 “outstanding” (Hosmer et al. 2013).

```
roc_obj <- roc(response = model_full$y,      # observed 0/1 outcome
               predictor = fitted(model_full), # predicted probabilities
               quiet    = TRUE)
auc(roc_obj)
```

```
#> Area under the curve: 0.715
```

```
ci.auc(roc_obj)      # DeLong 95% confidence interval
```

```
#> 95% CI: 0.683–0.747 (DeLong)
```

```
roc_df <- tibble(fpr = 1 - roc_obj$specificities,
                 tpr = roc_obj$sensitivities) |>
  arrange(fpr, tpr)

ggplot(roc_df, aes(x = fpr, y = tpr)) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed",
              colour = "grey50") +
  geom_step(linewidth = 0.9, colour = "#0D7377") +
  annotate("text", x = 0.65, y = 0.25,
           label = paste0("AUC = ", round(auc(roc_obj), 3))) +
  coord_equal() +
  labs(title = "ROC curve: treatment uptake model",
       x = "1 - specificity (false positive rate)",
       y = "Sensitivity (true positive rate)")
```

The AUC is 0.71 (95% CI 0.68 to 0.75): if we pick at random one patient on treatment and one not on treatment, the model assigns the higher probability to the treated patient about 71% of the time. This is acceptable discrimination for a model whose purpose is explanation rather than individual prediction.

Clinical interpretation

An AUC of about 0.71 means that the measured determinants explain part, but far from all, of why some patients take up treatment and others do not. The model is useful for understanding the population; it would not be good enough to decide which individual patients need extra support. Note also that the AUC is computed on the same data used to fit the model, so it is slightly optimistic; a prediction model would need internal validation, for example by bootstrapping (Harrell 2015).

ROC curve: treatment uptake model

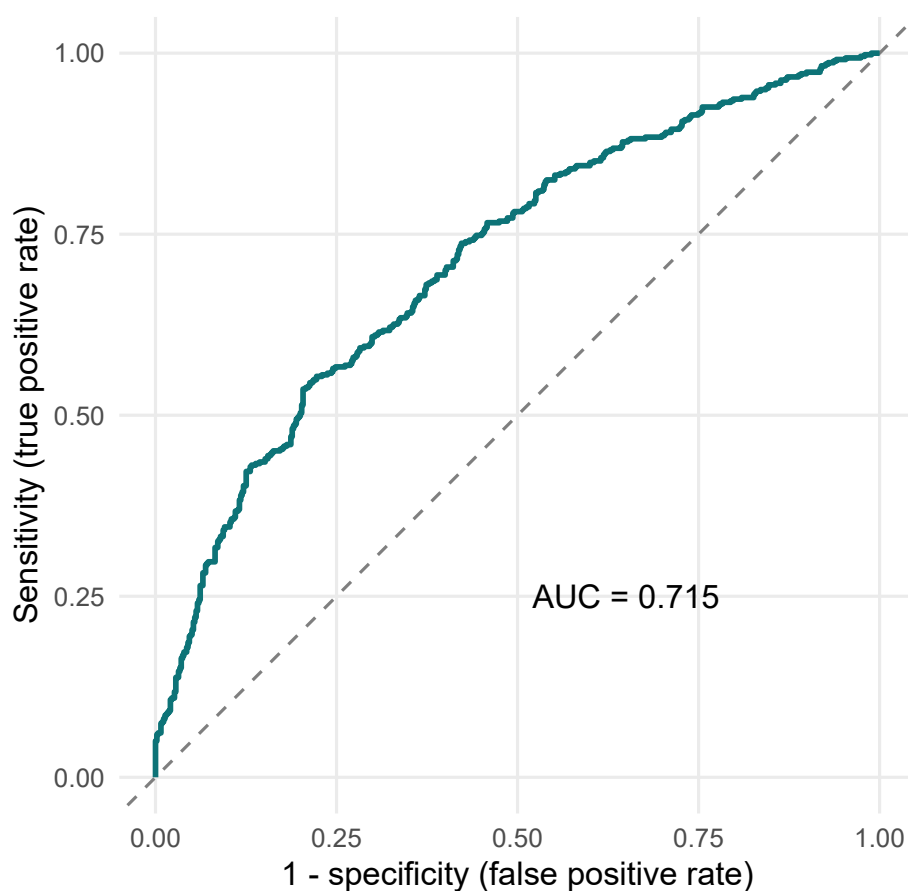


Figure 5.3: ROC curve for the multivariable logistic regression model of treatment uptake. The dashed diagonal shows a model with no discrimination (AUC = 0.5).

5.9.3 Calibration and the Hosmer–Lemeshow test

Discrimination is about ranking; *calibration* is about agreement. A well-calibrated model that predicts a 30% probability of uptake for a group of patients should see about 30% of them on treatment. The classical check groups patients by their predicted probability, usually into ten groups ("deciles of risk"), and compares the observed number of events O_g in each group with the expected number $E_g = \sum_{i \in g} \hat{p}_i$.

Hosmer and Lemeshow (1980) combined these comparisons into a goodness-of-fit statistic,

$$\hat{C} = \sum_{g=1}^G \frac{(O_g - E_g)^2}{E_g (1 - E_g/n_g)},$$

where n_g is the number of patients in group g . If the model fits, $\hat{C}$ approximately follows a χ^2 distribution with $G - 2$ degrees of freedom. A *small* p-value indicates poor fit. The test is available in add-on packages, but it is simple to compute directly, which also makes clear what it does.

```
calib <- tibble(y = model_full$y, p_hat = fitted(model_full)) |>
  mutate(decile = ntile(p_hat, 10)) |>           # 10 groups of equal size
  group_by(decile) |>
  summarise(n      = n(),
            mean_pred = mean(p_hat),           # mean predicted probability
            obs_prop  = mean(y),               # observed proportion
            expected   = sum(p_hat),           # E_g
            observed   = sum(y))               # O_g

kable(calib, digits = 3,
      caption = "Observed and expected uptake by decile of predicted risk.")
```

Table 5.7: Observed and expected uptake by decile of predicted risk.

decile	n	mean_pred	obs_prop	expected	observed
1	100	0.159	0.200	15.90	20
2	100	0.257	0.280	25.67	28
3	99	0.319	0.253	31.57	25
4	99	0.371	0.354	36.69	35
5	99	0.426	0.485	42.16	48
6	99	0.476	0.424	47.11	42
7	99	0.538	0.535	53.22	53
8	99	0.610	0.566	60.38	56
9	99	0.676	0.727	66.97	72
10	99	0.781	0.788	77.33	78

```
hl_stat <- with(calib, sum((observed - expected)^2 /
                          (expected * (1 - expected / n))))
hl_df    <- nrow(calib) - 2
hl_p     <- pchisq(hl_stat, df = hl_df, lower.tail = FALSE)
c(statistic = round(hl_stat, 2), df = hl_df, p = round(hl_p, 3))
```

```
#> statistic      df      p
#>      8.150    8.000 0.419
```

Across the ten groups, the observed and expected counts agree closely, and the Hosmer–Lemeshow statistic is 8.15 on 8 df ($p = 0.42$): no evidence of poor calibration. The same information is easier to see in a calibration plot.

```
ggplot(calib, aes(x = mean_pred, y = obs_prop)) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed",
             colour = "grey50") +
  geom_line(colour = "#0D7377") +
  geom_point(size = 2.5, colour = "#0D7377") +
  scale_x_continuous(labels = percent, limits = c(0, 1)) +
  scale_y_continuous(labels = percent, limits = c(0, 1)) +
  coord_equal() +
  labs(title = "Calibration by decile of predicted risk",
       x = "Mean predicted probability of uptake",
       y = "Observed proportion on treatment")
```

Calibration by decile of predicted risk

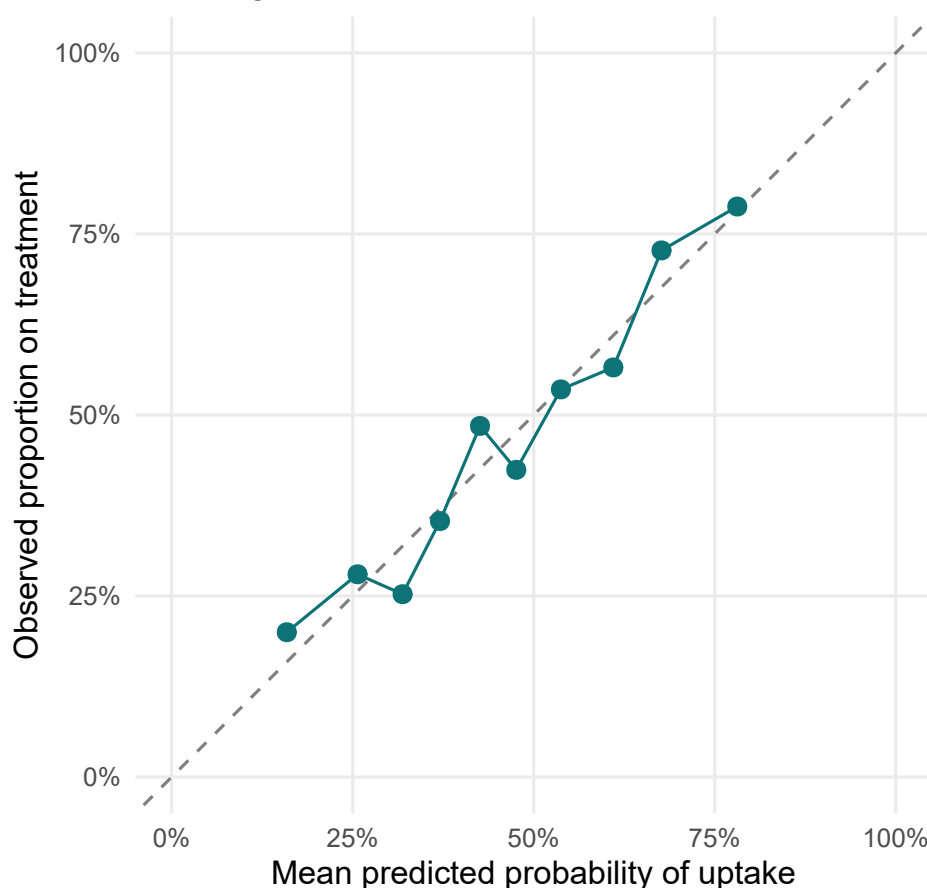


Figure 5.4: Calibration plot: observed proportion on treatment against mean predicted probability in ten groups of predicted risk. Points on the dashed line indicate perfect calibration.

Common mistake

Treating a non-significant Hosmer–Lemeshow test as proof of a good model. The test has low power in small samples and, in very large samples, flags trivial miscalibration; its result also depends on the number of groups chosen (Hosmer et al. 2013). Use it alongside a calibration plot, and remember that calibration in the data used to build the model says little about calibration in a new population.

5.9.4 Influential observations

A single unusual patient, or a few, can noticeably change a regression estimate. *Cook's distance* measures how much all the fitted values change when observation i is removed; `broom::augment()` computes it for every observation. A common screening threshold is $4/n$.

```
aug_glm <- augment(model_full)           # one row per patient used
cut_off <- 4 / nrow(aug_glm)
sum(aug_glm$.cooksdi > cut_off)          # number above the threshold
```

```
#> [1] 17
```

```
max(aug_glm$.cooksdi)                   # the largest Cook's distance
```

```
#> [1] 0.01534
```

```
# Who are the 10 most influential patients?
top10 <- order(aug_glm$.cooksdi, decreasing = TRUE)[1:10]
count(cc[top10, ], diabetes, treatment_uptake)
```

```
#>   diabetes treatment_uptake n
#> 1      No             Yes 2
#> 2     Yes             No 7
#> 3     Yes             Yes 1
```

17 of the 992 patients exceed the threshold, but the largest Cook's distance is only 0.015, far below the value of 1 that is sometimes used to indicate a truly dominating observation. Who are the most influential patients? Most of the ten largest values belong to patients with diabetes who are *not* on treatment. This makes sense: only 7 of the 28 diagnosed patients with diabetes are untreated, so each of them carries a great deal of the information about the diabetes coefficient.

What happens if we remove these ten patients, as an over-zealous analyst might do to "clean" the model?

```
model_sens <- update(model_full, data = cc[-top10, ])
tibble(term = names(coef(model_full))[-1],
       full = exp(coef(model_full))[-1],
       without_top10 = exp(coef(model_sens))[-1]) |>
  mutate(full = round(full, 2), without_top10 = signif(without_top10, 3))
```

```
#> # A tibble: 11 × 3
#>   term                full without_top10
#>   <chr>              <dbl>         <dbl>
#> 1 age                1.03           1.03
#> 2 sexMale            0.74           0.754
#> 3 educationPrimary  1.39           1.36
#> 4 educationSecondary 1.92           1.98
#> 5 educationTertiary  2.44           2.46
#> 6 residenceUrban    1.87           1.94
#> 7 diabetesYes      3.56 15300000
#> 8 family_history_htnYes 1.91           1.88
#> 9 health_insuranceYes  2.05           2.1
#> 10 knowledge_score    1.1           1.1
#> 11 distance_to_facility_km 0.98           0.982
```

Most ORs barely change, but the diabetes OR explodes to an absurd value of about 15 million. Once the untreated patients with diabetes in the data are removed, *every* remaining patient with diabetes is on treatment: diabetes then predicts the outcome perfectly. This is *complete separation* (Section 5.5), and the maximum likelihood estimate of the diabetes coefficient no longer exists. The example teaches two lessons. First, influential observations are often genuine and informative patients, not errors; deleting them can destroy the very information a coefficient depends on. Second, flagged observations should be *inspected* (are they data-entry errors, or real but unusual patients?), and any sensitivity analysis should be reported alongside the main analysis rather than replacing it.

5.10 Presenting the results

5.10.1 A publication-ready regression table

The standard way to present a multivariable logistic regression in a paper is a table with, for each predictor, the crude (univariable) OR and the adjusted (multivariable) OR, each with its 95% CI and p-value. Showing both lets readers see how much adjustment changes each estimate. Copying numbers from the console into a word processor is slow and error-prone; `gtsummary` builds the table directly from the model objects (Sjoberg et al. 2021).

`tbl_uvregression()` fits one univariable model per predictor, `tbl_regression()` formats the multivariable model, and `tbl_merge()` puts them side by side. We fit the univariable models to the same 992 complete cases (`cc`) so that crude and adjusted estimates are directly comparable.

```
# Readable labels for the predictors
labels_list <- list(
  age ~ "Age (years)",
  sex ~ "Sex",
  education ~ "Education",
  residence ~ "Residence",
  diabetes ~ "Diabetes",
  family_history_htn ~ "Family history of hypertension",
  health_insurance ~ "Health insurance",
  knowledge_score ~ "Knowledge score (per point)",
  distance_to_facility_km ~ "Distance to facility (per km)"
)
```

R

```
# Univariable (crude) ORs, one model per predictor
tbl_uni <- tbl_uvregression(
  data      = cc,
  method    = glm,
  y         = treatment_uptake,
  method.args = list(family = binomial),
  exponentiate = TRUE,
  label     = labels_list,
  hide_n    = TRUE
) |>
  modify_header(estimate ~ "**cOR**")

# Multivariable (adjusted) ORs from the pre-specified model
tbl_multi <- tbl_regression(model_full, exponentiate = TRUE,
  label = labels_list) |>
  modify_header(estimate ~ "**aOR**")

tbl_both <- tbl_merge(list(tbl_uni, tbl_multi),
  tab_spanner = c("**Crude**", "**Adjusted**"))

tbl_both |>
  as_kable(caption = paste("Crude and adjusted odds ratios for",
    "hypertension treatment uptake (n = 992)."))
```

Table 5.8: Crude and adjusted odds ratios for hypertension treatment uptake (n = 992).

Characteristic	cOR	95% CI	p-value	aOR	95% CI	p-value
Age (years)	1.03	1.02, 1.04	<0.001	1.03	1.02, 1.04	<0.001
Sex						
Female	—	—		—	—	
Male	0.75	0.58, 0.97	0.029	0.74	0.56, 0.97	0.031
Education						
None	—	—		—	—	
Primary	1.38	0.97, 1.98	0.078	1.39	0.95, 2.05	0.091
Secondary	1.88	1.28, 2.76	0.001	1.92	1.27, 2.90	0.002
Tertiary	2.11	1.32, 3.39	0.002	2.44	1.48, 4.06	<0.001
Residence						
Rural	—	—		—	—	
Urban	1.87	1.46, 2.42	<0.001	1.87	1.40, 2.49	<0.001
Diabetes						
No	—	—		—	—	
Yes	3.27	1.42, 8.44	0.008	3.56	1.46, 9.61	0.007
Family history of hypertension						
No	—	—		—	—	
Yes	1.65	1.27, 2.15	<0.001	1.91	1.44, 2.53	<0.001
Health insurance						
No	—	—		—	—	
Yes	1.67	1.28, 2.18	<0.001	2.05	1.54, 2.74	<0.001
Knowledge score (per point)	1.09	1.05, 1.13	<0.001	1.10	1.06, 1.14	<0.001
Distance to facility (per km)	0.97	0.95, 0.99	0.003	0.98	0.96, 1.01	0.13

The table has seven columns: the characteristic, then OR, 95% CI and p-value for the crude and adjusted analyses. Reference categories are shown with a dash, which reminds the reader what each OR is compared with. In a manuscript you would add a footnote: "cOR = crude odds ratio; aOR = odds ratio adjusted for all other variables in the table; CI = confidence interval. Complete-case analysis of 992 of 1,089 patients with diagnosed hypertension."

Good practice

In a manuscript, export the same table to Word or HTML with `gtsummary::as_flex_table()` or `as_gt()`, so that the numbers in the paper come directly from the code. If the data change, re-run the script and the table updates itself. Report p-values to two significant figures (and "<0.001" for very small values), never as stars or as "NS".

5.10.2 A forest plot of adjusted odds ratios

A *forest plot* displays each OR as a point with a horizontal line for its confidence interval, and a vertical reference line at OR = 1. Because ORs are ratios, the horizontal axis should be on a **logarithmic** scale: on a linear scale an OR of 0.5 (halving the odds) would appear much closer to 1 than an OR of 2 (doubling), although the two effects are of the same magnitude, and confidence intervals would look artificially asymmetric.

```
# Readable labels for each model term
term_labels <- c(
  age = "Age (per year)",
  sexMale = "Male (vs female)",
  educationPrimary = "Primary (vs none)",
  educationSecondary = "Secondary (vs none)",
  educationTertiary = "Tertiary (vs none)",
  residenceUrban = "Urban (vs rural)",
  diabetesYes = "Diabetes",
  family_history_htnYes = "Family history of HTN",
  health_insuranceYes = "Health insurance",
  knowledge_score = "Knowledge score (per point)",
  distance_to_facility_km = "Distance (per km)"
)

forest_df <- or_table |>
  filter(term != "(Intercept)") |>
  mutate(label = factor(term_labels[term],
    levels = rev(term_labels))) # keep model order

ggplot(forest_df, aes(x = estimate, y = label)) +
  geom_vline(xintercept = 1, linetype = "dashed", colour = "grey50") +
  geom_errorbar(aes(xmin = conf.low, xmax = conf.high),
    width = 0.25, colour = "#0D7377", orientation = "y") +
  geom_point(size = 2.6, colour = "#0D7377") +
  scale_x_log10(breaks = c(0.5, 0.75, 1, 1.5, 2, 3, 5, 10)) +
  labs(title = "Adjusted odds ratios for treatment uptake",
    x = "Adjusted odds ratio (log scale)", y = NULL)
```

The plot conveys the main messages at a glance: all the ORs except those for sex and distance lie to the right of 1; most intervals exclude 1; the diabetes interval is much the widest; and the distance OR sits just to the left of 1 with an interval that crosses it. Note that the per-unit ORs for continuous variables look small only because the units are small; a forest plot that mixes per-unit

Adjusted odds ratios for treatment uptake

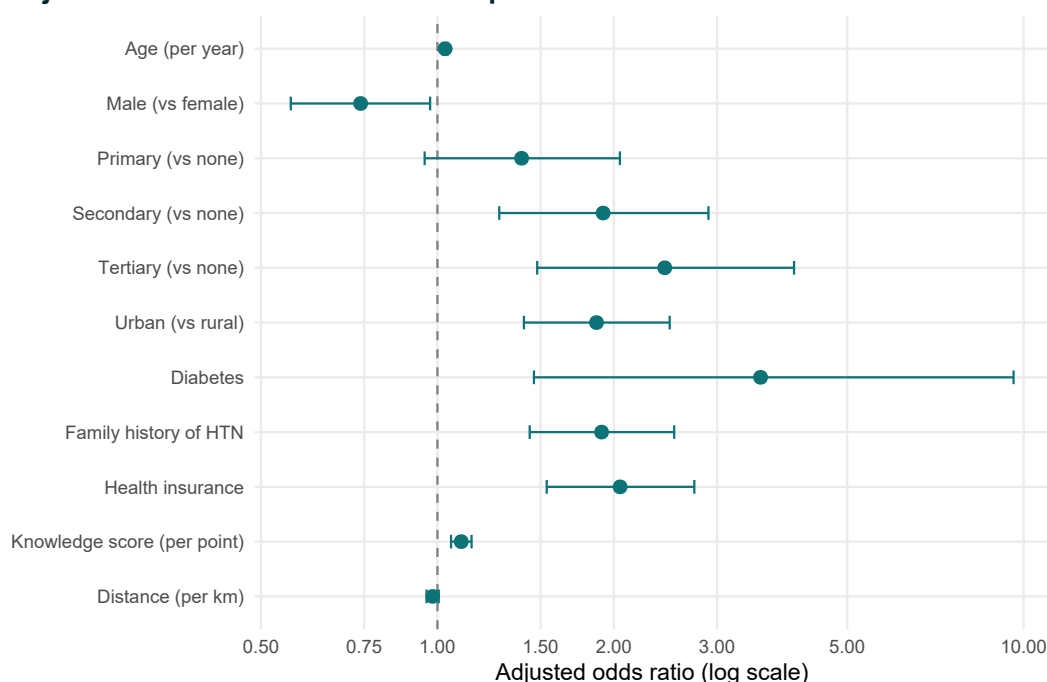


Figure 5.5: Forest plot of adjusted odds ratios (95% confidence intervals) for treatment uptake from the multivariable logistic regression model ($n = 992$). The horizontal axis is on a logarithmic scale; the dashed line marks $OR = 1$.

and per-category ORs should say so in its labels, or use rescaled ORs.

5.11 Writing the results

5.11.1 What to report

The STROBE statement (*Strengthening the Reporting of Observational Studies in Epidemiology*) lists the items that reports of cohort, case-control and cross-sectional studies should contain (Elm et al. 2007). For a regression analysis, the most relevant are:

- **Participants:** the number of individuals at each stage, including the number eligible, the number analysed, and the reasons for exclusion (here 1,089 diagnosed patients, of whom 992 had complete data).
- **Statistical methods:** the model, how variables were chosen, how continuous variables were handled, how missing data were addressed, and any sensitivity analyses.
- **Main results:** crude and adjusted estimates with their precision (95% CIs), making clear which confounders were adjusted for and why.
- **Category boundaries and units:** for continuous predictors, the unit to which each OR refers.
- **Other analyses:** interactions examined and sensitivity analyses.

5.11.2 A Methods paragraph

> We used multivariable logistic regression to estimate the association > of patient and access characteristics with treatment uptake among adults > with diagnosed hypertension. Predictors were specified before analysis on > the basis of the literature and clinical reasoning: age, sex, education, > residence, diabetes, family history of hypertension, health insurance, > hypertension knowledge score and distance to the facility. Continuous > predictors were modelled as linear terms after checking for non-linearity > with quadratic terms. Results are reported as odds ratios (ORs) with 95% > profile-likelihood confidence intervals; overall associations of > categorical variables were assessed with likelihood-ratio tests. > Discrimination was assessed with the area under the ROC curve and > calibration with the Hosmer–Lemeshow test. Analyses used complete cases > and were performed in R version 4.6.

5.11.3 A Results paragraph

The paragraph below is written with inline R code, so every number comes straight from the model objects created above.

> Of the 1,089 adults with diagnosed > hypertension, 508 (46.6%) were > receiving antihypertensive treatment; 992 had complete data and > were included in the multivariable model. After mutual adjustment, > treatment uptake was associated with diabetes (OR > 3.56, 95% CI 1.46 to 9.61), health insurance (OR > 2.05, 95% CI 1.54 to 2.74), a family history of hypertension (OR > 1.91, 95% CI 1.44 to 2.53) and urban rather than rural > residence (OR 1.87, 95% CI 1.40 to 2.49). The odds of uptake increased > with educational attainment (likelihood-ratio $p \Rightarrow 0.001$; OR for tertiary versus no education > 2.44, 95% CI 1.48 to 4.06), with age (OR per 10 years > 1.35, 95% CI 1.23 to 1.50) > and with hypertension knowledge (OR per 5 points > 1.60, 95% CI 1.31 to 1.96). > Men had lower odds of uptake than women (OR 0.74, 95% CI 0.56 to 0.97). > Distance to the facility was associated with lower uptake in the crude > analysis, but after adjustment (mainly for residence) the association was > weaker and compatible with no association (OR per 5 km > 0.91, 95% CI 0.81 to 1.03). > The model showed acceptable discrimination (AUC > 0.71, 95% CI 0.68 to 0.75) > and no evidence of poor calibration (Hosmer–Lemeshow $p \Rightarrow 0.42$).

Notice the features of this paragraph. It starts with the participants and the number analysed. Every OR has a 95% CI, and the reference category is clear ("urban rather than rural", "men ... than women"). Continuous predictors are reported per meaningful increment. A multi-level variable is summarised by its overall LR test plus the most informative comparison. A non-significant result (distance) is reported with its estimate and interval, not as "not significant" alone. And the wording describes *associations*, not effects.

Common mistake

Over-claiming causality. This is a cross-sectional study: exposure and outcome were measured at the same time, so we cannot tell whether, for example, knowledge led to treatment or treatment (and the counselling that accompanies it) led to knowledge. Words such as "effect", "impact", "leads to", "reduces" and "determines" imply causation. Prefer "was associated with", "had higher odds of" and "was more common among", and discuss reverse causation and residual confounding in the Discussion (Elm et al. 2007).

Clinical interpretation

If these findings were real, they would point to several groups at risk of not receiving treatment after diagnosis: men, rural residents, people without health insurance, those with little formal education and those with poor knowledge of hypertension. Some of these factors (insurance coverage, knowledge) are potentially modifiable through policy and health education. Remember, however, that the data in this book are simulated for teaching: the results illustrate methods, not real clinical findings.

5.11.4 Results, interpretation and discussion

A Results section reports what was found: numbers, intervals, and neutral wording. The interpretation of those numbers (what they mean for patients and services), the comparison with previous studies, and the limitations belong in the Discussion. For this analysis the limitations section would mention the cross-sectional design (no temporal ordering), self-reported uptake, the complete-case analysis, possible residual confounding by unmeasured factors such as income or severity of hypertension, and the clustering of patients within six facilities. Finally, the Discussion should avoid the Table 2 fallacy: the adjusted ORs are mutually adjusted associations, not a ranking of causal effects (Westreich and Greenland 2013).

5.12 Reproducible reporting

5.12.1 Why reproducibility matters

An analysis is *reproducible* if another person, given the same data and code, obtains the same results (Peng 2011). This is a lower bar than *replicability* (obtaining consistent results in new data), but it is surprisingly often not met. Numbers copied by hand from the console into a manuscript drift out of date as the analysis evolves; figures produced by clicking through menus cannot be regenerated; and a script that runs only on the author's computer, with files in `C:/Users/...`, cannot be checked by anyone else. In clinical research, where results may change practice, the ability to retrace every step from raw data to published number is part of scientific integrity.

5.12.2 Principles for a reproducible analysis

Wilson et al. (2017) describe "good enough" practices that any researcher can adopt, and Peng (2011) describes a spectrum from publication alone to fully linked and executable code and data. For an analysis like the one in this book, the essential practices are as follows.

- **Keep raw data read-only.** Never edit the raw file by hand. All cleaning is done by a script (Chapter 2) that writes a separate clean file, as `analysis_data.rds` was created.
- **Organise the project.** Use one folder per project, opened as an RStudio Project, with sub-folders such as `Data/` (raw and clean data), `Scripts/` (numbered analysis scripts), `Resources/` or `Output/` (generated tables and figures), and `Reports/` (R Markdown or Quarto documents).
- **Use relative paths.** Refer to files relative to the project folder ("`Data/analysis_data.rds`"), never with an absolute path; the `here` package helps when scripts live in sub-folders.
- **Script everything.** Every table and figure should be produced by code that is saved, not by interactive commands typed and forgotten.

- **Set the random seed.** Any analysis involving random numbers (bootstrapping, multiple imputation, random splits, simulation) must call `set.seed()` so that it gives identical results each time.
- **Record the software environment.** Report the versions of R and of the key packages, since results and defaults can change between versions.
- **Use version control** (such as Git) to keep a history of changes to code and text.

5.12.3 Setting the seed: an example

Maximum likelihood estimation is deterministic, so `model_full` gives the same coefficients every time. Resampling methods are not. A bootstrap confidence interval for the AUC, for example, draws random resamples of the patients.

```
set.seed(2026)
ci.auc(roc_obj, method = "bootstrap", boot.n = 500, progress = "none")
```

```
#> 95% CI: 0.682-0.747 (500 stratified bootstrap replicates)
```

```
set.seed(2026) # same seed, so exactly the same interval
ci.auc(roc_obj, method = "bootstrap", boot.n = 500, progress = "none")
```

```
#> 95% CI: 0.682-0.747 (500 stratified bootstrap replicates)
```

Run with the same seed, the two bootstrap intervals are identical; with a different seed (or none) they would differ slightly from run to run. Both are close to the analytical DeLong interval computed earlier.

5.12.4 From script to report: R Markdown and Quarto

The most powerful tool for reproducible reporting is the *dynamic document*, which interleaves text and code in a single file (Xie 2015). In an R Markdown (`.Rmd`) or Quarto (`.qmd`) document, prose is written in Markdown and analysis code is placed in *chunks*. When the document is rendered ("knitted"), R runs every chunk in a fresh session and inserts the output (tables, figures and numbers) into a Word, PDF or HTML report. This book itself was produced in this way.

A minimal analysis report has three parts:

1. A *header* (written in YAML) with the title, author, date and output format, for example output: `word_document` in R Markdown or format: `docx` in Quarto.
2. *Code chunks* that load the packages and data, fit the models and produce the tables and figures exactly as in this chapter.
3. *Text* with inline R code, so that numbers in sentences are computed rather than typed. The Results paragraph in Section 5.11 was written this way: an inline expression such as `nrow(dx)`, placed between backticks after the letter *r*, is replaced by its value (1089) when the document is rendered.

Because the report is regenerated from the data each time, a correction to the cleaning script, or

an updated data set, flows automatically into every number, table and figure. There is no step at which a number can be copied wrongly.

The code below shows how outputs would be saved from a script. In a real project they would go to an output folder; here they are written to a temporary directory so that nothing in the project is changed.

```

out_dir <- tempdir()      # in a project: "Resources" or "Output"

# Regression table and forest plot data as CSV
write_csv(or_table, file.path(out_dir, "regression_table.csv"))

# The forest plot as a high-resolution PNG
forest_plot <- ggplot(forest_df, aes(x = estimate, y = label)) +
  geom_vline(xintercept = 1, linetype = "dashed") +
  geom_errorbar(aes(xmin = conf.low, xmax = conf.high), width = 0.25,
               orientation = "y") +
  geom_point() +
  scale_x_log10()
ggsave(file.path(out_dir, "forest_plot.png"), forest_plot,
       width = 7, height = 5, dpi = 300)

list.files(out_dir, pattern = "regression_table|forest_plot")

#> [1] "forest_plot.png"      "regression_table.csv"

```

5.12.5 Recording the software environment

`sessionInfo()` prints the R version, the operating system and every loaded package with its version. In a report, you would include it in an appendix or supplementary file, for example with `writeln(capture.output(sessionInfo()), "session_info.txt")`. The full output is long; the essential information is shown below.

```

R.version.string

#> [1] "R version 4.6.1 (2026-06-24 ucrt)"

pkgs <- c("tidyverse", "broom", "gtsummary", "car", "pROC", "ggplot2")
tibble(package = pkgs,
       version = map_chr(pkgs, \(p) as.character(packageVersion(p))))

#> # A tibble: 6 × 2
#>   package version
#>   <chr>    <chr>
#> 1 tidyverse 2.0.0
#> 2 broom     1.0.13
#> 3 gtsummary 2.5.1
#> 4 car       3.1.5
#> 5 pROC      1.19.0.1
#> 6 ggplot2   4.0.3

```

Good practice

Before submitting a paper, restart R, clear the workspace, and run (or render) the whole analysis from the raw data to the final report in one go. If it runs without error and reproduces every number in the manuscript, your analysis is reproducible. If it does not, it is better that you find out now than a reviewer later.

5.13 Summary

This chapter brought together the skills of the previous chapters to answer the study's main question with regression models. Linear regression models the mean of a continuous outcome as a linear function of several predictors, estimated by least squares; its coefficients are differences in means, holding other predictors fixed, and its assumptions are checked with residual plots. For a binary outcome, logistic regression models the log-odds as a linear function of the predictors, estimated by maximum likelihood; exponentiated coefficients are odds ratios, which approximate risk ratios only when the outcome is rare.

We fitted a pre-specified multivariable logistic regression for treatment uptake among 992 patients with diagnosed hypertension and complete data, read the `summary()` output line by line, converted coefficients to odds ratios with profile-likelihood confidence intervals, rescaled continuous predictors to meaningful increments, and tested education as a whole with a likelihood-ratio test. Comparing crude and adjusted estimates showed that the apparent association between distance and uptake was largely explained by residence, a clear example of confounding, while the changes in the ORs for family history and insurance partly reflected non-collapsibility. The model had adequate events per variable, no collinearity, no evidence of non-linearity or poor calibration, and acceptable discrimination. Finally, we presented the results as a merged crude and adjusted table and a forest plot, wrote Methods and Results paragraphs in line with STROBE, and saw how dynamic documents, seeds and a recorded software environment make the whole analysis reproducible.

Key points

- Decide the purpose of a model (description, prediction or explanation) and its predictors before looking at the outcome data.
- In logistic regression, $\exp(\beta)$ is the odds ratio for a one-unit increase, holding the other predictors fixed; report continuous predictors per clinically meaningful increment, $\exp(c\beta)$.
- Odds ratios exaggerate risk ratios when the outcome is common; say "odds", and add predicted probabilities or risk ratios when readers need risks.
- Report the number of patients analysed: `glm()` silently drops anyone with a missing value in any model variable.
- Use likelihood-ratio tests (`drop1()`, `anova(test = "LRT")`) for multi-level factors and for comparing nested models fitted to the same patients.
- A change from crude to adjusted OR may reflect confounding, but ORs are non-collapsible; reason causally, and do not interpret every adjusted OR as a causal effect (the Table 2 fallacy).
- Avoid stepwise selection in explanatory models; check events per variable, collinearity (VIF), linearity, discrimination (AUC), calibration and influential observations.

- Present crude and adjusted ORs with 95% CIs in a `gtsummary` table and a log-scale forest plot; write results as associations, following STROBE.
- Make the analysis reproducible: scripts, relative paths, `set.seed()`, dynamic documents and a record of software versions.

5.14 Further reading

- Hosmer et al. (2013), *Applied Logistic Regression*: the standard reference on logistic regression, covering model building, interpretation of odds ratios, goodness of fit and diagnostics in depth.
- Vittinghoff et al. (2012), *Regression Methods in Biostatistics*: a clear, example-driven account of linear, logistic, Poisson and Cox regression, with an excellent chapter on choosing predictors for different purposes.
- Harrell (2015), *Regression Modeling Strategies*: the definitive guide to building prediction models responsibly, including the case against stepwise selection, splines, validation and calibration.
- Westreich and Greenland (2013) and VanderWeele (2019): two short, readable papers on how to choose adjustment variables and how (not) to interpret a multivariable model's coefficients.
- Elm et al. (2007), the STROBE statement: the checklist to follow when writing up any observational study.

5.15 Exercises

Exercise 5.1

Using the full `analysis_data` (all 1,500 adults), fit a linear regression of systolic blood pressure (`sbp_mmhg`) on `age`, `bmi` and `sex`.

1. Report the coefficient for `age` per 10 years, with its 95% CI.
2. Interpret the coefficient for `sexMale` in one sentence.
3. How much of the variation in SBP does the model explain?
4. Draw the residuals-versus-fitted plot. Are the assumptions reasonable?

Exercise 5.2

Among the diagnosed patients (`dx`), cross-tabulate `family_history_htn` against `treatment_uptake`.

1. Calculate by hand the proportion and the odds of uptake in each group, and the crude odds ratio.
2. Fit `glm(treatment_uptake ~ family_history_htn, family = binomial)` and confirm that `exp(coef())` gives the same OR.
3. Calculate the crude risk ratio and compare it with the OR. Why do they differ? Check your answer with the Zhang and Yu formula.

Exercise 5.3

From `model_full`, calculate the adjusted odds ratio and 95% CI for:

1. a 10-year increase in age;
2. a 5-point increase in knowledge score;
3. a 10 km increase in distance to the facility.

Which of these increments do you consider most clinically meaningful, and why should the increment be chosen before the analysis?

Exercise 5.4

Investigate whether the association between health insurance and uptake is confounded.

1. Using the complete-case data `cc`, fit the crude model for `health_insurance` and compare its OR with the adjusted OR from `model_full`. Calculate the percentage change in the log OR.
2. Describe in words a causal diagram for insurance and uptake. Which of the other model variables might be confounders, and could any be mediators?
3. Is the change you observed best explained by confounding or by non-collapsibility? Justify your answer.

Exercise 5.5

Education is an ordered variable.

1. Use `drop1()` to obtain the likelihood-ratio test for education in `model_full`, and state the null hypothesis it tests.
2. Create a numeric version of education (0 = None, 1 = Primary, 2 = Secondary, 3 = Tertiary) and refit the model with it in place of the factor. Interpret the OR "per level of education".
3. Compare the two models with AIC. Which would you report, and why?

Exercise 5.6

Patients were recruited from six facilities.

1. Add `facility` to `model_full` and test it with a likelihood-ratio test.
2. Does adding facility change any of the other adjusted ORs by more than 10%?
3. Compare the AUC of the two models. What do you conclude about the role of facility in this analysis?

Exercise 5.7

A health economist suggests that health insurance might matter more for rural patients, who face higher costs of travel.

1. Add the interaction `health_insurance:residence` to `model_full` and test it with a likelihood-ratio test.
2. From the interaction model, calculate the OR for insurance separately in rural and in urban patients.
3. Write two sentences reporting the result, as you would in a paper.

Exercise 5.8

A secondary outcome is blood pressure control (`bp_controlled`), defined only among patients on treatment.

1. Restrict the data to diagnosed patients with `treatment_uptake == "Yes"` and tabulate `bp_controlled`. How many events are there?
2. Using the 10 events-per-variable guideline, how many parameters can a logistic model for BP control support?
3. Pre-specify and fit a model with no more than that number of parameters (for example age, sex, BMI and adherence). Report the adjusted ORs with 95% CIs and the AUC.
4. Write a short Results paragraph (about 100 words) that reports the findings appropriately, including the limitations imposed by the small number of events.

Solutions to the Exercises

This appendix gives worked solutions to the exercises at the end of each chapter. Try each exercise yourself before reading the solution: there is often more than one correct way to write the code, and comparing your approach with ours is part of the learning.

A.1 Chapter 1: Getting Started with R and RStudio

Exercise 1.1

We store each measurement in an object whose name includes its unit, then write the formulas in terms of those objects. Height must be converted from centimetres to metres before squaring.

```
weight_kg <- 82
height_cm <- 168
sbp <- 148
dbp <- 96

bmi <- weight_kg / (height_cm / 100)^2
round(bmi, 1)
```

```
#> [1] 29.1
```

```
map <- dbp + (sbp - dbp) / 3
round(map, 1)
```

```
#> [1] 113.3
```

```
# Without parentheses: ^ is evaluated first, so this is
# weight / (height / 100^2) = 82 / (168 / 10000)
weight_kg / height_cm / 100^2
```

```
#> [1] 4.881e-05
```

The patient's BMI is 29.1 kg/m², just below the conventional threshold for obesity (30 kg/m²), and the MAP is 113.3 mmHg. Without the parentheses R computes 100² first (exponentiation has the highest priority), then divides from left to right, so the expression becomes 82 / 168 / 10,000, a meaningless number close to zero. No error message warns you: wrong parentheses produce wrong numbers silently.

Exercise 1.2

```
dbp <- c(88, 92, 79, 101, 95, NA, 84, 90)
length(dbp)
```

R

```
#> [1] 8
```

```
mean(dbp) # NA: one value is unknown
```

R

```
#> [1] NA
```

```
mean(dbp, na.rm = TRUE) # mean of the 7 observed values
```

R

```
#> [1] 89.86
```

```
sum(dbp >= 90, na.rm = TRUE) # number of readings >= 90
```

R

```
#> [1] 4
```

```
mean(dbp >= 90, na.rm = TRUE) # proportion of observed readings
```

R

```
#> [1] 0.5714
```

```
dbp[3:5]
```

R

```
#> [1] 79 101 95
```

```
dbp[!is.na(dbp)]
```

R

```
#> [1] 88 92 79 101 95 84 90
```

The vector has eight elements, one of them missing. Without `na.rm = TRUE`, `mean()` returns NA, because the mean of a set that includes an unknown value is itself unknown; with it, R averages the seven observed values (89.9 mmHg). Four readings are at least 90 mmHg, which is 4 of 7 observed readings, or 57%. Note that the comparison `dbp >= 90` gives NA for the missing reading, so `na.rm = TRUE` is needed in `sum()` and `mean()` too. The logical index `!is.na(dbp)` ("not missing") keeps the seven observed readings.

Exercise 1.3

```
glucose <- c("5.4", "6.1", "<2.0", "7.3", "999", "4.8")
class(glucose)
```

R

```
#> [1] "character"
```

```
glucose_num <- as.numeric(glucose)
glucose_num
```

```
#> [1] 5.4 6.1 NA 7.3 999.0 4.8
```

```
mean(glucose_num, na.rm = TRUE)
```

```
#> [1] 204.5
```

```
mean(glucose_num[glucose_num != 999], na.rm = TRUE)
```

```
#> [1] 5.9
```

The vector is character because its values are written in quotation marks; and even without the quotes, the entry "<2.0" could not be a number, so a column containing it would be read as text. `as.numeric()` converts the digits successfully but turns "<2.0" into NA (with the warning "NAs introduced by coercion"), because "less than 2.0" is not a number. That is a loss of information: the result is known to be low, not unknown. The value 999 is almost certainly a missing-value code, since a fasting glucose of 999 mmol/L is impossible. Left in, it raises the mean of the five numeric values to 204.5 mmol/L; without it, the mean of the four real values is 5.9 mmol/L.

Exercise 1.4

```
htn <- read_csv("Data/hypertension_phc_raw.csv")
htn_xl <- read_excel("Data/hypertension_phc_raw.xlsx", sheet = "data")
```

```
dim(htn)
```

```
#> [1] 1503 36
```

```
dim(htn_xl)
```

```
#> [1] 1503 36
```

```
identical(names(htn), names(htn_xl))
```

```
#> [1] TRUE
```

```
n_distinct(htn$patient_id)
```

```
#> [1] 1500
```

Both objects have 1,503 rows and 36 columns, with identical column names. The study enrolled 1,500 patients, and there are exactly 1,500 distinct patient identifiers, so three identifiers appear twice: the file contains three duplicate records, which must be removed before analysis (Chapter 2).

Exercise 1.5

```
glimpse(select(htn, age, sbp_mmhg, bmi, sex, facility, education,
               diabetes, treatment_uptake, enroll_date))
```

R

```
#> Rows: 1,503
#> Columns: 9
#> $ age          <dbl> 74, 56, 54, 33, 86, 70, 45, 59, 46, 61, 70, 56, 45...
#> $ sbp_mmhg     <dbl> 140, 185, 147, 124, 131, 153, 162, 108, 152, 127, ...
#> $ bmi          <dbl> 22.6, 26.7, 21.4, 28.4, 21.6, 31.6, NA, 25.2, 33.0...
#> $ sex          <chr> "Female", "Female", "Female", "Female", "Male", "M...
#> $ facility     <chr> "Igoma HC", "Kisesa HC", "Bugando PHC", "Ilemela H...
#> $ education    <chr> "Primary", "Primary", "None", "Primary", "Primary"...
#> $ diabetes     <chr> "No", "No", "No", "No", "0", "No", "0", "No", "0",...
#> $ treatment_uptake <chr> "Yes", "No", "N", "Yes", "No", "No", "1", "No", "0...
#> $ enroll_date  <chr> "01/02/2024", "2024-09-03", "2024-11-26", "2024-10..."
```

Numeric (<dbl>) variables include age, sbp_mmhg and bmi (also height_cm, weight_kg and the laboratory values). Character (<chr>) variables include sex, facility and education. The variables diabetes and treatment_uptake (and also health_insurance, family_history_htn and htn_diagnosed) should be Yes/No factors, but they mix the codes "Yes"/"No", "Y"/"N" and "1"/"0"; because "Yes" cannot be a number, readr stores the whole column as text. enroll_date is character because its values use different formats ("01/02/2024", "2024-09-03"), so readr could not recognise a single date format and kept the text unchanged.

Exercise 1.6

```
summary(select(htn, sbp_mmhg, dbp_mmhg, height_cm))
```

R

```
#>      sbp_mmhg      dbp_mmhg      height_cm
#> Min.   : 0      Min.   : 5.0      Min.   : 17
#> 1st Qu.:126      1st Qu.: 79.0      1st Qu.:158
#> Median :139      Median : 86.0      Median :164
#> Mean   :140      Mean   : 85.9      Mean   :164
#> 3rd Qu.:153      3rd Qu.: 93.0      3rd Qu.:169
#> Max.   :700      Max.   :121.0      Max.   :193
```

```
table(htn$diabetes)
```

R

```
#>
#>      0      1      N      No      Y      Yes
#> 230      7     143    1095      4      24
```

```
table(htn$htn_diagnosed)
```

R

```
#>
#>    0    1    N  No    Y  Yes
#> 73 158  44 296 110 822
```

```
sum(is.na(htn$weight_kg))
```

R

```
#> [1] 45
```

Systolic pressure ranges from 0 to 700 mmHg: both extremes are impossible. Diastolic pressure ranges from 5 to 121 mmHg; the minimum of 5 mmHg is implausible. Height ranges from 17 to about 193 cm; 17 cm is impossible for an adult and is probably 170 cm with a misplaced decimal point. Both diabetes and htn_diagnosed use six different codes (0, 1, N, No, Y, Yes) for what should be two categories. Finally, 45 values of weight_kg are missing.

Exercise 1.7

```
htn |>
  filter(facility == "Nyamagana PHC") |>
  mutate(map_mmhg = round((sbp_mmhg - dbp_mmhg) / 3, 1)) |>
  select(patient_id, age, sbp_mmhg, dbp_mmhg, map_mmhg) |>
  arrange(desc(map_mmhg)) |>
  head(5)
```

R

```
#> # A tibble: 5 × 5
#>   patient_id age sbp_mmhg dbp_mmhg map_mmhg
#>   <chr>      <dbl>   <dbl>   <dbl>   <dbl>
#> 1 PHC-0741    63    171    116    134.3
#> 2 PHC-1014    59    149    119    129
#> 3 PHC-0810    65    183    102    129
#> 4 PHC-0701    62    163    110    127.7
#> 5 PHC-1106    53    159    109    125.7
```

```
htn |>
  filter(diabetes == "Yes") |>
  count(facility)
```

R

```
#> # A tibble: 6 × 2
#>   facility      n
#>   <chr>      <int>
#> 1 Bugando PHC      6
#> 2 Buzuruga PHC      3
#> 3 Igoma HC          3
#> 4 Ilemela HC          6
#> 5 Kisesa HC          2
#> 6 Nyamagana PHC      4
```

The highest MAP values at Nyamagana PHC lie between about 126 and 134 mmHg, from patients with systolic pressures of 149 to 183 mmHg and diastolic pressures above 100 mmHg. These are high but clinically possible (severe hypertension), so unlike an SBP of 700 mmHg they should not be treated as errors. The second pipeline counts records whose diabetes value is exactly

"Yes". Only 24 records in total are coded "Yes", whereas Exercise 1.6 showed that 11 more are coded "1" or "Y", so a count based on a single spelling underestimates the number of patients with diabetes by almost a third (24 instead of 35). This is one more reason to standardise codes before any analysis.

Exercise 1.8

```
ggplot(htn, aes(x = sbp_mmhg)) +  
  geom_histogram(binwidth = 10, fill = "#1F6F8B", colour = "white") +  
  labs(x = "Systolic blood pressure (mmHg)", y = "Number of patients")
```

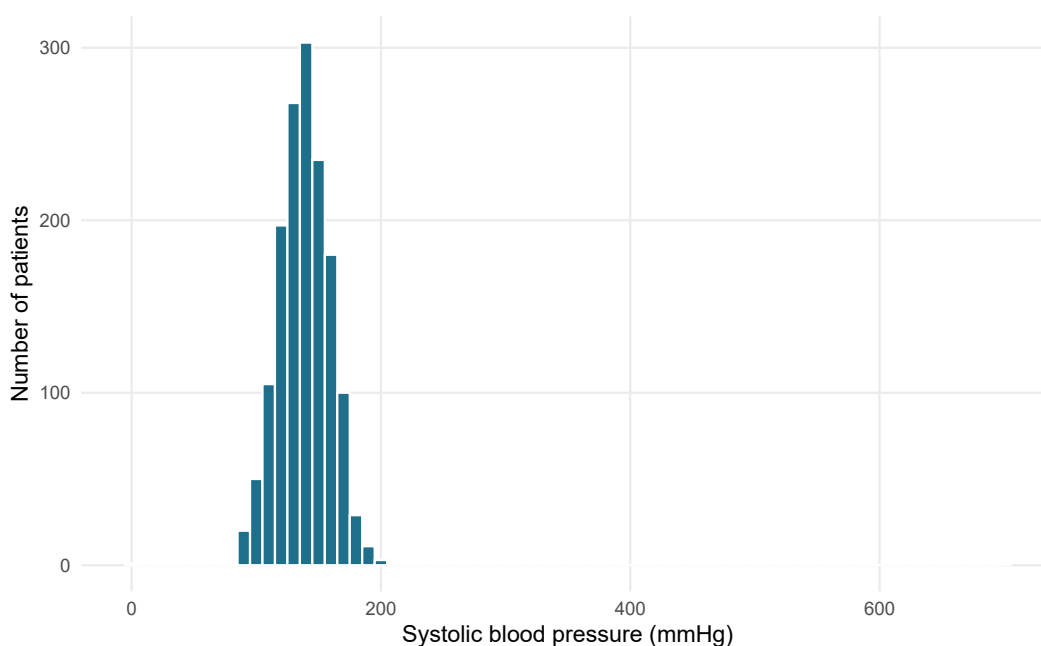


Figure A.1: Histogram of systolic blood pressure in the raw data (n = 1,503). Two impossible values (0 and 700 mmHg) stretch the horizontal axis and compress the real distribution into a narrow band.

```
htn_plausible <- htn |>  
  filter(sbp_mmhg >= 60, sbp_mmhg <= 260)  
nrow(htn) - nrow(htn_plausible) # records removed
```

```
#> [1] 2
```

```
summary(htn_plausible$sbp_mmhg)
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.   
#>    90   126    139    139   153    201
```

```
ggplot(htn_plausible, aes(x = sbp_mmhg)) +  
  geom_histogram(binwidth = 5, fill = "#1F6F8B", colour = "white") +  
  labs(x = "Systolic blood pressure (mmHg)", y = "Number of patients")
```

In the first histogram the two extreme values force the axis to run from 0 to 700 mmHg, so the real data occupy a narrow band and their shape is hard to see. Filtering removed two records (the

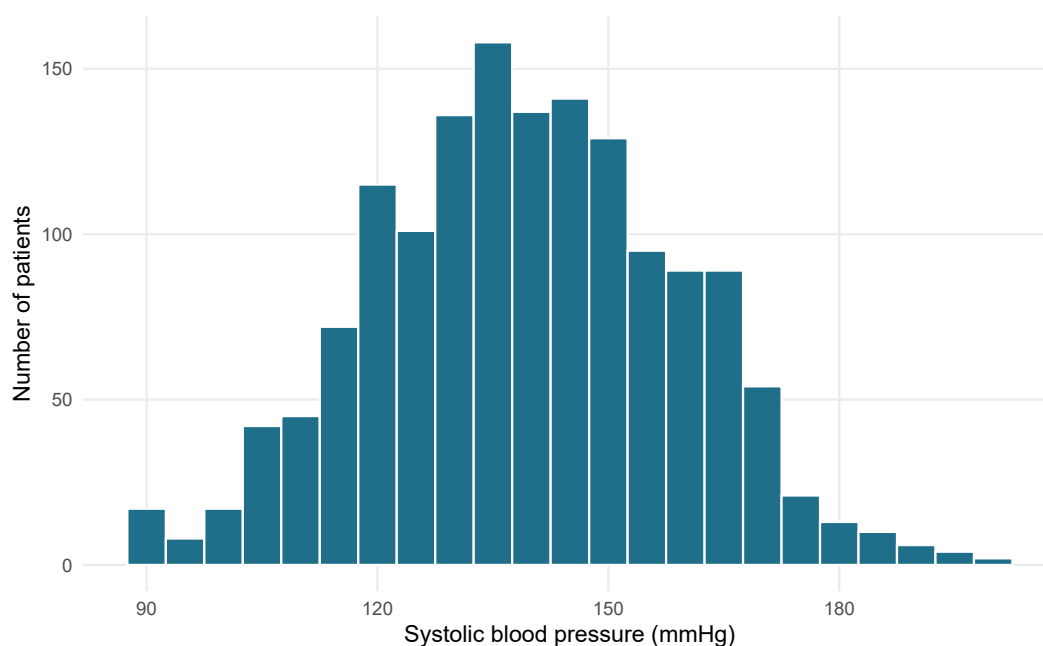


Figure A.2: Histogram of systolic blood pressure after keeping only plausible values between 60 and 260 mmHg. The distribution is unimodal and slightly right-skewed, centred near 140 mmHg.

values 0 and 700 mmHg; no SBP values were missing, so no further rows were lost). The plausible values form a single-peaked distribution centred near 139 mmHg (median), with most readings between about 100 and 190 mmHg and a slightly longer tail towards high pressures. The minimum plausible value is 90 mmHg and the maximum 201 mmHg. Just under half of the patients have a systolic pressure of 140 mmHg or more (the median is 139 mmHg), as expected in a population with many hypertensive patients. In practice, implausible values should be set to missing in a documented cleaning step rather than silently filtered out, as shown in Chapter 2.

A.2 Chapter 2: Understanding and Cleaning Clinical Data

The solutions below start either from the raw file or from the clean analysis file, as each exercise requires. We load the packages and both datasets once.

```
library(tidyverse)
library(lubridate)

na_codes <- c("", "NA", "999", "-99")
raw_default <- read_csv("Data/hypertension_phc_raw.csv")
raw <- read_csv("Data/hypertension_phc_raw.csv", na = na_codes)
analysis_data <- readRDS("Data/analysis_data.rds")

# The Yes/No helper from Section 2.6
to_yesno <- function(x) {
  x <- str_to_lower(str_trim(as.character(x)))
  case_when(
    x %in% c("yes", "y", "1", "true") ~ "Yes",
    x %in% c("no", "n", "0", "false") ~ "No",
    TRUE ~ NA_character_
  )
}
```

```
}
```

Exercise 2.1

We write a small summary function and apply it to both imports, so that the two rows of the result are directly comparable.

```
# 1-2. Fasting glucose under the two imports
glucose_summary <- function(d) {
  summarise(d,
    mean = mean(fasting_glucose_mmol_l, na.rm = TRUE),
    median = median(fasting_glucose_mmol_l, na.rm = TRUE),
    max = max(fasting_glucose_mmol_l, na.rm = TRUE),
    n_missing = sum(is.na(fasting_glucose_mmol_l)))
}
bind_rows(default = glucose_summary(raw_default),
  declared = glucose_summary(raw), .id = "import")
```

```
#> # A tibble: 2 × 5
#>   import      mean median    max n_missing
#>   <chr>      <dbl> <dbl> <dbl>    <int>
#> 1 default  32.521    5.4  999      35
#> 2 declared  5.4489    5.4  10.1     75
```

With the default import, the 40 values coded 999 are treated as real glucose concentrations. The **mean** is the most affected summary: it rises from 5.4 to 32.5 mmol/L, because the mean uses the size of every value and a few enormous values dominate it. The **maximum** is also wrong (999 instead of 10.1). The **median** is unchanged at 5.4, because the 40 sentinels, although extreme, are only 2.7 per cent of the values and do not move the middle of the distribution; this robustness is one reason medians are preferred for skewed data. The default import reports 35 missing values and the declared import 75, the difference being the 40 sentinels.

For question 3, we import with only the standard missing codes and convert 999 to NA in the glucose column alone, using `na_if()`.

```
# 3. Treat 999 as missing only in the column where it is a sentinel
raw_selective <- read_csv("Data/hypertension_phc_raw.csv",
  na = c("", "NA")) |>
  mutate(fasting_glucose_mmol_l = na_if(fasting_glucose_mmol_l, 999))

sum(is.na(raw_selective$fasting_glucose_mmol_l))
```

```
#> [1] 75
```

```
max(raw_selective$creatinine_umol_l) # creatinine untouched
```

```
#> [1] 141
```

Glucose now has the same 75 missing values as before, while creatinine is untouched (its maximum, 141 µmol/L, shows no column was altered by mistake; in a dataset where creatinine genuinely

reached 999 it would be kept). This selective approach is the safe default whenever a code could be real in some columns.

Exercise 2.2

We list the repeated IDs, show their records with `janitor::get_dupes()`, and compare the number of exact duplicate rows with the number of surplus IDs.

```
# 1. IDs that occur more than once, and their records
raw |> count(patient_id) |> filter(n > 1)
```

```
#> # A tibble: 3 × 2
#>   patient_id     n
#>   <chr>       <int>
#> 1 PHC-0011         2
#> 2 PHC-0251         2
#> 3 PHC-0881         2
```

```
janitor::get_dupes(raw, patient_id) |>
  select(patient_id, dupe_count, facility, age, sex, sbp_mmhg)
```

```
#> # A tibble: 6 × 6
#>   patient_id dupe_count facility    age sex    sbp_mmhg
#>   <chr>       <int> <chr>    <dbl> <chr>    <dbl>
#> 1 PHC-0011         2 Ilemela HC    51 Male    128
#> 2 PHC-0011         2 Ilemela HC    51 Male    128
#> 3 PHC-0251         2 Bugando PHC    51 Female  131
#> 4 PHC-0251         2 Bugando PHC    51 Female  131
#> 5 PHC-0881         2 Kisesa HC     45 Female  140
#> 6 PHC-0881         2 Kisesa HC     45 Female  140
```

```
# 2. Exact duplicates versus surplus IDs
sum(duplicated(raw))
```

```
#> [1] 3
```

```
nrow(raw) - n_distinct(raw$patient_id)
```

```
#> [1] 3
```

Three IDs (PHC-0011, PHC-0251 and PHC-0881) occur twice, and `get_dupes()` shows the two records of each side by side, with a `dupe_count` column. There are 3 exact duplicate rows and 3 surplus IDs (1,503 rows minus 1,500 distinct IDs). Because the two numbers agree, every repeated ID is an exact copy, and there are no conflicting records that would need checking against source documents.

A suitable sentence for a methods section: *"The raw dataset contained 1,503 records; three were exact duplicates of other records (identical in all variables) and were removed, leaving 1,500 participants with unique identifiers."*

Exercise 2.3

We tabulate the raw codes, then apply the helper after removing duplicates and check the mapping with a cross-tabulation of old against new values.

```
# 1. Codes used before cleaning
table(raw$health_insurance, useNA = "ifany")
```

R

```
#>
#>   0    1    N   No    Y   Yes
#> 150   70   97  743   49  394
```

```
table(raw$htn_diagnosed, useNA = "ifany")
```

R

```
#>
#>   0    1    N   No    Y   Yes
#>  73 158   44 296 110  822
```

```
# 2. Apply the helper and check the mapping
clean_bin <- distinct(raw) |>
  mutate(health_insurance_new = to_yesno(health_insurance),
         htn_diagnosed_new = to_yesno(htn_diagnosed))

table(before = clean_bin$health_insurance,
      after = clean_bin$health_insurance_new)
```

R

```
#>      after
#> before No Yes
#>   0   150   0
#>   1    70   0
#>   N    96   0
#>  No   742   0
#>   Y    49   0
#>  Yes   393
```

```
table(before = clean_bin$htn_diagnosed,
      after = clean_bin$htn_diagnosed_new)
```

R

```
#>      after
#> before No Yes
#>   0    73   0
#>   1    158  0
#>   N    43   0
#>  No   295   0
#>   Y    110  0
#>  Yes    821
```

Both variables use six codes: 0/1, N/Y and No/Yes. The cross-tabulations show that every code maps to exactly one clean value (all the 0, N and No codes to "No" and all the 1, Y and Yes codes to "Yes") and that no value became missing. The counts are slightly smaller than in the first two tables because the three duplicate records have been removed.

```
# 3. Proportions in the clean data
analysis_data |>
  summarise(insured_pct = round(100 * mean(health_insurance == "Yes"), 1),
            diagnosed_pct = round(100 * mean(htn_diagnosed == "Yes"), 1))
```

```
#> # A tibble: 1 × 2
#>   insured_pct diagnosed_pct
#>       <dbl>         <dbl>
#> 1       34.1           72.6
```

In the clean data, 34.1 per cent of patients have health insurance and 72.6 per cent (1,089 of 1,500) have been diagnosed with hypertension. Because `health_insurance` and `htn_diagnosed` have no missing values, `mean(x == "Yes")` gives the proportion directly; with missing values you would need `na.rm = TRUE` and should report the denominator.

Exercise 2.4

Cross-variable checks compare variables that must stand in a fixed relationship to each other.

```
# 1. Diastolic at least as high as systolic
analysis_data |>
  summarise(dbp_ge_sbp = sum(dbp_mmhg >= sbp_mmhg, na.rm = TRUE))
```

```
#> # A tibble: 1 × 1
#>   dbp_ge_sbp
#>       <int>
#> 1         11
```

```
# 2. Pulse pressure
analysis_data <- analysis_data |>
  mutate(pulse_pressure = sbp_mmhg - dbp_mmhg)
summary(analysis_data$pulse_pressure)
```

```
#>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
#>  -18.0   39.0   53.0   53.3   68.0  129.0      3
```

```
sum(analysis_data$pulse_pressure < 20, na.rm = TRUE)
```

```
#> [1] 92
```

Eleven patients have a diastolic pressure equal to or higher than their systolic pressure. That is not physiologically possible for a correctly measured brachial blood pressure: systolic pressure is, by definition, the peak pressure of the cardiac cycle. These records have each passed the single-variable range checks, which is exactly why cross-variable checks are needed. The pulse pressure summary confirms the problem from a different angle: the minimum is -18 mmHg. A further 92 patients have a pulse pressure below 20 mmHg, which is unusual (normal pulse pressure is around 40 mmHg) and might reflect transposed or mistyped readings. In a real study these records would be queried with the facilities. The data in this book are simulated and the readings are left unchanged in `analysis_data`, but if you were cleaning these data for publication you would

add a rule (for example, setting both pressures to NA when diastolic is at least systolic) to the cleaning script and report it.

```
# 3. Consistency rules from the data dictionary
analysis_data |>
  group_by(htn_diagnosed) |>
  summarise(n = n(),
            min_months = min(months_since_diagnosis),
            max_months = max(months_since_diagnosis))
```

```
#> # A tibble: 2 × 4
#>   htn_diagnosed      n min_months max_months
#>   <fct>          <int>    <dbl>    <dbl>
#> 1 No              411         0         0
#> 2 Yes             1089         1        119
```

```
analysis_data |>
  count(treatment_uptake, adherence_recorded = !is.na(adherence))
```

```
#> # A tibble: 2 × 3
#>   treatment_uptake adherence_recorded      n
#>   <fct>          <lgl>          <int>
#> 1 No              FALSE          992
#> 2 Yes              TRUE           508
```

Both rules hold. All 411 patients who are not diagnosed have `months_since_diagnosis` equal to 0, and the 1,089 diagnosed patients have values from 1 to 119 months. Adherence is recorded for all 508 treated patients and for none of the 992 untreated patients.

Exercise 2.5

Adding `right = FALSE` makes each interval include its lower limit, for example `[18.5, 25)`, which matches the WHO definition.

```
# 1. WHO-consistent categories: intervals closed on the left
analysis_data <- analysis_data |>
  mutate(bmi_cat_who = cut(bmi,
                           breaks = c(-Inf, 18.5, 25, 30, Inf),
                           labels = c("Underweight", "Normal",
                                       "Overweight", "Obese"),
                           right = FALSE))

# 2. Compare the two versions
table(default = analysis_data$bmi_cat, who = analysis_data$bmi_cat_who)
```

```
#>           who
#> default Underweight Normal Overweight Obese
#> Underweight      82      2         0      0
#> Normal           0     466        10      0
#> Overweight       0      0        568      9
#> Obese            0      0         0     316
```

```
analysis_data |>
  filter(bmi_cat != bmi_cat_who) |>
  count(bmi, bmi_cat, bmi_cat_who)
```

```
#> # A tibble: 3 × 4
#>   bmi bmi_cat    bmi_cat_who     n
#>   <dbl> <fct>      <fct>      <int>
#> 1  18.5 Underweight Normal         2
#> 2   25   Normal    Overweight    10
#> 3   30  Overweight  Obese         9
```

The cross-tabulation is almost diagonal: 21 patients change category, all of them moving one category up. The second table shows why: they are exactly the patients whose BMI, rounded to one decimal place, sits on a cut-point. Two patients with a BMI of 18.5 move from "Underweight" to "Normal", ten with a BMI of 25.0 from "Normal" to "Overweight" and nine with a BMI of 30.0 from "Overweight" to "Obese". The difference is small here, but in a study reporting the prevalence of obesity it would change the estimate, and it would make the results disagree with studies that applied the WHO definition correctly.

```
# 3. Age groups, checked against age
analysis_data <- analysis_data |>
  mutate(age_group = cut(age, breaks = c(18, 40, 60, Inf),
    labels = c("18-39", "40-59", "60+"),
    right = FALSE))

analysis_data |>
  group_by(age_group) |>
  summarise(n = n(), min_age = min(age), max_age = max(age))
```

```
#> # A tibble: 4 × 4
#>   age_group     n min_age max_age
#>   <fct>      <int>   <dbl>   <dbl>
#> 1 18-39      274     18     39
#> 2 40-59      772     40     59
#> 3 60+       452     60     95
#> 4 <NA>        2    NA     NA
```

The same `right = FALSE` idea defines age groups whose labels mean what they say: [18, 40) is 18 to 39 years. The minimum and maximum ages within each group confirm the definition (18–39, 40–59 and 60–95), and the two patients whose age was set to missing during validation correctly have a missing age group.

Exercise 2.6

Regular expressions describe each format: `\\d{4}` (with the backslash doubled inside an R string) is four digits and `[A-Za-z]{3}` three letters; `^` and `$` anchor the pattern to the start and end of the value.

```
# 1. Count each date format
dates <- distinct(raw)$enroll_date
c(iso = sum(str_detect(dates, "^\\d{4}-\\d{2}-\\d{2}$")),
  slash = sum(str_detect(dates, "^\\d{2}/\\d{2}/\\d{4}$")),
```

```
month_name = sum(str_detect(dates, "^\\d{2}-[A-Za-z]{3}-\\d{4}$"))
```

```
#>      iso      slash month_name
#>    1050      347      103
```

```
# 2. Month-first interpretation of the slash format
wrong <- parse_date_time(dates, orders = c("ymd", "mdy", "d-b-Y")) |>
  as_date()
sum(is.na(wrong)) # failures
```

```
#> [1] 221
```

```
sum(wrong != analysis_data$enroll_date, na.rm = TRUE) # silently different
```

```
#> [1] 137
```

The three formats account for all 1,500 dates (1,050 + 347 + 103). With month-first parsing of the slash format, 221 dates fail to parse: they have a first number greater than 12 (such as 18/02/2024), which cannot be a month. That failure is at least visible. More dangerous are the 137 dates that parse successfully but differ from the correct dates. Most of them (115) are slash dates in which both numbers are 12 or less, so that 01/02/2024 becomes 2 January instead of 1 February; the remaining 22 are month-name dates that `parse_date_time()` matched to a different order once the order list changed. Without a comparison against the correctly parsed dates, none of these 137 errors would have been noticed. This is why the order must be chosen from knowledge of the data, and why the result must be checked.

```
# 3. Enrolments per quarter
analysis_data |>
  mutate(enroll_quarter = quarter(enroll_date)) |>
  count(enroll_quarter)
```

```
#> # A tibble: 4 × 2
#>   enroll_quarter     n
#>   <int> <int>
#> 1         1   392
#> 2         2   412
#> 3         3   421
#> 4         4   275
```

Enrolment was evenly spread over the first three quarters (392 to 421 patients) and lower in the fourth quarter (275), because recruitment ended on 2 December.

Exercise 2.7

We create a missingness indicator and compare it across groups of observed variables.

```
analysis_data <- analysis_data |>
  mutate(chol_missing = is.na(total_chol_mmol_l))
```

```
# 1. Percentage missing by diagnosis and by residence
analysis_data |>
  group_by(htn_diagnosed) |>
  summarise(n = n(), pct_missing = round(100 * mean(chol_missing), 1))
```

```
#> # A tibble: 2 × 3
#>   htn_diagnosed     n pct_missing
#>   <fct>         <int>     <dbl>
#> 1 No             411         3.9
#> 2 Yes           1089         6.8
```

```
analysis_data |>
  group_by(residence) |>
  summarise(n = n(), pct_missing = round(100 * mean(chol_missing), 1))
```

```
#> # A tibble: 2 × 3
#>   residence     n pct_missing
#>   <fct>      <int>     <dbl>
#> 1 Rural     703         5.4
#> 2 Urban     797         6.5
```

```
# 2. Mean age by missingness
analysis_data |>
  group_by(chol_missing) |>
  summarise(n = n(), mean_age = round(mean(age, na.rm = TRUE), 1))
```

```
#> # A tibble: 2 × 3
#>   chol_missing     n mean_age
#>   <lgl>         <int>     <dbl>
#> 1 FALSE      1410         52
#> 2 TRUE         90        55.9
```

Cholesterol is missing for 6.8 per cent of diagnosed patients and 3.9 per cent of undiagnosed patients, and for 6.5 per cent of urban and 5.4 per cent of rural residents. Patients with missing cholesterol are on average about four years older (55.9 versus 52.0 years). The differences are modest, and Chapter 4 shows how to test whether differences of this size could be due to chance, but they point in a consistent direction: missingness appears to be related to observed characteristics (diagnosis, age). If so, the data are not MCAR, and an analysis restricted to patients with a cholesterol result would slightly over-represent younger, undiagnosed patients. MAR, with missingness depending on age and diagnosis, is a more plausible working assumption, and any analysis using cholesterol should at least include these variables in the model or in an imputation model.

The data cannot tell us whether cholesterol is MNAR. That would require knowing whether patients with missing values tend to have higher or lower cholesterol than observed patients of the same age and diagnosis, and their cholesterol is precisely what we do not observe. Judgements about MNAR rest on knowledge of how the data were collected (for example, whether blood was drawn only from patients who looked unwell) and are explored with sensitivity analyses. Because these data are simulated, the pattern illustrates the reasoning rather than a real clinical finding.

Exercise 2.8

The function collects the steps of the chapter in the same order. It relies on the `to_yn()` helper defined at the top of this appendix; in a real project you would keep both functions in the same script file.

```
clean_htn <- function(path) {
  # Import with missing-value codes; remove exact duplicates; trim text
  dat <- read_csv(path, na = c("", "NA", "999", "-99"),
    show_col_types = FALSE) |>
    distinct() |>
    mutate(across(where(is.character), str_trim))

  # Categories: sex and the Yes/No variables
  dat <- dat |>
    mutate(
      sex = case_when(
        str_to_lower(sex) %in% c("female", "f") ~ "Female",
        str_to_lower(sex) %in% c("male", "m") ~ "Male",
        TRUE ~ NA_character_),
      across(c(diabetes, family_history_htn, health_insurance,
        htn_diagnosed, treatment_uptake), to_yn)
    )

  # Validation: impossible values become NA
  dat <- dat |>
    mutate(
      age = if_else(between(age, 18, 110), age, NA_real_),
      sbp_mmhg = if_else(between(sbp_mmhg, 70, 260), sbp_mmhg, NA_real_),
      dbp_mmhg = if_else(between(dbp_mmhg, 40, 150), dbp_mmhg, NA_real_),
      height_cm = if_else(between(height_cm, 120, 210), height_cm, NA_real_),
      weight_kg = if_else(between(weight_kg, 30, 200), weight_kg, NA_real_)
    )

  # Derived variables and dates
  dat <- dat |>
    mutate(
      bmi = round(weight_kg / (height_cm / 100)^2, 1),
      bmi_cat = cut(bmi, breaks = c(-Inf, 18.5, 25, 30, Inf),
        labels = c("Underweight", "Normal",
          "Overweight", "Obese")),
      bp_category = case_when(
        is.na(sbp_mmhg) | is.na(dbp_mmhg) ~ NA_character_,
        sbp_mmhg >= 140 | dbp_mmhg >= 90 ~ "Hypertension",
        sbp_mmhg >= 130 | dbp_mmhg >= 80 ~ "Elevated",
        TRUE ~ "Normal"),
      enroll_date = as_date(parse_date_time(
        enroll_date, orders = c("ymd", "dmy", "d-b-Y")))
    )

  # Factors with deliberate levels
  yn <- c("No", "Yes")
  dat <- dat |>
    mutate(
      facility = factor(facility),
```

```

sex = factor(sex, levels = c("Female", "Male")),
residence = factor(residence, levels = c("Rural", "Urban")),
education = factor(education, ordered = TRUE,
                    levels = c("None", "Primary", "Secondary",
                               "Tertiary")),
occupation = factor(occupation),
marital_status = factor(marital_status),
physical_activity = factor(physical_activity, ordered = TRUE,
                           levels = c("Low", "Moderate", "High")),
smoking = factor(smoking, levels = c("Never", "Former", "Current")),
alcohol = factor(alcohol, levels = c("None", "Moderate", "Heavy")),
bp_category = factor(bp_category,
                     levels = c("Normal", "Elevated", "Hypertension")),
health_insurance = factor(health_insurance, levels = yn),
family_history_htn = factor(family_history_htn, levels = yn),
diabetes = factor(diabetes, levels = yn),
htn_diagnosed = factor(htn_diagnosed, levels = yn),
treatment_uptake = factor(treatment_uptake, levels = yn),
adherence = factor(na_if(adherence, ""), levels = c("Poor", "Good")),
bp_controlled = factor(na_if(bp_controlled, ""), levels = yn)
)

# Assertions: fail loudly if a new problem appears
stopifnot(
  n_distinct(dat$patient_id) == nrow(dat),
  !anyNA(dat$sex),
  !anyNA(dat$enroll_date),
  !anyNA(dat$treatment_uptake),
  all(between(dat$sbp_mmhg, 70, 260), na.rm = TRUE)
)
dat
}

my_clean <- clean_htn("Data/hypertension_phc_raw.csv")
all.equal(my_clean, readRDS("Data/analysis_data.rds"))

#> [1] TRUE

```

`all.equal()` returns `TRUE`: the function reproduces the analysis dataset exactly. The assertions at the end check that the patient ID is unique, that sex, enrolment date and the outcome have no missing values, and that systolic pressures are in range; if a future version of the raw file contained, say, a new spelling of sex or a new date format, the function would stop with an error naming the failed condition instead of returning quietly flawed data.

A function is more useful than a long script when a corrected raw file arrives for three reasons. It can be rerun on the new file with one line, without editing anything. It can be applied to several files (for example, an interim and a final data export) and the results compared. And because everything happens inside the function, no intermediate objects are left in the workspace to be confused with the clean data. Packaging a cleaning pipeline as a function is a small step towards the reproducible workflows recommended by Wilson et al. (2017).

A.3 Chapter 3: Descriptive Statistics, Tables and Figures

All solutions start from the clean analysis file and, where the outcome is involved, from the population of diagnosed patients.

```
analysis_data <- readRDS("Data/analysis_data.rds")
diagnosed <- analysis_data |> filter(htn_diagnosed == "Yes")
teal <- "#0D7377"
```

R

Exercise 3.1

We compute the four statistics for each variable, together with the number of missing values.

```
analysis_data |>
  select(age, distance_to_facility_km) |>
  pivot_longer(everything(), names_to = "variable", values_to = "value") |>
  group_by(variable) |>
  summarise(
    missing = sum(is.na(value)),
    mean    = mean(value, na.rm = TRUE),
    sd      = sd(value, na.rm = TRUE),
    median  = median(value, na.rm = TRUE),
    iqr     = IQR(value, na.rm = TRUE)
  ) |>
  kable(digits = 2, caption = "Summaries of age and distance to the facility.")
```

R

Table A.1: Summaries of age and distance to the facility.

variable	missing	mean	sd	median	iqr
age	2	52.28	14.10	52.00	19.0
distance_to_facility_km	60	7.82	6.09	6.45	6.3

Age has 2 missing values and distance 60. For **age**, the mean (52.3 years) and median (52 years) almost coincide and the SD (14.1) is small relative to the mean, so the distribution is roughly symmetric: report **mean (SD)**, 52.3 (14.1) years. For **distance**, the mean (7.8 km) is well above the median (6.45 km) and the SD (6.1 km) is close to the mean of a variable that cannot be negative, the signature of right skew (see the histogram in Section 3.5): report **median (IQR)**: 6.45 km (IQR 4.0 to 10.3 km, an interquartile range of 6.3 km).

Exercise 3.2

```
ldl <- analysis_data$ldl_mmol_l

mean(ldl) # the na.rm trap: NA

#> [1] NA
```

R

```
c(n_valid = sum(!is.na(ldl)),
  n_missing = sum(is.na(ldl)),
  pct_miss = 100 * mean(is.na(ldl)),
  mean     = mean(ldl, na.rm = TRUE),
```

R

```
sd      = sd(ldl, na.rm = TRUE),
cv_pct  = 100 * sd(ldl, na.rm = TRUE) / mean(ldl, na.rm = TRUE)) |>
round(2)
```

```
#>   n_valid n_missing pct_miss   mean    sd  cv_pct
#>   1395.00    105.00     7.00    2.70   1.03   38.26
```

Without `na.rm = TRUE`, `mean()` returns NA because 105 LDL values (7.0%) are missing. Among the 1,395 patients with a result, mean LDL cholesterol was 2.70 mmol/L with SD 1.03 mmol/L, a coefficient of variation of about 38%, so LDL is relatively more variable than total cholesterol (CV about 19%). A sentence for a paper: *"LDL cholesterol was available for 1,395 of 1,500 participants (93.0%); mean 2.70 mmol/L (SD 1.03)."*

Exercise 3.3

```
table(analysis_data$smoking, useNA = "ifany")
```

R

```
#>
#>   Never   Former Current   <NA>
#>    899    295    261     45
```

```
table(analysis_data$bp_category, useNA = "ifany")
```

R

```
#>
#>   Normal   Elevated Hypertension   <NA>
#>    144     357     996           3
```

```
analysis_data |>
  tabyl(smoking) |>
  adorn_pct_formatting(digits = 1)
```

R

```
#>   smoking   n percent valid_percent
#>   Never 899   59.9%      61.8%
#>   Former 295   19.7%      20.3%
#>   Current 261   17.4%      17.9%
#>   <NA> 45     3.0%       -
```

```
analysis_data |>
  tabyl(bp_category) |>
  adorn_pct_formatting(digits = 1)
```

R

```
#>   bp_category   n percent valid_percent
#>   Normal 144     9.6%      9.6%
#>   Elevated 357   23.8%     23.8%
#>   Hypertension 996 66.4%     66.5%
#>   <NA> 3       0.2%       -
```

Smoking status is missing for 45 patients and blood-pressure category for 3. There are 261

current smokers: (a) 17.4% of all 1,500 patients (percent), and (b) 17.9% of the 1,455 with known smoking status (`valid_percent`). The difference is small here because only 3% are missing, but the denominator should always be stated.

Exercise 3.4

```
ins_tab <- table(Insurance = diagnosed$health_insurance,
                Treated    = diagnosed$treatment_uptake)
addmargins(ins_tab)
```

```
#>           Treated
#> Insurance   No  Yes Sum
#>       No   419 306 725
#>       Yes   162 202 364
#>       Sum   581 508 1089
```

```
round(100 * prop.table(ins_tab, margin = 1), 1) # row % (within insurance)
```

```
#>           Treated
#> Insurance   No  Yes
#>       No   57.8 42.2
#>       Yes   44.5 55.5
```

```
round(100 * prop.table(ins_tab, margin = 2), 1) # column % (within uptake)
```

```
#>           Treated
#> Insurance   No  Yes
#>       No   72.1 60.2
#>       Yes   27.9 39.8
```

Among the 364 insured diagnosed patients, 55.5% were on treatment, compared with 42.2% of the 725 uninsured. These are the **row percentages**, computed within categories of the explanatory variable (insurance). They answer the question "is insurance associated with uptake?" directly, because they compare the outcome between the exposure groups. The column percentages (39.8% of treated and 27.9% of untreated patients were insured) describe the make-up of the outcome groups, which is what a Table 1 by treatment uptake shows, but they do not state the uptake rate in each insurance group.

Exercise 3.5

```
diagnosed |>
  group_by(treatment_uptake) |>
  summarise(
    n      = sum(!is.na(sbp_mmhg)),
    mean   = mean(sbp_mmhg, na.rm = TRUE),
    sd     = sd(sbp_mmhg, na.rm = TRUE),
    se     = sd / sqrt(n),
    lower  = mean - qt(0.975, n - 1) * se,
    upper  = mean + qt(0.975, n - 1) * se
```

```
) |>
kable(digits = 2, caption = "Mean SBP (mmHg) with 95% CI by uptake.")
```

Table A.2: Mean SBP (mmHg) with 95% CI by uptake.

treatment_uptake	n	mean	sd	se	lower	upper
No	581	142.2	19.81	0.82	140.6	143.8
Yes	507	148.1	18.34	0.81	146.5	149.7

```
diagnosed |>
  group_by(residence) |>
  summarise(n = n(), treated = sum(treatment_uptake == "Yes")) |>
  mutate(pct = 100 * treated / n,
         se = sqrt((pct / 100) * (1 - pct / 100) / n),
         lower = pct - 100 * 1.96 * se,
         upper = pct + 100 * 1.96 * se) |>
  select(-se) |>
  kable(digits = 1, caption = "Treatment uptake (%) with 95% CI by residence.")
```

Table A.3: Treatment uptake (%) with 95% CI by residence.

residence	n	treated	pct	lower	upper
Rural	501	195	38.9	34.7	43.2
Urban	588	313	53.2	49.2	57.3

Untreated diagnosed patients had mean SBP 142.2 mmHg (95% CI about 140.6 to 143.8) and treated patients 148.1 mmHg (about 146.5 to 149.7). Uptake was 38.9% (95% CI about 34.7% to 43.2%) among rural and 53.2% (about 49.2% to 57.3%) among urban diagnosed patients; the intervals do not overlap, suggesting a real difference that Chapter 4 will test formally.

The **SD** (about 18 to 20 mmHg) measures how much individual patients' SBP varies around their group mean; it would stay about the same if we recruited more patients. The **SE** (about 0.8 mmHg) measures how precisely each group *mean* is estimated; it is the SD divided by $\sqrt{n}$ and would shrink with a larger sample. The SD describes patients, the SE describes the uncertainty of an estimate.

Exercise 3.6

```
diagnosed |>
  select(age, sex, residence, education, health_insurance,
         family_history_htn, knowledge_score, comorbidity_count,
         treatment_uptake) |>
  tbl_summary(
    by = treatment_uptake,
    type = comorbidity_count ~ "continuous", # treat the count as numeric
    statistic = list(
      c(age, knowledge_score) ~ "{mean} ({sd})",
      comorbidity_count ~ "{median} ({p25}, {p75})",
      all_categorical() ~ "{n} ({p}%)",
    ),
    digits = list(c(age, knowledge_score) ~ 1, comorbidity_count ~ 0),
    label = list(
```

```

age ~ "Age, years",
sex ~ "Sex",
residence ~ "Residence",
education ~ "Education",
health_insurance ~ "Health insurance",
family_history_htn ~ "Family history of hypertension",
knowledge_score ~ "Knowledge score (0-20)",
comorbidity_count ~ "Number of comorbidities"
),
missing = "ifany",
missing_text = "Missing"
) |>
add_overall(last = FALSE) |>
modify_header(label = "***Characteristic**") |>
as_kable(caption = "Diagnosed patients by treatment uptake.")

```

Table A.4: Diagnosed patients by treatment uptake.

Characteristic	Overall N = 1,089	No N = 581	Yes N = 508
Age, years	53.5 (14.0)	51.1 (13.4)	56.3 (14.1)
Missing	1	1	0
Sex			
Female	629 (58%)	315 (54%)	314 (62%)
Male	460 (42%)	266 (46%)	194 (38%)
Residence			
Rural	501 (46%)	306 (53%)	195 (38%)
Urban	588 (54%)	275 (47%)	313 (62%)
Education			
None	201 (19%)	126 (22%)	75 (15%)
Primary	444 (42%)	252 (44%)	192 (39%)
Secondary	292 (27%)	137 (24%)	155 (31%)
Tertiary	128 (12%)	55 (9.6%)	73 (15%)
Missing	24	11	13
Health insurance	364 (33%)	162 (28%)	202 (40%)
Family history of hypertension	394 (36%)	183 (31%)	211 (42%)
Knowledge score (0-20)	10.4 (3.5)	9.9 (3.4)	10.9 (3.4)
Missing	29	16	13
Number of comorbidities	0 (0, 1)	0 (0, 1)	0 (0, 1)

Without `type = comorbidity_count ~ "continuous"`, `gtsummary` would treat a count with only four distinct values as categorical and show `n (%)` for each value, which is in fact a perfectly good alternative here: the median (IQR) of 0 (0, 1) is identical in both groups and hides any difference.

Two characteristics that differ: treated patients were **older** (mean about 56 versus 51 years) and more often had a **family history of hypertension** (about 42% versus 31%); they were also more often insured, urban and better educated, and had a slightly higher knowledge score. These differences are clinically plausible: older patients and those with a family history are more likely to be recognised as high-risk and started on treatment, and insurance, education and knowledge plausibly lower the financial and informational barriers to care. They are unadjusted, descriptive

differences in simulated data, to be examined with tests (Chapter 4) and multivariable models (Chapter 5).

Exercise 3.7

```
ggplot(diagnosed, aes(x = treatment_uptake, y = knowledge_score)) +
  geom_jitter(width = 0.2, height = 0.2, alpha = 0.25, size = 0.8,
    na.rm = TRUE) +
  geom_boxplot(width = 0.4, outlier.shape = NA, fill = teal, alpha = 0.3,
    na.rm = TRUE) +
  labs(title = "Treated patients score slightly higher on knowledge",
    x = "On antihypertensive treatment", y = "Knowledge score (0-20)")
```

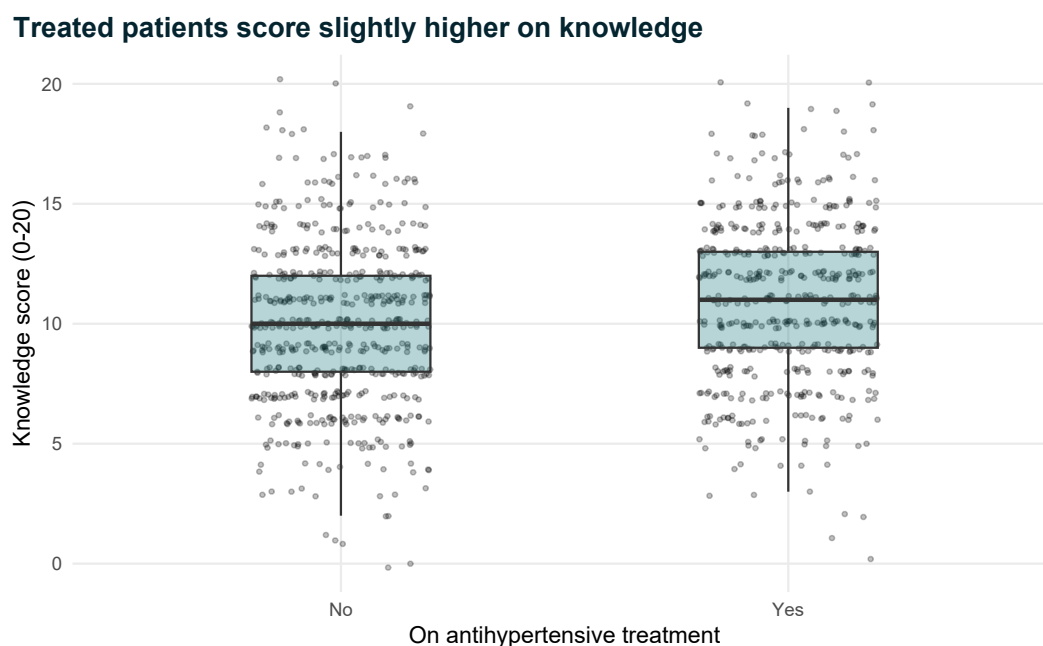


Figure A.3: Hypertension knowledge score (0-20) among diagnosed patients not on treatment and on treatment; boxes show the median and quartiles, points show individual patients.

The knowledge score is an integer, so a small vertical jitter (`height = 0.2`) stops patients with the same score from being drawn on top of each other. The median score is 11 among treated and 10 among untreated patients, but the two distributions overlap almost entirely.

```
diagnosed |>
  filter(!is.na(education)) |>
  group_by(education) |>
  summarise(pct = mean(treatment_uptake == "Yes")) |>
  ggplot(aes(x = education, y = pct)) +
  geom_col(fill = teal, width = 0.65) +
  geom_text(aes(label = percent(pct, accuracy = 0.1)), vjust = -0.4) +
  scale_y_continuous(labels = label_percent(), limits = c(0, 0.7)) +
  labs(title = "Treatment uptake rises with education",
    x = "Highest education level", y = "Diagnosed patients on treatment")
```

Because education is an ordered factor, the bars appear from none to tertiary without extra code. Uptake rises from about 37% among patients with no schooling to about 57% among those with tertiary education, a gradient consistent with education easing access to and use of care. A bar

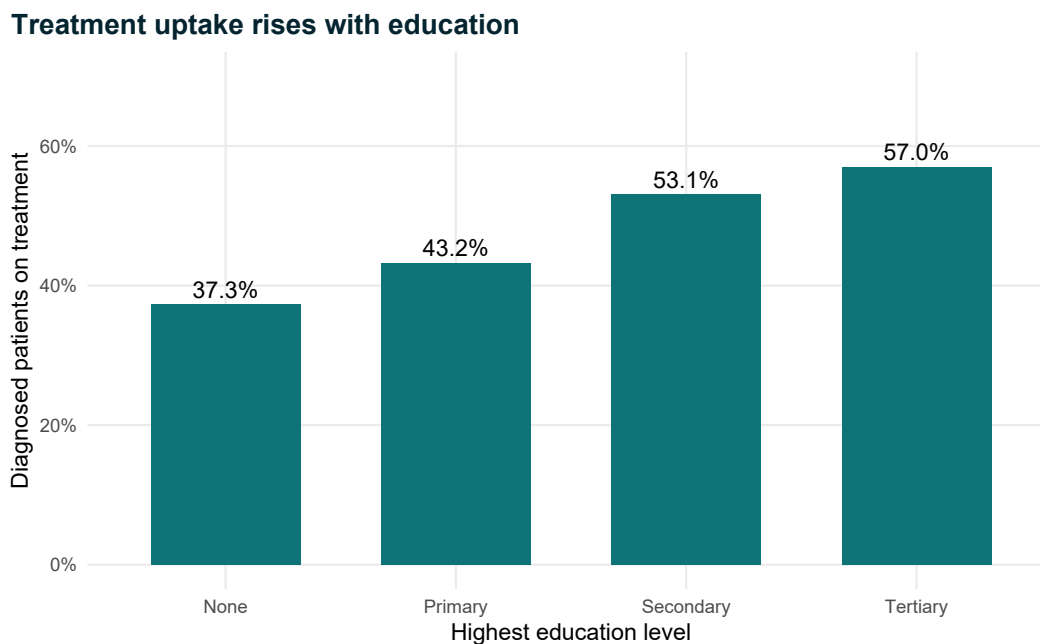


Figure A.4: Percentage of diagnosed hypertensive patients on treatment by highest education level (24 patients with missing education excluded); uptake rises steadily with education.

chart is appropriate here because each bar is a *percentage*, whose length from zero is meaningful.

Exercise 3.8

```

facility_summary <- diagnosed |>
  group_by(facility) |>
  summarise(
    n          = n(),
    treated     = sum(treatment_uptake == "Yes"),
    median_dist = median(distance_to_facility_km, na.rm = TRUE),
    pct_insured = 100 * mean(health_insurance == "Yes")
  ) |>
  rowwise() |>                                # one prop.test() per facility
  mutate(
    ci      = list(prop.test(treated, n, correct = FALSE)$conf.int),
    pct     = 100 * treated / n,
    lower   = 100 * ci[1],
    upper   = 100 * ci[2]
  ) |>
  ungroup() |>
  select(facility, n, pct, lower, upper, median_dist, pct_insured) |>
  arrange(desc(pct))

facility_summary |>
  kable(digits = 1,
        caption = "Uptake (% , Wilson 95% CI) and access by facility.")

```

```

ggplot(facility_summary, aes(x = median_dist, y = pct / 100)) +
  geom_point(size = 3, colour = teal) +
  geom_text(aes(label = facility), vjust = -1, size = 3.2) +

```

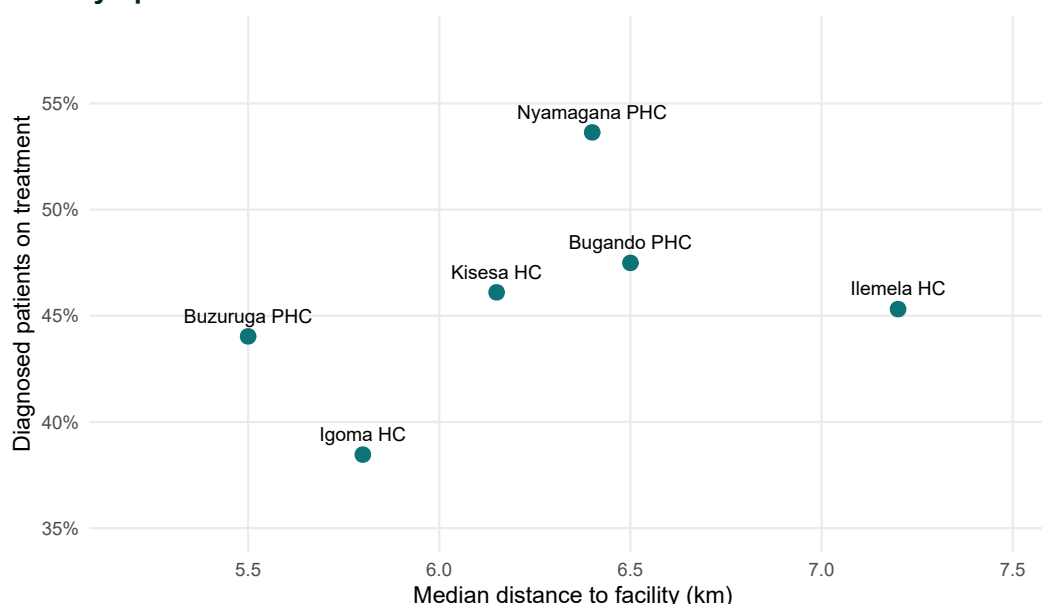
Table A.5: Uptake (%), Wilson 95% CI and access by facility.

facility	n	pct	lower	upper	median_dist	pct_insured
Nyamagana PHC	220	53.6	47.0	60.1	6.4	36.8
Bugando PHC	259	47.5	41.5	53.6	6.5	34.0
Kisesa HC	154	46.1	38.4	54.0	6.2	33.1
Ilemela HC	192	45.3	38.4	52.4	7.2	35.9
Buzuruga PHC	134	44.0	35.9	52.5	5.5	26.1
Igoma HC	130	38.5	30.5	47.0	5.8	30.8

```

scale_y_continuous(labels = label_percent(), limits = c(0.35, 0.58)) +
scale_x_continuous(limits = c(5.2, 7.5)) +
labs(title = "Facility uptake and median distance",
     x = "Median distance to facility (km)",
     y = "Diagnosed patients on treatment")

```

Facility uptake and median distance**Figure A.5:** Treatment uptake among diagnosed patients against median distance to the facility, one point per facility; with only six facilities no clear relationship is visible.

`rowwise()` makes `mutate()` work one facility at a time, so `prop.test()` receives that facility's counts; the Wilson interval is stored as a list column and then split into its lower and upper limits. Uptake ranges from 38.5% (Igoma HC) to 53.6% (Nyamagana PHC), with Wilson intervals similar to the Wald intervals of Section 3.8 because each facility has more than 100 diagnosed patients. In the plot there is no clear pattern: the facilities with the shortest median distance (Buzuruga PHC and Igoma HC) do not have the highest uptake.

Great caution is needed for three reasons. First, there are only **six** data points, so any apparent relationship could easily arise by chance. Second, this is an **ecological** comparison of facility averages: a relationship between facilities need not hold for individual patients (the *ecological fallacy*), and indeed at the patient level Table 1 in Section 3.10 showed that treated patients lived somewhat closer to the facility. Third, facilities differ in many ways at once (insurance coverage, staffing, catchment population), so distance cannot be isolated from other facility characteristics

without a patient-level multivariable analysis that accounts for clustering (Chapter 5).

A.4 Chapter 4: Common Statistical Tests in Medical Research

All solutions start from the clean data and the analytic population of diagnosed patients.

```
library(tidyverse)
library(broom)
library(patchwork)

course_teal <- "#0D7377"

analysis_data <- readRDS("Data/analysis_data.rds")
dx <- filter(analysis_data, htn_diagnosed == "Yes")
```

Exercise 4.1

1. Size of the analytic population and uptake.

```
nrow(dx)
```

```
#> [1] 1089
```

```
dx |> count(treatment_uptake) |> mutate(percent = round(100 * n / sum(n), 1))
```

```
#> # A tibble: 2 × 3
#>   treatment_uptake     n percent
#>   <fct>           <int>   <dbl>
#> 1 No              581    53.4
#> 2 Yes              508    46.6
```

There are 1,089 diagnosed patients, of whom 508 (46.6%) are on treatment.

2. Uptake among all 1,500 patients.

```
analysis_data |>
  count(htn_diagnosed, treatment_uptake) |>
  mutate(percent_of_all = round(100 * n / sum(n), 1))
```

```
#> # A tibble: 3 × 4
#>   htn_diagnosed treatment_uptake     n percent_of_all
#>   <fct>           <fct>           <int>       <dbl>
#> 1 No            No              411        27.4
#> 2 Yes            No              581        38.7
#> 3 Yes            Yes              508        33.9
```

```
round(100 * mean(analysis_data$treatment_uptake == "Yes"), 1)
```

```
#> [1] 33.9
```

Across all 1,500 patients only 33.9% are "on treatment", because the 411 undiagnosed patients are all recorded as not treated. Treatment uptake is defined only for people who know they have hypertension, so including undiagnosed patients mixes two different processes (being diagnosed and being treated once diagnosed) and gives a misleadingly low figure; the correct denominator is the 1,089 diagnosed patients.

3. Standard deviation and standard error of the knowledge score.

```
dx |>
  summarise(n      = sum(!is.na(knowledge_score)),
            mean    = mean(knowledge_score, na.rm = TRUE),
            sd      = sd(knowledge_score, na.rm = TRUE),
            se      = sd / sqrt(n))

#> # A tibble: 1 × 4
#>       n    mean    sd    se
#>   <int> <dbl> <dbl> <dbl>
#> 1  1060  10.4   3.46  0.106
```

The mean knowledge score is 10.4 points with a standard deviation of 3.5 points: individual patients typically differ from the mean by about 3.5 points. The standard error, 0.11 points, describes the precision of the *mean*: if the study were repeated, sample means would typically vary by about 0.1 points. The SD describes patients; the SE describes the estimate and shrinks as $1/\sqrt{n}$.

Exercise 4.2

1. Histogram and Q–Q plot of BMI.

```
p_hist <- ggplot(dx, aes(x = bmi)) +
  geom_histogram(binwidth = 1, fill = course_teal, colour = "white") +
  labs(x = "BMI (kg/m²)", y = "Patients")
p_qq <- ggplot(dx, aes(sample = bmi)) +
  stat_qq(size = 0.6, alpha = 0.5, colour = course_teal) +
  stat_qq_line() +
  labs(x = "Theoretical quantiles", y = "Sample quantiles")
p_hist + p_qq
```

The histogram is symmetric and bell-shaped, and the Q–Q plot is close to a straight line, with very slight deviations only at the extremes.

2. Shapiro–Wilk test.

```
shapiro.test(dx$bmi)

#>
#>      Shapiro-Wilk normality test
#>
#> data:  dx$bmi
#> W = 0.997, p-value = 0.052
```

$W = 0.997$ and $p = 0.052$: no strong evidence against normality. With more than 1,000 patients, however, the test would flag even trivial departures, and the plots (together with the central limit theorem) are the more useful guide. Either way, BMI can be treated as approximately normal.

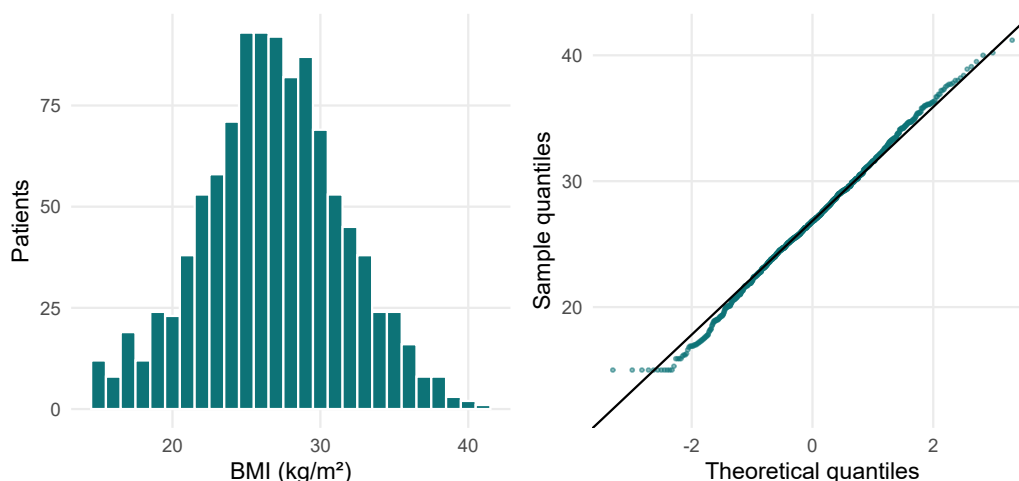


Figure A.6: Histogram and normal Q–Q plot of body mass index among diagnosed patients.

3. Comparing BMI by treatment uptake. A Welch t-test is appropriate.

```
t.test(bmi ~ treatment_uptake, data = dx)
```

R

```
#>
#>      Welch Two Sample t-test
#>
#> data:  bmi by treatment_uptake
#> t = -0.504, df = 1045, p-value = 0.61
#> alternative hypothesis: true difference in means between group No and group
#>   Yes is not equal to 0
#> 95 percent confidence interval:
#>  -0.72587  0.42912
#> sample estimates:
#>  mean in group No mean in group Yes
#>      26.763      26.912
```

```
wilcox.test(bmi ~ treatment_uptake, data = dx) # a check
```

R

```
#>
#>      Wilcoxon rank sum test with continuity correction
#>
#> data:  bmi by treatment_uptake
#> W = 135646, p-value = 0.52
#> alternative hypothesis: true location shift is not equal to 0
```

Mean BMI was 26.8 kg/m² in untreated and 26.9 kg/m² in treated patients; the difference (No minus Yes) is -0.15 kg/m² (95% CI -0.73 to 0.43 ; $p = 0.61$). The Wilcoxon test agrees ($p = 0.52$). The confidence interval is narrow and contains only differences far too small to matter clinically, so in this case we can reasonably conclude that BMI is not materially different between the groups, not merely that “the test was not significant”.

Exercise 4.3

1. Summary by group.

```
dx |>
  group_by(treatment_uptake) |>
  summarise(n = sum(!is.na(sbp_mmhg)),
            mean = mean(sbp_mmhg, na.rm = TRUE),
            sd = sd(sbp_mmhg, na.rm = TRUE))
```

```
#> # A tibble: 2 × 4
#>   treatment_uptake     n  mean    sd
#>   <fct>           <int> <dbl> <dbl>
#> 1 No              581  142.  19.8
#> 2 Yes             507  148.  18.3
```

2. Welch t-test.

```
t.test(sbp_mmhg ~ treatment_uptake, data = dx)
```

```
#>
#>      Welch Two Sample t-test
#>
#> data:  sbp_mmhg by treatment_uptake
#> t = -5.14, df = 1082, p-value = 3.3e-07
#> alternative hypothesis: true difference in means between group No and group
#>   Yes is not equal to 0
#> 95 percent confidence interval:
#>  -8.2128 -3.6727
#> sample estimates:
#> mean in group No mean in group Yes
#>           142.16           148.10
```

Patients on treatment had a higher mean systolic blood pressure than those not on treatment (148.1 vs 142.2 mmHg; difference 5.9 mmHg, 95% CI 3.7 to 8.2 mmHg; Welch t-test $p < 0.001$).

3. Interpretation. A difference of about 6 mmHg in mean systolic pressure is clinically relevant at the population level (reductions of this size are associated with appreciable reductions in cardiovascular risk). The direction is at first surprising, since treatment should *lower* blood pressure. In a cross-sectional study, however, we cannot tell what came first: patients with higher blood pressure are more likely to have been offered and to have accepted treatment (reverse causation, or "confounding by indication"), and treated patients may not yet be controlled. A single cross-sectional comparison cannot evaluate the effect of treatment on blood pressure. (The data are simulated, so the pattern illustrates the reasoning rather than a real finding.)

Exercise 4.4

1. Boxplot and summary.

```
dx |>
  filter(!is.na(bmi_cat), !is.na(sbp_mmhg)) |>
  ggplot(aes(x = bmi_cat, y = sbp_mmhg)) +
  geom_jitter(width = 0.2, alpha = 0.15, size = 0.8, colour = course_teal) +
```

```
geom_boxplot(fill = NA, outlier.shape = NA, width = 0.5) +
labs(x = "BMI category", y = "Systolic BP (mmHg)")
```

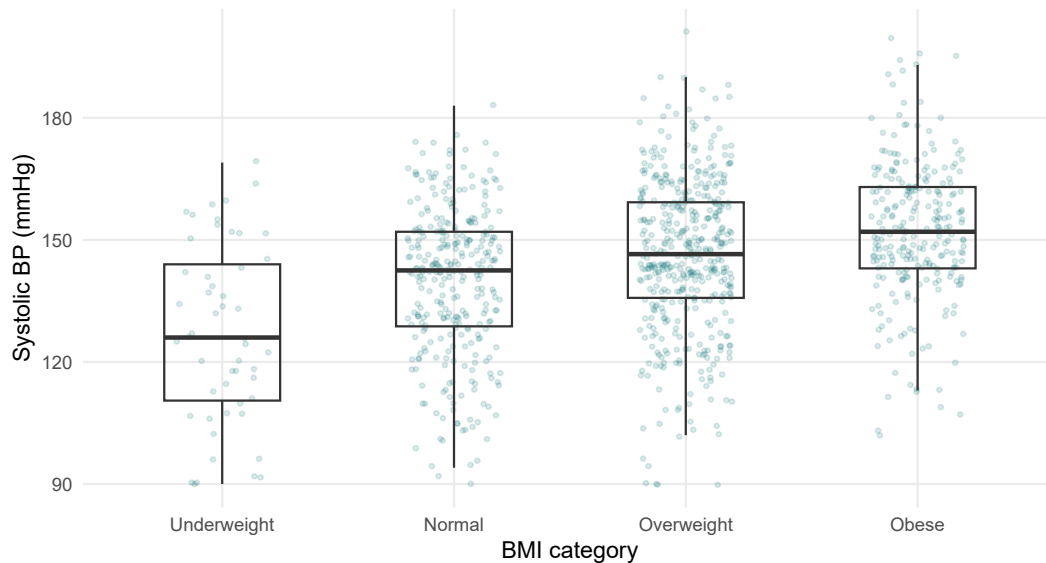


Figure A.7: Systolic blood pressure by BMI category among diagnosed patients.

```
dx |>
  filter(!is.na(bmi_cat), !is.na(sbp_mmhg)) |>
  group_by(bmi_cat) |>
  summarise(n = n(), mean = mean(sbp_mmhg), sd = sd(sbp_mmhg))
```

```
#> # A tibble: 4 × 4
#>   bmi_cat      n mean  sd
#>   <fct>    <int> <dbl> <dbl>
#> 1 Underweight    51  127.  22.1
#> 2 Normal       308  140.  18.2
#> 3 Overweight    440  146.  18.9
#> 4 Obese        256  152.  16.9
```

Mean systolic pressure rises steadily from about 127 mmHg in underweight to 152 mmHg in obese patients.

2. One-way ANOVA.

```
aov_sbp <- aov(sbp_mmhg ~ bmi_cat, data = dx)
summary(aov_sbp)
```

```
#>               Df Sum Sq Mean Sq F value Pr(>F)
#> bmi_cat         3   39096    13032    38.5 <2e-16 ***
#> Residuals     1051  355674      338
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> 34 observations deleted due to missingness
```

The between-groups line has $4 - 1 = 3$ degrees of freedom and the residual line 1,051 ($N - k$ with $N = 1,055$). $F = 38.5$ with $p < 2 \times 10^{-16}$: very strong evidence that mean systolic pressure differs between BMI categories. 34 patients were excluded because BMI category or

systolic pressure was missing.

3. Kruskal–Wallis test.

```
kruskal.test(sbp_mmhg ~ bmi_cat, data = dx)

#>
#>      Kruskal-Wallis rank sum test
#>
#> data:  sbp_mmhg by bmi_cat
#> Kruskal-Wallis chi-squared = 89.6, df = 3, p-value <2e-16
```

The rank-based test agrees ($\chi^2 = 89.6$, 3 df, $p < 0.001$).

4. Bonferroni-adjusted pairwise comparisons.

```
pairwise.t.test(dx$sbp_mmhg, dx$bmi_cat, p.adjust.method = "bonferroni")

#>
#>      Pairwise comparisons using t tests with pooled SD
#>
#> data:  dx$sbp_mmhg and dx$bmi_cat
#>
#>           Underweight Normal Overweight
#> Normal      3e-05      -      -
#> Overweight  2e-11      2e-05      -
#> Obese       <2e-16      1e-14     1e-04
#>
#> P value adjustment method: bonferroni
```

All six adjusted p-values are below 0.001: every BMI category differs from every other, in a consistent gradient (higher BMI, higher systolic pressure). Tukey's method (`TukeyHSD(aov_sbp)`) would give the same conclusion and, in addition, confidence intervals for each pairwise difference.

Exercise 4.5

1. Table and row percentages.

```
tab_res <- table(Residence = dx$residence, Uptake = dx$treatment_uptake)
addmargins(tab_res)
```

```
#>           Uptake
#> Residence   No  Yes  Sum
#>   Rural    306  195  501
#>   Urban    275  313  588
#>   Sum      581  508 1089
```

```
round(100 * prop.table(tab_res, margin = 1), 1)
```

```
#>           Uptake
#> Residence   No  Yes
#>   Rural    61.1 38.9
#>   Urban    46.8 53.2
```

Uptake was 53.2% among urban and 38.9% among rural patients.

2. Chi-square test and expected counts.

```
chisq.test(tab_res)
```

```
#>
#>      Pearson's Chi-squared test with Yates' continuity correction
#>
#> data:  tab_res
#> X-squared = 21.7, df = 1, p-value = 3.2e-06
```

```
chisq.test(tab_res)$expected
```

```
#>      Uptake
#> Residence    No    Yes
#>   Rural 267.29 233.71
#>   Urban 313.71 274.29
```

$\chi^2 = 21.7$ on 1 df, $p < 0.001$. All expected counts are above 230, so the chi-square approximation is entirely adequate.

3. Risk difference, relative risk and odds ratio (urban vs rural).

```
a <- tab_res["Urban", "Yes"]; b <- tab_res["Urban", "No"] # exposed
cc <- tab_res["Rural", "Yes"]; dd <- tab_res["Rural", "No"] # unexposed
n1 <- a + b; n0 <- cc + dd
p1 <- a / n1; p0 <- cc / n0

rd <- p1 - p0
se_rd <- sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
rr <- p1 / p0
se_log_rr <- sqrt(1 / a + 1 / n1 + 1 / cc + 1 / n0)
or <- (a * dd) / (b * cc)
se_log_or <- sqrt(1 / a + 1 / b + 1 / cc + 1 / dd)

tibble(
  measure = c("Risk difference", "Relative risk", "Odds ratio"),
  estimate = c(rd, rr, or),
  lower = c(rd - 1.96 * se_rd, exp(log(rr) - 1.96 * se_log_rr),
            exp(log(or) - 1.96 * se_log_or)),
  upper = c(rd + 1.96 * se_rd, exp(log(rr) + 1.96 * se_log_rr),
            exp(log(or) + 1.96 * se_log_or))
) |>
knitr::kable(digits = 3,
  caption = "Urban versus rural residence and treatment uptake.")
```

Table A.6: Urban versus rural residence and treatment uptake.

measure	estimate	lower	upper
Risk difference	0.143	0.084	0.202
Relative risk	1.368	1.197	1.563
Odds ratio	1.786	1.402	2.275

Urban patients were 14.3 percentage points more likely to be on treatment than rural patients (95% CI 8.4 to 20.2); their risk of uptake was about 1.37 times as high (95% CI 1.20 to 1.56), and their odds about 1.79 times as high (95% CI 1.40 to 2.28). All three confidence intervals exclude the null value.

4. Why the OR is further from 1 than the RR. The odds ratio approximates the relative risk only when the outcome is rare. Uptake is common (around 40–55%), and when risks are high the odds $p/(1 - p)$ change much faster than the risks themselves, so the OR (1.79) is noticeably further from 1 than the RR (1.37). Reporting the OR as if it were "1.8 times as likely" would exaggerate the association.

Exercise 4.6

1. Cross-product OR and logistic regression.

```
tab_diab <- table(Diabetes = dx$diabetes, Uptake = dx$treatment_uptake)
tab_diab
```

```
#>      Uptake
#> Diabetes No Yes
#>      No  574 487
#>      Yes   7  21
```

```
or_table <- (tab_diab["Yes", "Yes"] * tab_diab["No", "No"]) /
  (tab_diab["Yes", "No"] * tab_diab["No", "Yes"])
or_table
```

```
#> [1] 3.5359
```

```
m_diab <- glm(treatment_uptake ~ diabetes, data = dx, family = binomial)
tidy(m_diab, exponentiate = TRUE, conf.int = TRUE) |>
  select(term, estimate, p.value, conf.low, conf.high)
```

```
#> # A tibble: 2 × 5
#>   term          estimate p.value conf.low conf.high
#>   <chr>          <dbl>   <dbl>   <dbl>   <dbl>
#> 1 (Intercept)    0.848 0.00763    0.752    0.957
#> 2 diabetesYes    3.54 0.00416    1.56    9.04
```

The two agree exactly: $(21 \times 574)/(7 \times 487) = 3.54$, and the exponentiated coefficient of `diabetesYes` is also 3.54. Logistic regression with a single binary predictor reproduces the 2×2 table.

2. Fisher's exact test.

```
fisher.test(tab_diab)
```

```
#>
#>      Fisher's Exact Test for Count Data
#>
#> data:  tab_diab
```

```
#> p-value = 0.0033
#> alternative hypothesis: true odds ratio is not equal to 1
#> 95 percent confidence interval:
#>  1.4313 9.9226
#> sample estimates:
#> odds ratio
#>      3.5321
```

Fisher's test reports an odds ratio of 3.53. It is a *conditional maximum likelihood* estimate (computed conditional on the table margins), not the simple cross-product ratio, so it differs slightly, particularly when some cells are small. Its exact interval (1.43 to 9.92) is also a little wider than the profile-likelihood interval from the regression (1.56 to 9.04). The p-values (0.003 and 0.004) lead to the same conclusion.

3. Why the interval is wide. The standard error of the log odds ratio is $\sqrt{1/a + 1/b + 1/c + 1/d}$, which is dominated by the smallest cells. Only 28 patients have diabetes, and only 7 of them are untreated, so $1/7$ makes the standard error large (about 0.44, compared with 0.13 for health insurance, whose smallest cell is 162). The data are compatible with anything from a modest to a very large association.

Exercise 4.7

1. Scatter plot.

```
ggplot(dx, aes(x = age, y = knowledge_score)) +
  geom_jitter(width = 0, height = 0.2, alpha = 0.3, size = 1,
             colour = course_teal) +
  geom_smooth(method = "lm", formula = y ~ x, colour = "black") +
  labs(x = "Age (years)", y = "Knowledge score (0-20)")
```

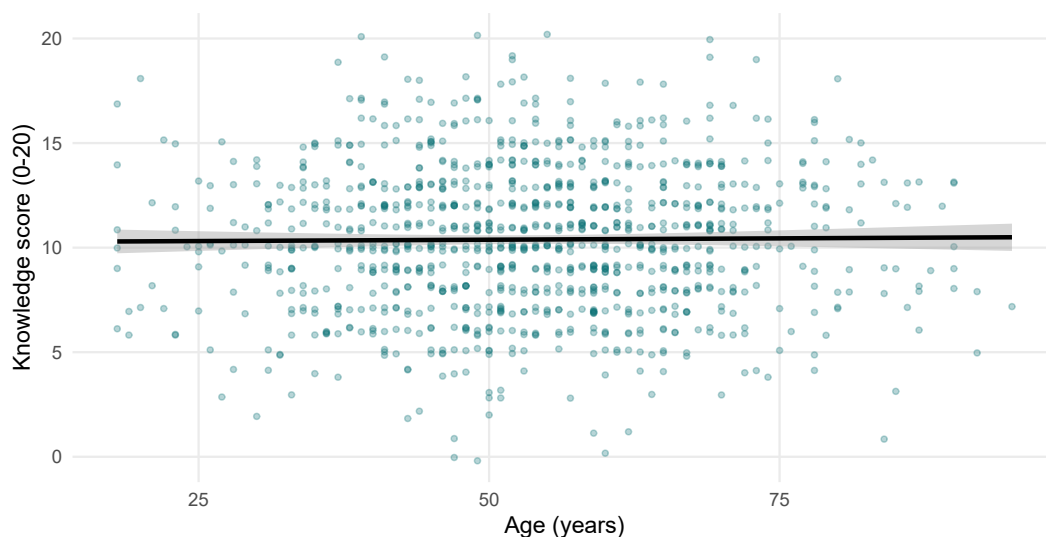


Figure A.8: Hypertension knowledge score against age among diagnosed patients, with a least-squares line.

The cloud is shapeless and the fitted line is flat.

2. Correlation with age.

```
cor.test(dx$age, dx$knowledge_score)
```

R

```
#>
#>      Pearson's product-moment correlation
#>
#> data:  dx$age and dx$knowledge_score
#> t = 0.343, df = 1057, p-value = 0.73
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  -0.049719  0.070749
#> sample estimates:
#>      cor
#> 0.010553
```

```
cor.test(dx$age, dx$knowledge_score, method = "spearman", exact = FALSE)
```

R

```
#>
#>      Spearman's rank correlation rho
#>
#> data:  dx$age and dx$knowledge_score
#> S = 1.95e+08, p-value = 0.66
#> alternative hypothesis: true rho is not equal to 0
#> sample estimates:
#>      rho
#> 0.013459
```

Pearson's $r = 0.011$ (95% CI -0.050 to 0.071 ; $p = 0.73$) and Spearman's $r_s = 0.013$ ($p = 0.66$). There is no evidence of a linear or monotonic association, and the confidence interval shows that any linear correlation is at most very weak (the largest plausible value, 0.07 , would correspond to age explaining less than 1% of the variation).

3. Correlation with distance.

```
cor.test(dx$distance_to_facility_km, dx$knowledge_score)
```

R

```
#>
#>      Pearson's product-moment correlation
#>
#> data:  dx$distance_to_facility_km and dx$knowledge_score
#> t = -0.318, df = 1014, p-value = 0.75
#> alternative hypothesis: true correlation is not equal to 0
#> 95 percent confidence interval:
#>  -0.071439  0.051555
#> sample estimates:
#>      cor
#> -0.0099799
```

```
cor.test(dx$distance_to_facility_km, dx$knowledge_score,
         method = "spearman", exact = FALSE)
```

R

```
#>
#>      Spearman's rank correlation rho
#>
```

```
#> data: dx$distance_to_facility_km and dx$knowledge_score
#> S = 1.78e+08, p-value = 0.52
#> alternative hypothesis: true rho is not equal to 0
#> sample estimates:
#>      rho
#> -0.020245
```

Both coefficients are close to zero ($r = -0.010$, $r_s = -0.020$). Spearman's coefficient is more appropriate here, because distance is strongly right-skewed and a few very large distances could unduly influence Pearson's r ; the rank-based coefficient is not affected by them.

4. Comment. The statement makes two errors. First, a non-significant p-value is not evidence of no relationship (absence of evidence is not evidence of absence); here, however, the narrow confidence interval does support the conclusion that any *linear* association is very weak. Second, and more importantly, the absence of a correlation between age and knowledge says nothing about whether older (or younger) patients would *benefit* from targeted education: that depends on their needs and on uptake, which is related to age (Section 4.4). Correlation also only captures linear (Pearson) or monotonic (Spearman) patterns; a U-shaped relationship could be missed, which is why we plot first.

Exercise 4.8

1. Eight chi-square tests.

```
vars <- c("sex", "residence", "education", "occupation",
          "marital_status", "smoking", "alcohol", "facility")

screen <- tibble(
  variable = vars,
  p_raw = map_dbl(vars, function(v) {
    chisq.test(table(dx[[v]], dx$treatment_uptake))$p.value
  })
)
```

2. Bonferroni and Holm adjustment.

```
screen |>
  mutate(p_bonferroni = p.adjust(p_raw, method = "bonferroni"),
         p_holm       = p.adjust(p_raw, method = "holm")) |>
  arrange(p_raw) |>
  # show very small p-values as "<0.001" rather than rounding them to 0
  mutate(across(starts_with("p_"),
                 \(x) scales::pvalue(x, accuracy = 0.001))) |>
  knitr::kable(
    caption = "Unadjusted and adjusted p-values for eight chi-square
              tests of association with treatment uptake.")
```

Before adjustment, three variables have $p < 0.05$: residence, education and sex. After Bonferroni or Holm adjustment only residence and education remain below 0.05; the association with sex ($p = 0.014$ unadjusted) does not survive. Holm's method is a step-down refinement of Bonferroni that is uniformly more powerful while still controlling the family-wise error rate: it compares the smallest p-value with α/m , the next with $\alpha/(m - 1)$, and so on.

3. Chance of at least one false positive.

Table A.7: Unadjusted and adjusted p-values for eight chi-square tests of association with treatment uptake.

variable	p_raw	p_bonferroni	p_holm
residence	<0.001	<0.001	<0.001
education	<0.001	0.002	0.002
sex	0.014	0.108	0.081
alcohol	0.123	0.984	0.615
facility	0.135	>0.999	0.615
smoking	0.230	>0.999	0.691
occupation	0.353	>0.999	0.706
marital_status	0.885	>0.999	0.885

```
1 - 0.95^8
```

R

```
#> [1] 0.33658
```

If all eight null hypotheses were true and the tests independent, the probability of at least one p-value below 0.05 would be about 34%. A paper that screens many variables and reports only the "significant" ones is therefore likely to report some false-positive associations; it should report all tests performed, describe the analysis as exploratory, and either adjust for multiplicity or interpret isolated small p-values with caution.

A.5 Chapter 5: Regression Modelling and Reporting Results

These solutions use the same packages, data and main model as Chapter 5. The setup code below recreates the analytic population (`dx`), the pre-specified multivariable model (`model_full`) and its complete-case data (`cc`). Remember that the data are simulated for teaching: the results illustrate methods, not real clinical findings.

```
analysis_data <- readRDS("Data/analysis_data.rds")

# Diagnosed patients; education as an unordered factor (reference "None")
dx <- analysis_data |>
  filter(htn_diagnosed == "Yes") |>
  mutate(education = factor(education, ordered = FALSE))

model_full <- glm(
  treatment_uptake ~ age + sex + education + residence + diabetes +
    family_history_htn + health_insurance + knowledge_score +
    distance_to_facility_km,
  data = dx, family = binomial
)
cc <- model.frame(model_full) # the 992 complete cases
```

R

Exercise 5.1

We fit the linear model to all adults with complete data on the four variables and extract the coefficients with confidence intervals.

```
lm_sex <- lm(sbp_mmhg ~ age + bmi + sex, data = analysis_data)
summary(lm_sex)$coefficients |> round(3)
```

```
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    87.309      3.214   27.168   0.000
#> age             0.260      0.034    7.714   0.000
#> bmi            1.451      0.098   14.870   0.000
#> sexMale         0.325      0.966    0.336   0.737
```

```
# Age per 10 years, with 95% CI
age10 <- 10 * c(coef(lm_sex)["age"], confint(lm_sex)["age", ]
round(age10, 2)
```

```
#>   age  2.5 % 97.5 %
#>  2.60  1.94  3.26
```

```
c(n = nobs(lm_sex), R2 = round(summary(lm_sex)$r.squared, 3),
  adj_R2 = round(summary(lm_sex)$adj.r.squared, 3))
```

```
#>      n      R2   adj_R2
#> 1449.000  0.161  0.159
```

1. Each 10-year increase in age is associated with a mean SBP 2.6 mmHg higher (95% CI 1.9 to 3.3), holding BMI and sex fixed.
2. Men have a mean SBP 0.3 mmHg higher than women of the same age and BMI (95% CI -1.6 to 2.2). The interval includes zero, so the data are compatible with no difference in mean SBP by sex.
3. The model explains 16.1% of the variance in SBP, essentially the same as the model without sex in the chapter. Adding sex contributes little.
4. The residual plot is below.

```
augment(lm_sex) |>
  ggplot(aes(.fitted, .resid)) +
  geom_point(alpha = 0.25, colour = "#0D7377") +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_smooth(method = "loess", se = FALSE, colour = "#E07A1F") +
  labs(x = "Fitted SBP (mmHg)", y = "Residual (mmHg)")
```

The residuals form a band of roughly constant width around zero, with no clear curvature or funnel shape, so the linearity and equal-variance assumptions look reasonable. (A Q–Q plot, as in the chapter, would be used to check normality.)

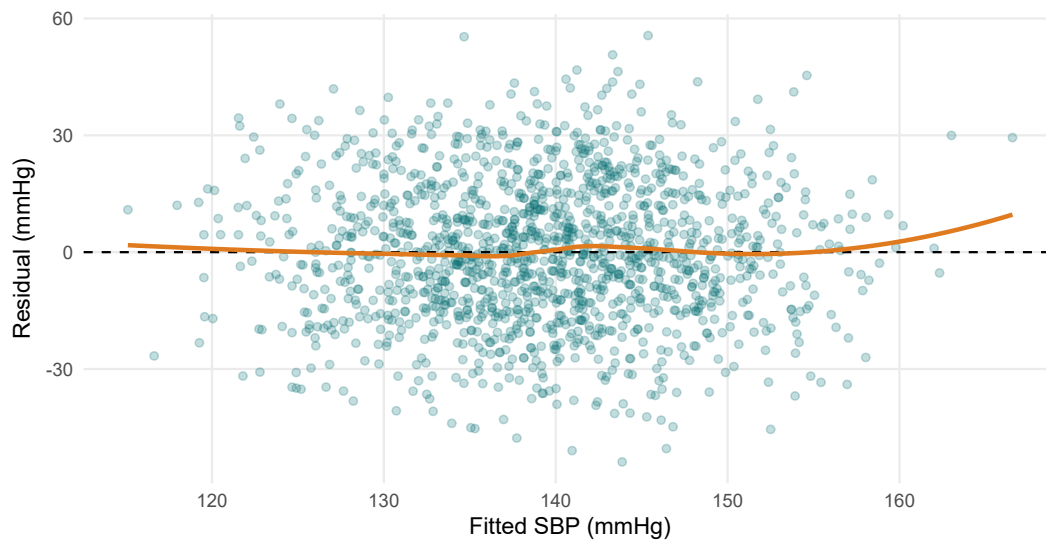


Figure A.9: Residuals against fitted values for the linear regression of SBP on age, BMI and sex.

Exercise 5.2

```
tab_fh <- table(FamilyHistory = dx$family_history_htn,
                Uptake         = dx$treatment_uptake)
tab_fh
```

```
#>           Uptake
#> FamilyHistory No Yes
#>           No  398 297
#>           Yes  183 211
```

```
fh <- dx |>
  group_by(family_history_htn) |>
  summarise(n = n(), p = mean(treatment_uptake == "Yes")) |>
  mutate(odds = p / (1 - p))
kable(fh, digits = 3, caption = "Uptake by family history of hypertension.")
```

Table A.8: Uptake by family history of hypertension.

family_history_htn	n	p	odds
No	695	0.427	0.746
Yes	394	0.536	1.153

```
or_fh <- fh$odds[2] / fh$odds[1]      # crude odds ratio
rr_fh <- fh$p[2] / fh$p[1]           # crude risk ratio
c(OR = round(or_fh, 3), RR = round(rr_fh, 3))
```

```
#>   OR   RR
#> 1.545 1.253
```

```
glm_fh <- glm(treatment_uptake ~ family_history_htn, data = dx,
              family = binomial)
exp(coef(glm_fh))
```

```
#>           (Intercept) family_history_htnYes
#>           0.7462           1.5451
```

```
# Zhang and Yu conversion from OR to RR, using the risk in the reference group
p0 <- fh$p[1]
round(or_fh / ((1 - p0) + p0 * or_fh), 3)
```

```
#> [1] 1.253
```

1. Without a family history, 42.7% of patients are on treatment (odds 0.75); with a family history, 53.6% (odds 1.15). The crude OR is 1.55.
2. The exponentiated coefficient from `glm()` is identical: a one-predictor logistic regression reproduces the 2×2 table.
3. The crude RR is 1.25, smaller than the OR. Uptake is common (around 40–55%), so the odds are much larger than the risks and the OR lies further from 1 than the RR. The Zhang and Yu formula, with $p_0 = 0.427$, recovers the RR exactly for a crude OR.

Exercise 5.3

We multiply each coefficient and its profile-likelihood confidence limits by the chosen increment before exponentiating.

```
ci_log <- confint(model_full)
tibble(term = c("age", "knowledge_score", "distance_to_facility_km"),
       increment = c(10, 5, 10)) |>
  mutate(OR = exp(increment * coef(model_full)[term]),
         lower = exp(increment * ci_log[term, 1]),
         upper = exp(increment * ci_log[term, 2])) |>
  kable(digits = 2, caption = "Adjusted ORs for chosen increments.")
```

Table A.9: Adjusted ORs for chosen increments.

term	increment	OR	lower	upper
age	10	1.35	1.23	1.50
knowledge_score	5	1.60	1.31	1.96
distance_to_facility_km	10	0.83	0.65	1.05

A decade of age is a natural clinical unit, and a 5-point difference in knowledge (a quarter of the 0–20 scale, close to the interquartile range) is easy to interpret. For distance, 10 km is roughly the difference between living next to the clinic and living at the upper quartile of distance in this sample. The increments must be chosen before the analysis (and stated in the Methods) because otherwise an analyst could pick whichever unit made an association look most impressive; the choice changes the size of the OR but not its p-value.

Exercise 5.4

```

glm_ins_cc <- glm(treatment_uptake ~ health_insurance, data = cc,
                  family = binomial)
b_crude <- coef(glm_ins_cc)["health_insuranceYes"]
b_adj <- coef(model_full)["health_insuranceYes"]

c(crude_OR = exp(b_crude), adjusted_OR = exp(b_adj),
  pct_change_logOR = 100 * (b_adj - b_crude) / b_crude) |>
  unname() |> setNames(c("crude_OR", "adjusted_OR", "pct_change_logOR")) |>
  round(2)

```

```

#>      crude_OR      adjusted_OR pct_change_logOR
#>          1.67          2.05          40.24

```

1. The OR moves from 1.67 (crude) to 2.05 (adjusted), a change of about 40% in the log OR, well above the 10% change-in-estimate threshold.
2. A plausible causal diagram: insurance increases uptake (by reducing the cost of medicines and visits). Education, residence and age could influence both whether a person is insured and whether they take up treatment, so they are potential confounders. Knowledge score could be a mediator if insurance schemes provide health education, or a confounder if better-informed people seek insurance; the direction is uncertain. Diabetes and family history are unlikely to be caused by insurance and mainly affect the outcome.
3. To decide between confounding and non-collapsibility we can check how strongly insurance is related to the other predictors, and compute a *marginal* (population-averaged) OR from the adjusted model by standardisation: predict every patient's probability of uptake with insurance set to "Yes" and to "No", average each, and form the OR of the two averages. Standardisation removes confounding by the model covariates but, unlike the conditional OR, is collapsible.

```

# Is insurance associated with the other predictors? (ORs near 1 = weakly)
glm(health_insurance ~ age + sex + education + residence + diabetes +
    family_history_htn + knowledge_score + distance_to_facility_km,
    data = cc, family = binomial) |>
  tidy(exponentiate = TRUE) |>
  filter(term != "(Intercept)") |>
  select(term, OR = estimate, p = p.value) |>
  mutate(OR = round(OR, 2), p = round(p, 3))

```

```

#> # A tibble: 10 × 3
#>   term                OR      p
#>   <chr>             <dbl> <dbl>
#> 1 age              0.99 0.237
#> 2 sexMale          1.01 0.939
#> 3 educationPrimary 1.2  0.341
#> 4 educationSecondary 1.21 0.362
#> 5 educationTertiary 1.09 0.736
#> 6 residenceUrban    0.8  0.128
#> 7 diabetesYes      0.4  0.101
#> 8 family_history_htnYes 0.82 0.178
#> 9 knowledge_score   0.97 0.161
#> 10 distance_to_facility_km 0.99 0.243

```

```
# Marginal OR by standardisation over the observed covariates
all_yes <- mutate(cc, health_insurance = "Yes") # everyone insured
all_no  <- mutate(cc, health_insurance = "No")  # no one insured
p_yes  <- mean(predict(model_full, newdata = all_yes, type = "response"))
p_no   <- mean(predict(model_full, newdata = all_no, type = "response"))
c(p_insured = p_yes, p_uninsured = p_no,
  marginal_OR = (p_yes / (1 - p_yes)) / (p_no / (1 - p_no))) |> round(3)

#>   p_insured p_uninsured marginal_OR
#>      0.564      0.409      1.869
```

Insurance is only weakly related to most of the other predictors, but insured patients are somewhat less likely to be urban, to have diabetes or to have a family history, all of which favour uptake. This produces some negative confounding of the crude OR. The standardised (marginal) OR, 1.87, removes that confounding while staying on the population-averaged scale of the crude OR. We can therefore split the change on the log-odds scale into two parts:

- crude to marginal, from 1.67 to 1.87: the change due to **confounding** by the measured covariates;
- marginal to conditional, from 1.87 to 2.05: the change due to **non-collapsibility**.

```
logs <- c(crude = b_crude, marginal = log(marg_or), conditional = b_adj)
c(confounding_part = unname(logs[2] - logs[1]),
  noncollapsibility_part = unname(logs[3] - logs[2])) |> round(3)

#>      confounding_part noncollapsibility_part
#>           0.114           0.092
```

The two components are of similar size. So the answer is "both": about half of the change from the crude to the adjusted OR reflects confounding (insured patients have fewer of the other characteristics that favour uptake), and about half reflects non-collapsibility. Both ORs are valid, but they answer different questions: the conditional OR compares patients with the same covariate values; the marginal OR compares the whole population with and without insurance.

Exercise 5.5

```
drop1(model_full, test = "LRT")["education", ]

#> Single term deletions
#>
#> Model:
#> treatment_uptake ~ age + sex + education + residence + diabetes +
#>   family_history_htn + health_insurance + knowledge_score +
#>   distance_to_facility_km
#>           Df Deviance   AIC   LRT Pr(>Chi)
#> education   3      1238 1256 16.3    0.001 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

1. The LR test compares `model_full` with the same model without the three education indicators. The null hypothesis is that all three education coefficients are zero, that is, that the odds of uptake are the same at every level of education once the other variables are accounted for. It is rejected ($p \approx 0.001$).

```
cc_num <- cc |> mutate(edu_num = as.integer(education) - 1) # 0, 1, 2, 3

model_trend <- glm(
  treatment_uptake ~ age + sex + edu_num + residence + diabetes +
    family_history_htn + health_insurance + knowledge_score +
    distance_to_facility_km,
  data = cc_num, family = binomial
)
tidy(model_trend, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term == "edu_num") |>
  select(term, OR = estimate, conf.low, conf.high, p.value)
```

```
#> # A tibble: 1 × 5
#>   term      OR conf.low conf.high  p.value
#>   <chr>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1 edu_num 1.36     1.17     1.58 0.0000673
```

```
AIC(model_full, model_trend)
```

```
#>           df  AIC
#> model_full 12 1246
#> model_trend 10 1242
```

1. With education coded 0–3, each step up the education scale (None to Primary, Primary to Secondary, Secondary to Tertiary) is associated with 1.36 times the odds of uptake. This assumes the same OR for every step, so the OR for Tertiary versus None is $1.36^3 = 2.49$.
2. The trend model has AIC 1242.0 against 1245.9 for the factor model: it fits as well with two fewer parameters, and is therefore preferred by AIC (a difference of about 4 points). The category ORs in the chapter (1.39, 1.92, 2.44) do increase roughly geometrically. Either choice is defensible; what matters is that it is made *before* looking at the results. The factor version is easier for readers to interpret and makes no assumption about equal steps; the trend version is more parsimonious and gives a single, more precise estimate. A sensible approach is to report the categories in the table and mention the trend test in the text.

Exercise 5.6

```
model_fac <- update(model_full, . ~ . + facility)
anova(model_full, model_fac, test = "LRT")[, 3:5]
```

```
#>   Df Deviance Pr(>Chi)
#> 1
#> 2 5         7.6      0.18
```

```
# Percentage change in each shared OR when facility is added
shared <- names(coef(model_full))[-1]
tibble(term = shared,
        OR_without = exp(coef(model_full)[shared]),
        OR_with = exp(coef(model_fac)[shared])) |>
mutate(pct_change = 100 * (OR_with - OR_without) / OR_without) |>
kable(digits = 2, caption = "Adjusted ORs with and without facility.")
```

Table A.10: Adjusted ORs with and without facility.

term	OR_without	OR_with	pct_change
age	1.03	1.03	0.03
sexMale	0.74	0.74	0.33
educationPrimary	1.39	1.40	0.68
educationSecondary	1.92	1.96	2.37
educationTertiary	2.44	2.48	1.65
residenceUrban	1.87	1.87	-0.05
diabetesYes	3.56	3.74	5.21
family_history_htnYes	1.91	1.92	0.75
health_insuranceYes	2.05	2.04	-0.57
knowledge_score	1.10	1.10	0.20
distance_to_facility_km	0.98	0.98	-0.16

```
auc_without <- auc(roc(model_full$y, fitted(model_full), quiet = TRUE))
auc_with <- auc(roc(model_fac$y, fitted(model_fac), quiet = TRUE))
round(c(without = auc_without, with = auc_with), 3)
```

```
#> without    with
#>   0.715    0.721
```

1. Adding the five facility indicators reduces the deviance by 7.6 on 5 df ($p = 0.18$): no clear evidence that uptake differs between facilities after adjustment.
2. The largest change in any other OR is 5.2% (for diabetesYes); no OR changes by more than 10%. Facility does not confound the other associations.
3. The AUC rises only slightly, from 0.715 to 0.721, as expected whenever terms are added to a model evaluated on its own data. Facility adds little. However, it remains a design variable: patients within the same facility may be correlated, and a sensitivity analysis with facility as a fixed effect (or with cluster-robust standard errors) is worth reporting to show that the conclusions do not depend on ignoring clustering.

Exercise 5.7

```
model_ir <- update(model_full, . ~ . + health_insurance:residence)
anova(model_full, model_ir, test = "LRT")[, 3:5]
```

```
#>   Df Deviance Pr(>Chi)
#> 1
```

```
#> 2 1 0.303 0.58
```

To obtain the OR for insurance in each residence group, note that in the interaction model the log OR for insurance is β_{ins} among rural patients (the reference) and $\beta_{\text{ins}} + \beta_{\text{int}}$ among urban patients. The variance of the sum is $\text{Var}(\beta_{\text{ins}}) + \text{Var}(\beta_{\text{int}}) + 2 \text{Cov}(\beta_{\text{ins}}, \beta_{\text{int}})$, taken from the variance–covariance matrix `vcov()`.

```
b <- coef(model_ir)
V <- vcov(model_ir)
ins <- "health_insuranceYes"
int <- "residenceUrban:health_insuranceYes"

log_or <- c(rural = unname(b[ins]),
            urban = unname(b[ins] + b[int]))
se <- c(rural = sqrt(V[ins, ins]),
        urban = sqrt(V[ins, ins] + V[int, int] + 2 * V[ins, int]))

tibble(residence = names(log_or),
       OR = exp(log_or),
       lower = exp(log_or - 1.96 * se),
       upper = exp(log_or + 1.96 * se)) |>
  kable(digits = 2,
        caption = "OR for health insurance by residence (Wald CIs).")
```

Table A.11: OR for health insurance by residence (Wald CIs).

residence	OR	lower	upper
rural	1.88	1.24	2.86
urban	2.21	1.48	3.30

1. A possible write-up: "The association between health insurance and treatment uptake did not differ materially by residence (likelihood-ratio test for interaction $p = 0.58$); the adjusted OR for insurance was 1.88 (95% CI 1.24 to 2.86) among rural and 2.21 (1.48 to 3.30) among urban patients. We therefore report the model without the interaction term." Note that the two stratum-specific intervals overlap widely; the economist's hypothesis is not supported by these data, although the study was not powered to detect a modest interaction.

Exercise 5.8

```
treated <- dx |> filter(treatment_uptake == "Yes")
table(treated$bp_controlled, useNA = "ifany")
```

```
#>
#> No Yes
#> 470 38
```

```
table(Diabetes = treated$diabetes, Controlled = treated$bp_controlled)
```

```
#>           Controlled
#> Diabetes   No Yes
#>           No  449  38
#>           Yes   21   0
```

1. Only 38 of the 508 treated patients have controlled blood pressure. Here the *event* is the less frequent category, controlled BP, so the effective sample size is 38, not 508.
2. With 10 events per parameter, the model can support about 3 parameters. Even this is optimistic, and the estimates will be imprecise.
3. We pre-specify three parameters: age, BMI and adherence (good versus poor), all of which are clinically plausible determinants of BP control. Sex is omitted to respect the EPV limit. Diabetes cannot be used: none of the treated patients with diabetes has controlled BP, so the model would suffer from separation.

```
model_bp <- glm(bp_controlled ~ age + bmi + adherence,
  data = treated, family = binomial)
nobs(model_bp)
```

```
#> [1] 494
```

```
tidy(model_bp, exponentiate = TRUE, conf.int = TRUE) |>
  filter(term != "(Intercept)") |>
  select(term, OR = estimate, conf.low, conf.high, p.value) |>
  mutate(across(OR:conf.high, \(x) round(x, 2)), p.value = round(p.value, 3))
```

```
#> # A tibble: 3 × 5
#>   term          OR conf.low conf.high p.value
#>   <chr>        <dbl>   <dbl>   <dbl>   <dbl>
#> 1 age          0.97     0.95     0.99    0.013
#> 2 bmi          0.88     0.81     0.95    0.001
#> 3 adherenceGood 0.36     0.16     0.76    0.011
```

```
auc_bp <- auc(roc(model_bp$y, fitted(model_bp), quiet = TRUE))
round(auc_bp, 3)
```

```
#> [1] 0.739
```

1. A possible Results paragraph: "Among the 508 patients on antihypertensive treatment, 38 (7.5%) had controlled blood pressure. Because of the small number of patients with controlled blood pressure, the model was restricted to three pre-specified predictors. Higher BMI was associated with lower odds of control (OR per 5 kg/m² 0.52, 95% CI 0.36 to 0.76), as was older age (OR per 10 years 0.73, 95% CI 0.57 to 0.93). Good adherence was associated with lower odds of control than poor adherence (OR 0.36, 95% CI 0.16 to 0.76), an unexpected finding. The model's discrimination was acceptable (AUC 0.74). With only 38 events, estimates are imprecise and should be interpreted cautiously."

The adherence result runs against clinical expectation. In a real study it would prompt a check of how adherence and BP control were measured (for example, poorly controlled patients may be counselled more and then report better adherence: reverse causation in a cross-sectional design). Here it is simply a feature of the simulated data, and a reminder that implausible results deserve scrutiny rather than a creative explanation. Note also that the profile-likelihood CIs for rescaled ORs were obtained by multiplying the log-scale limits by the increment, as in the chapter.

References

- Agresti, A. (2013). *Categorical Data Analysis*. 3rd ed. Hoboken, NJ: Wiley.
- Altman, D. G. (1991). *Practical Statistics for Medical Research*. London: Chapman and Hall.
- Altman, D. G. and J. M. Bland (1994). "Quartiles, Quintiles, Centiles, and Other Quantiles". In: *BMJ* 309.6960, p. 996. doi: [10.1136/bmj.309.6960.996](https://doi.org/10.1136/bmj.309.6960.996).
- Altman, D. G. and J. M. Bland (1995a). "Absence of Evidence Is Not Evidence of Absence". In: *BMJ* 311.7003, p. 485. doi: [10.1136/bmj.311.7003.485](https://doi.org/10.1136/bmj.311.7003.485).
- Altman, D. G. and J. M. Bland (1995b). "Statistics Notes: The Normal Distribution". In: *BMJ* 310.6975, p. 298. doi: [10.1136/bmj.310.6975.298](https://doi.org/10.1136/bmj.310.6975.298).
- Bland, J. M. and D. G. Altman (1995). "Multiple Significance Tests: The Bonferroni Method". In: *BMJ* 310.6973, p. 170. doi: [10.1136/bmj.310.6973.170](https://doi.org/10.1136/bmj.310.6973.170).
- Bland, J. M. and D. G. Altman (2000). "The Odds Ratio". In: *BMJ* 320.7247, p. 1468. doi: [10.1136/bmj.320.7247.1468](https://doi.org/10.1136/bmj.320.7247.1468).
- Bland, M. (2015). *An Introduction to Medical Statistics*. 4th ed. Oxford: Oxford University Press.
- Broman, K. W. and K. H. Woo (2018). "Data Organization in Spreadsheets". In: *The American Statistician* 72.1, pp. 2–10. doi: [10.1080/00031305.2017.1375989](https://doi.org/10.1080/00031305.2017.1375989).
- Buuren, S. van and K. Groothuis-Oudshoorn (2011). "mice: Multivariate Imputation by Chained Equations in R". In: *Journal of Statistical Software* 45.3, pp. 1–67. doi: [10.18637/jss.v045.i03](https://doi.org/10.18637/jss.v045.i03).
- Elm, E. von, D. G. Altman, M. Egger, S. J. Pocock, P. C. Gøtzsche, and J. P. Vandenbroucke (2007). "The Strengthening the Reporting of Observational Studies in Epidemiology (STROBE) Statement: Guidelines for Reporting Observational Studies". In: *The Lancet* 370.9596, pp. 1453–1457. doi: [10.1016/S0140-6736\(07\)61602-X](https://doi.org/10.1016/S0140-6736(07)61602-X).
- Greenland, S., J. Pearl, and J. M. Robins (1999). "Causal Diagrams for Epidemiologic Research". In: *Epidemiology* 10.1, pp. 37–48. doi: [10.1097/00001648-199901000-00008](https://doi.org/10.1097/00001648-199901000-00008).
- Greenland, S., S. J. Senn, K. J. Rothman, J. B. Carlin, C. Poole, S. N. Goodman, and D. G. Altman (2016). "Statistical Tests, P Values, Confidence Intervals, and Power: A Guide to Misinterpretations". In: *European Journal of Epidemiology* 31.4, pp. 337–350. doi: [10.1007/s10654-016-0149-3](https://doi.org/10.1007/s10654-016-0149-3).
- Grolemund, G. and H. Wickham (2011). "Dates and Times Made Easy with lubridate". In: *Journal of Statistical Software* 40.3, pp. 1–25. doi: [10.18637/jss.v040.i03](https://doi.org/10.18637/jss.v040.i03).
- Hanley, J. A. and B. J. McNeil (1982). "The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve". In: *Radiology* 143.1, pp. 29–36. doi: [10.1148/radiology.143.1.7063747](https://doi.org/10.1148/radiology.143.1.7063747).
- Harrell, F. E. (2015). *Regression Modeling Strategies: With Applications to Linear Models, Logistic and Ordinal Regression, and Survival Analysis*. 2nd ed. Cham: Springer.
- Heinze, G., C. Wallisch, and D. Dunkler (2018). "Variable Selection – A Review and Recommendations for the Practicing Statistician". In: *Biometrical Journal* 60.3, pp. 431–449. doi: [10.1002/bimj.201700067](https://doi.org/10.1002/bimj.201700067).
- Hosmer, D. W. and S. Lemeshow (1980). "Goodness of Fit Tests for the Multiple Logistic Regression Model". In: *Communications in Statistics – Theory and Methods* 9.10, pp. 1035–1046. doi: [10.1080/03610928008827941](https://doi.org/10.1080/03610928008827941).

- Hosmer, D. W., S. Lemeshow, and R. X. Sturdivant (2013). *Applied Logistic Regression*. 3rd ed. Hoboken, NJ: Wiley.
- Kirkwood, B. R. and J. A. C. Sterne (2003). *Essential Medical Statistics*. 2nd ed. Oxford: Blackwell Science.
- Kruskal, W. H. and W. A. Wallis (1952). "Use of Ranks in One-Criterion Variance Analysis". In: *Journal of the American Statistical Association* 47.260, pp. 583–621. doi: [10.1080/01621459.1952.10483441](https://doi.org/10.1080/01621459.1952.10483441).
- Little, R. J. A. and D. B. Rubin (2019). *Statistical Analysis with Missing Data*. 3rd ed. Hoboken, NJ: Wiley.
- Lumley, T., P. Diehr, S. Emerson, and L. Chen (2002). "The Importance of the Normality Assumption in Large Public Health Data Sets". In: *Annual Review of Public Health* 23, pp. 151–169. doi: [10.1146/annurev.publhealth.23.100901.140546](https://doi.org/10.1146/annurev.publhealth.23.100901.140546).
- Mann, H. B. and D. R. Whitney (1947). "On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other". In: *The Annals of Mathematical Statistics* 18.1, pp. 50–60. doi: [10.1214/aoms/1177730491](https://doi.org/10.1214/aoms/1177730491).
- Mills, K. T., A. Stefanescu, and J. He (2020). "The Global Epidemiology of Hypertension". In: *Nature Reviews Nephrology* 16.4, pp. 223–237. doi: [10.1038/s41581-019-0244-2](https://doi.org/10.1038/s41581-019-0244-2).
- NCD Risk Factor Collaboration (NCD-RisC) (2021). "Worldwide Trends in Hypertension Prevalence and Progress in Treatment and Control from 1990 to 2019: A Pooled Analysis of 1201 Population-Representative Studies with 104 Million Participants". In: *The Lancet* 398.10304, pp. 957–980. doi: [10.1016/S0140-6736\(21\)01330-1](https://doi.org/10.1016/S0140-6736(21)01330-1).
- Pearson, K. (1900). "On the Criterion that a Given System of Deviations from the Probable in the Case of a Correlated System of Variables is Such that it Can be Reasonably Supposed to have Arisen from Random Sampling". In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 50.302, pp. 157–175. doi: [10.1080/14786440009463897](https://doi.org/10.1080/14786440009463897).
- Peduzzi, P., J. Concato, E. Kemper, T. R. Holford, and A. R. Feinstein (1996). "A Simulation Study of the Number of Events per Variable in Logistic Regression Analysis". In: *Journal of Clinical Epidemiology* 49.12, pp. 1373–1379. doi: [10.1016/S0895-4356\(96\)00236-3](https://doi.org/10.1016/S0895-4356(96)00236-3).
- Peng, R. D. (2011). "Reproducible Research in Computational Science". In: *Science* 334.6060, pp. 1226–1227. doi: [10.1126/science.1213847](https://doi.org/10.1126/science.1213847).
- R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria.
- Rothman, K. J., S. Greenland, and T. L. Lash (2008). *Modern Epidemiology*. 3rd ed. Philadelphia: Lippincott Williams & Wilkins.
- Rubin, D. B. (1976). "Inference and Missing Data". In: *Biometrika* 63.3, pp. 581–592. doi: [10.1093/biomet/63.3.581](https://doi.org/10.1093/biomet/63.3.581).
- Shapiro, S. S. and M. B. Wilk (1965). "An Analysis of Variance Test for Normality (Complete Samples)". In: *Biometrika* 52.3/4, pp. 591–611. doi: [10.2307/2333709](https://doi.org/10.2307/2333709).
- Sjoberg, D. D., K. Whiting, M. Curry, J. A. Lavery, and J. Larmarange (2021). "Reproducible Summary Tables with the gtsummary Package". In: *The R Journal* 13.1, pp. 570–580. doi: [10.32614/RJ-2021-053](https://doi.org/10.32614/RJ-2021-053).
- Spearman, C. (1904). "The Proof and Measurement of Association between Two Things". In: *The American Journal of Psychology* 15.1, pp. 72–101. doi: [10.2307/1412159](https://doi.org/10.2307/1412159).
- Sterne, J. A. C., I. R. White, J. B. Carlin, M. Spratt, P. Royston, M. G. Kenward, A. M. Wood, and J. R. Carpenter (2009). "Multiple Imputation for Missing Data in Epidemiological and Clinical Research: Potential and Pitfalls". In: *BMJ* 338, b2393. doi: [10.1136/bmj.b2393](https://doi.org/10.1136/bmj.b2393).
- Student (1908). "The Probable Error of a Mean". In: *Biometrika* 6.1, pp. 1–25. doi: [10.2307/2331554](https://doi.org/10.2307/2331554).

- VanderWeele, T. J. (2019). "Principles of Confounder Selection". In: *European Journal of Epidemiology* 34.3, pp. 211–219. doi: [10.1007/s10654-019-00494-6](https://doi.org/10.1007/s10654-019-00494-6).
- Vittinghoff, E., D. V. Glidden, S. C. Shiboski, and C. E. McCulloch (2012). *Regression Methods in Biostatistics: Linear, Logistic, Survival, and Repeated Measures Models*. 2nd ed. New York: Springer.
- Wasserstein, R. L. and N. A. Lazar (2016). "The ASA Statement on p-Values: Context, Process, and Purpose". In: *The American Statistician* 70.2, pp. 129–133. doi: [10.1080/00031305.2016.1154108](https://doi.org/10.1080/00031305.2016.1154108).
- Weissgerber, T. L., N. M. Milic, S. J. Winham, and V. D. Garovic (2015). "Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm". In: *PLOS Biology* 13.4, e1002128. doi: [10.1371/journal.pbio.1002128](https://doi.org/10.1371/journal.pbio.1002128).
- Welch, B. L. (1947). "The Generalization of 'Student's' Problem when Several Different Population Variances are Involved". In: *Biometrika* 34.1/2, pp. 28–35. doi: [10.2307/2332510](https://doi.org/10.2307/2332510).
- Westreich, D. and S. Greenland (2013). "The Table 2 Fallacy: Presenting and Interpreting Confounder and Modifier Coefficients". In: *American Journal of Epidemiology* 177.4, pp. 292–298. doi: [10.1093/aje/kws412](https://doi.org/10.1093/aje/kws412).
- Wickham, H. (2014). "Tidy Data". In: *Journal of Statistical Software* 59.10, pp. 1–23. doi: [10.18637/jss.v059.i10](https://doi.org/10.18637/jss.v059.i10).
- Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. 2nd ed. New York: Springer.
- Wickham, H., M. Çetinkaya-Rundel, and G. Grolemund (2023). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 2nd ed. Sebastopol, CA: O'Reilly Media.
- Wickham, H. et al. (2019). "Welcome to the tidyverse". In: *Journal of Open Source Software* 4.43, p. 1686. doi: [10.21105/joss.01686](https://doi.org/10.21105/joss.01686).
- Wilson, G., J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal (2017). "Good Enough Practices in Scientific Computing". In: *PLOS Computational Biology* 13.6, e1005510. doi: [10.1371/journal.pcbi.1005510](https://doi.org/10.1371/journal.pcbi.1005510).
- World Health Organization (2021). *Guideline for the Pharmacological Treatment of Hypertension in Adults*. Geneva: World Health Organization.
- Xie, Y. (2015). *Dynamic Documents with R and knitr*. 2nd ed. Boca Raton, FL: Chapman and Hall/CRC.
- Zhang, J. and K. F. Yu (1998). "What's the Relative Risk? A Method of Correcting the Odds Ratio in Cohort Studies of Common Outcomes". In: *JAMA* 280.19, pp. 1690–1691. doi: [10.1001/jama.280.19.1690](https://doi.org/10.1001/jama.280.19.1690).